

Transformer

Wei Xu

(many slides from Greg Durrett)

Administrivia

- ▶ Midterm 10/28 (close-book, close-note)
- ▶ Practice Midterm has been released on Ed
- ▶ Course Project:
 - Group of 2-4 people
 - A proposal of 1-2 pages (team forming + topic)
- ▶ Access to the PACE's Instructional GPU Cluster:
https://gatech.service-now.com/home?id=kb_article_view&sysparm_article=KB0042102

This Lecture

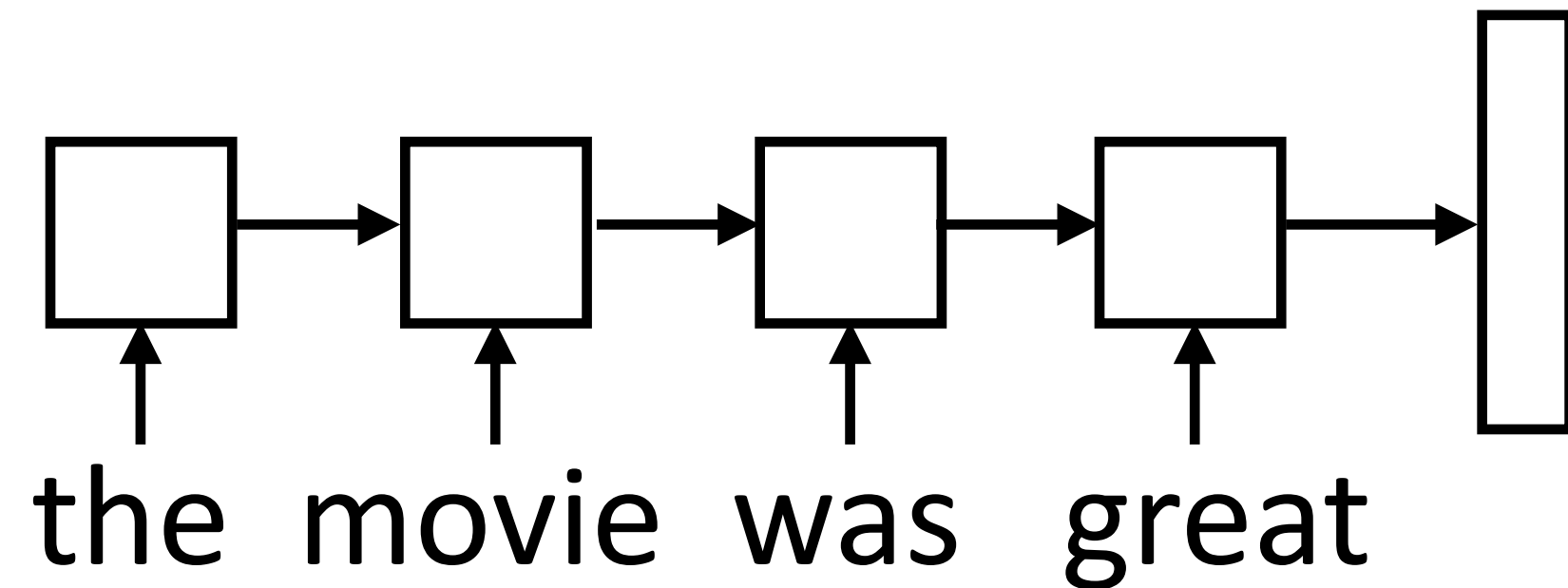
- ▶ Transformer architecture
- ▶ Layer Normalization
- ▶ Optimizers: SGD, Momentum, AdaGrad, Adam, ...

Readings

- ▶ “The Annotated Transformer” by Sasha Rush
<https://nlp.seas.harvard.edu/2018/04/03/attention.html>
- ▶ “The Illustrated Transformer” by Jay Lamar
<http://jalammar.github.io/illustrated-transformer/>
- ▶ Jurafsky+Martin Chapter 7

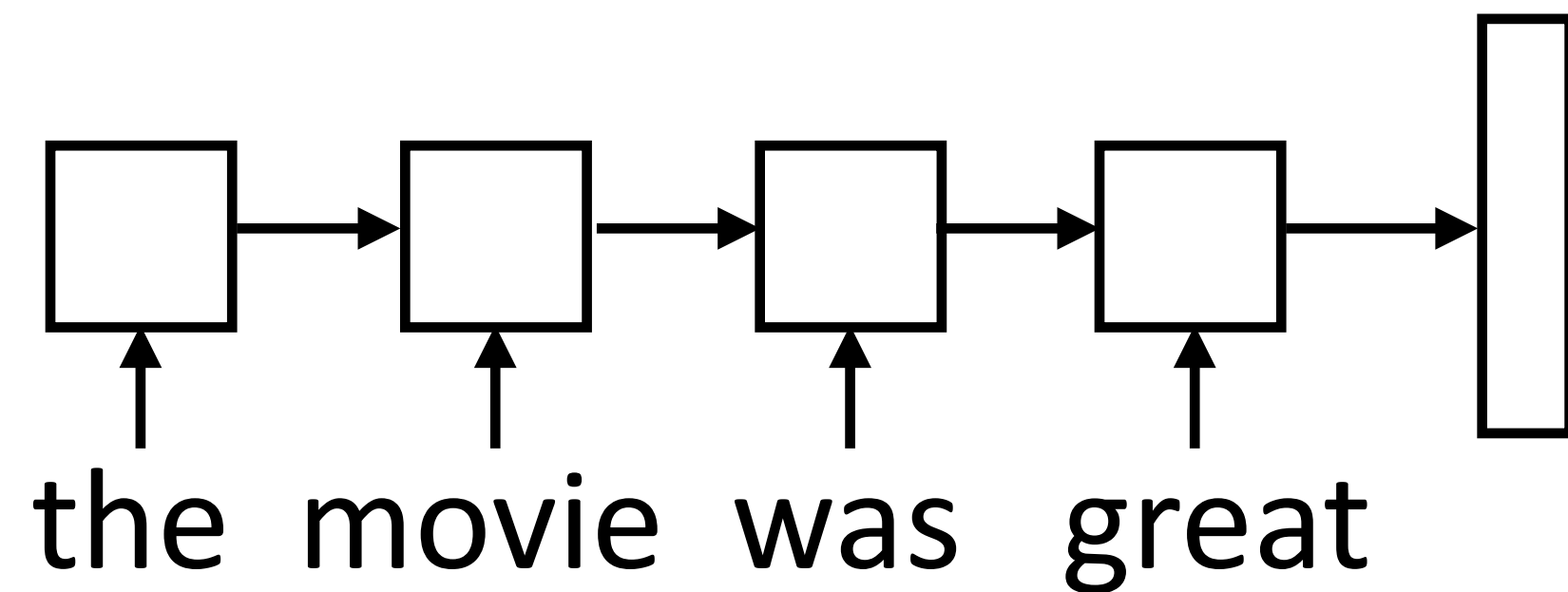
Recap: Encoder-Decoder

- ▶ Generate next word conditioned on previous word as well as hidden state



Recap: Encoder-Decoder

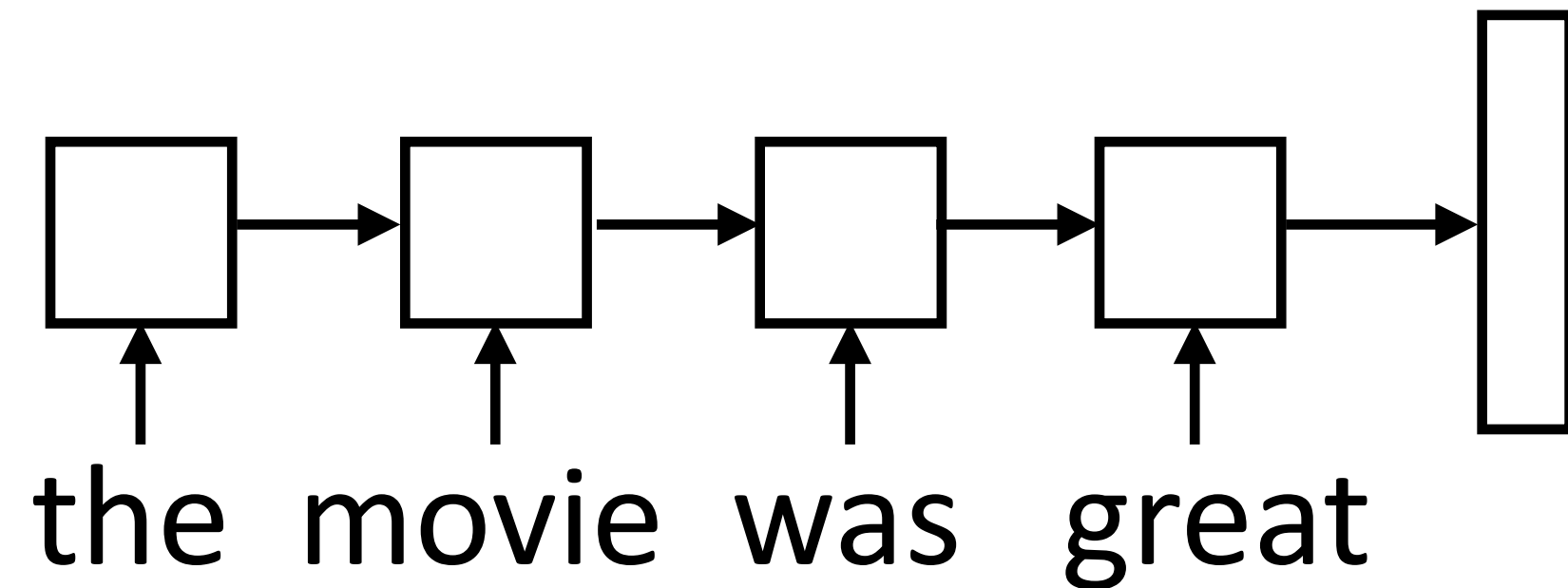
- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary



$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h})$$

Recap: Encoder-Decoder

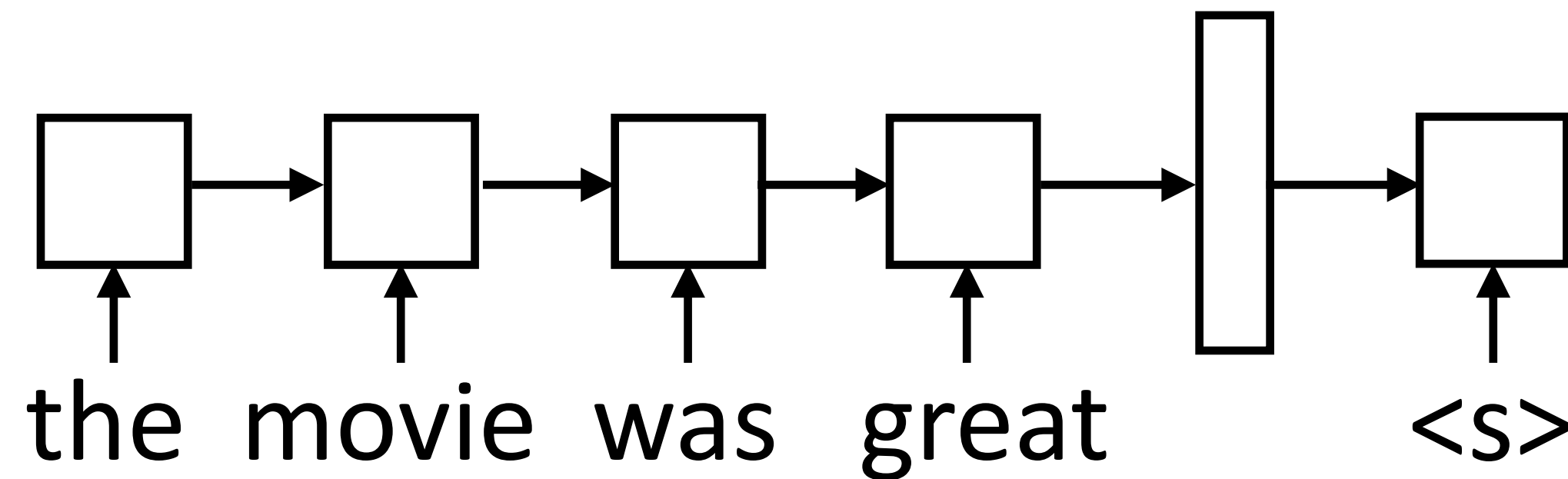
- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary



$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h}) \qquad P(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^n P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$$

Recap: Encoder-Decoder

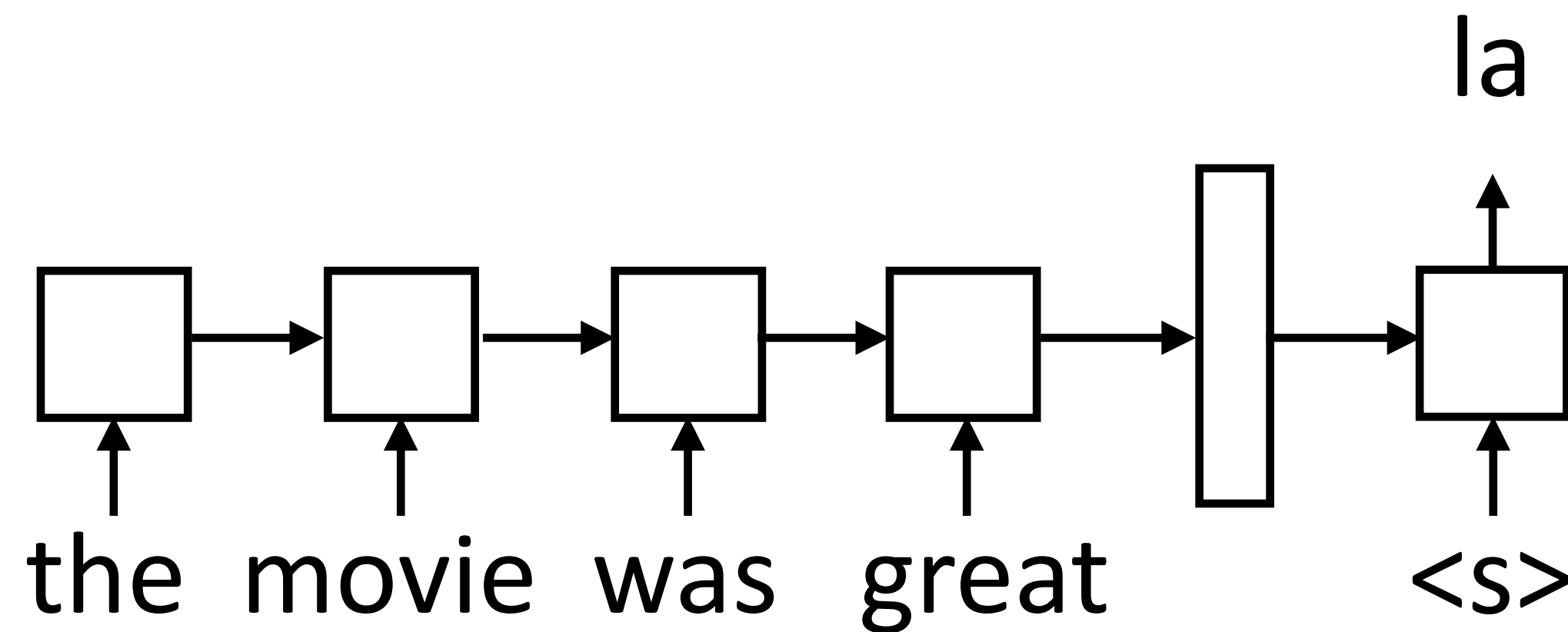
- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary



$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h}) \qquad P(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^n P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$$

Recap: Encoder-Decoder

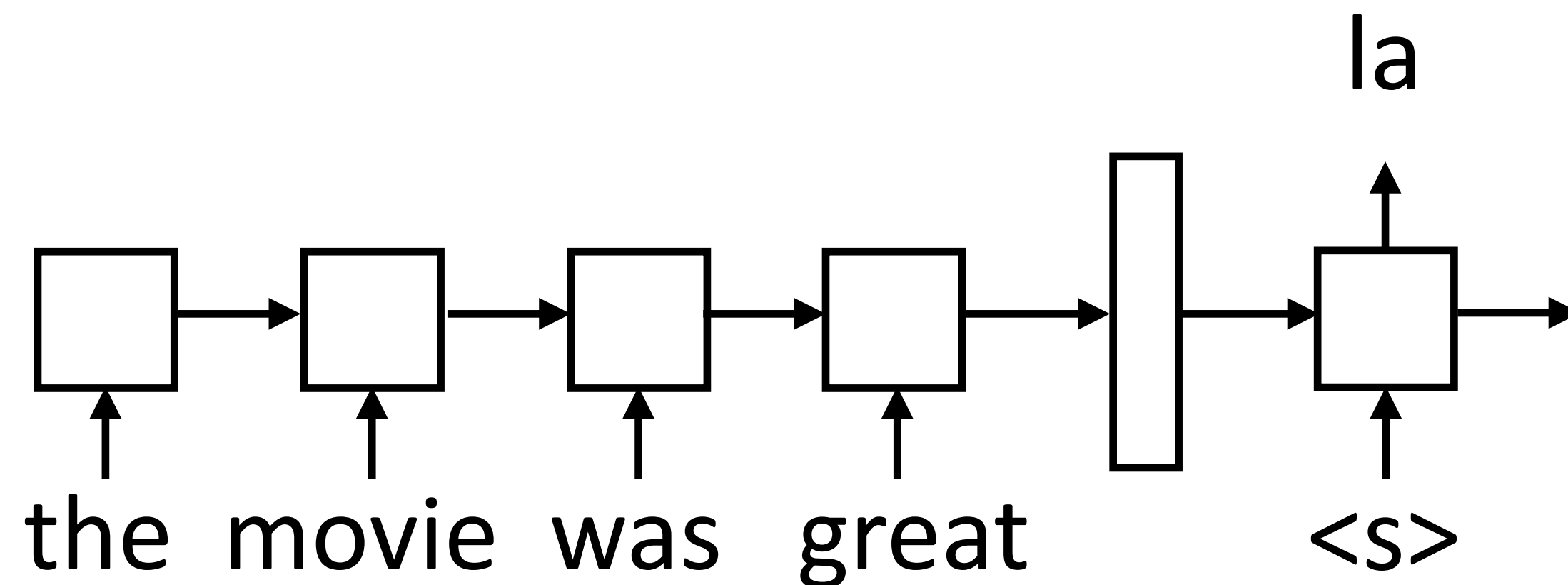
- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary



$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h}) \qquad P(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^n P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$$

Recap: Encoder-Decoder

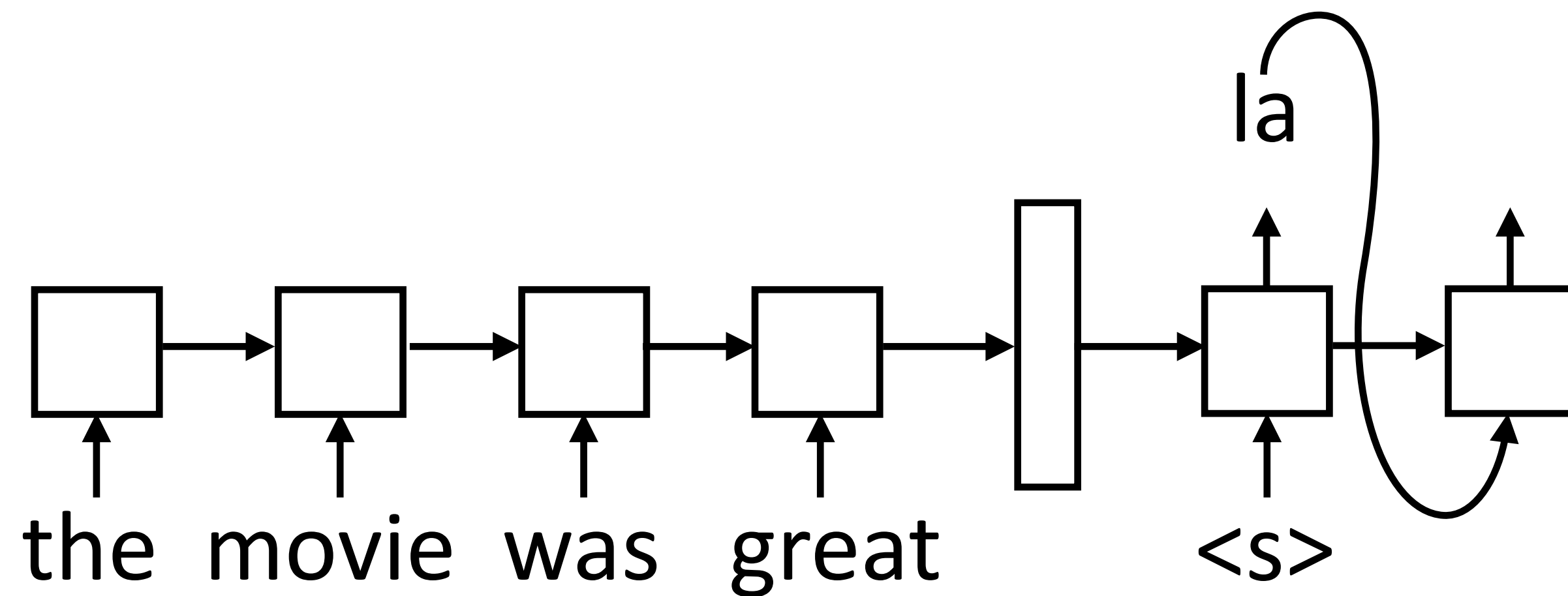
- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary



$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h}) \qquad P(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^n P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$$

Recap: Encoder-Decoder

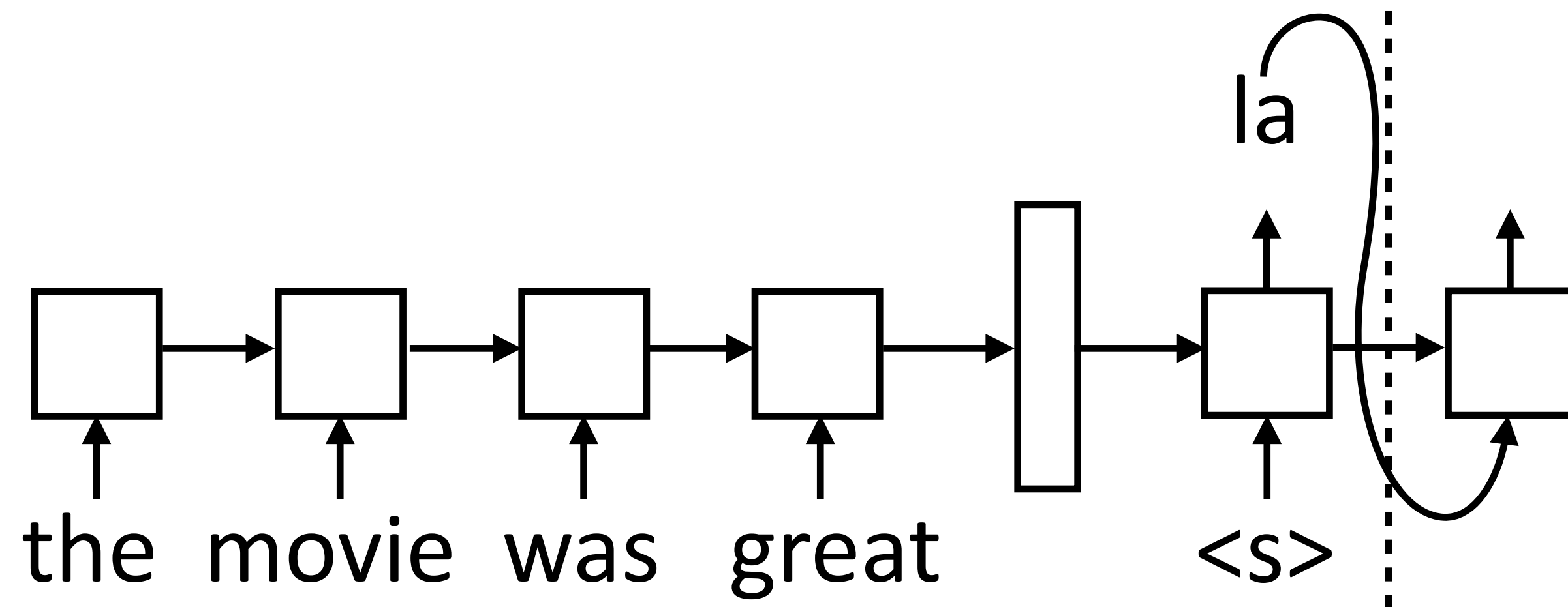
- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary



$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h}) \quad P(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^n P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$$

Recap: Encoder-Decoder

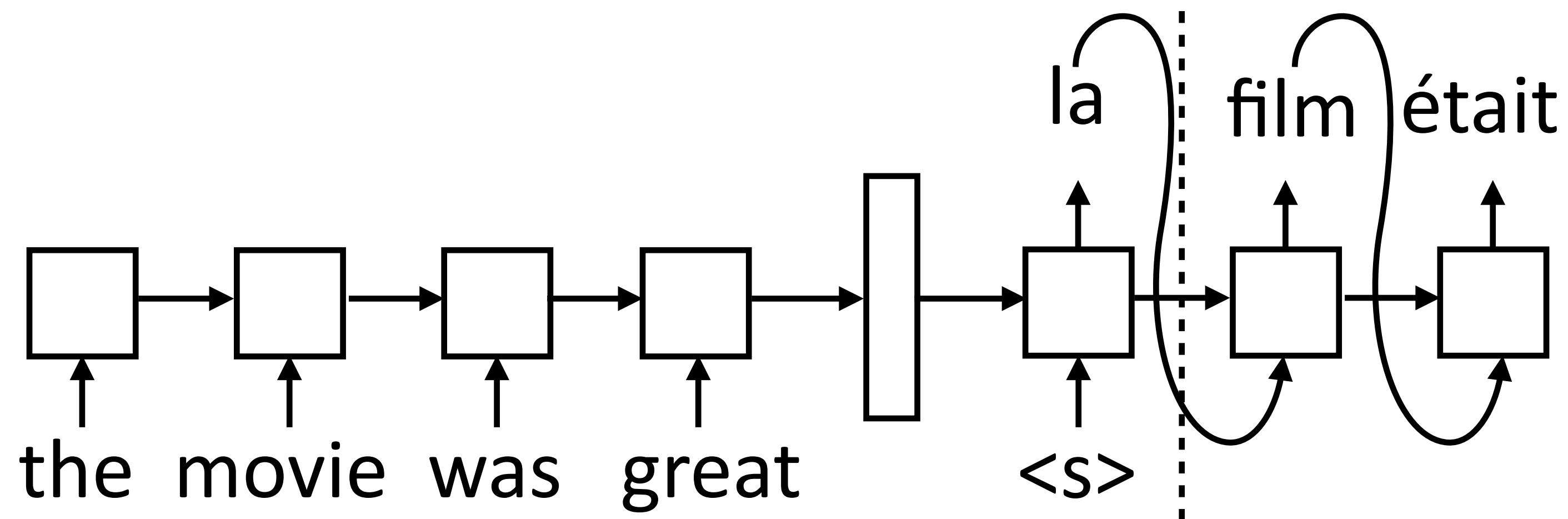
- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary



$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h}) \quad P(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^n P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$$

Recap: Encoder-Decoder

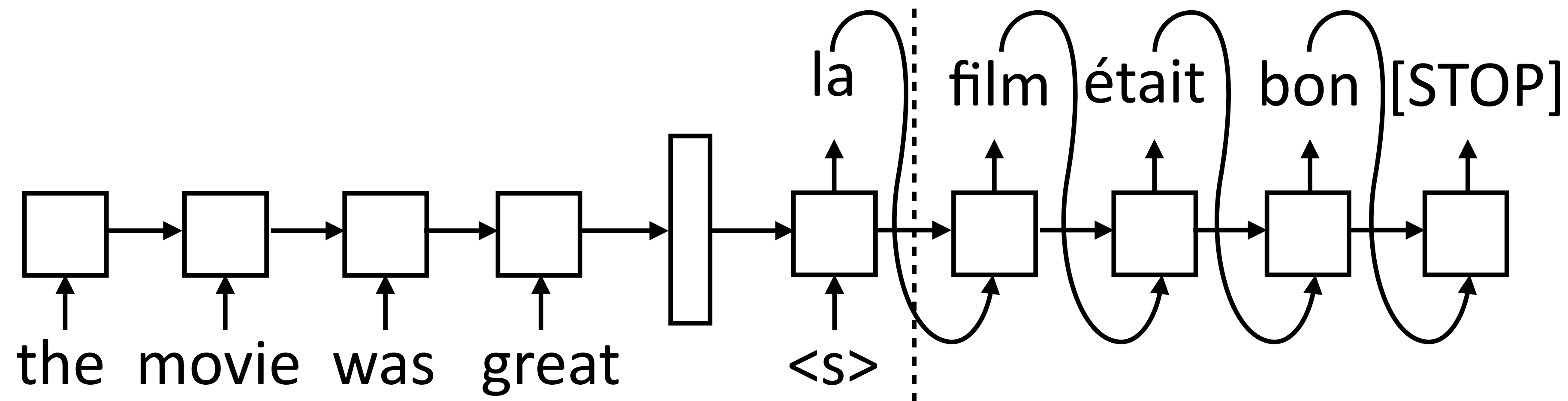
- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary



$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h}) \qquad P(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^n P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$$

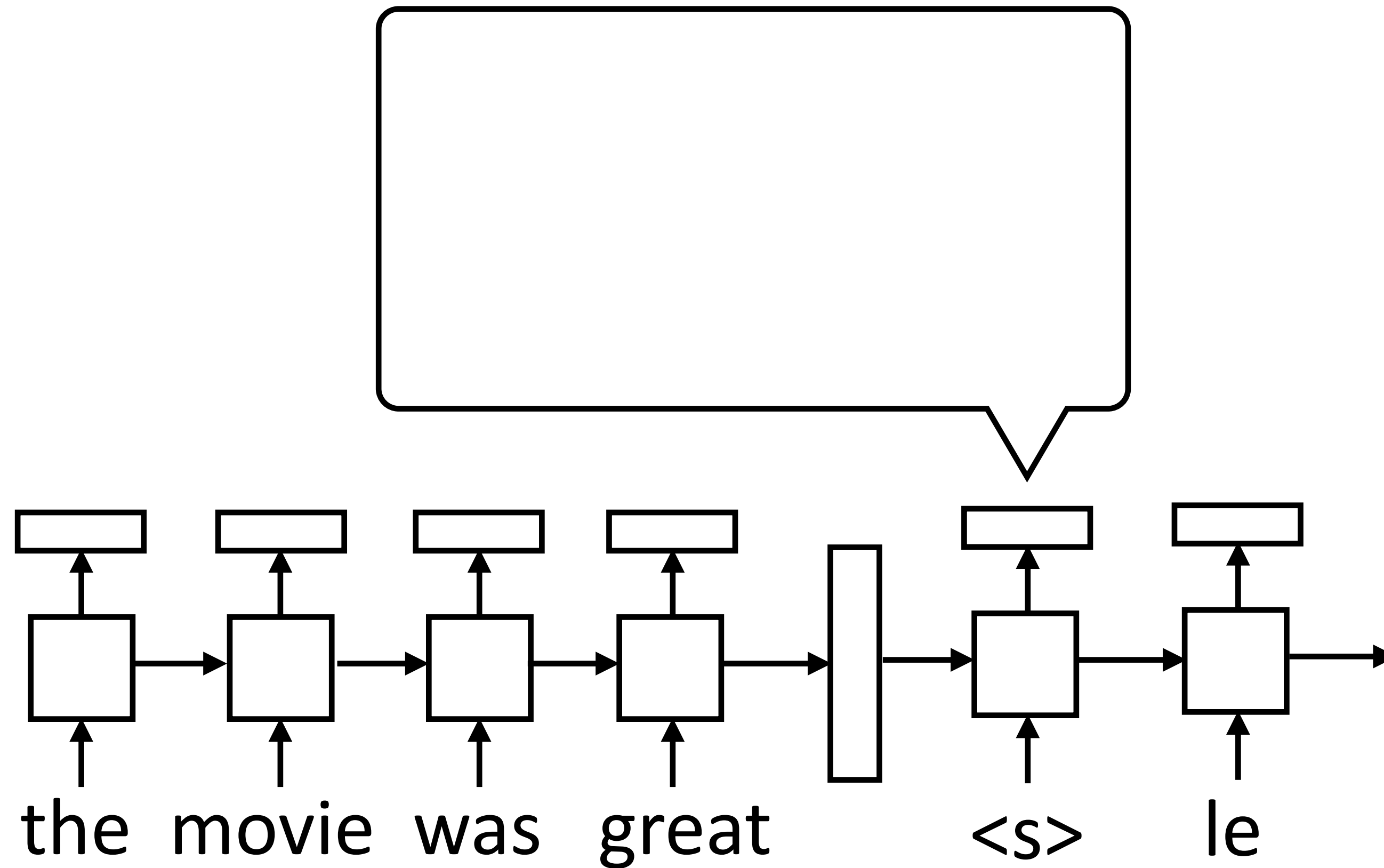
Recap: Encoder-Decoder

- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary

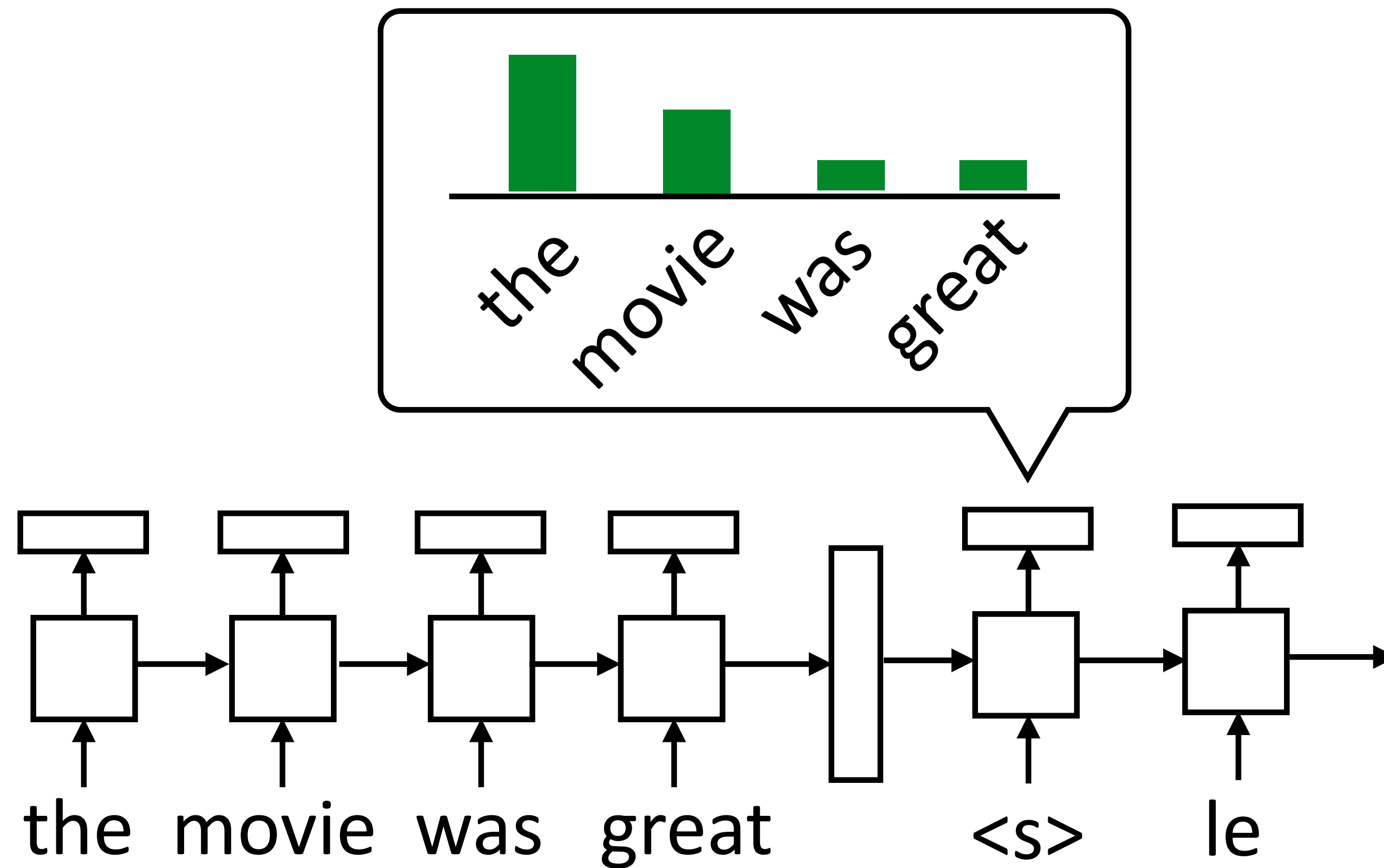


$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h}) \qquad P(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^n P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$$

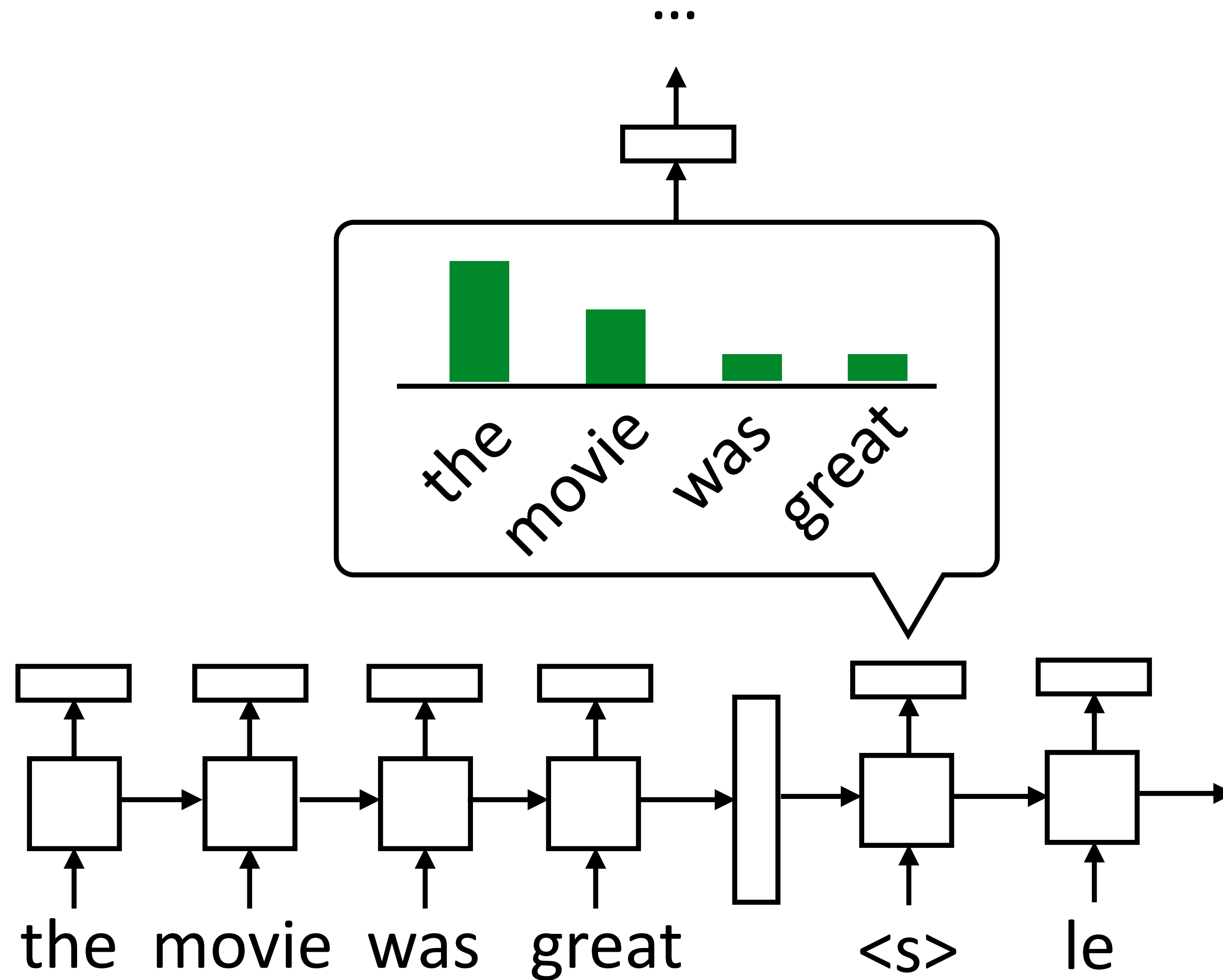
Recap: Attention



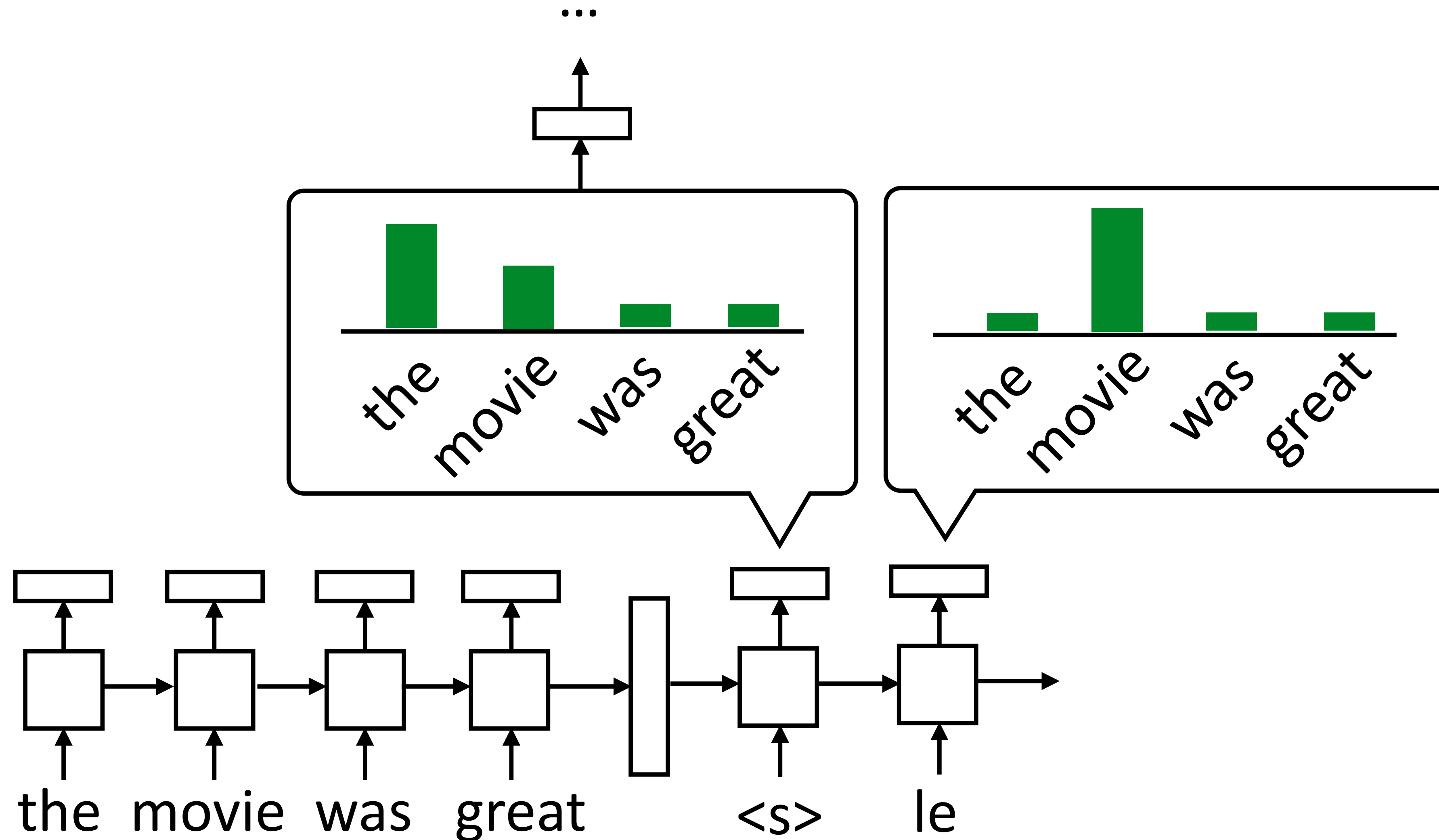
Recap: Attention



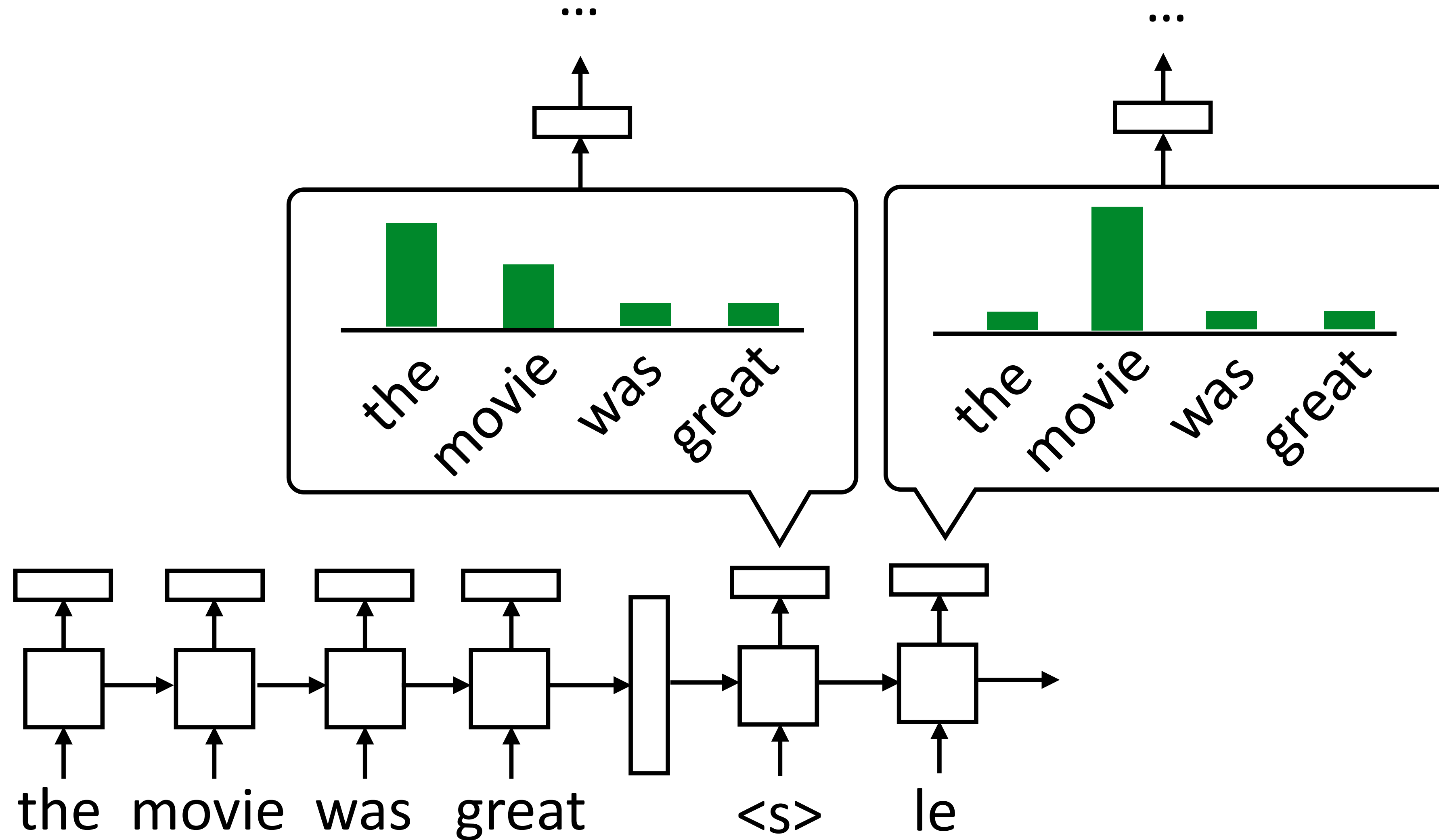
Recap: Attention



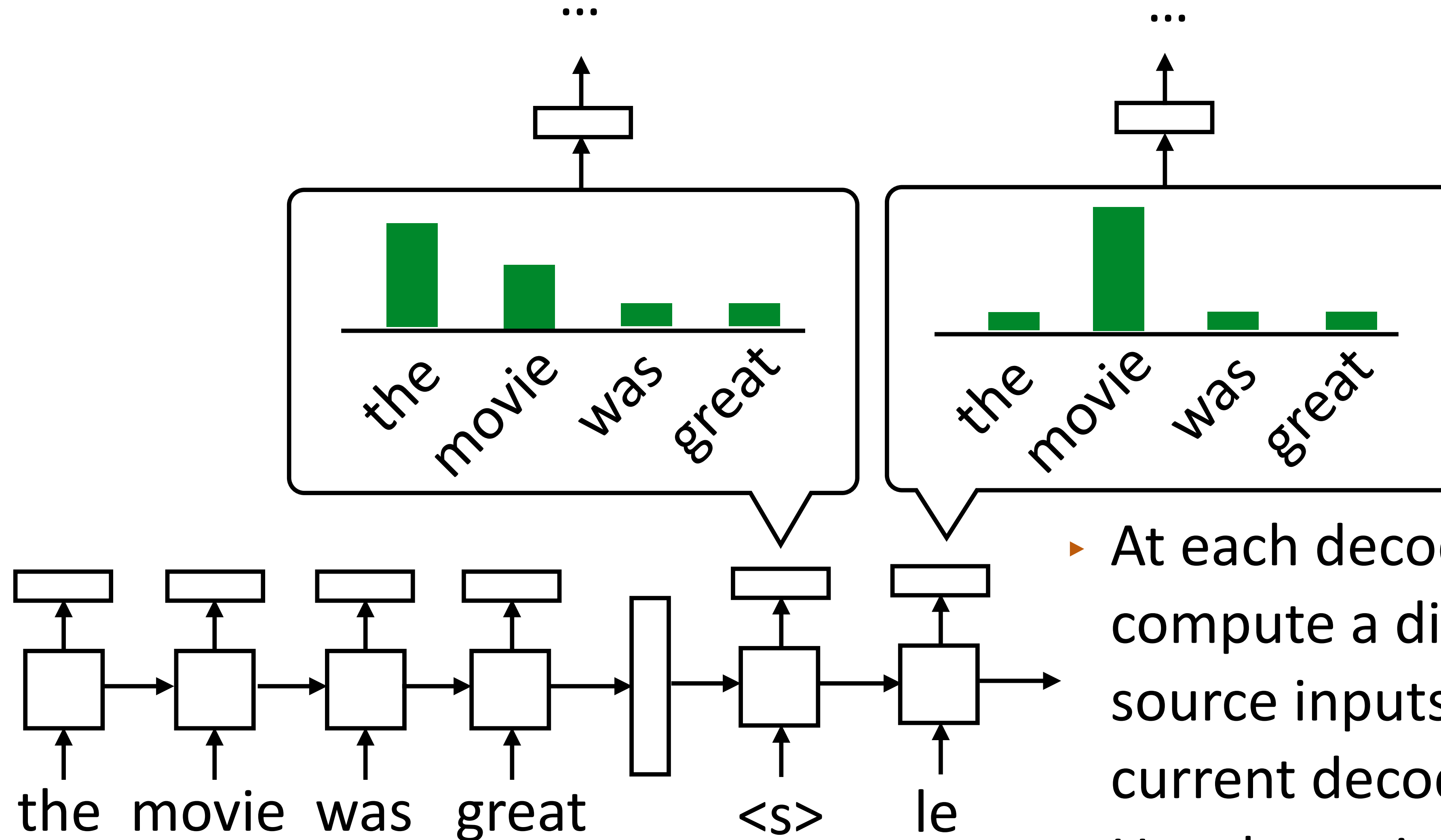
Recap: Attention



Recap: Attention



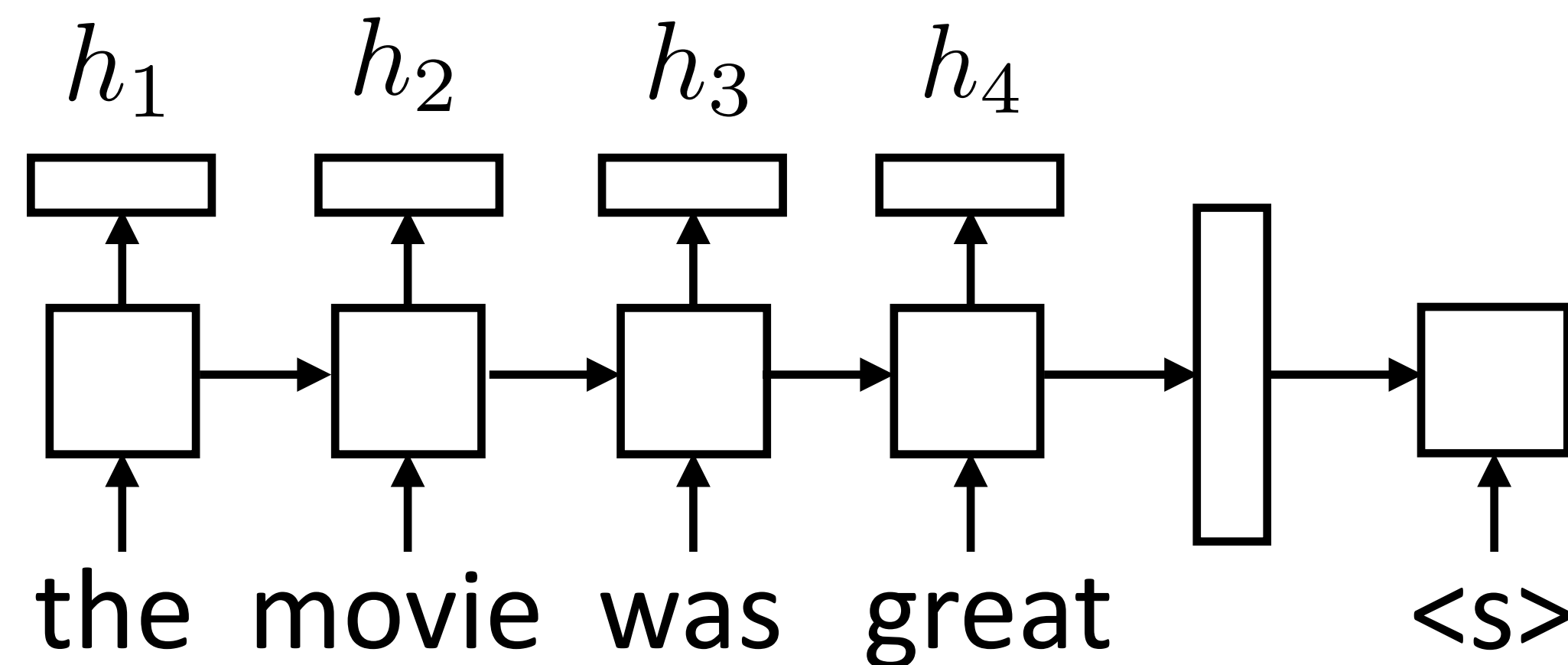
Recap: Attention



- ▶ At each decoder state, compute a distribution over source inputs based on current decoder state
- ▶ Use the weighted sum of input tokens to predict output

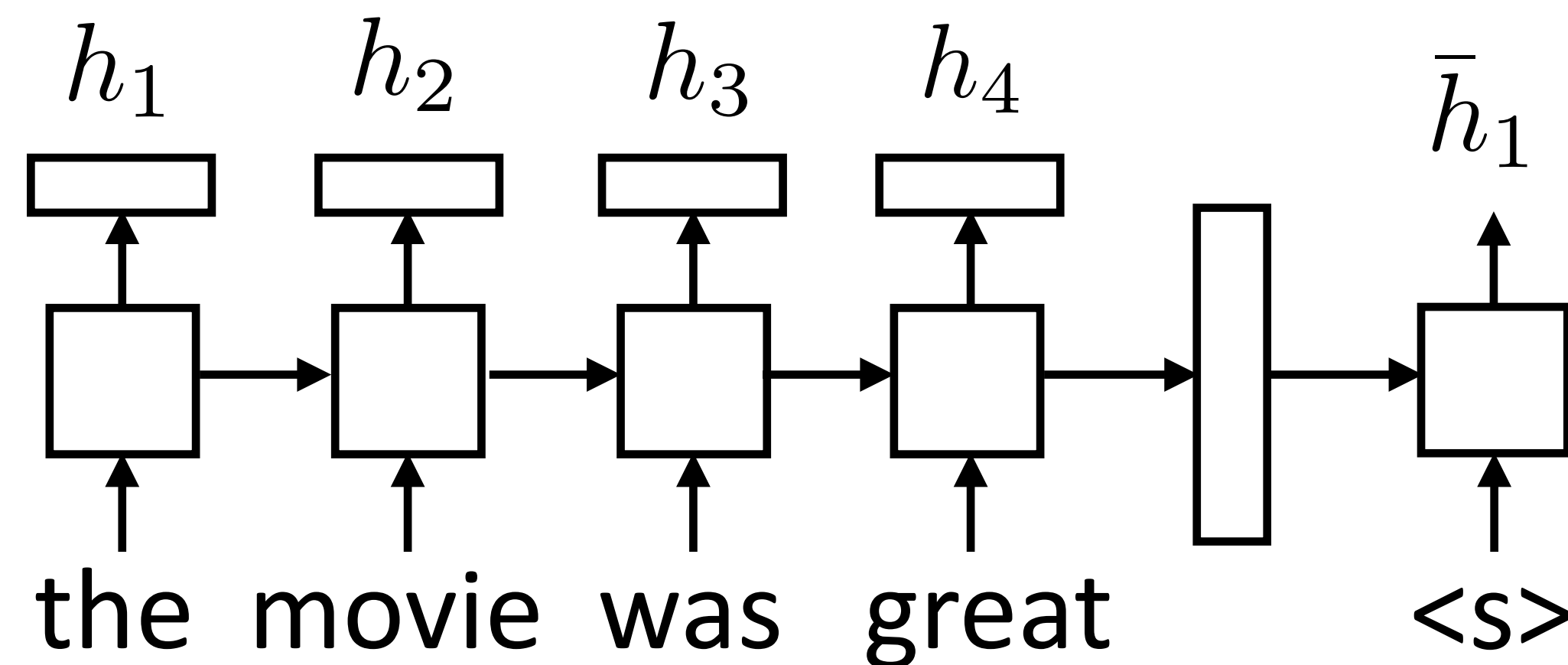
Recap: Attention

- ▶ For each decoder state, compute weighted sum of input states



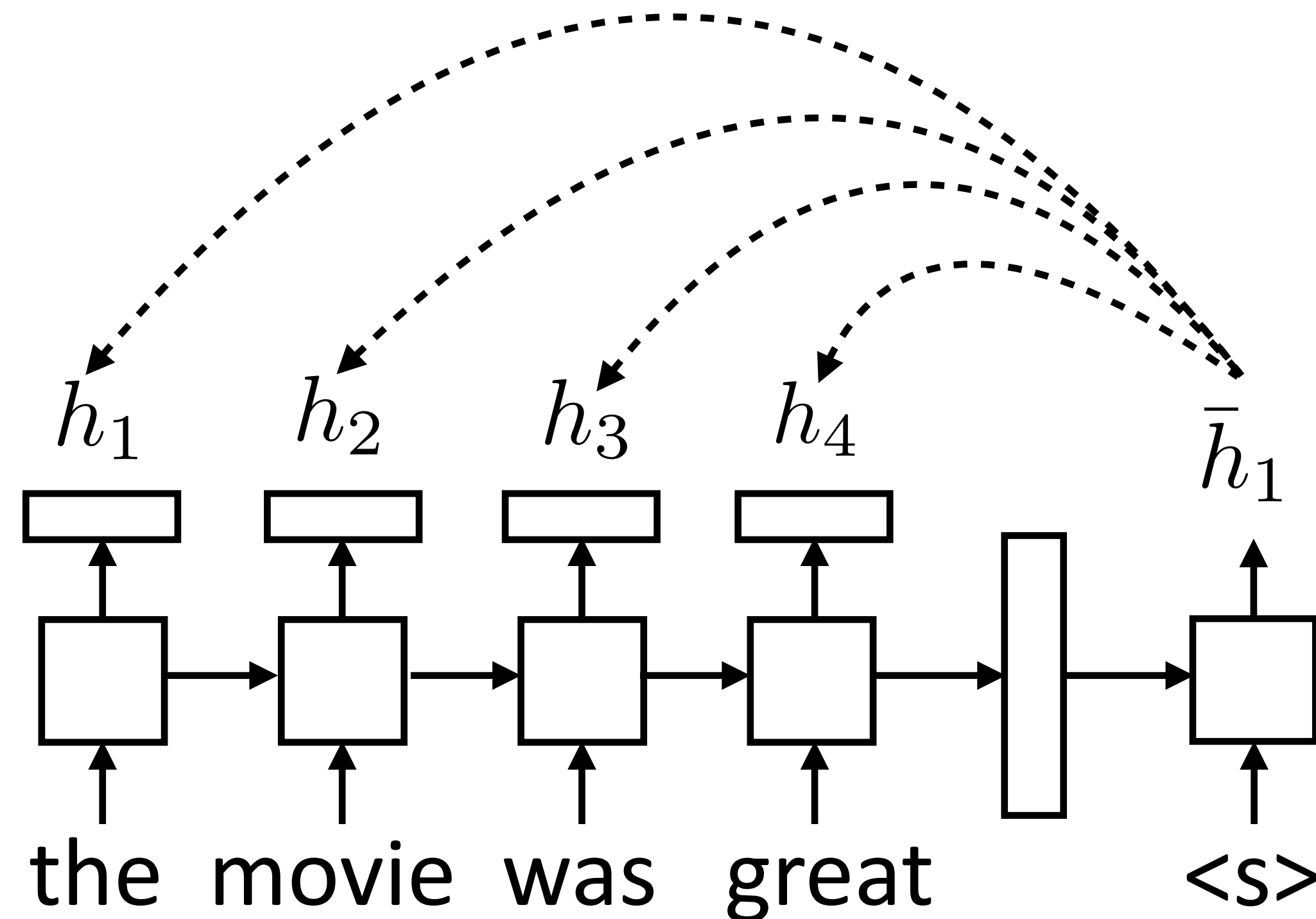
Recap: Attention

- ▶ For each decoder state, compute weighted sum of input states



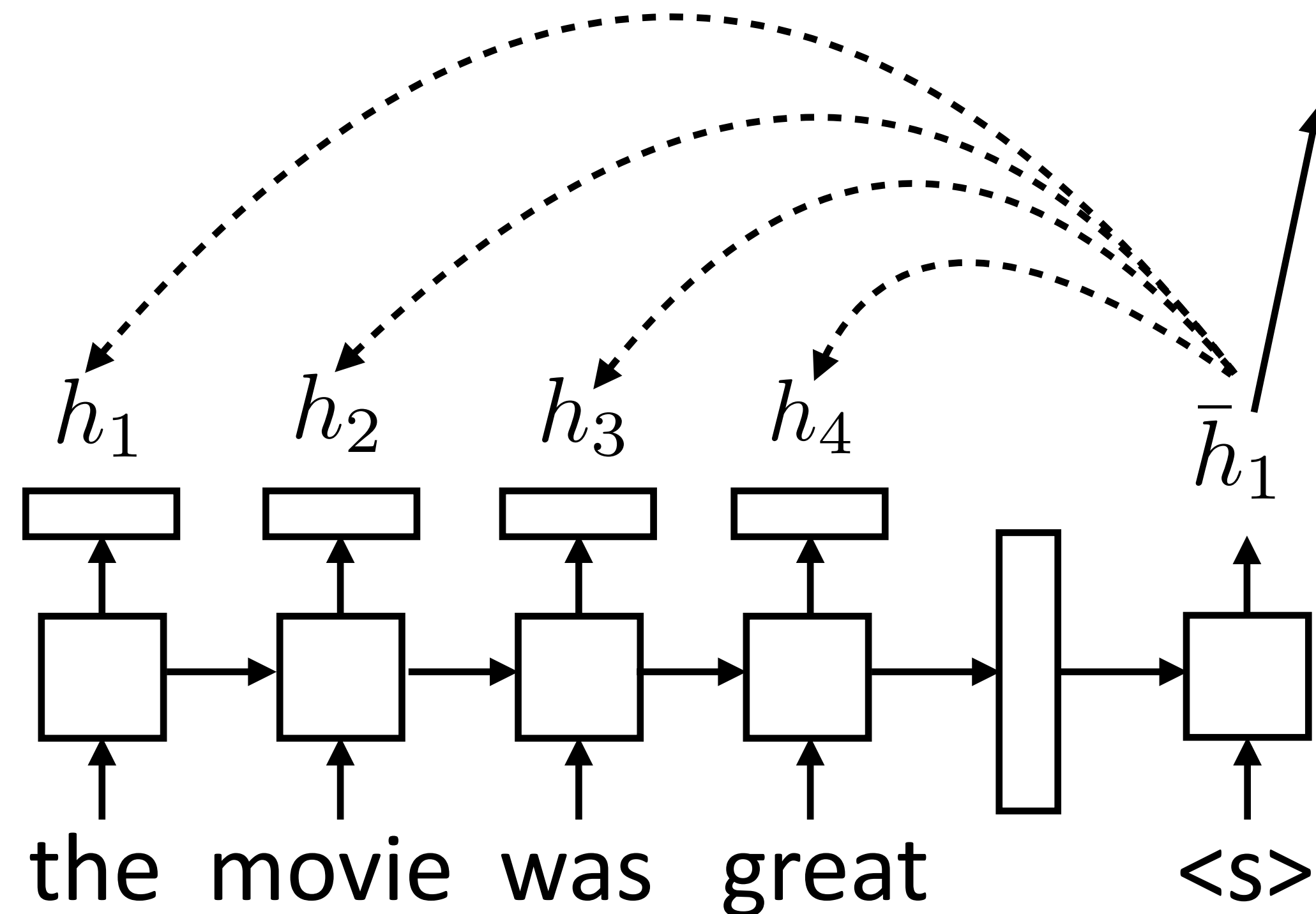
Recap: Attention

- For each decoder state, compute weighted sum of input states



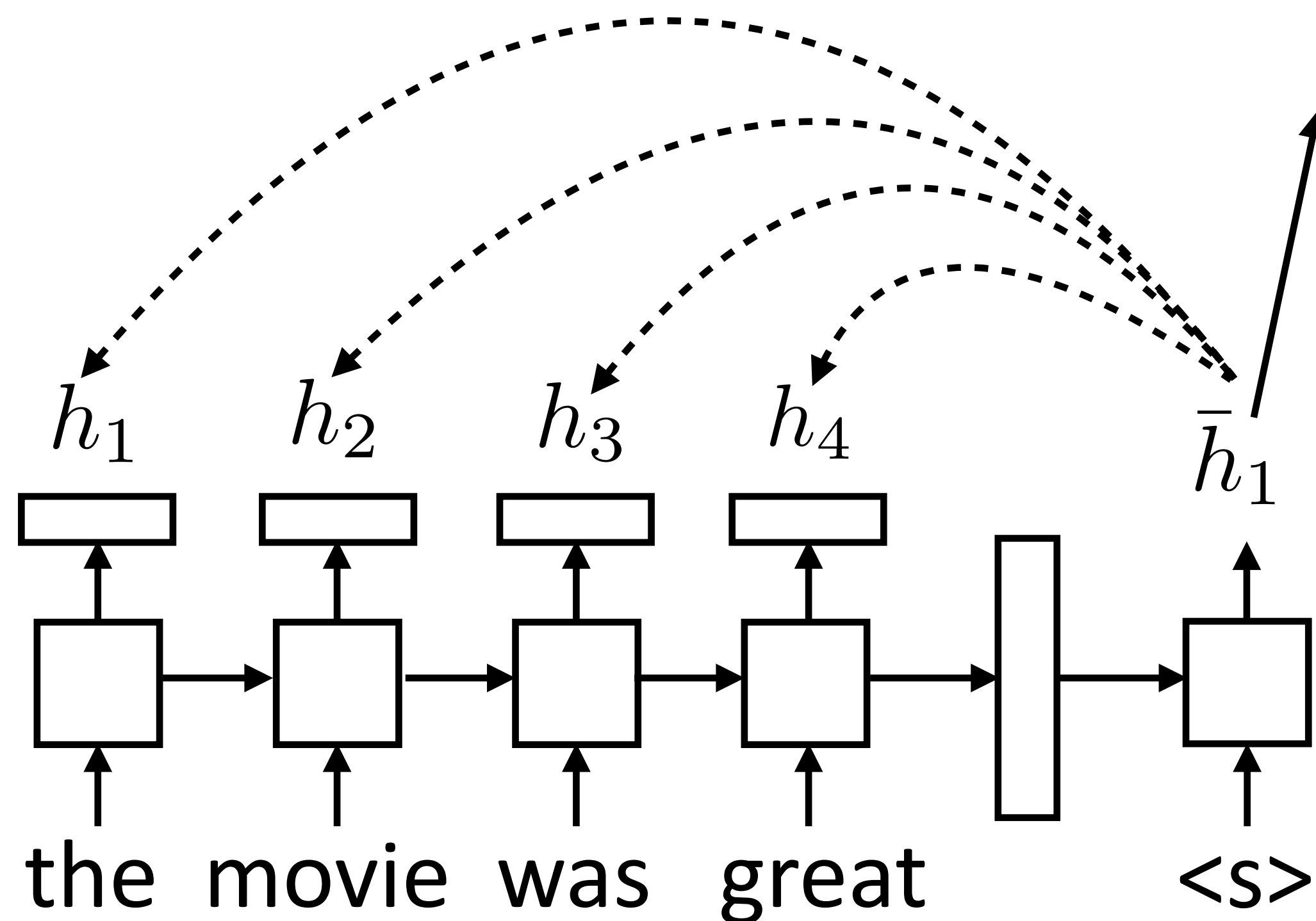
Recap: Attention

- For each decoder state, compute weighted sum of input states



Recap: Attention

- ▶ For each decoder state, compute weighted sum of input states

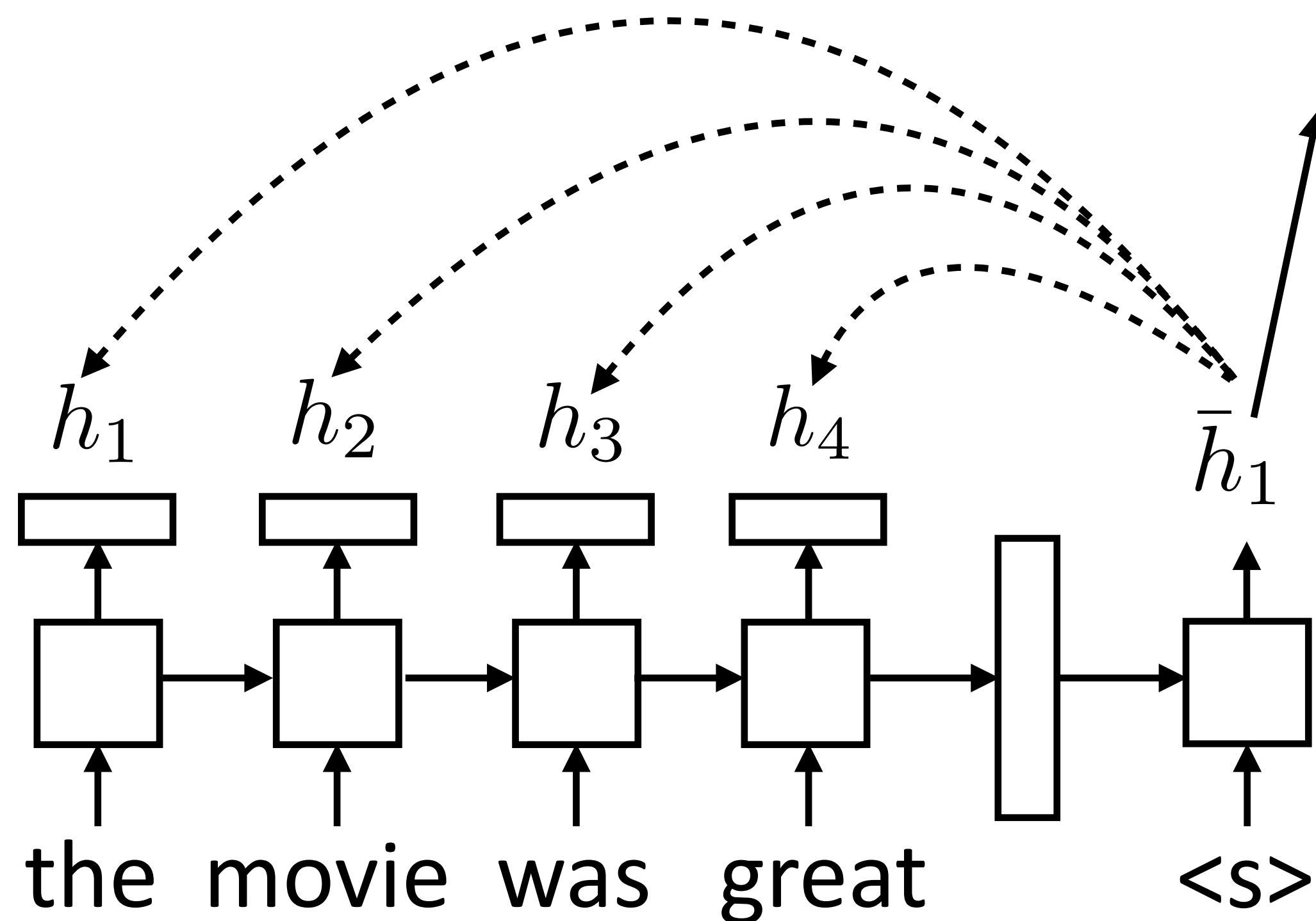


$$e_{ij} = f(\bar{h}_i, h_j)$$

- ▶ Some function f (e.g., dot product)

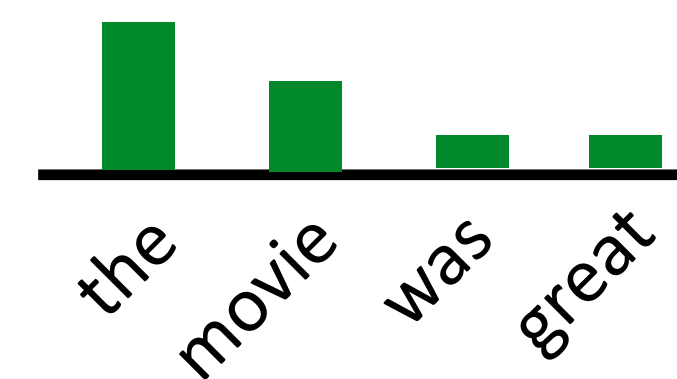
Recap: Attention

- For each decoder state, compute weighted sum of input states



$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

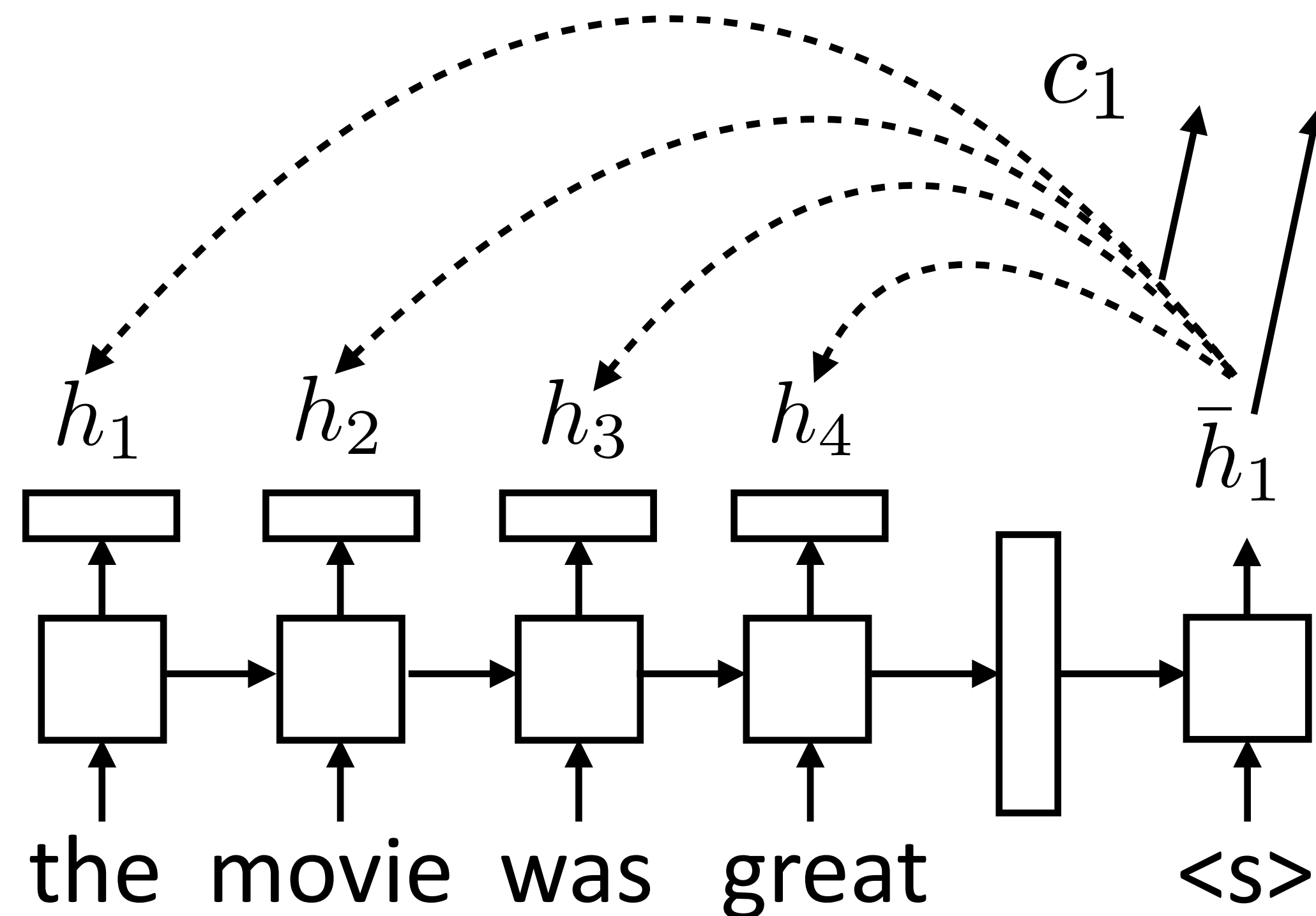
$$e_{ij} = f(\bar{h}_i, h_j)$$



- Some function f (e.g., dot product)

Recap: Attention

- ▶ For each decoder state, compute weighted sum of input states

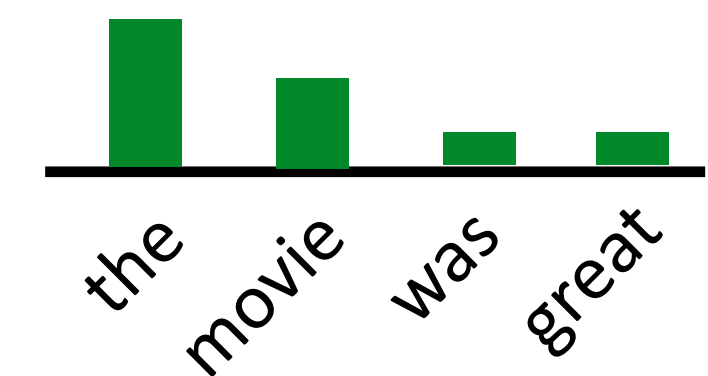


$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

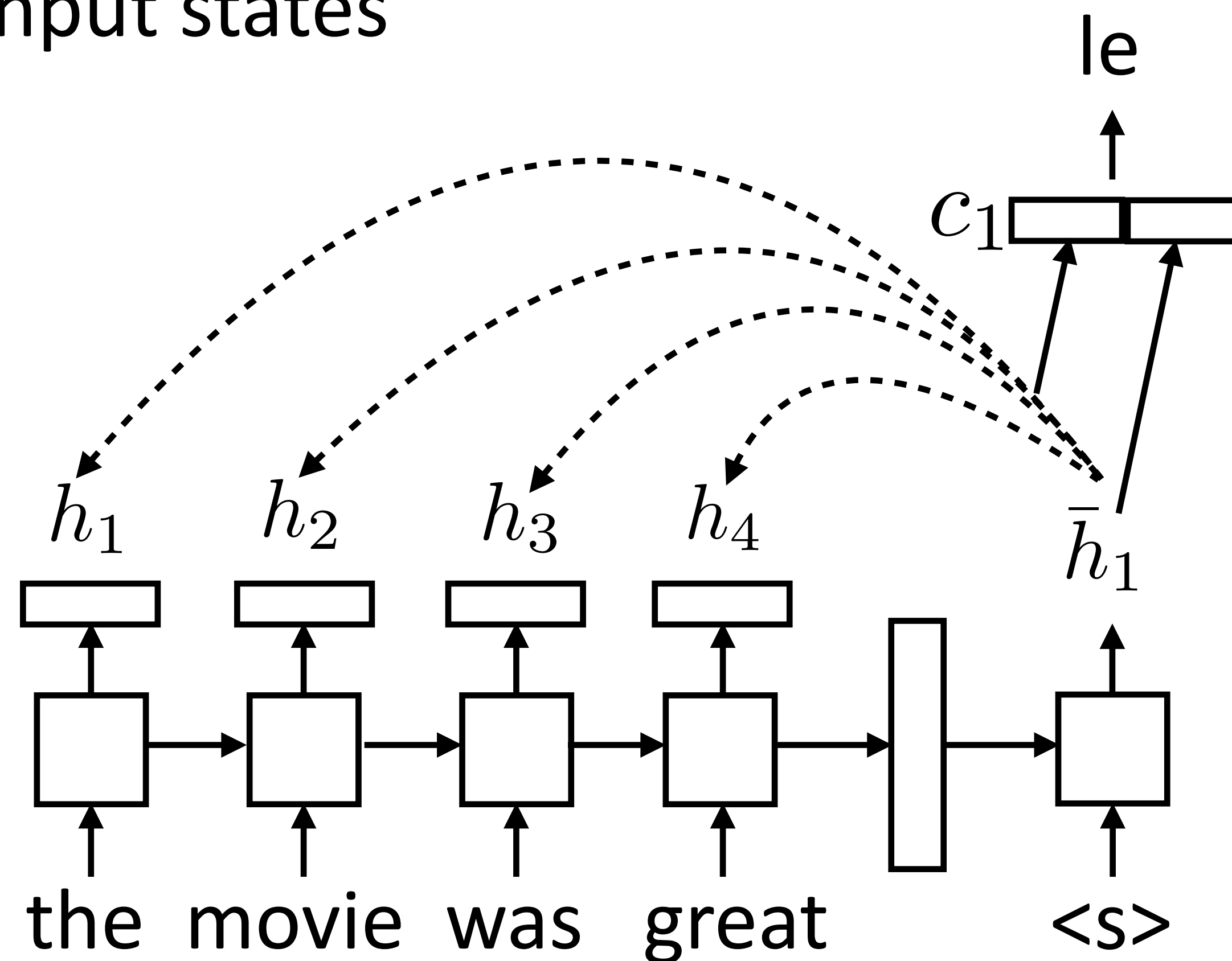
- ▶ Weighted sum of input hidden states (vector)



- ▶ Some function f (e.g., dot product)

Recap: Attention

- For each decoder state, compute weighted sum of input states

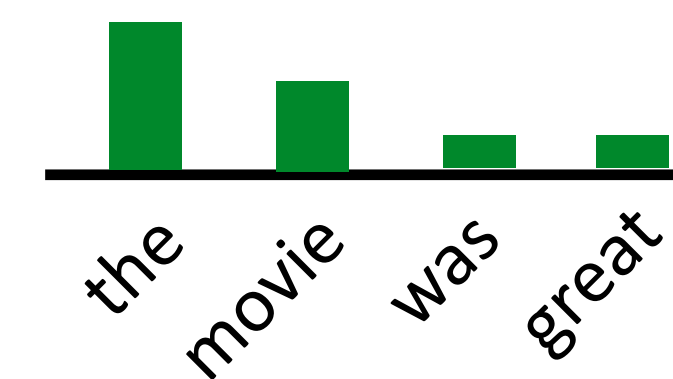


$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

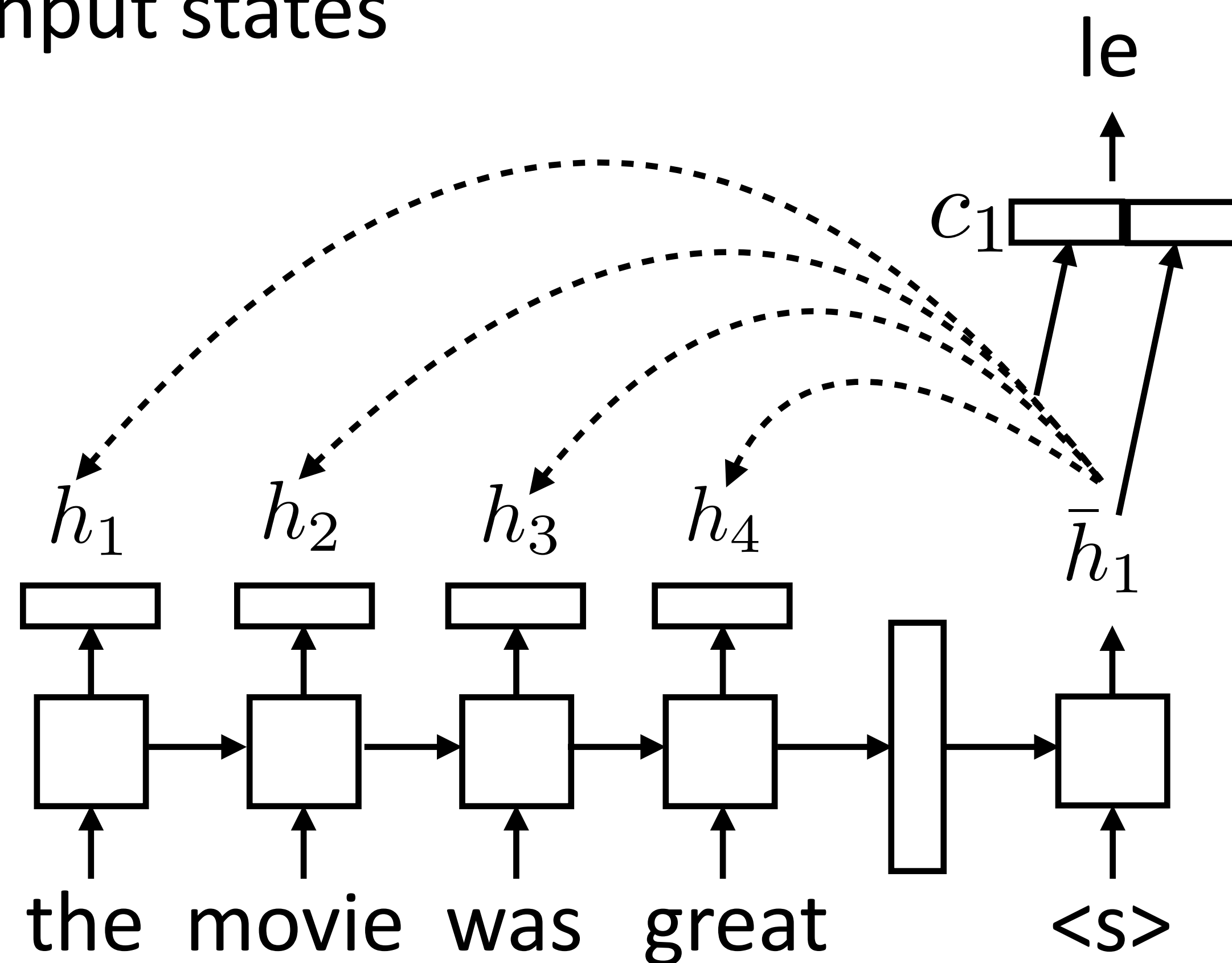
- Weighted sum of input hidden states (vector)



- Some function f (e.g., dot product)

Recap: Attention

- For each decoder state, compute weighted sum of input states



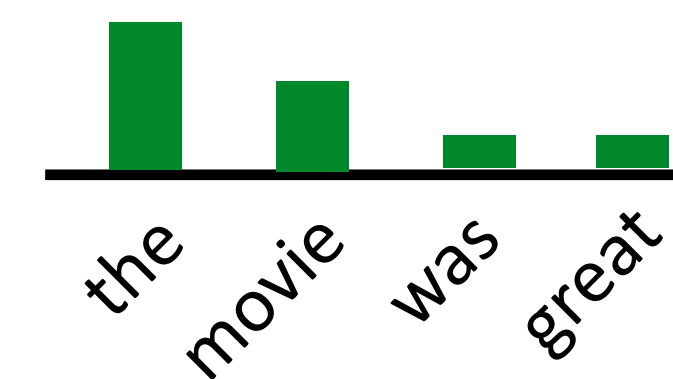
$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W[c_i; \bar{h}_i])$$

$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

- Weighted sum of input hidden states (vector)



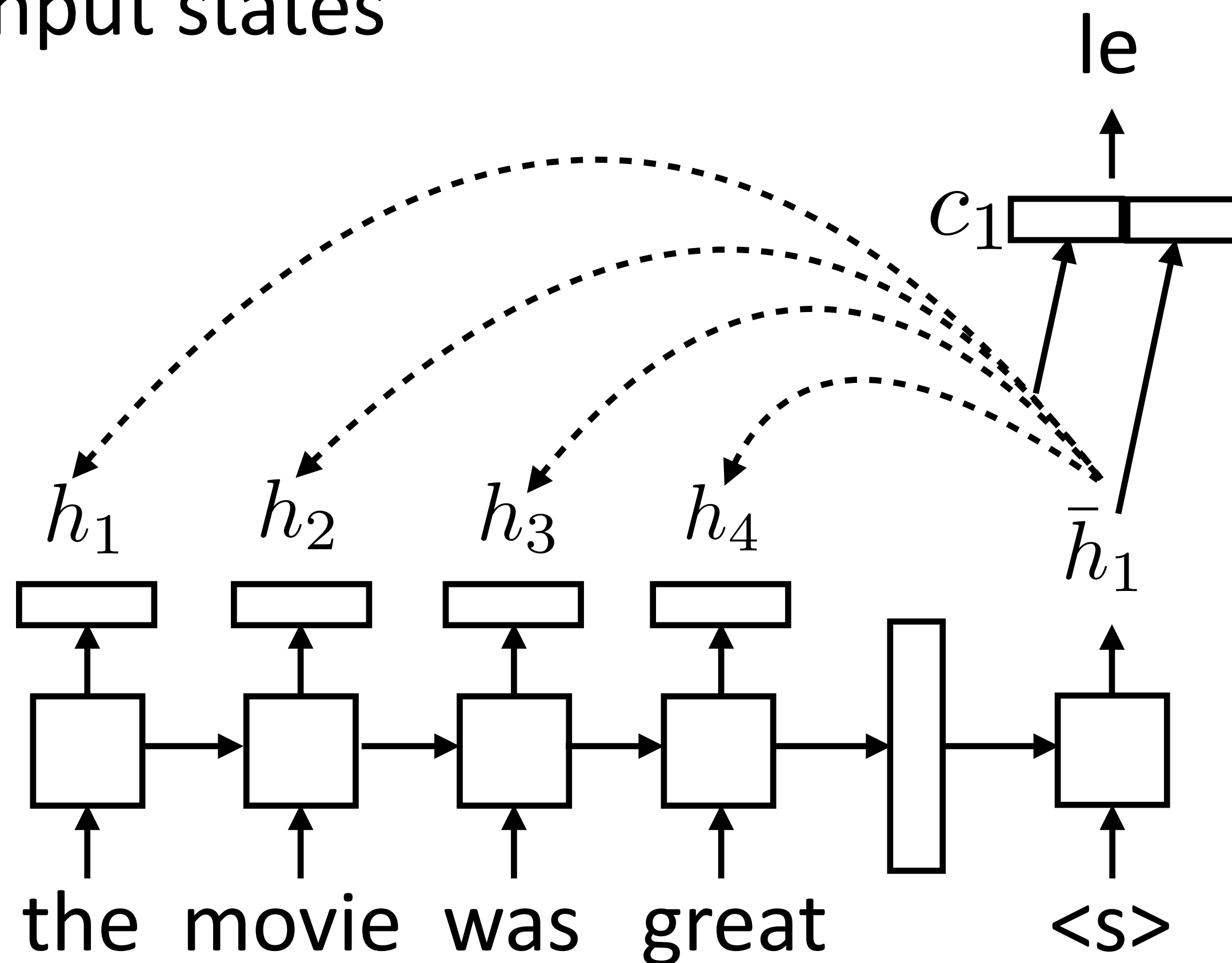
- Some function f (e.g., dot product)

Recap: Attention

- For each decoder state, compute weighted sum of input states

- No attn: $P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h}_i)$

$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W [c_i; \bar{h}_i])$$

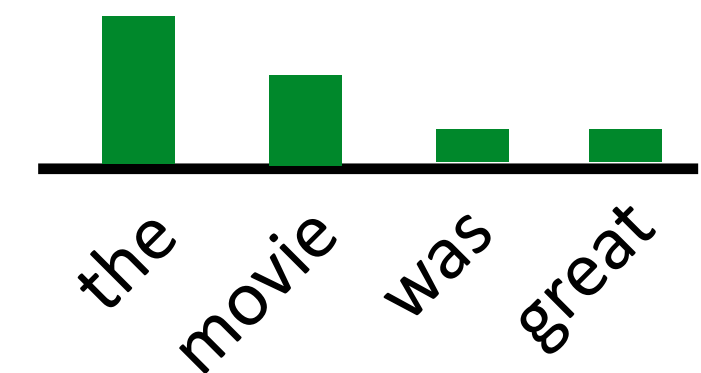


$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

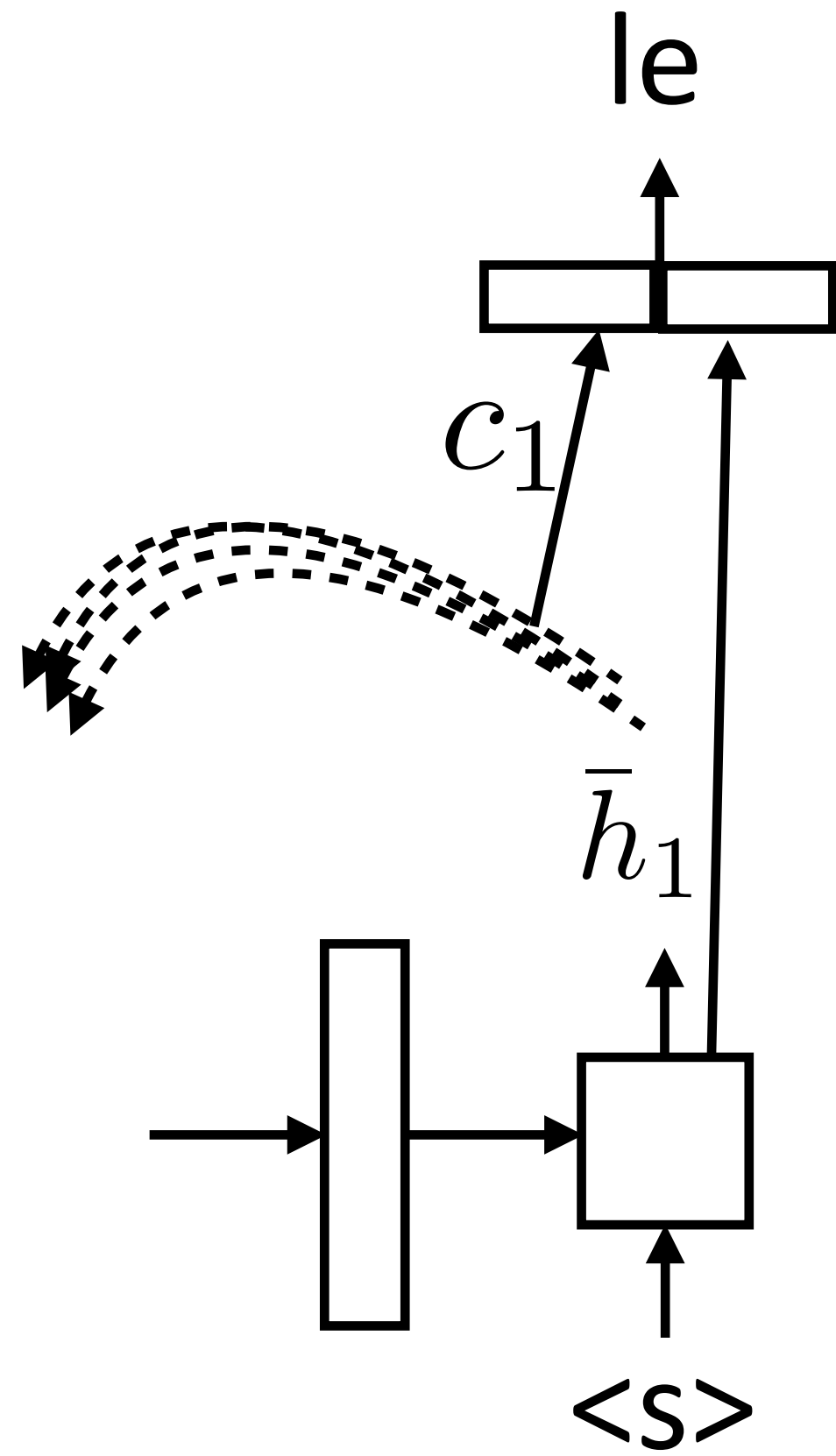
$$e_{ij} = f(\bar{h}_i, h_j)$$

- Weighted sum of input hidden states (vector)



- Some function f (e.g., dot product)

Attention

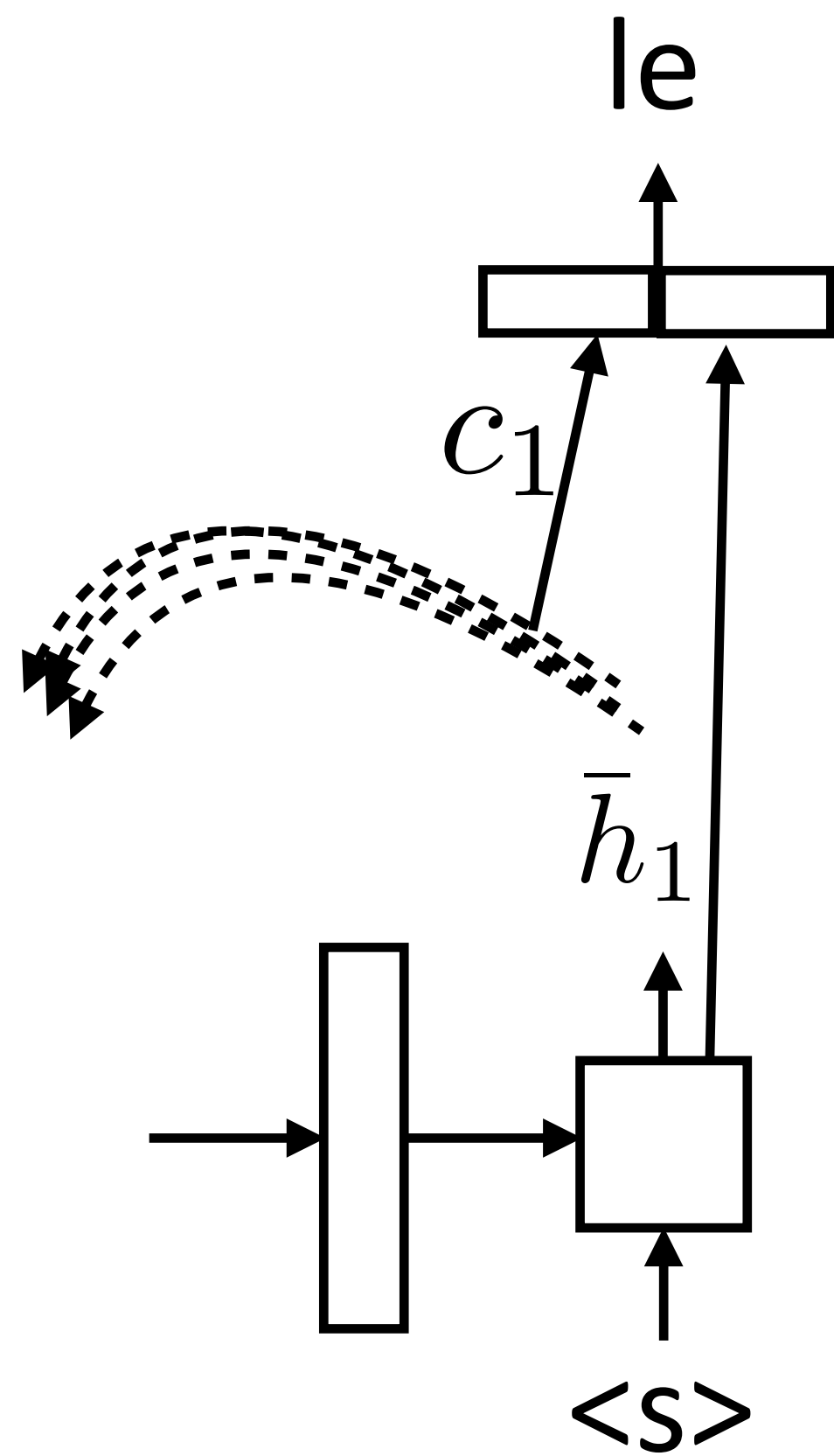


$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

Attention



$$c_i = \sum_j \alpha_{ij} h_j$$

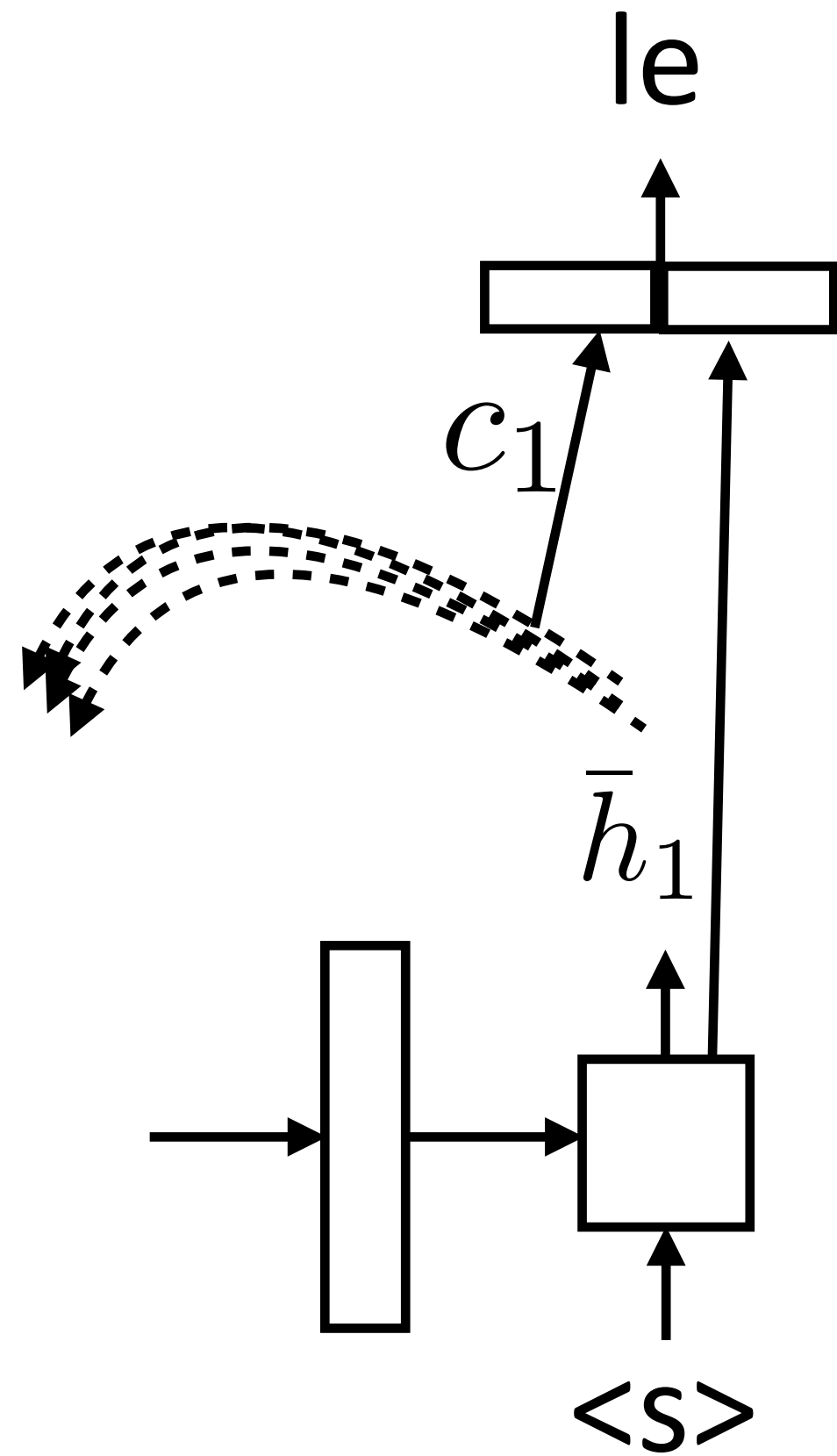
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

$$f(\bar{h}_i, h_j) = \tanh(W[\bar{h}_i; h_j])$$

► Bahdanau+ (2014): additive

Attention



$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

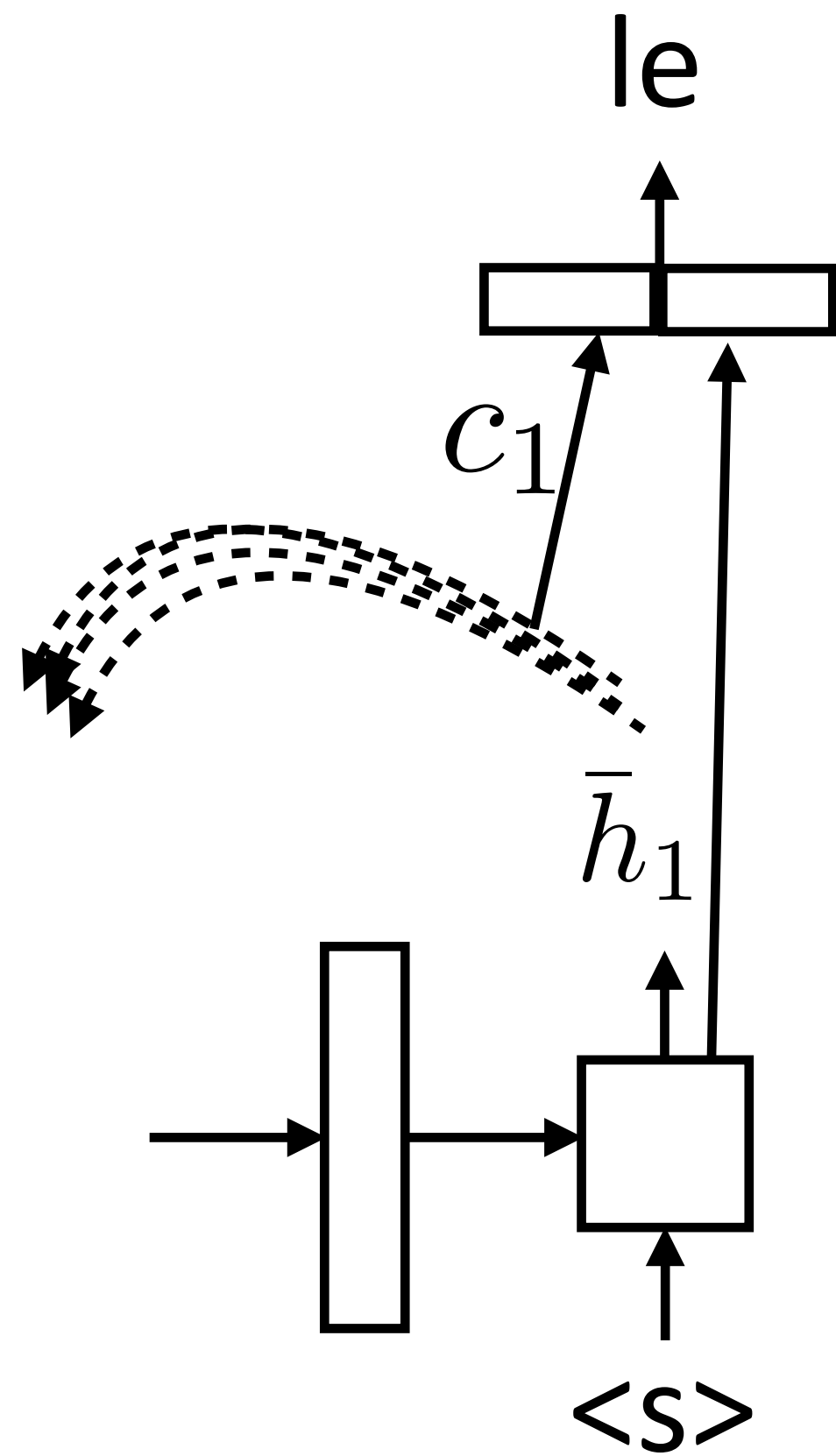
$$f(\bar{h}_i, h_j) = \tanh(W[\bar{h}_i; h_j])$$

► Bahdanau+ (2014): additive

$$f(\bar{h}_i, h_j) = \bar{h}_i \cdot h_j$$

► Luong+ (2015): dot product

Attention



$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

$$f(\bar{h}_i, h_j) = \tanh(W[\bar{h}_i; h_j])$$

► Bahdanau+ (2014): additive

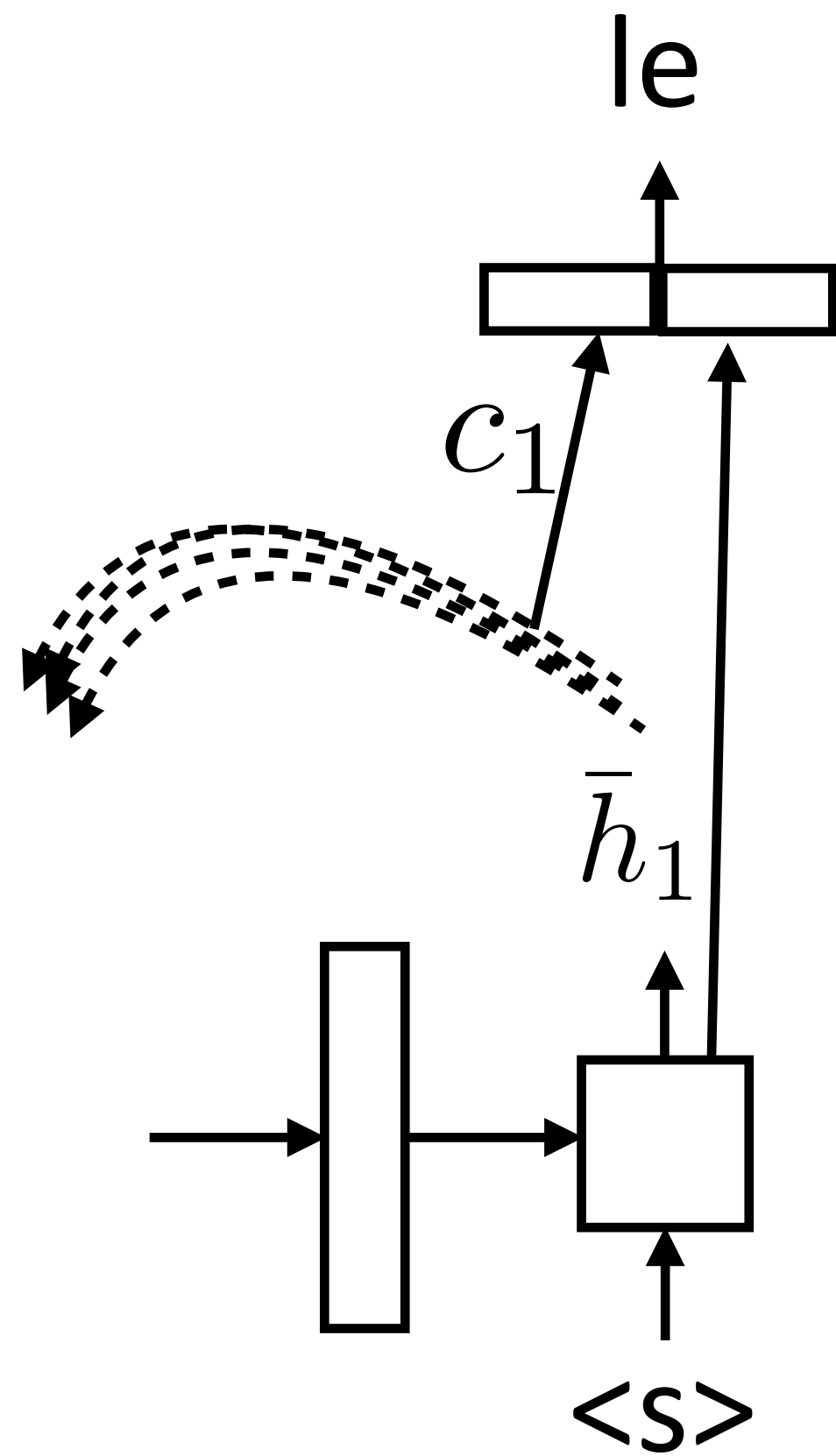
$$f(\bar{h}_i, h_j) = \bar{h}_i \cdot h_j$$

► Luong+ (2015): dot product

$$f(\bar{h}_i, h_j) = \bar{h}_i^\top W h_j$$

► Luong+ (2015): bilinear

Attention



$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

$$f(\bar{h}_i, h_j) = \tanh(W[\bar{h}_i; h_j])$$

► Bahdanau+ (2014): additive

$$f(\bar{h}_i, h_j) = \bar{h}_i \cdot h_j$$

► Luong+ (2015): dot product

$$f(\bar{h}_i, h_j) = \bar{h}_i^\top W h_j$$

► Luong+ (2015): bilinear

► Note that this all uses outputs of hidden layers

Attention is All You Need

Attention Is All You Need

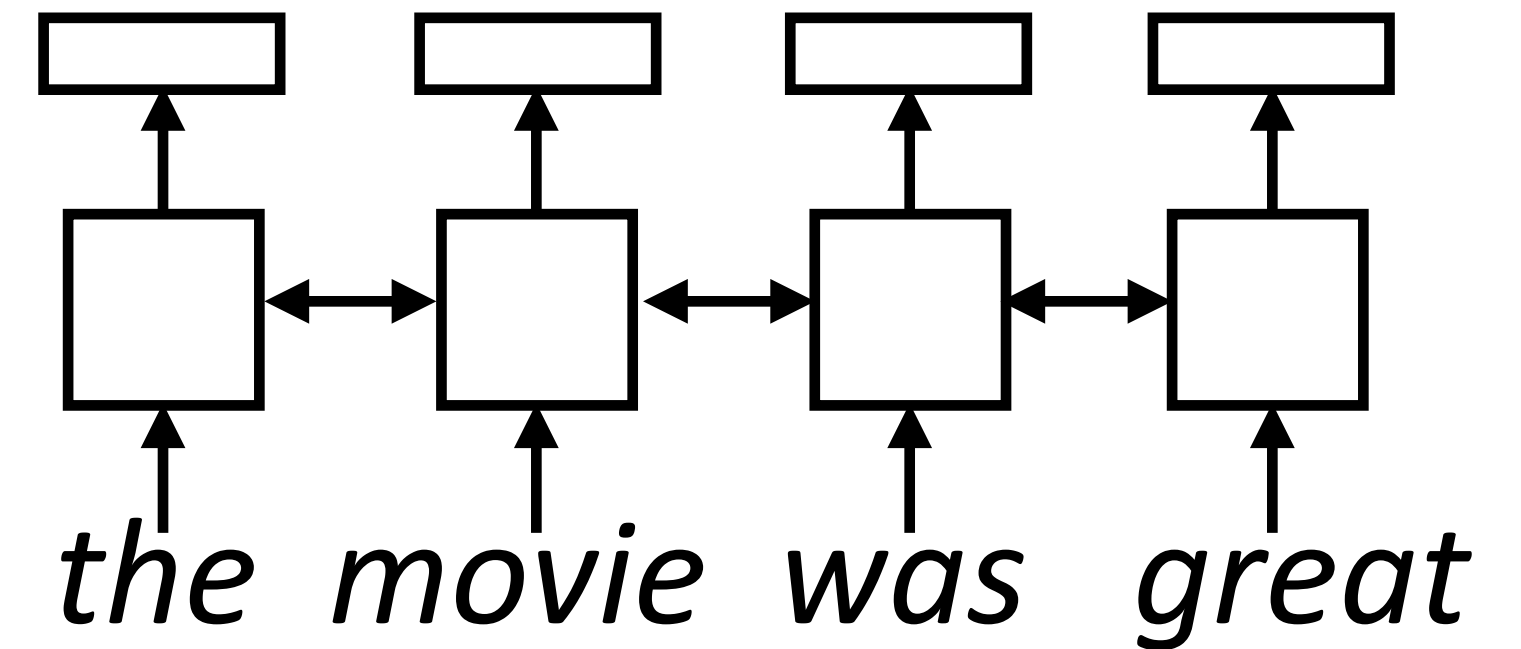
Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com	Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu	Lukasz Kaiser* Google Brain lukaszkaizer@google.com	
Illia Polosukhin* † illia.polosukhin@gmail.com			

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

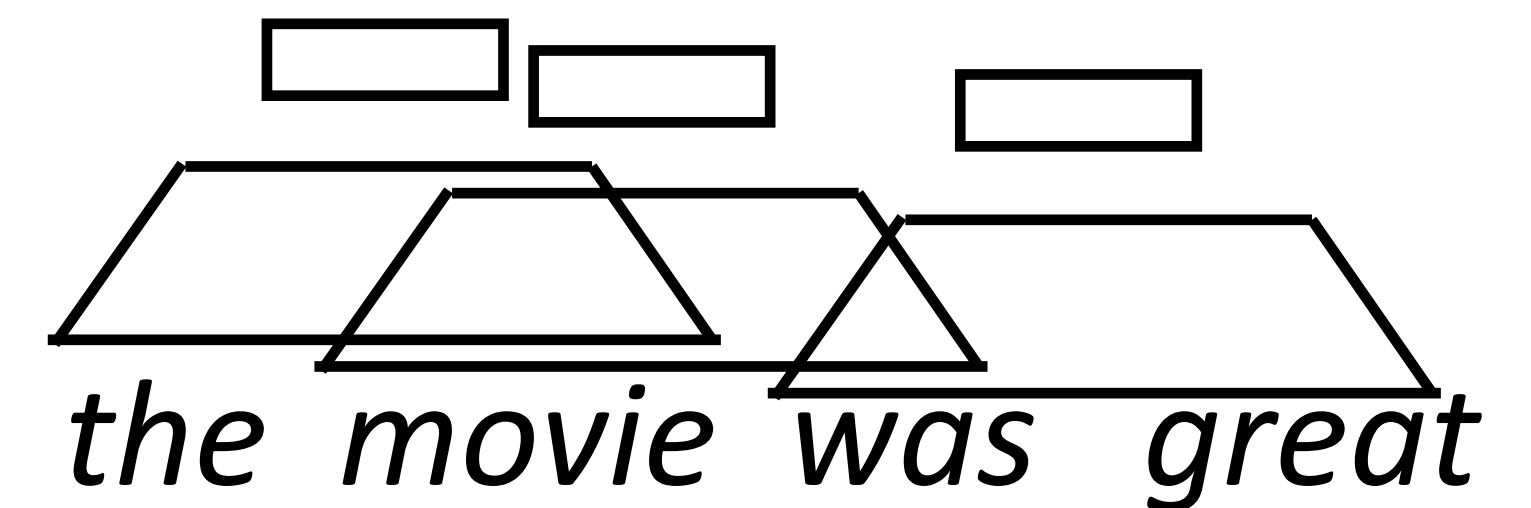
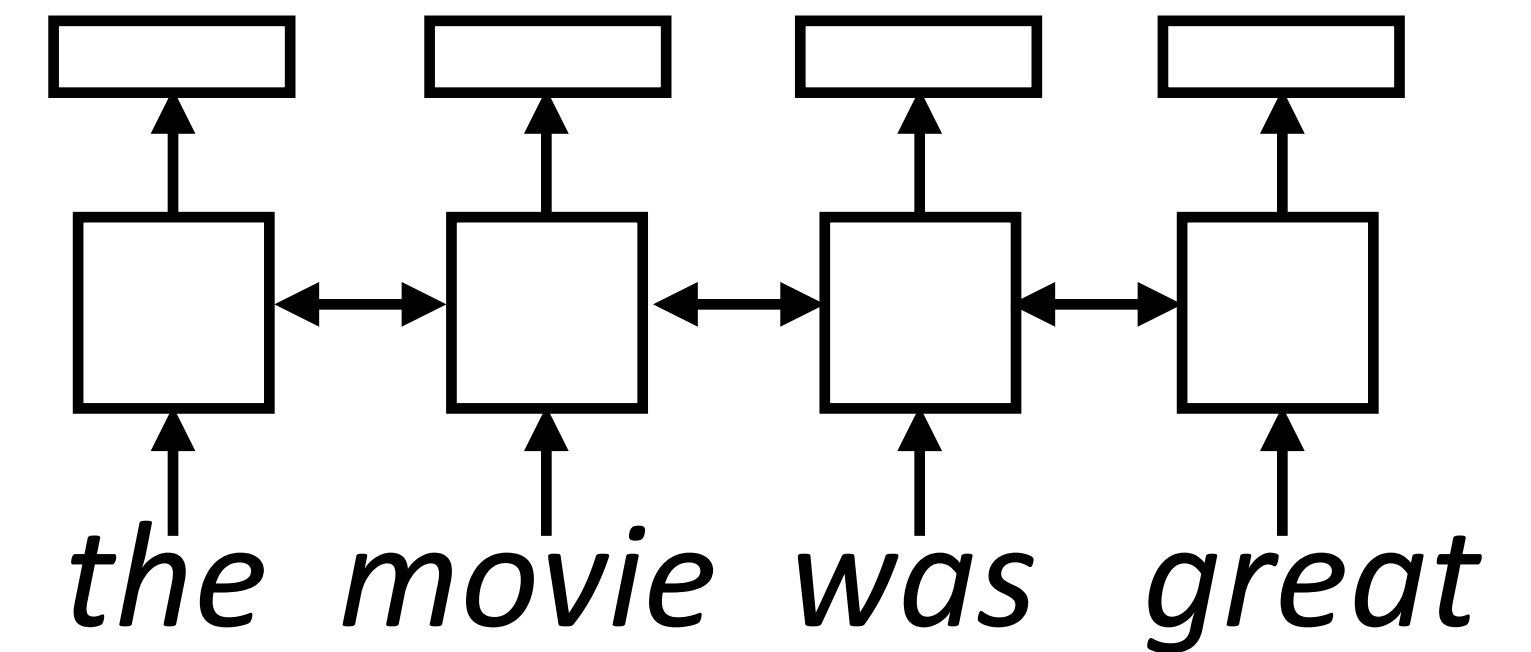
Sentence Encoders

- ▶ LSTM abstraction: maps each vector in a sentence to a new, context-aware vector



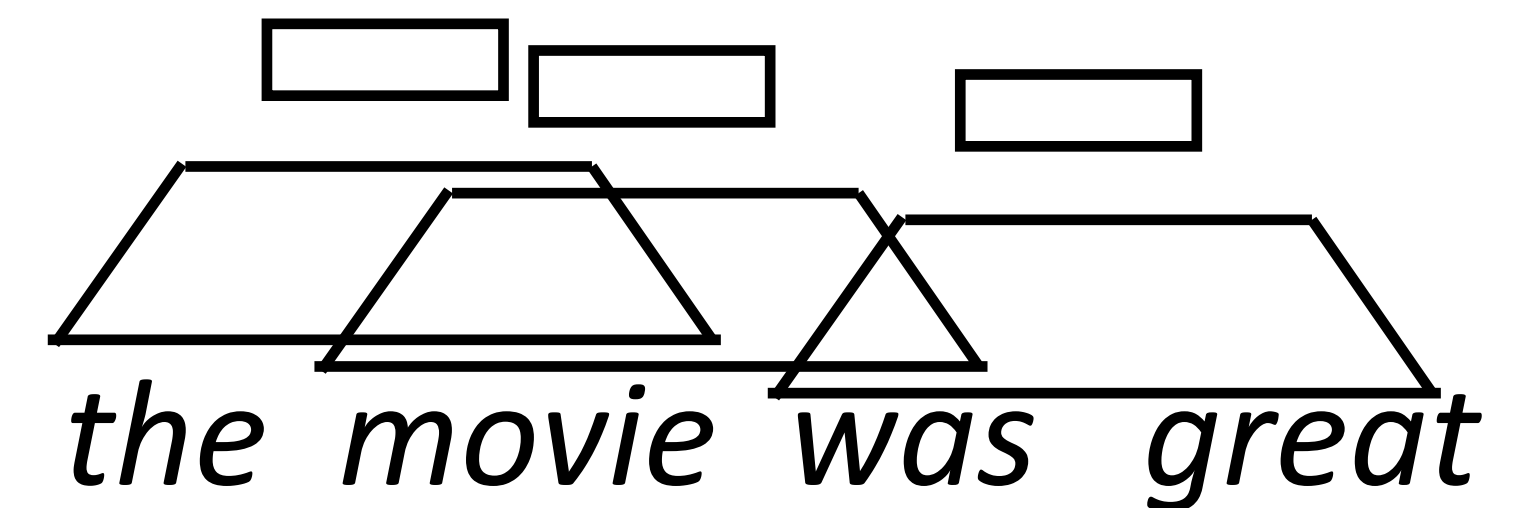
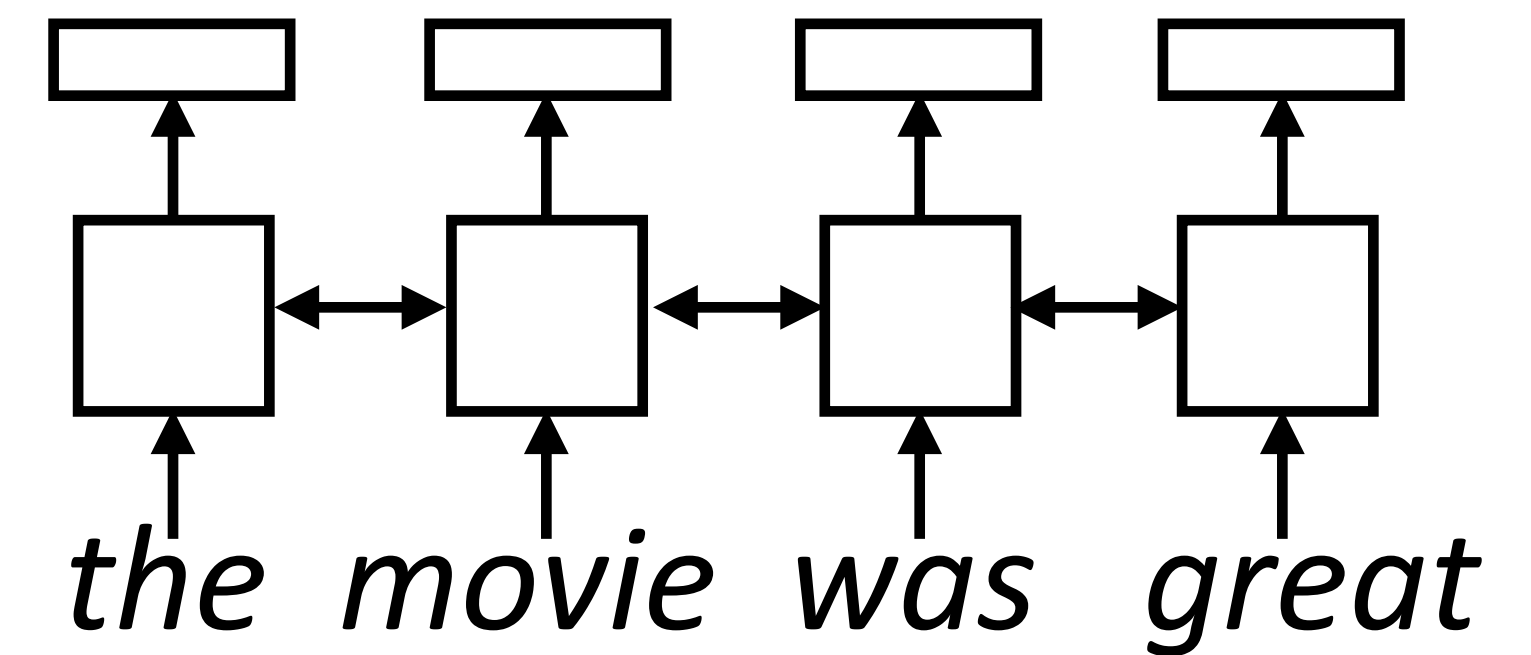
Sentence Encoders

- ▶ LSTM abstraction: maps each vector in a sentence to a new, context-aware vector
- ▶ CNNs do something similar with filters



Sentence Encoders

- ▶ LSTM abstraction: maps each vector in a sentence to a new, context-aware vector
- ▶ CNNs do something similar with filters
- ▶ Attention can give us a third way to do this



Self-Attention

Self-Attention

- ▶ Assume we're using GloVe/word2vec embeddings — what do we want our neural network to do?

The ballerina is very excited that she will dance in the show.

- ▶ What words need to be contextualized here?

Self-Attention

- ▶ Assume we're using GloVe/word2vec embeddings — what do we want our neural network to do?

*The ballerina is very excited that **she** will dance in the **show**.*

- ▶ What words need to be contextualized here?

Self-Attention

- ▶ Assume we're using GloVe/word2vec embeddings — what do we want our neural network to do?

*The ballerina is very excited that **she** will dance in the **show**.*

A blue curved line with arrows at both ends connects the word 'she' (in blue) to the word 'show' (in red). The line starts above 'she', goes up and over, and then comes down to point at 'show'. This illustrates a self-attention mechanism where the model focuses on the relationship between these two words within the same sentence.

- ▶ What words need to be contextualized here?

Self-Attention

- ▶ Assume we're using GloVe/word2vec embeddings — what do we want our neural network to do?

*The ballerina is very excited that **she** will dance in the **show**.*



- ▶ What words need to be contextualized here?
 - ▶ Pronouns need to look at antecedents

Self-Attention

- ▶ Assume we're using GloVe/word2vec embeddings — what do we want our neural network to do?

*The ballerina is very excited that **she** will dance in the **show**.*



- ▶ What words need to be contextualized here?
 - ▶ Pronouns need to look at antecedents

Self-Attention

- ▶ Assume we're using GloVe/word2vec embeddings — what do we want our neural network to do?

*The ballerina is very excited that **she** will dance in the **show**.*



The diagram shows two curved arrows originating from the word 'she' in the sentence. A blue arrow points to the word 'ballerina', and a red arrow points to the word 'show'. This illustrates how the model's attention mechanism weights different parts of the input sequence to understand the context of the word 'she'.

- ▶ What words need to be contextualized here?
 - ▶ Pronouns need to look at antecedents
 - ▶ Ambiguous words should look at context

Self-Attention

- ▶ Assume we're using GloVe/word2vec embeddings — what do we want our neural network to do?

*The ballerina is very excited that **she** will dance in the **show**.*



- ▶ What words need to be contextualized here?
 - ▶ Pronouns need to look at antecedents
 - ▶ Ambiguous words should look at context
 - ▶ Words should look at syntactic parents/children

Self-Attention

- ▶ Assume we're using GloVe/word2vec embeddings — what do we want our neural network to do?

*The ballerina is very excited that **she** will dance in the **show**.*

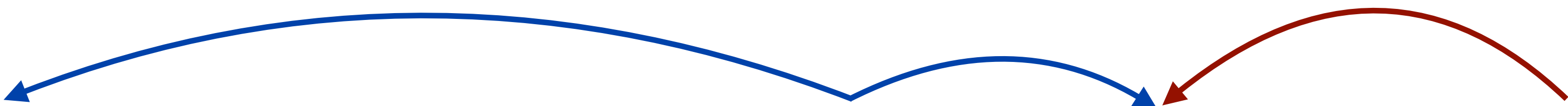


- ▶ What words need to be contextualized here?
 - ▶ Pronouns need to look at antecedents
 - ▶ Ambiguous words should look at context
 - ▶ Words should look at syntactic parents/children
- ▶ Problem: LSTMs and CNNs don't do this

Self-Attention

- ▶ Want:

*The ballerina is very excited that **she** will dance in the **show**.*

A diagram illustrating global self-attention. A red curved arrow originates from the word 'show' and points back to the word 'she'. A blue curved arrow originates from the word 'she' and points back to the word 'The'.

- ▶ LSTMs/CNNs: tend to look at local context

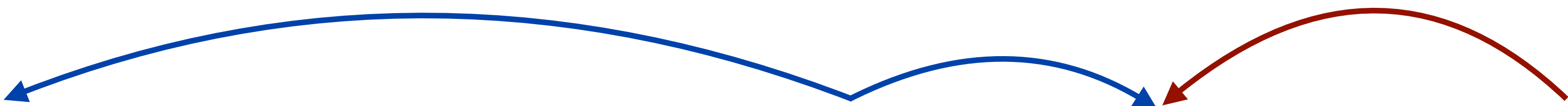
*The ballerina is very excited that **she** will dance in the **show**.*

A diagram illustrating local self-attention. Multiple red curved arrows originate from the word 'show' and point to nearby words: 'the', 'dance', 'will', and 'she'. Multiple blue curved arrows originate from the word 'she' and point to nearby words: 'that', 'is', 'very', and 'excited'.

Self-Attention

- ▶ Want:

*The ballerina is very excited that **she** will dance in the **show**.*

A diagram illustrating long-range dependencies in a sentence. Two curved arrows are shown above the text. The first arrow is blue and starts from the word 'she' and points to the word 'The'. The second arrow is red and starts from the word 'show' and points to the word 'that'.

- ▶ LSTMs/CNNs: tend to look at local context

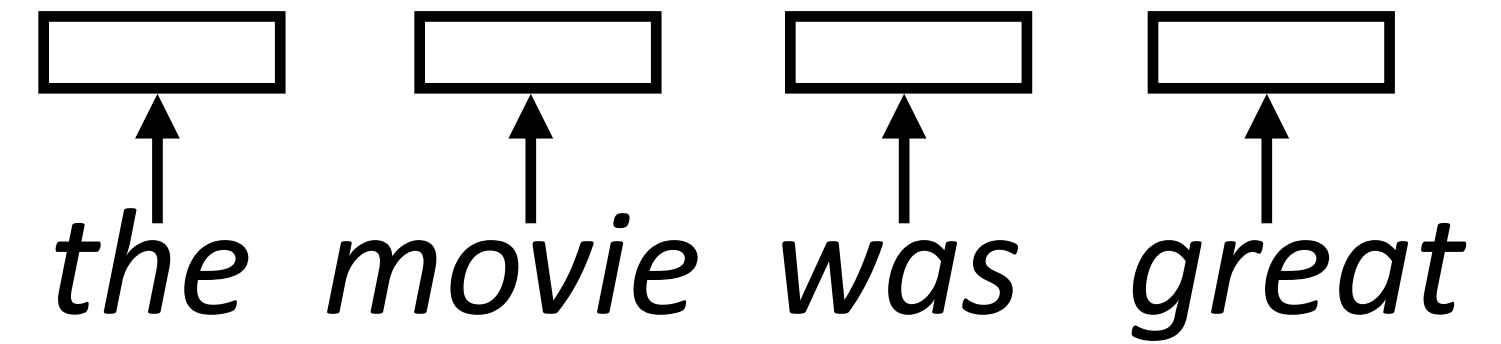
*The ballerina is very excited that **she** will dance in the **show**.*

A diagram illustrating local context dependencies in a sentence. Multiple curved arrows are shown above the text. Blue arrows connect 'she' to 'is', 'is' to 'very', 'very' to 'excited', 'excited' to 'that', 'that' to 'will', and 'will' to 'dance'. Red arrows connect 'show' to 'in' and 'in' to 'the'.

- ▶ To appropriately contextualize embeddings, we need to pass information over long distances dynamically for each word

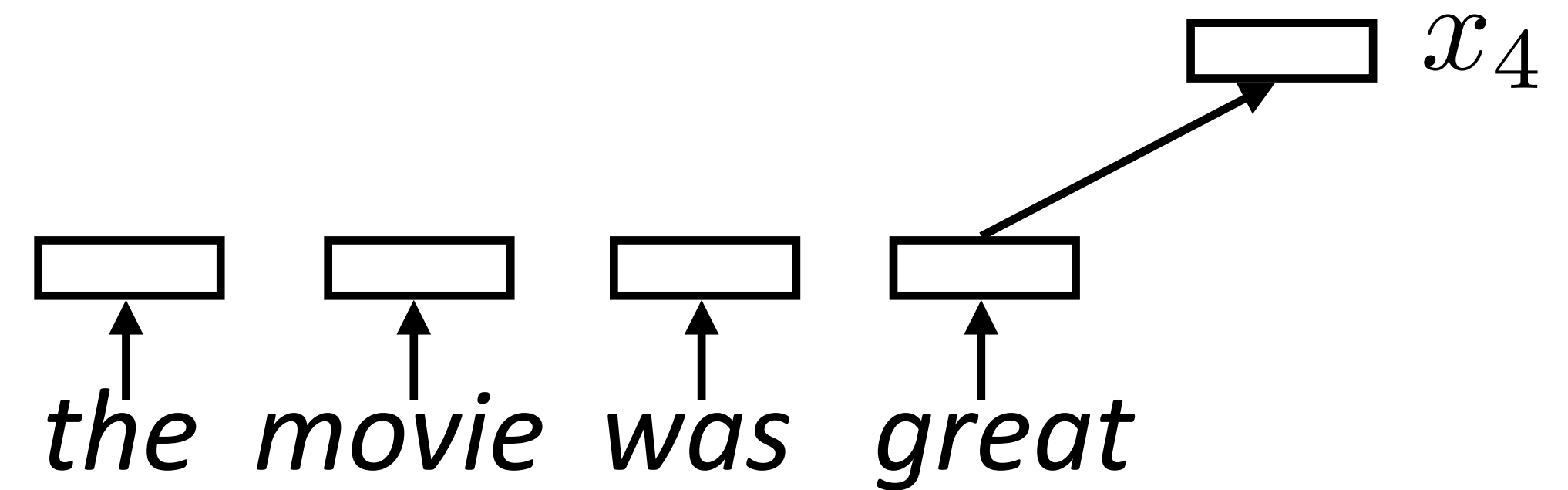
Self-Attention

- ▶ Each word forms a “query” which then computes attention over each word



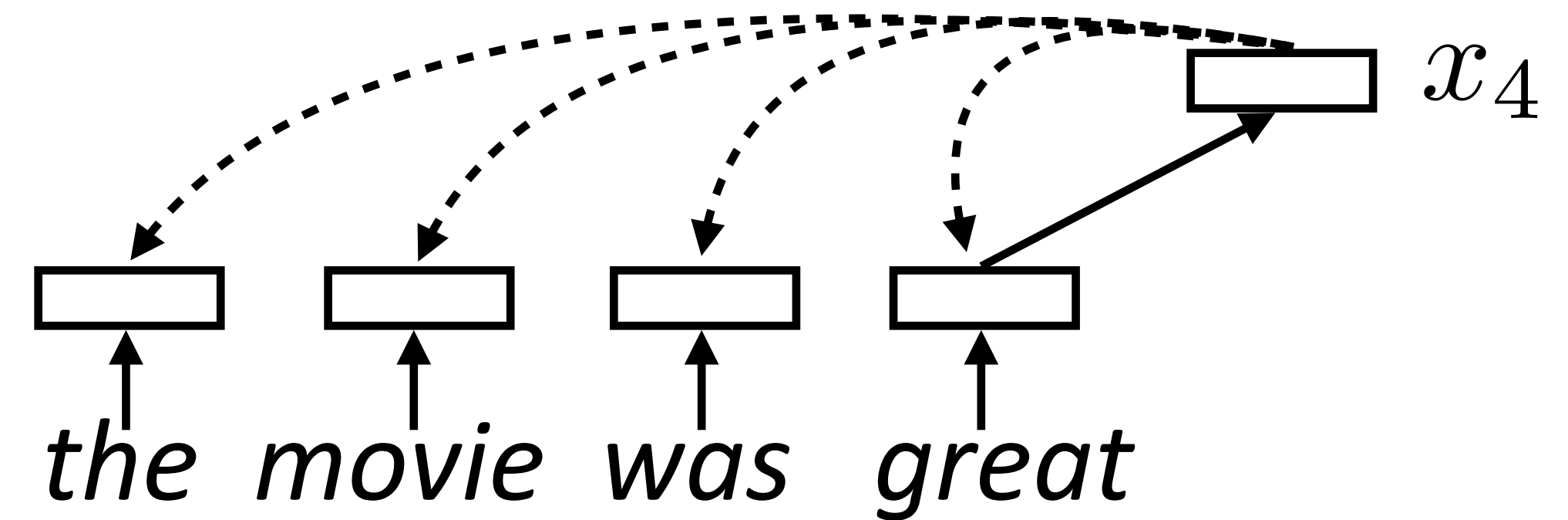
Self-Attention

- ▶ Each word forms a “query” which then computes attention over each word



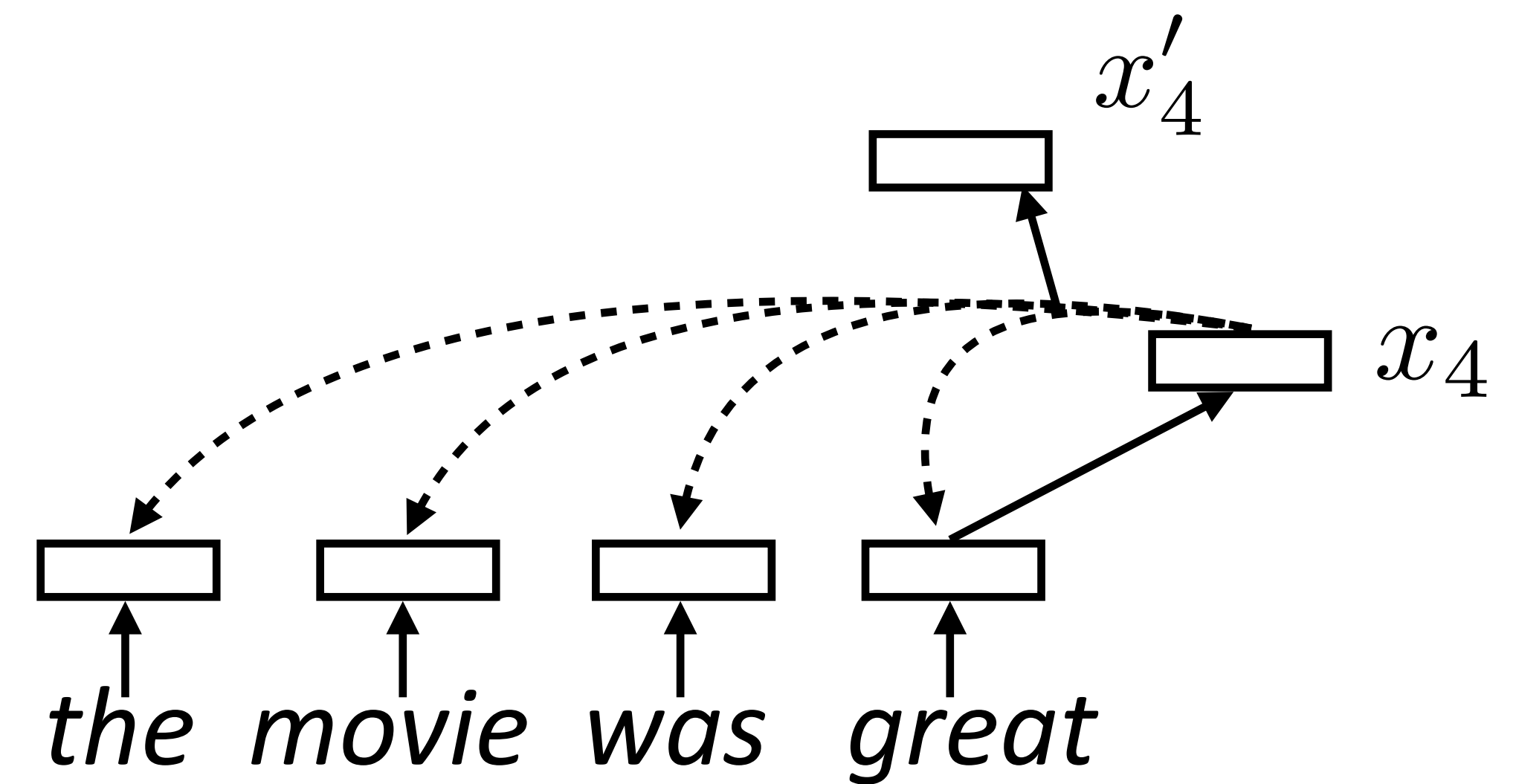
Self-Attention

- ▶ Each word forms a “query” which then computes attention over each word



Self-Attention

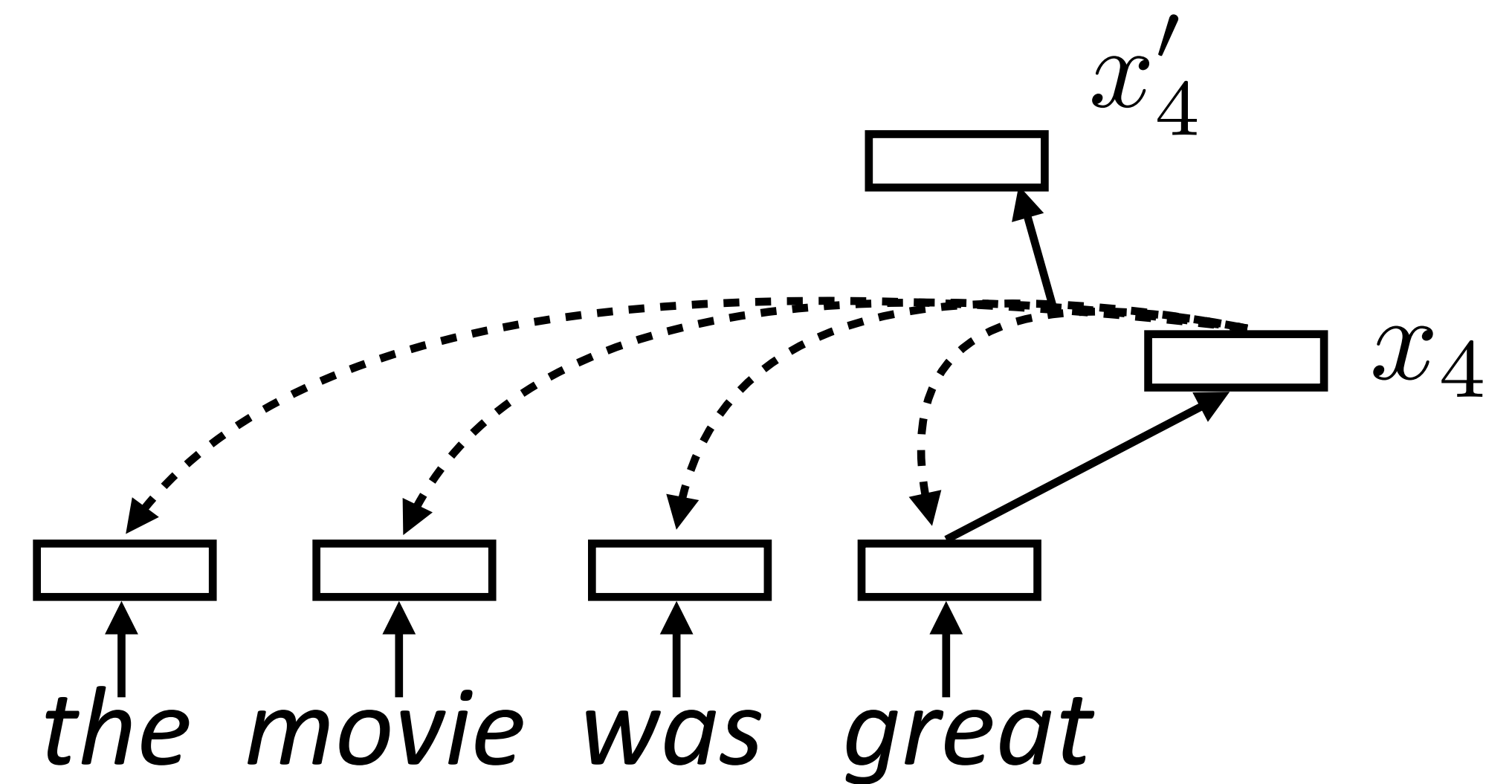
- ▶ Each word forms a “query” which then computes attention over each word



Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

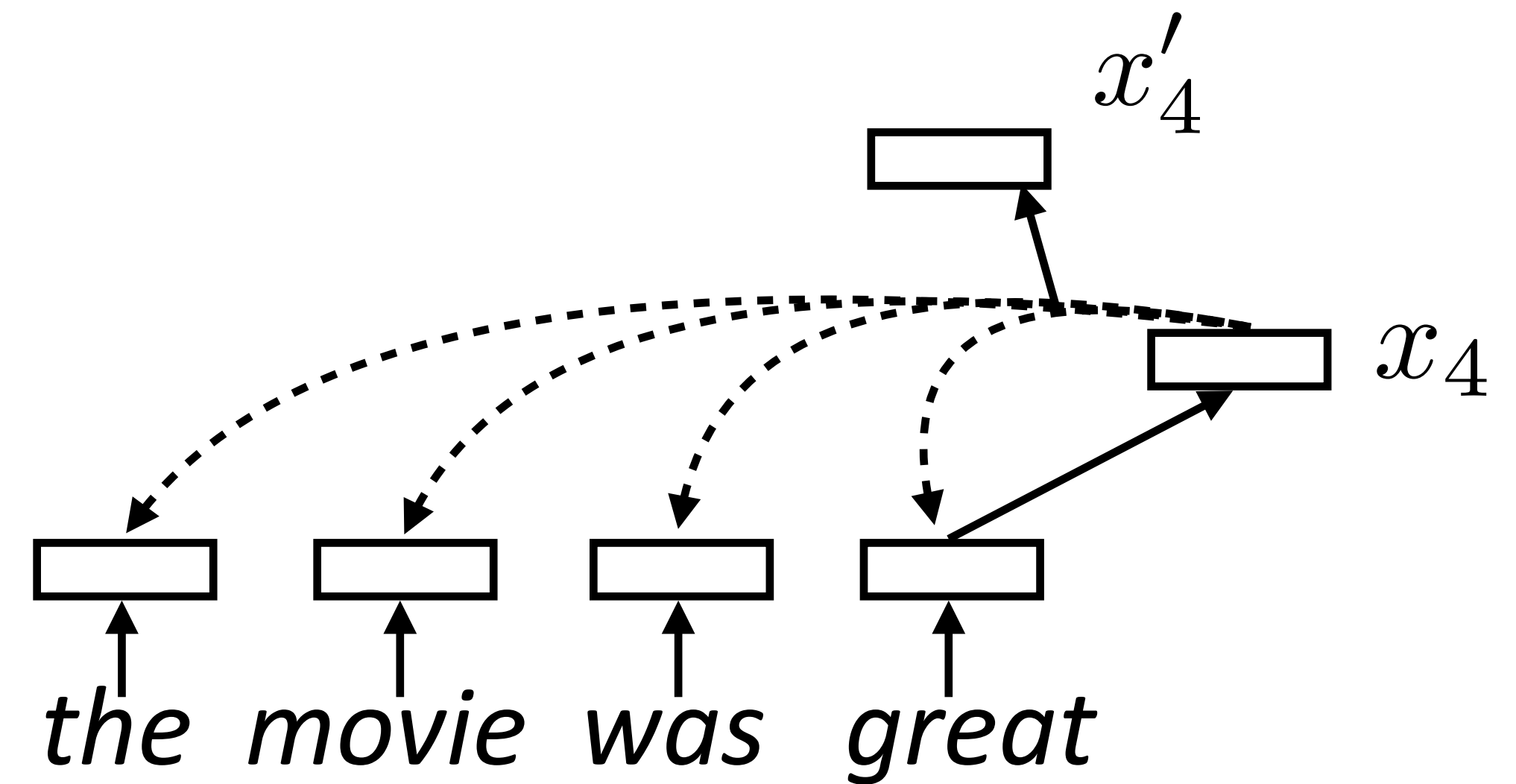


Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

$$x'_i = \sum_{j=1}^n \alpha_{i,j} x_j \quad \text{vector} = \text{sum of scalar} * \text{vector}$$

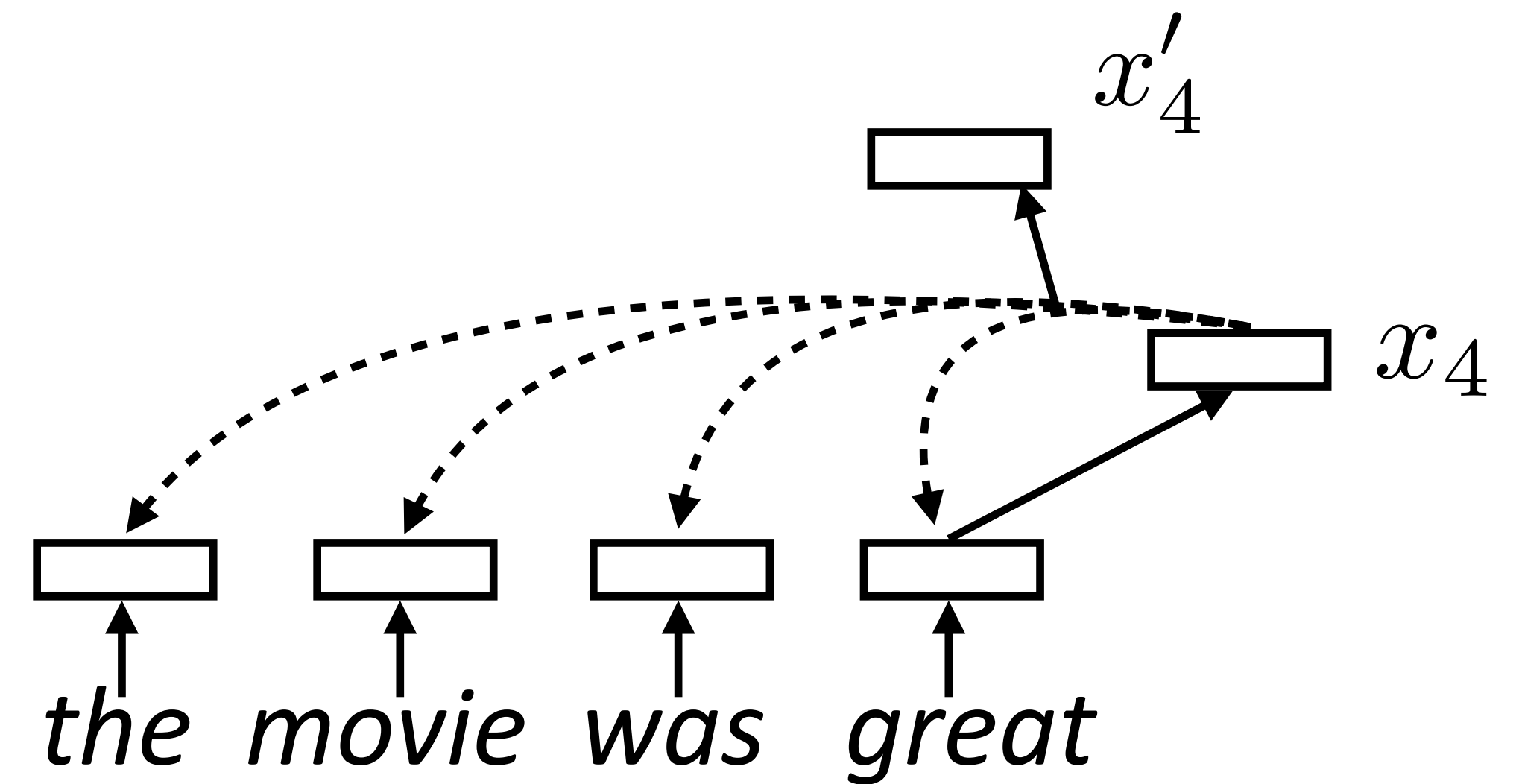


Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

$$x'_i = \sum_{j=1}^n \alpha_{i,j} x_j \quad \text{vector} = \text{sum of scalar} * \text{vector}$$



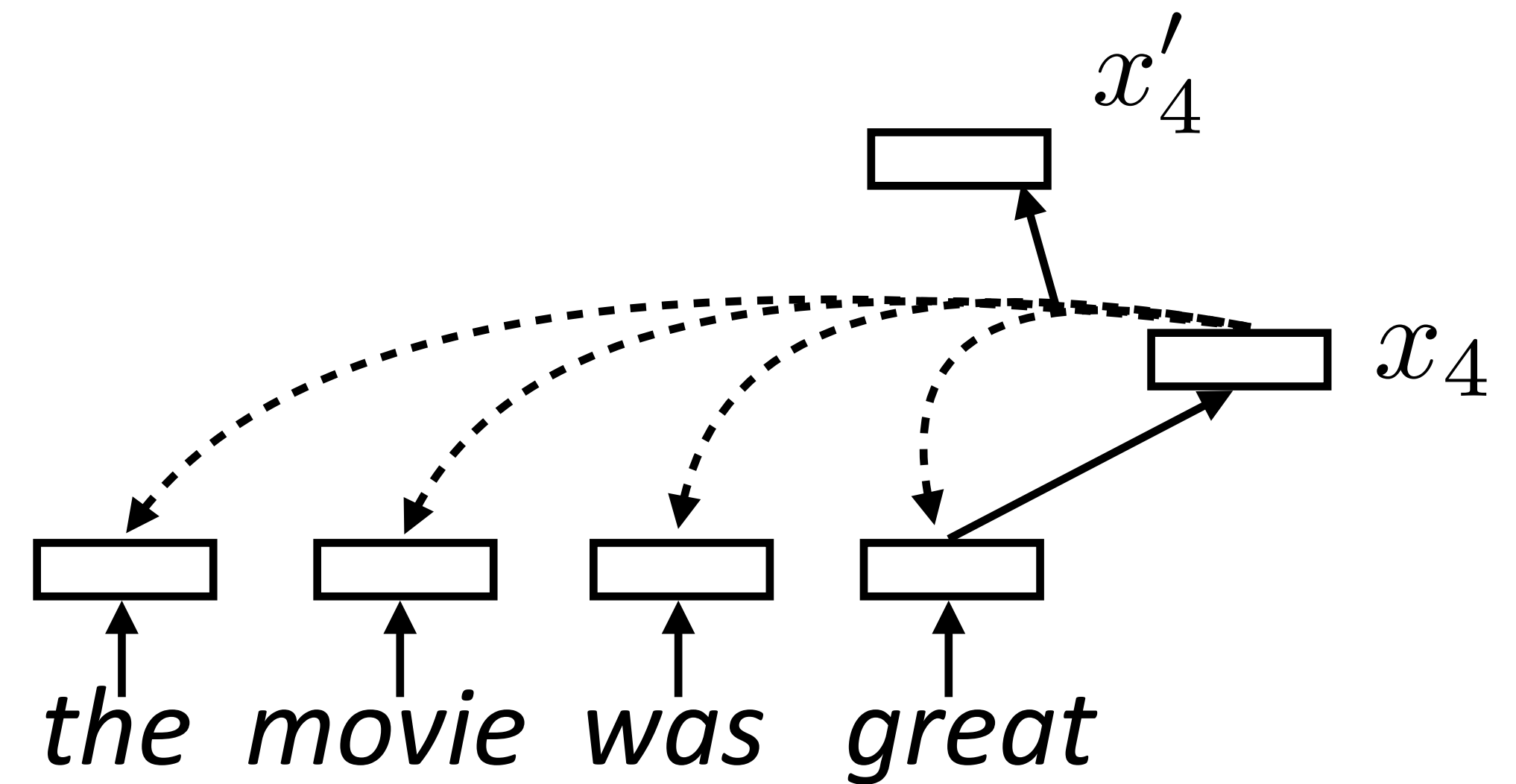
- Multiple “heads” use different sets of parameters W_k and V_k to get different attention values + transform vectors

Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

$$x'_i = \sum_{j=1}^n \alpha_{i,j} x_j \quad \text{vector} = \text{sum of scalar} * \text{vector}$$



- Multiple “heads” use different sets of parameters W_k and V_k to get different attention values + transform vectors

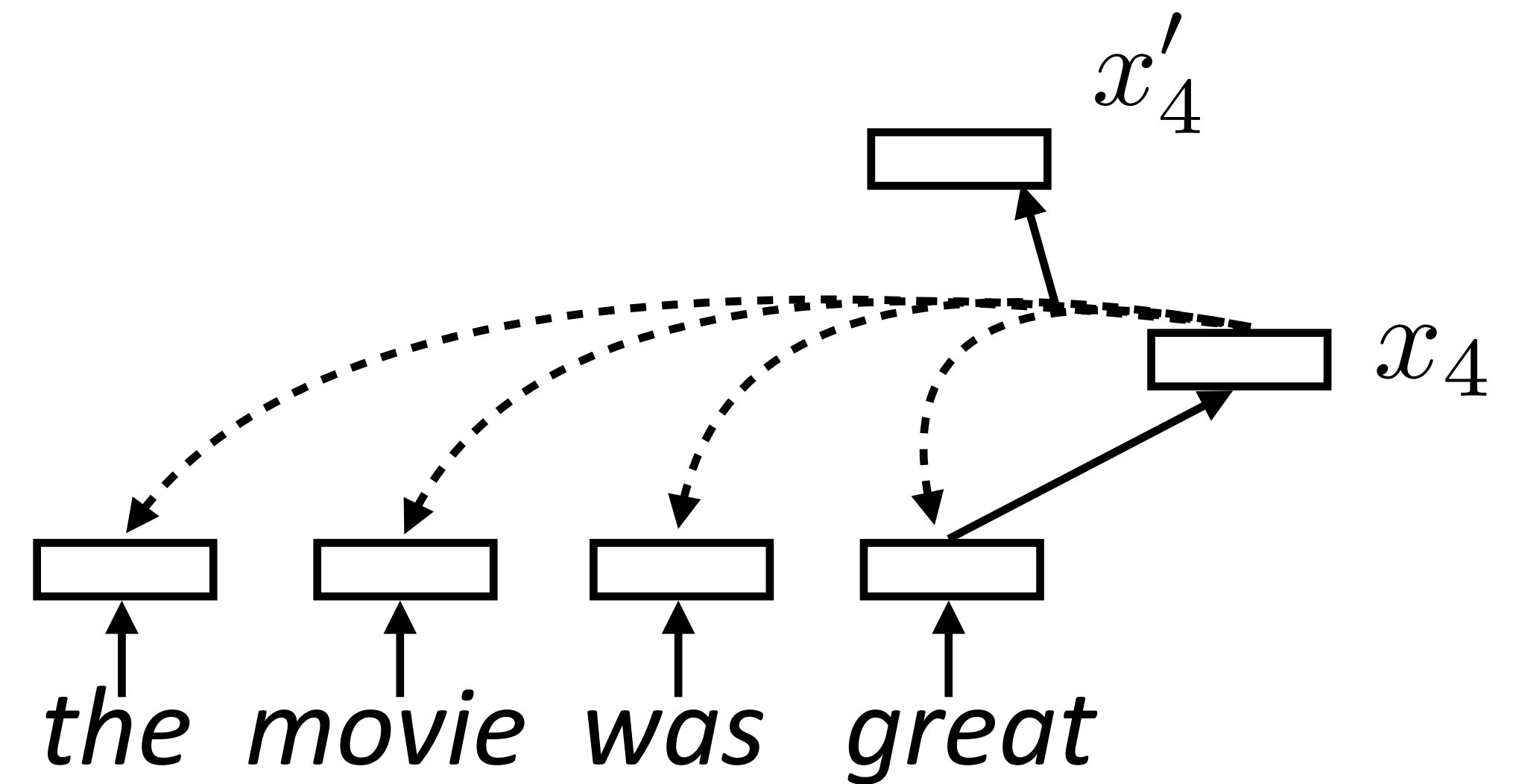
$$\alpha_{k,i,j} = \text{softmax}(x_i^\top W_k x_j)$$

Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

$$x'_i = \sum_{j=1}^n \alpha_{i,j} x_j \quad \text{vector} = \text{sum of scalar} * \text{vector}$$



- Multiple “heads” use different sets of parameters W_k and V_k to get different attention values + transform vectors

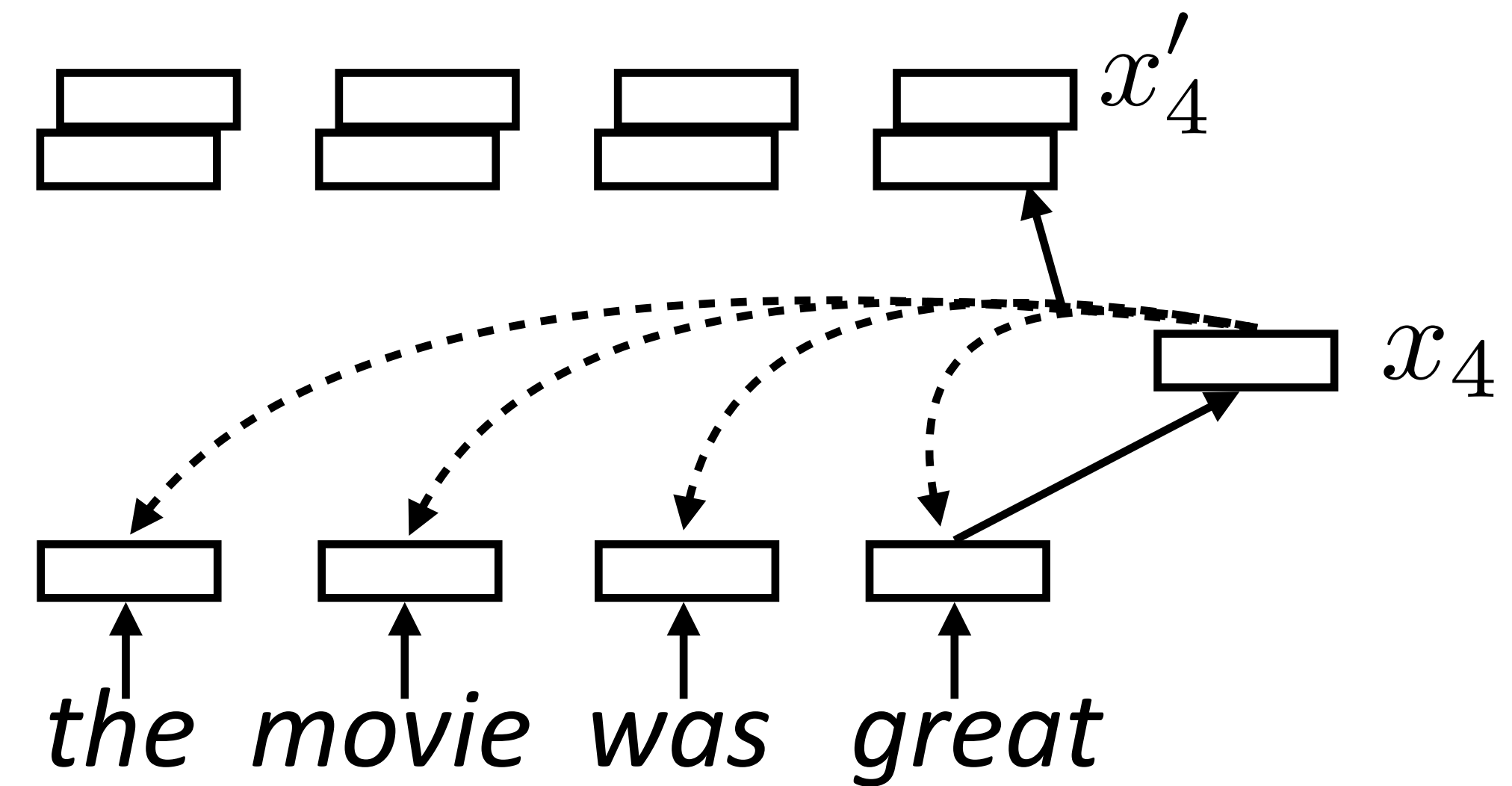
$$\alpha_{k,i,j} = \text{softmax}(x_i^\top W_k x_j) \quad x'_{k,i} = \sum_{j=1}^n \alpha_{k,i,j} V_k x_j$$

Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

$$x'_i = \sum_{j=1}^n \alpha_{i,j} x_j \quad \text{vector} = \text{sum of scalar} * \text{vector}$$

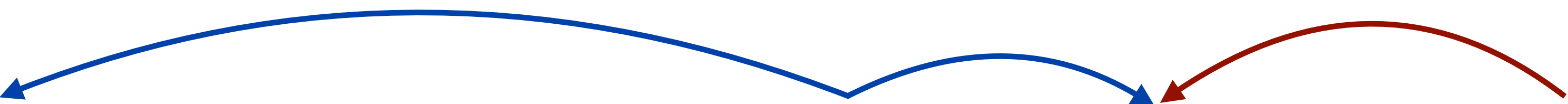


- Multiple “heads” use different sets of parameters W_k and V_k to get different attention values + transform vectors

$$\alpha_{k,i,j} = \text{softmax}(x_i^\top W_k x_j) \quad x'_{k,i} = \sum_{j=1}^n \alpha_{k,i,j} V_k x_j$$

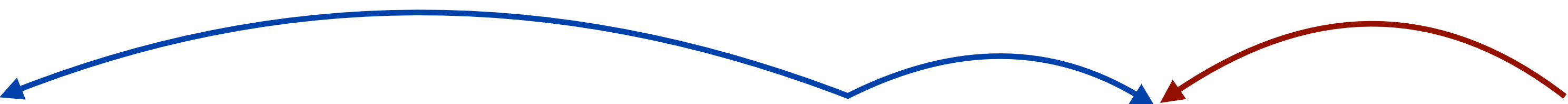
What can self-attention do?

*The ballerina is very excited that **she** will dance in the **show**.*



What can self-attention do?

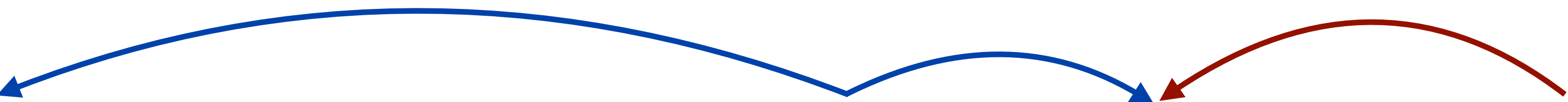
*The ballerina is very excited that **she** will dance in the **show**.*



0	0.5	0	0	0.1	0.1	0	0.1	0.2	0	0	0
---	-----	---	---	-----	-----	---	-----	-----	---	---	---

What can self-attention do?

*The ballerina is very excited that **she** will dance in the **show**.*



0	0.5	0	0	0.1	0.1	0	0.1	0.2	0	0	0
---	-----	---	---	-----	-----	---	-----	-----	---	---	---

0	0.1	0	0	0	0	0	0	0.5	0	0.4	0
---	-----	---	---	---	---	---	---	-----	---	-----	---

What can self-attention do?

*The ballerina is very excited that **she** will dance in the **show**.*

0	0.5	0	0	0.1	0.1	0	0.1	0.2	0	0	0
---	-----	---	---	-----	-----	---	-----	-----	---	---	---

0	0.1	0	0	0	0	0	0	0.5	0	0.4	0
---	-----	---	---	---	---	---	---	-----	---	-----	---

- ▶ Attend nearby + to semantically related terms

What can self-attention do?

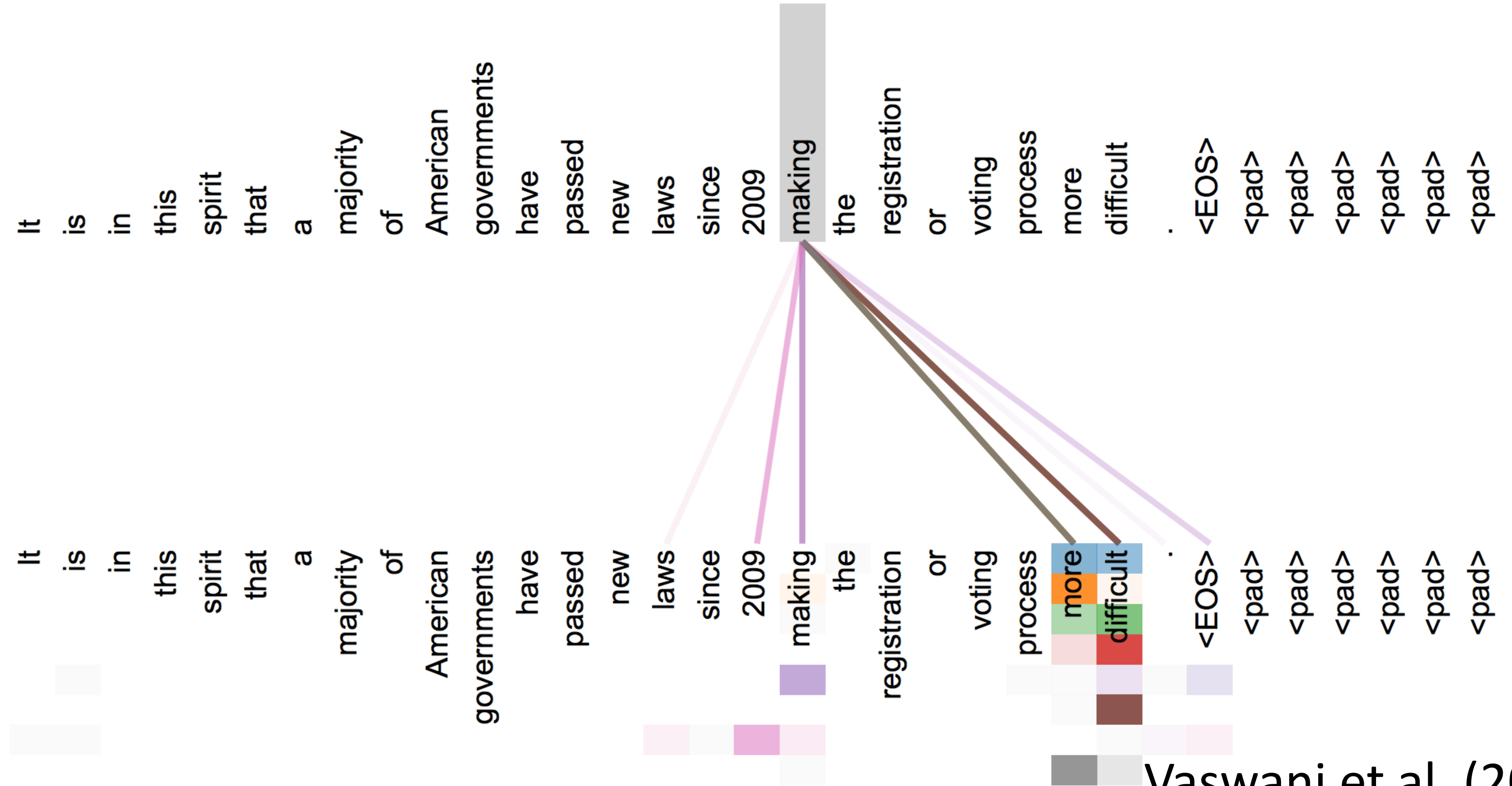
*The ballerina is very excited that **she** will dance in the **show**.*



0	0.5	0	0	0.1	0.1	0	0.1	0.2	0	0	0
0	0.1	0	0	0	0	0	0	0.5	0	0.4	0

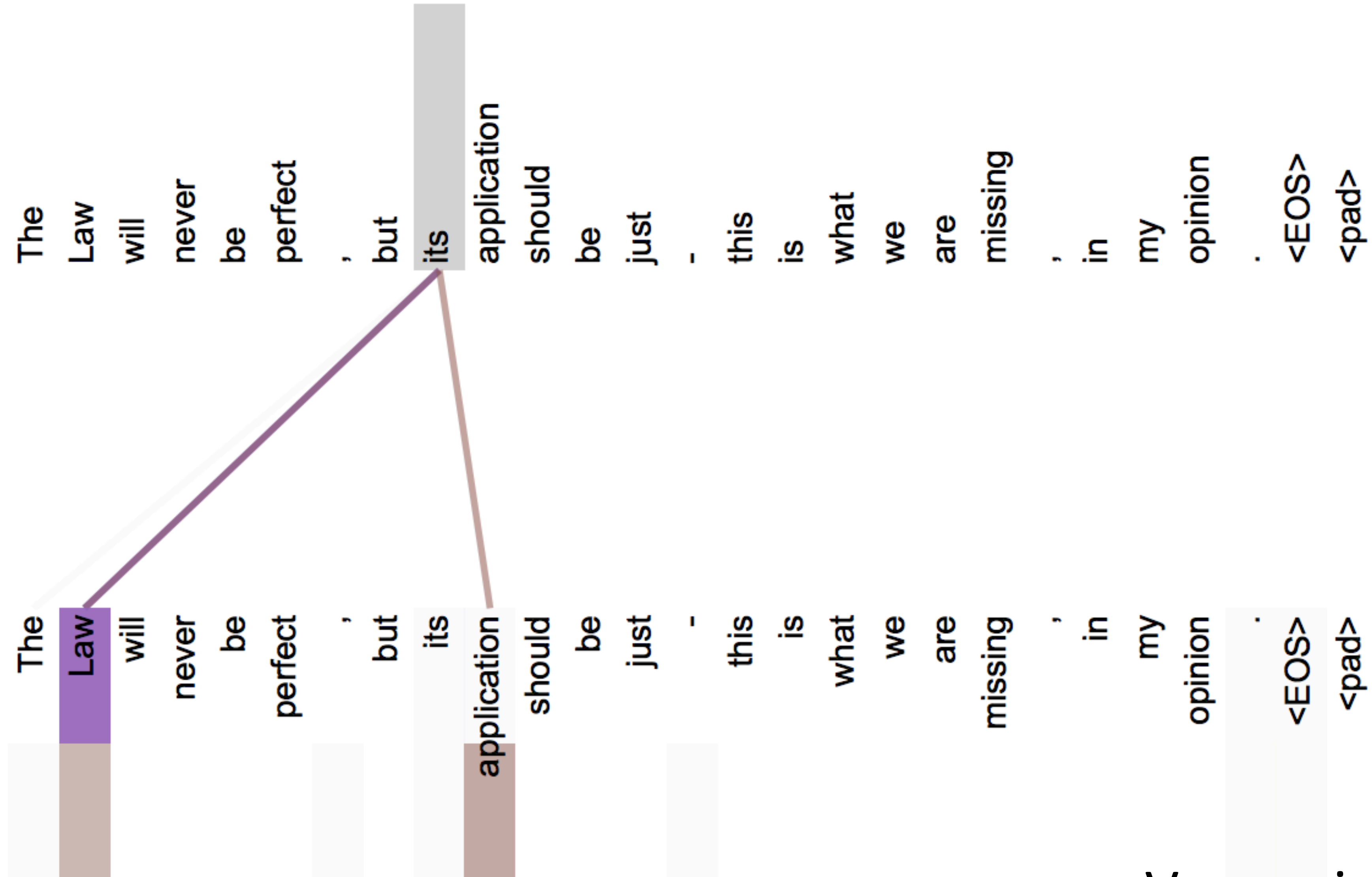
- ▶ Attend nearby + to semantically related terms
- ▶ Why multiple heads? Softmaxes end up being peaked, single distribution cannot easily put weight on multiple things

Visualization



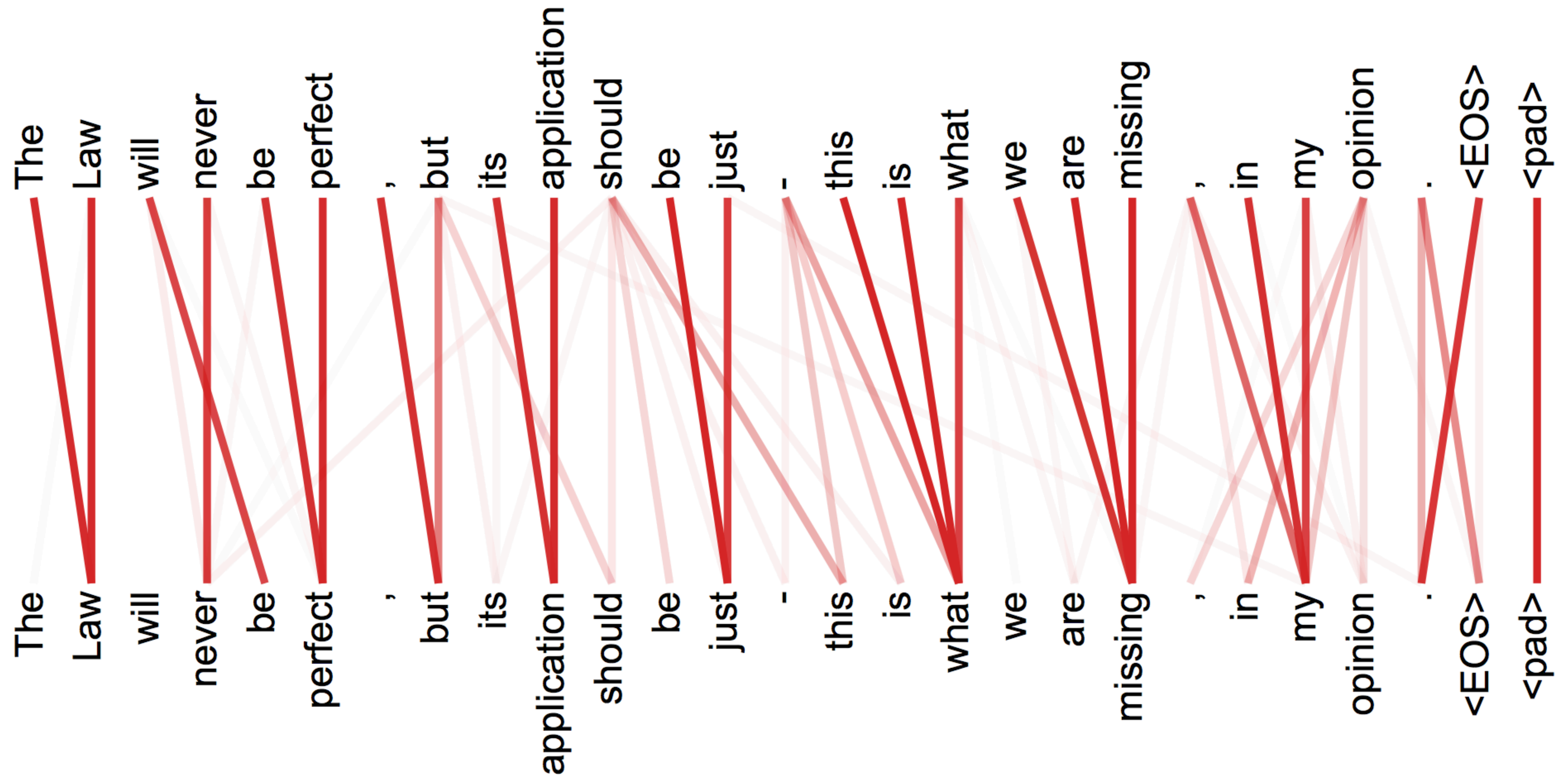
Vaswani et al. (2017)

Visualization



Vaswani et al. (2017)

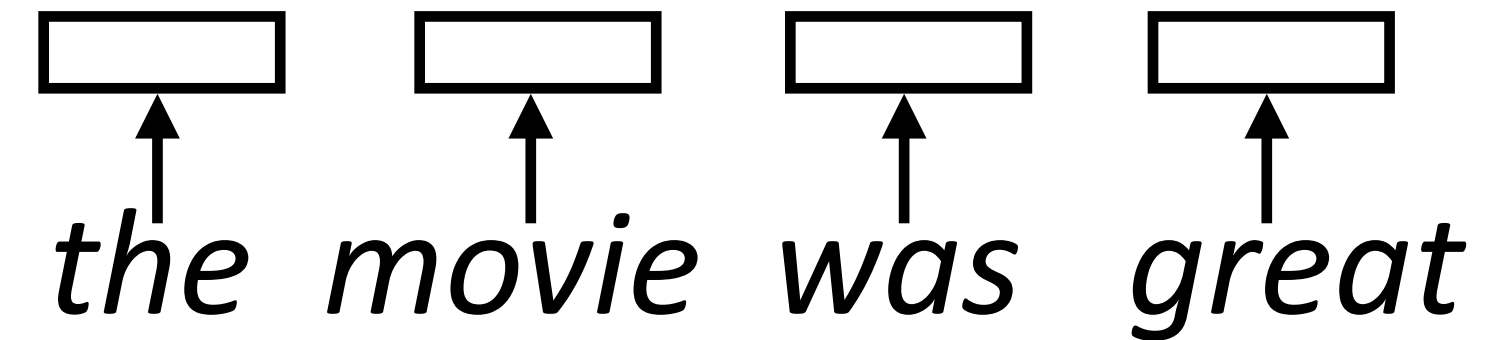
Visualization



Vaswani et al. (2017)

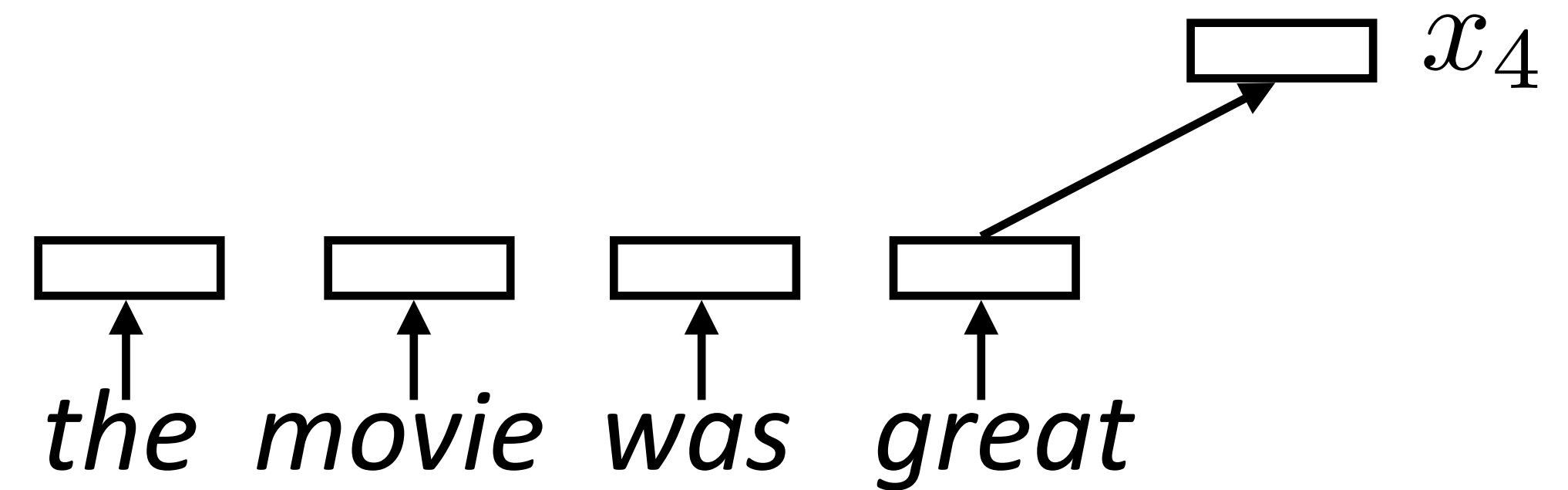
Self-Attention

- ▶ Each word forms a “query” which then computes attention over each word



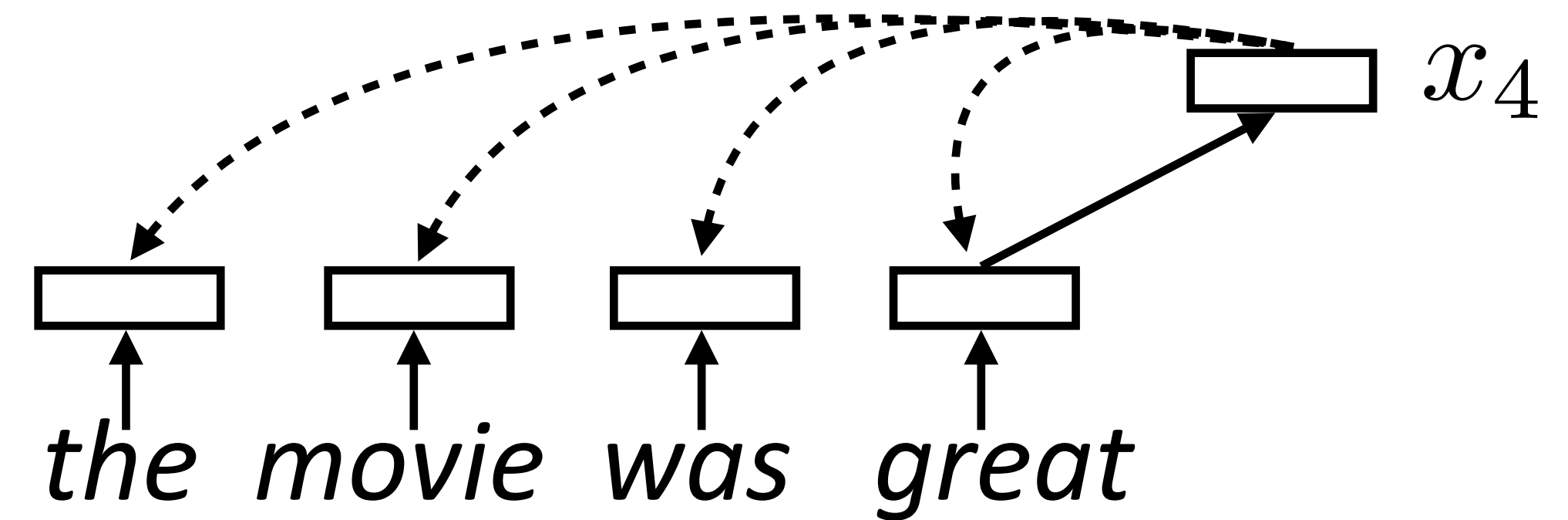
Self-Attention

- ▶ Each word forms a “query” which then computes attention over each word



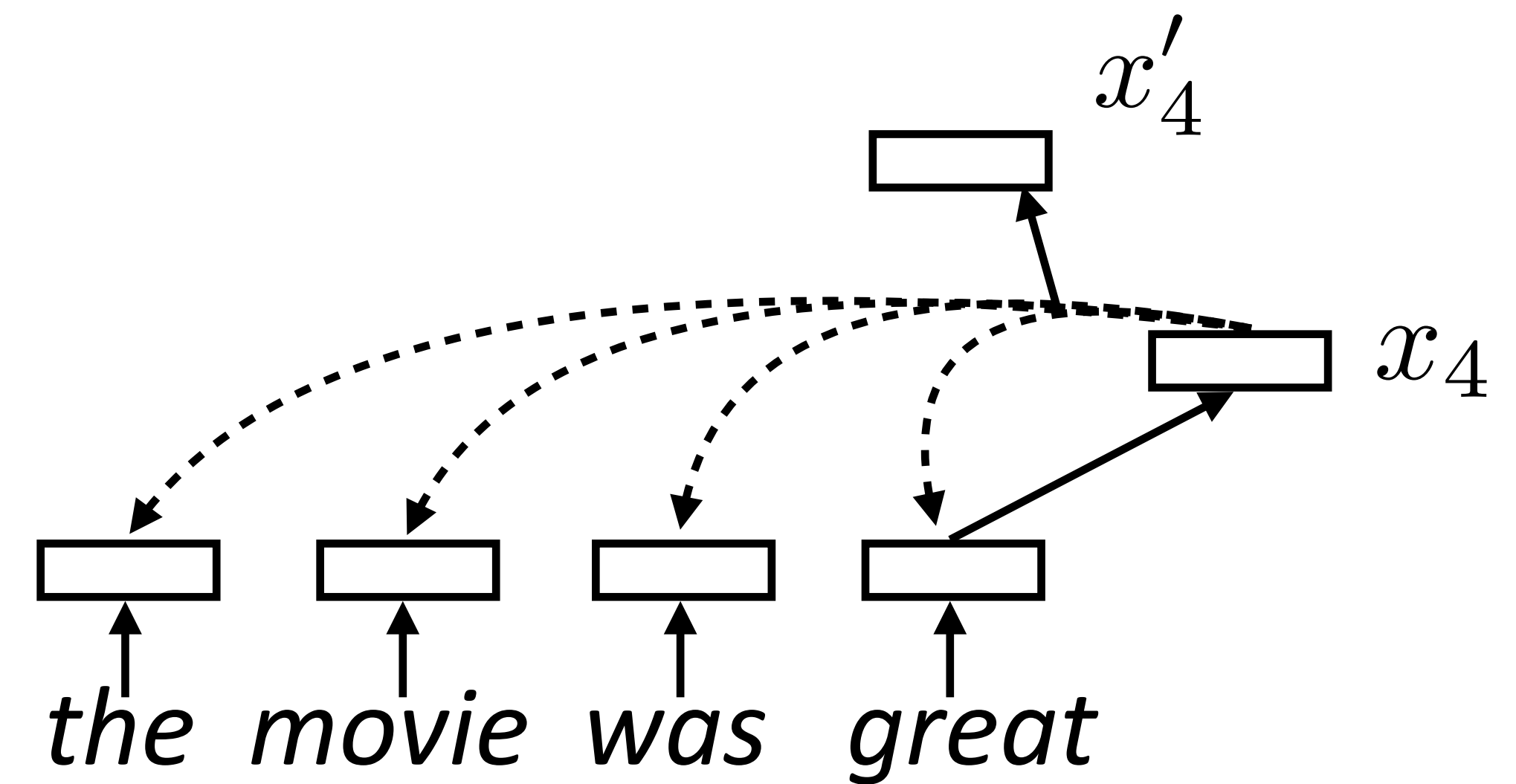
Self-Attention

- ▶ Each word forms a “query” which then computes attention over each word



Self-Attention

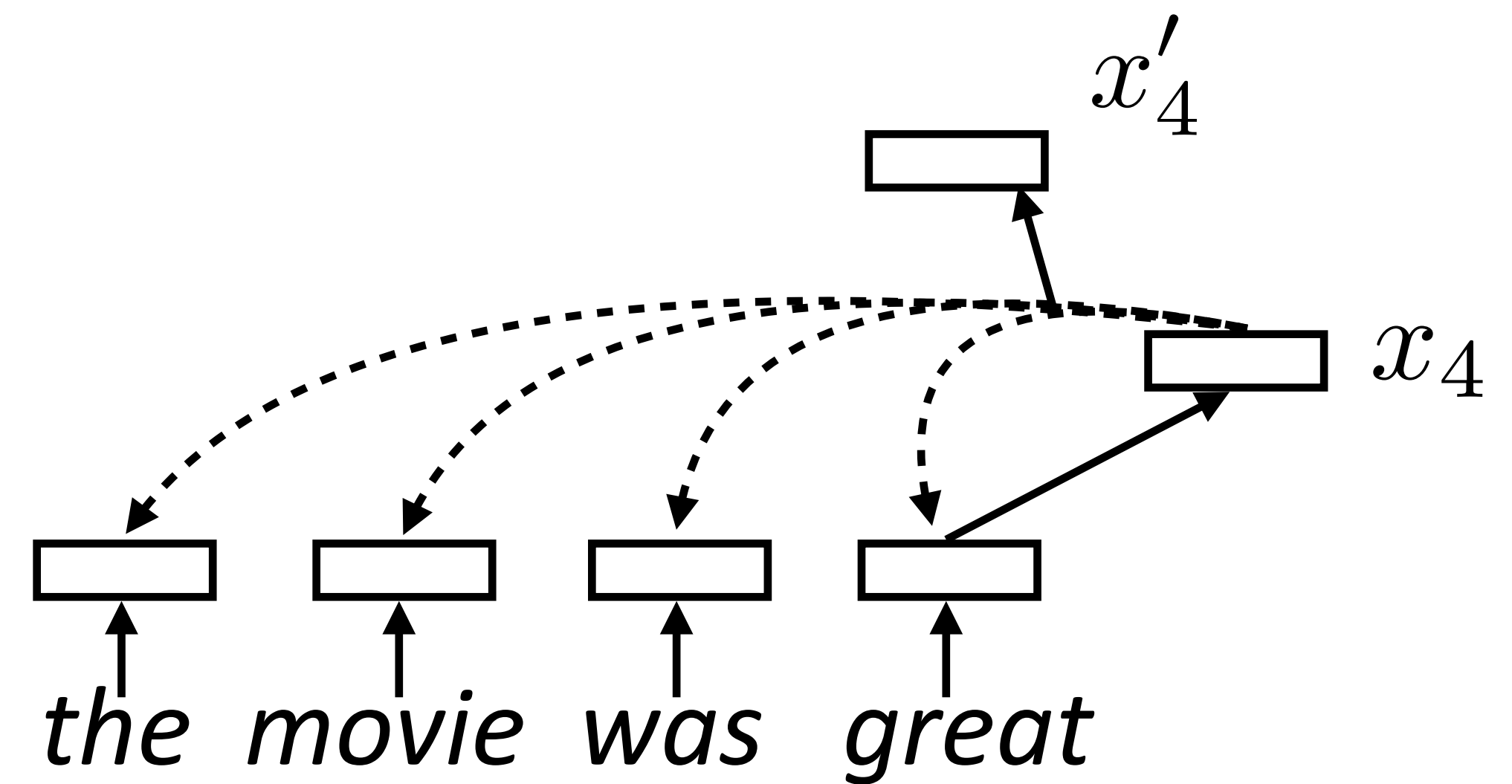
- ▶ Each word forms a “query” which then computes attention over each word



Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

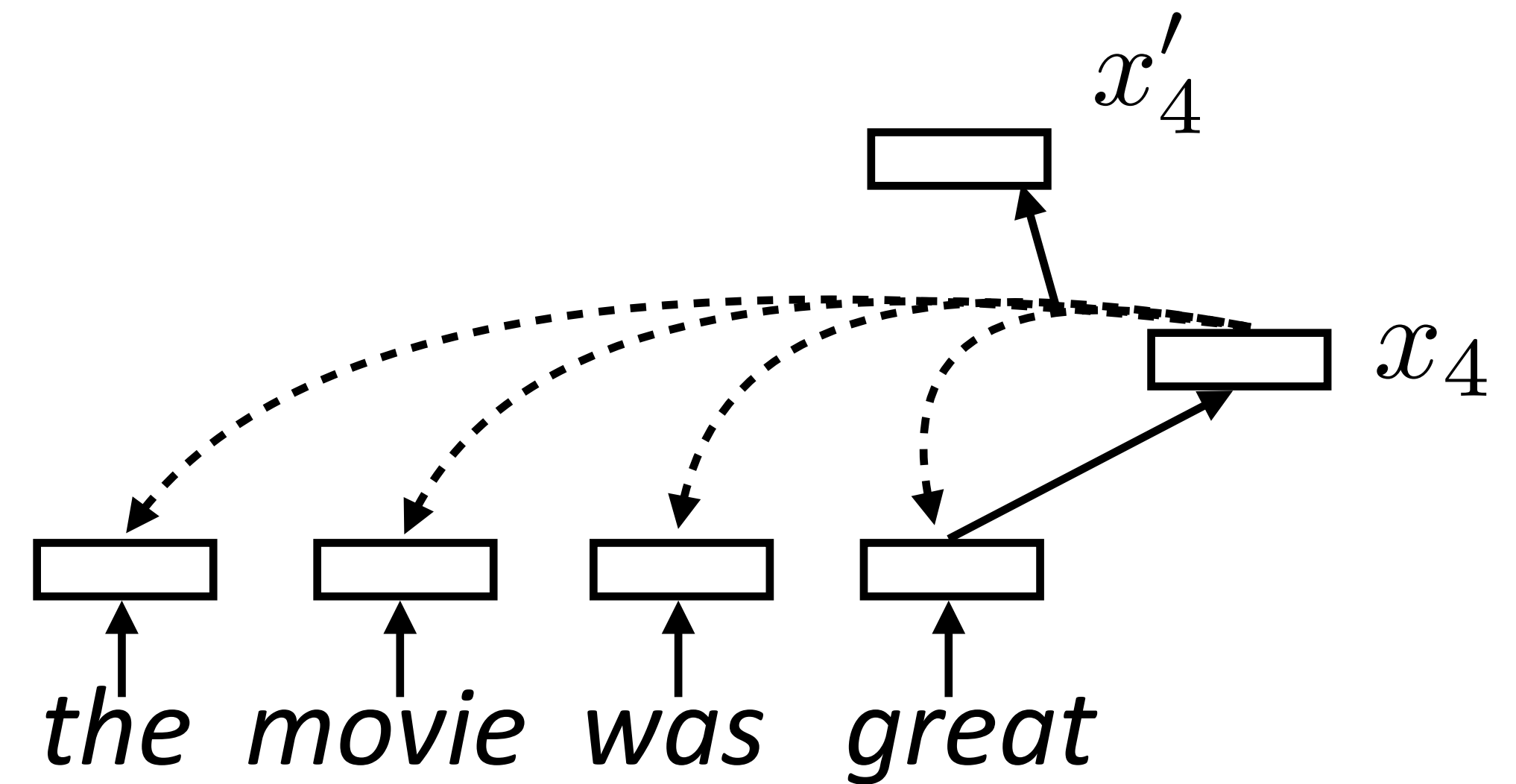


Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

$$x'_i = \sum_{j=1}^n \alpha_{i,j} x_j \quad \text{vector} = \text{sum of scalar} * \text{vector}$$

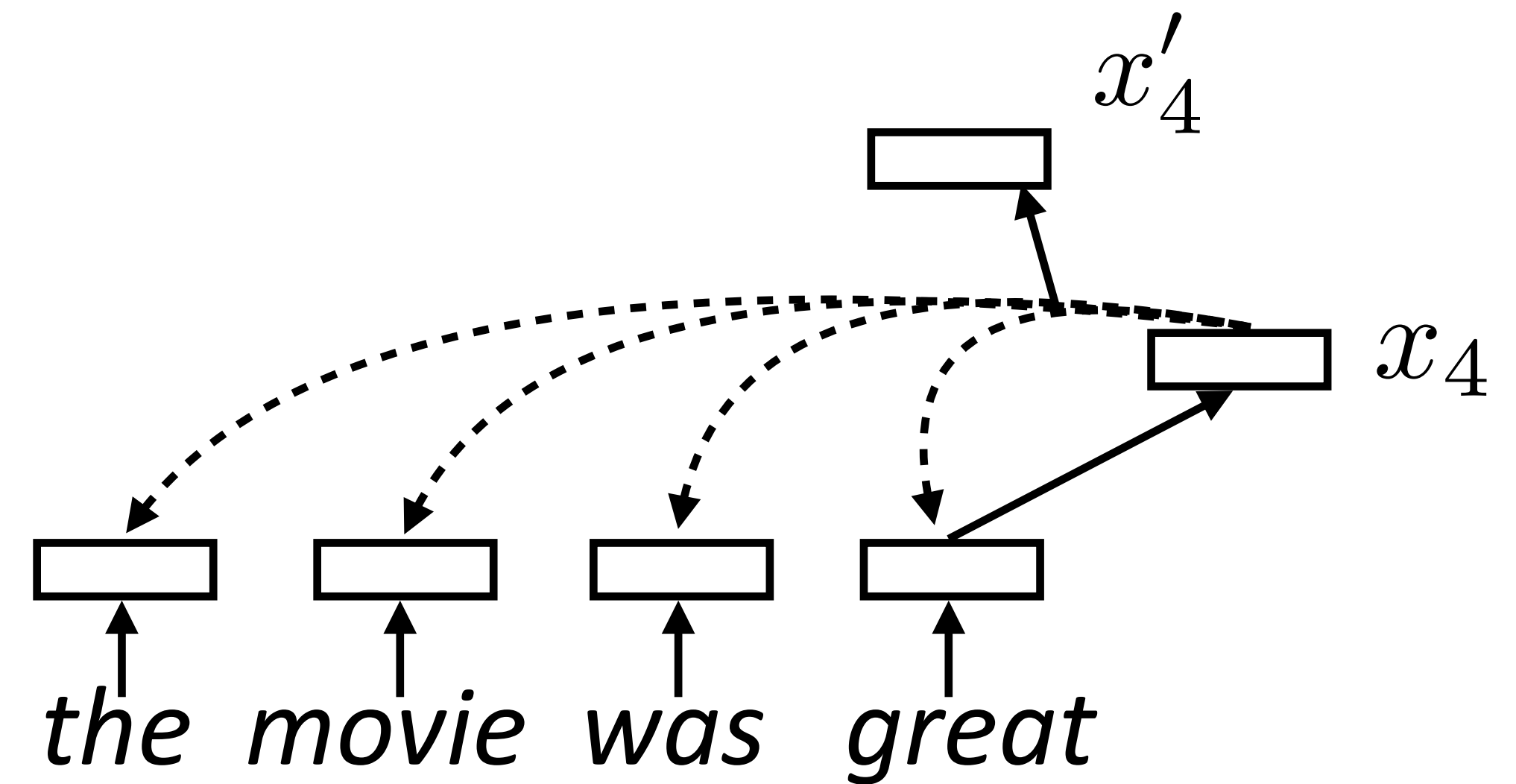


Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

$$x'_i = \sum_{j=1}^n \alpha_{i,j} x_j \quad \text{vector} = \text{sum of scalar} * \text{vector}$$



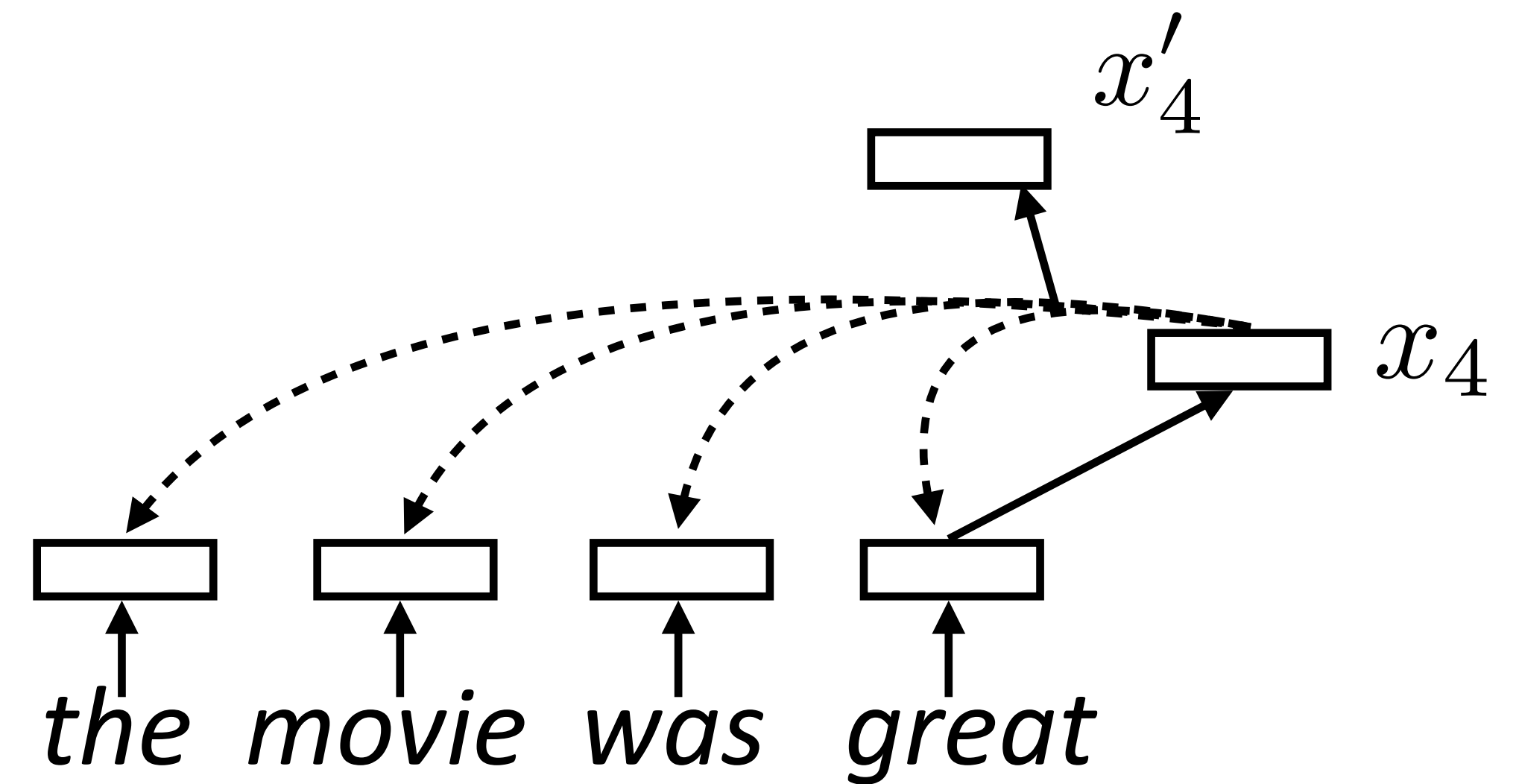
- Multiple “heads” analogous to different convolutional filters. Use parameters W_k and V_k to get different attention values + transform vectors

Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

$$x'_i = \sum_{j=1}^n \alpha_{i,j} x_j \quad \text{vector} = \text{sum of scalar} * \text{vector}$$



- Multiple “heads” analogous to different convolutional filters. Use parameters W_k and V_k to get different attention values + transform vectors

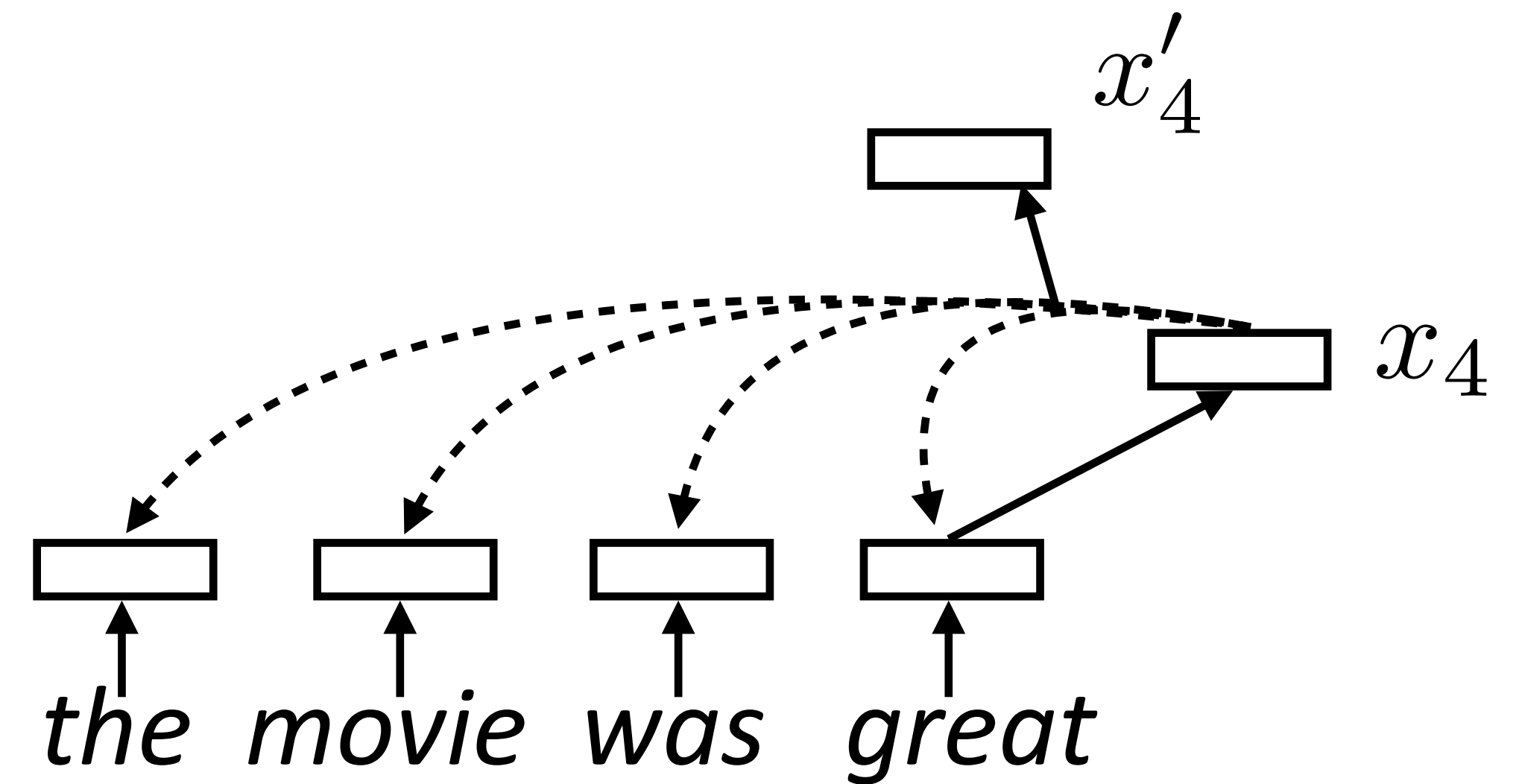
$$\alpha_{k,i,j} = \text{softmax}(x_i^\top W_k x_j)$$

Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

$$x'_i = \sum_{j=1}^n \alpha_{i,j} x_j \quad \text{vector} = \text{sum of scalar} * \text{vector}$$



- Multiple “heads” analogous to different convolutional filters. Use parameters W_k and V_k to get different attention values + transform vectors

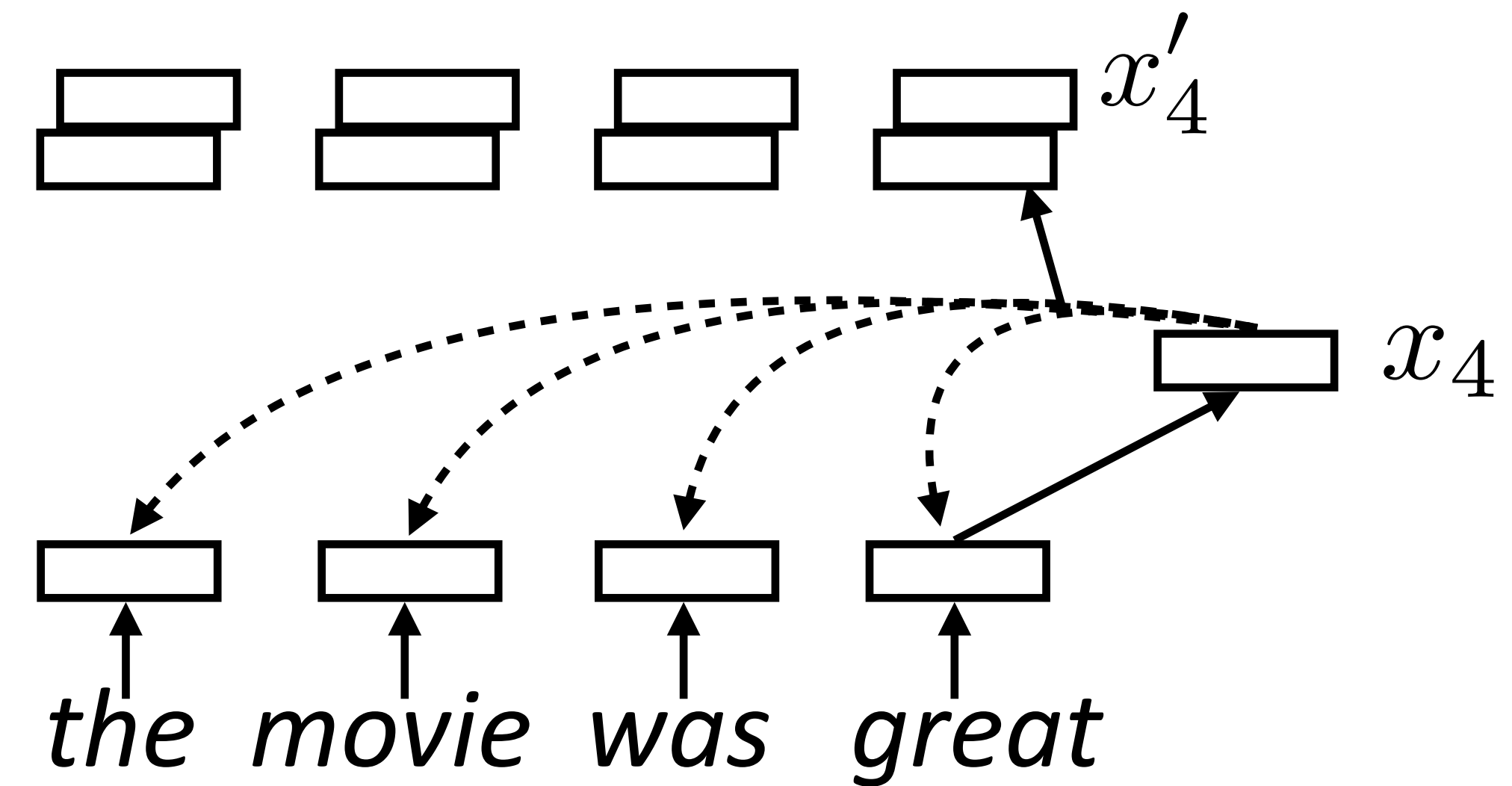
$$\alpha_{k,i,j} = \text{softmax}(x_i^\top W_k x_j) \quad x'_{k,i} = \sum_{j=1}^n \alpha_{k,i,j} V_k x_j$$

Self-Attention

- Each word forms a “query” which then computes attention over each word

$$\alpha_{i,j} = \text{softmax}(x_i^\top x_j) \quad \text{scalar}$$

$$x'_i = \sum_{j=1}^n \alpha_{i,j} x_j \quad \text{vector} = \text{sum of scalar} * \text{vector}$$



- Multiple “heads” analogous to different convolutional filters. Use parameters W_k and V_k to get different attention values + transform vectors

$$\alpha_{k,i,j} = \text{softmax}(x_i^\top W_k x_j) \quad x'_{k,i} = \sum_{j=1}^n \alpha_{k,i,j} V_k x_j$$

Multi-Head Self Attention

- ▶ Multiple “heads” - actual implementation that uses matrix operations

Multi-Head Self Attention

- ▶ Multiple “heads” - actual implementation that uses matrix operations
- ▶ Let $X = [\text{sent len}, \text{embedding dim}]$ be the input sentence

Multi-Head Self Attention

- ▶ Multiple “heads” - actual implementation that uses matrix operations
- ▶ Let $X = [\text{sent len}, \text{embedding dim}]$ be the input sentence
- ▶ Query $Q = XW^Q$: these are like the **decoder hidden state** in attention

Multi-Head Self Attention

- ▶ Multiple “heads” - actual implementation that uses matrix operations
- ▶ Let $X = [\text{sent len}, \text{embedding dim}]$ be the input sentence
- ▶ Query $Q = XW^Q$: these are like the **decoder hidden state** in attention
- ▶ Keys $K = XW^K$: these control what gets attended to, along with the query

Multi-Head Self Attention

- ▶ Multiple “heads” - actual implementation that uses matrix operations
- ▶ Let $X = [\text{sent len}, \text{embedding dim}]$ be the input sentence
- ▶ Query $Q = XW^Q$: these are like the **decoder hidden state** in attention
- ▶ Keys $K = XW^K$: these control what gets attended to, along with the query
- ▶ Values $V = XW^V$: these vectors get summed up to form the output

Multi-Head Self Attention

- ▶ Multiple “heads” - actual implementation that uses matrix operations
- ▶ Let $X = [\text{sent len}, \text{embedding dim}]$ be the input sentence
- ▶ Query $Q = XW^Q$: these are like the **decoder hidden state** in attention
- ▶ Keys $K = XW^K$: these control what gets attended to, along with the query
- ▶ Values $V = XW^V$: these vectors get summed up to form the output

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

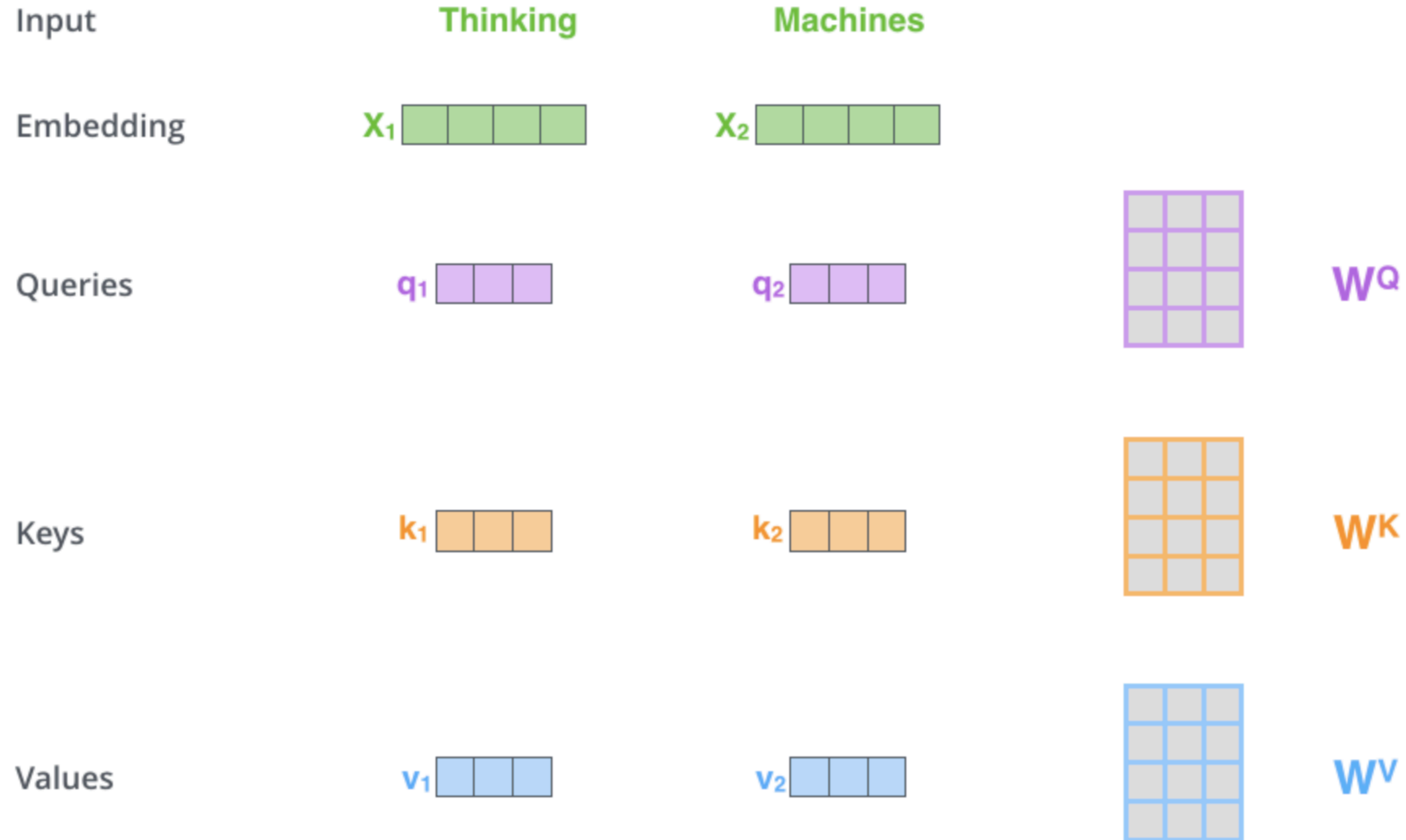
Multi-Head Self Attention

- ▶ Multiple “heads” - actual implementation that uses matrix operations
- ▶ Let $X = [\text{sent len}, \text{embedding dim}]$ be the input sentence
- ▶ Query $Q = XW^Q$: these are like the **decoder hidden state** in attention
- ▶ Keys $K = XW^K$: these control what gets attended to, along with the query
- ▶ Values $V = XW^V$: these vectors get summed up to form the output

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

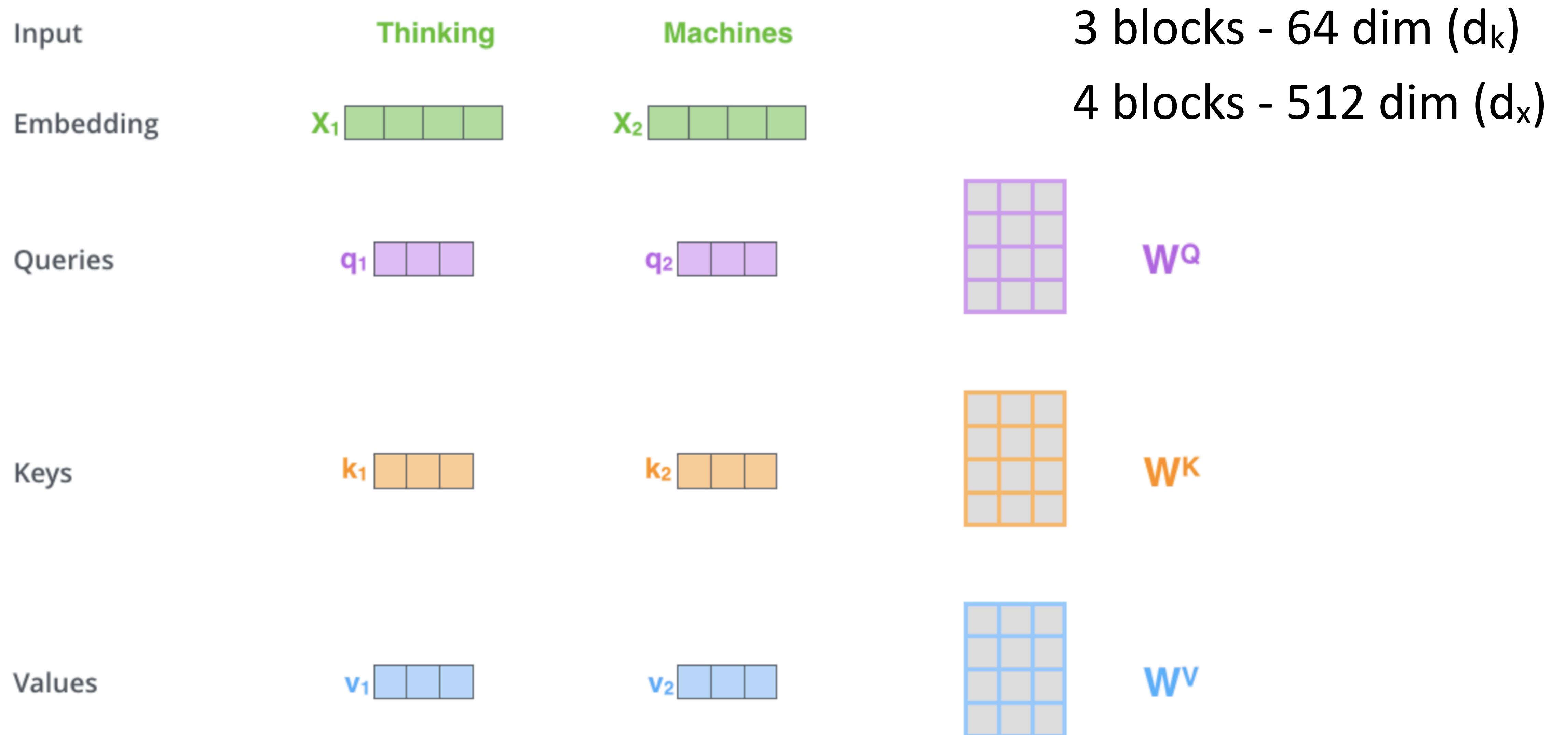
← dim of keys

Multi-Head Self Attention



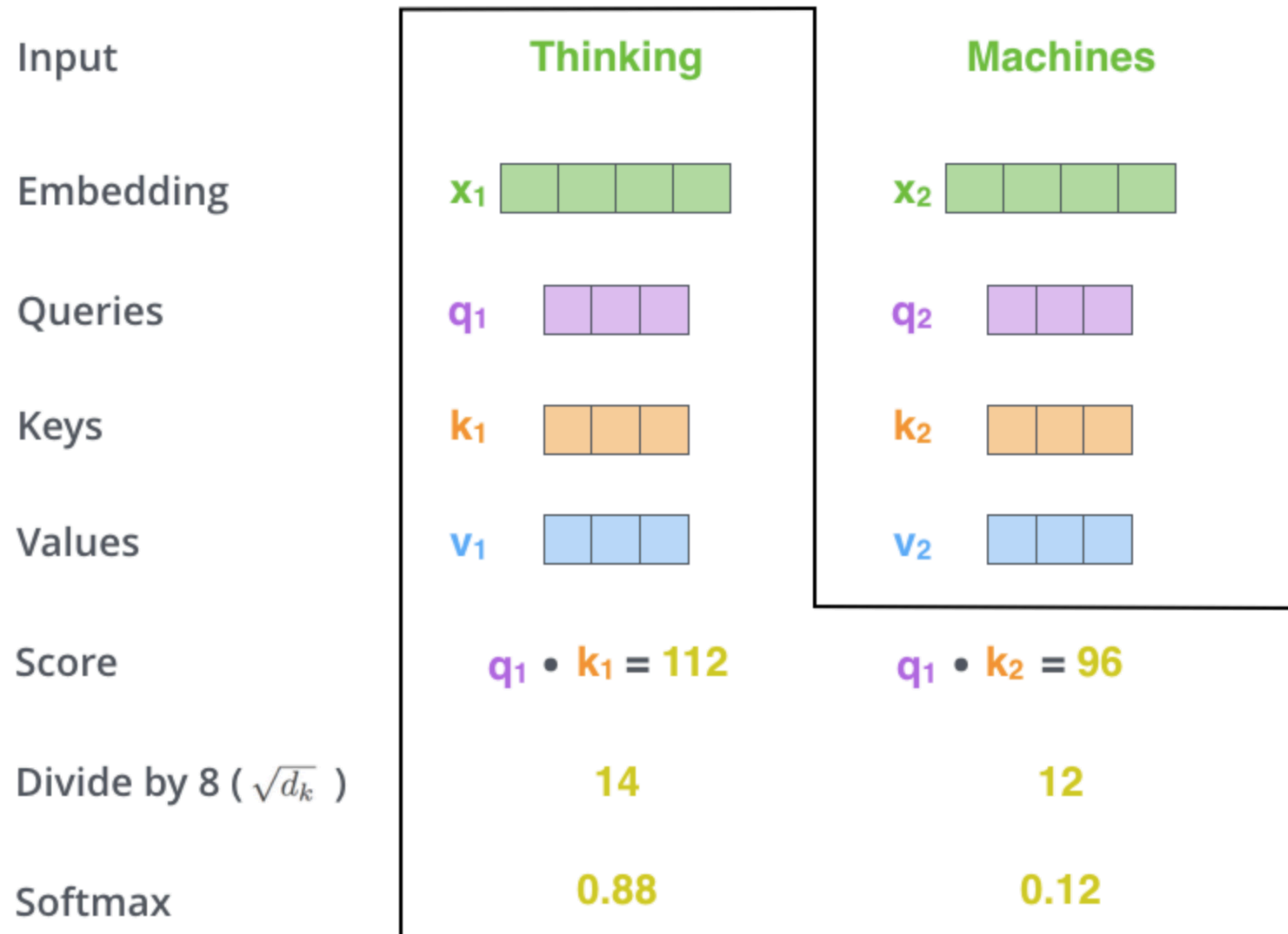
Credit: Alammam, *The Illustrated Transformer*

Multi-Head Self Attention



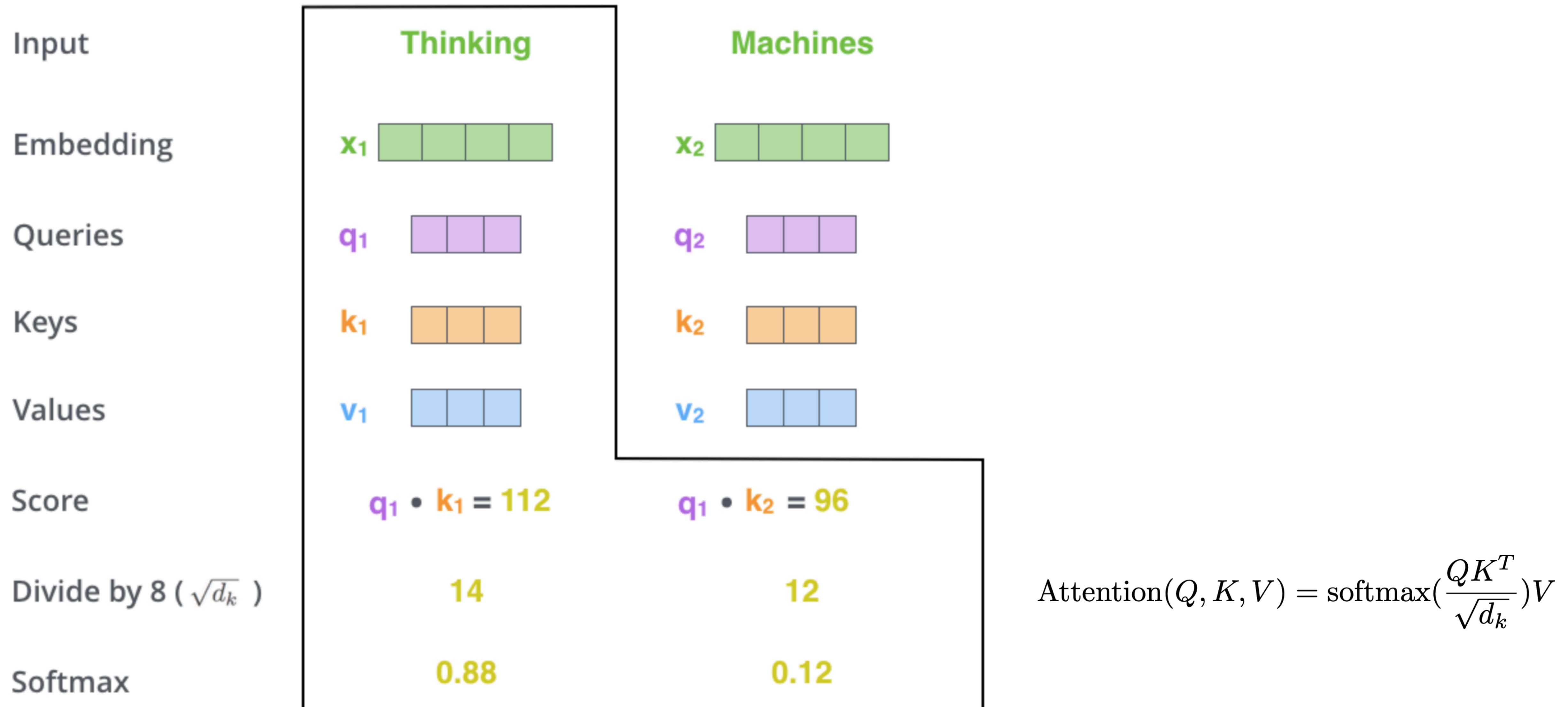
Credit: Alammam, *The Illustrated Transformer*

Multi-Head Self Attention



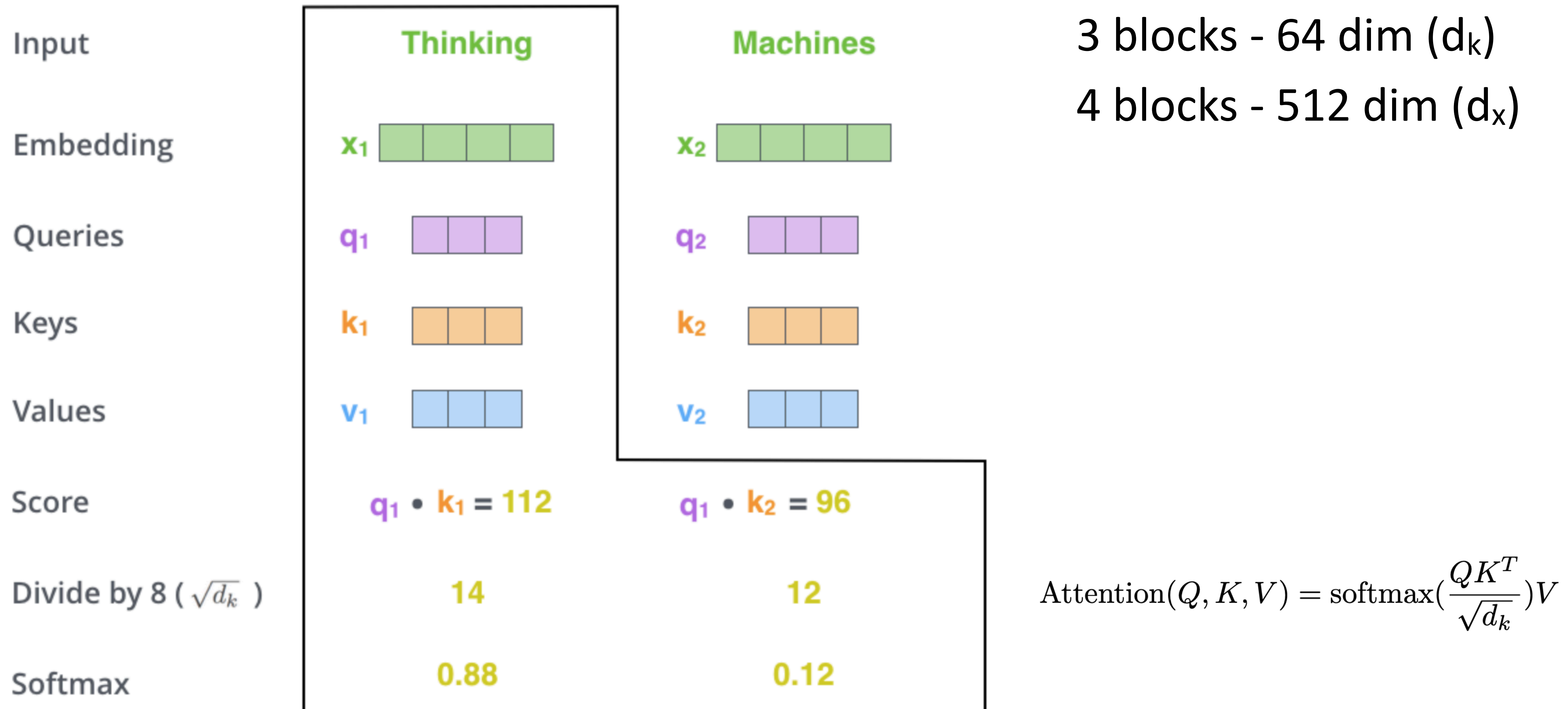
Credit: Alammam, *The Illustrated Transformer*

Multi-Head Self Attention



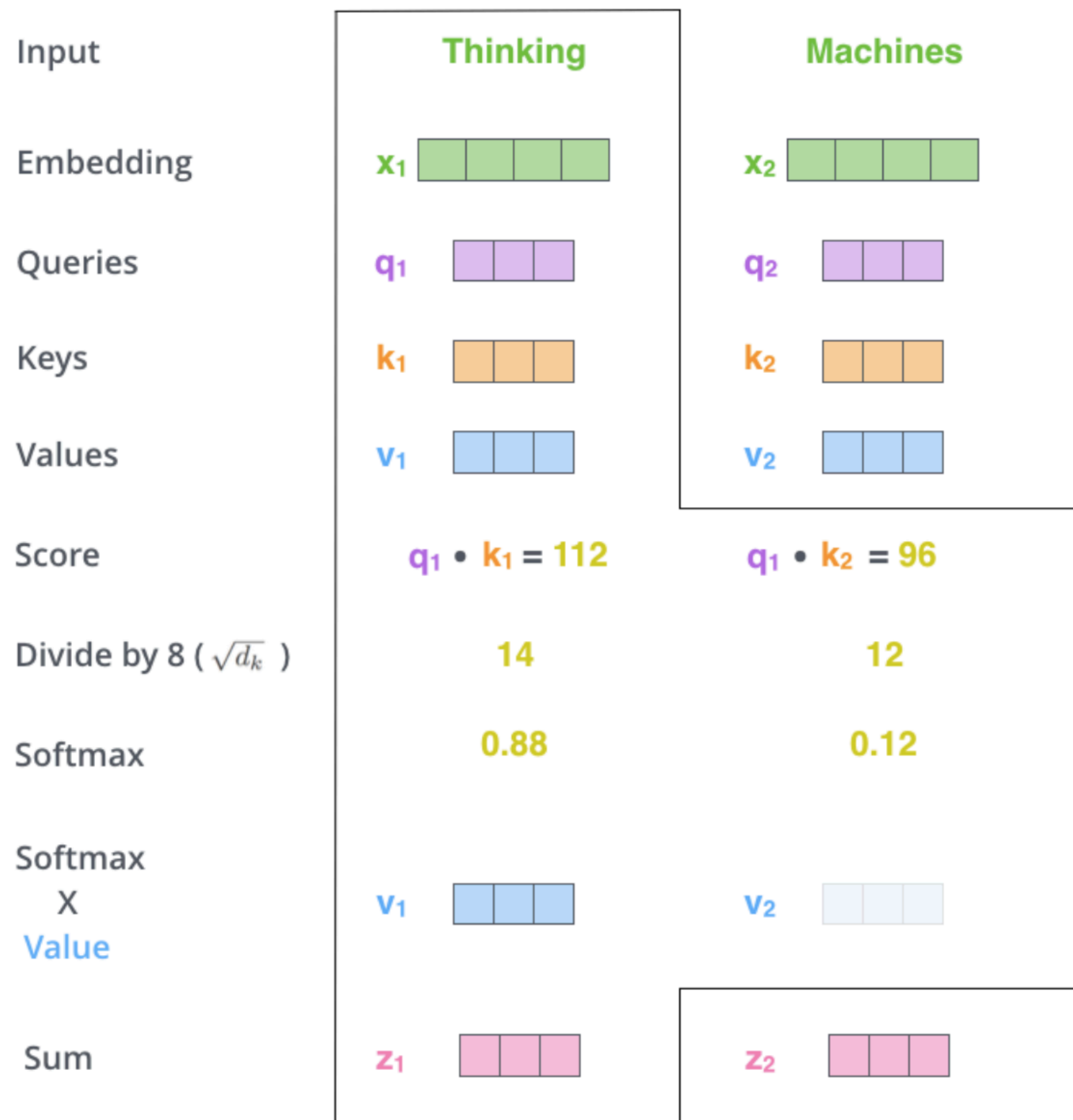
Credit: Alammam, *The Illustrated Transformer*

Multi-Head Self Attention



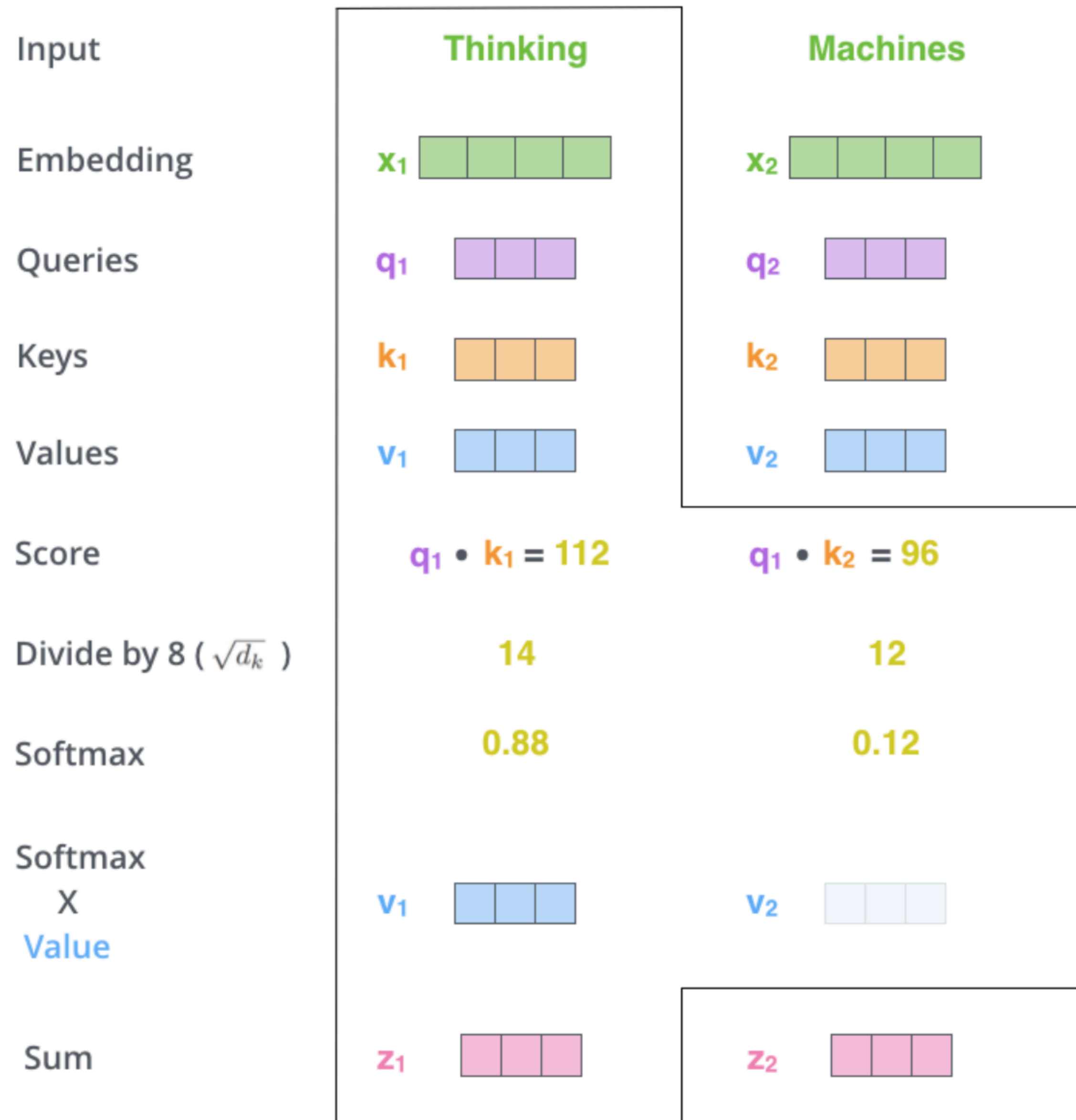
Credit: Alammam, *The Illustrated Transformer*

Multi-Head Self Attention



Credit: Alammam, *The Illustrated Transformer*

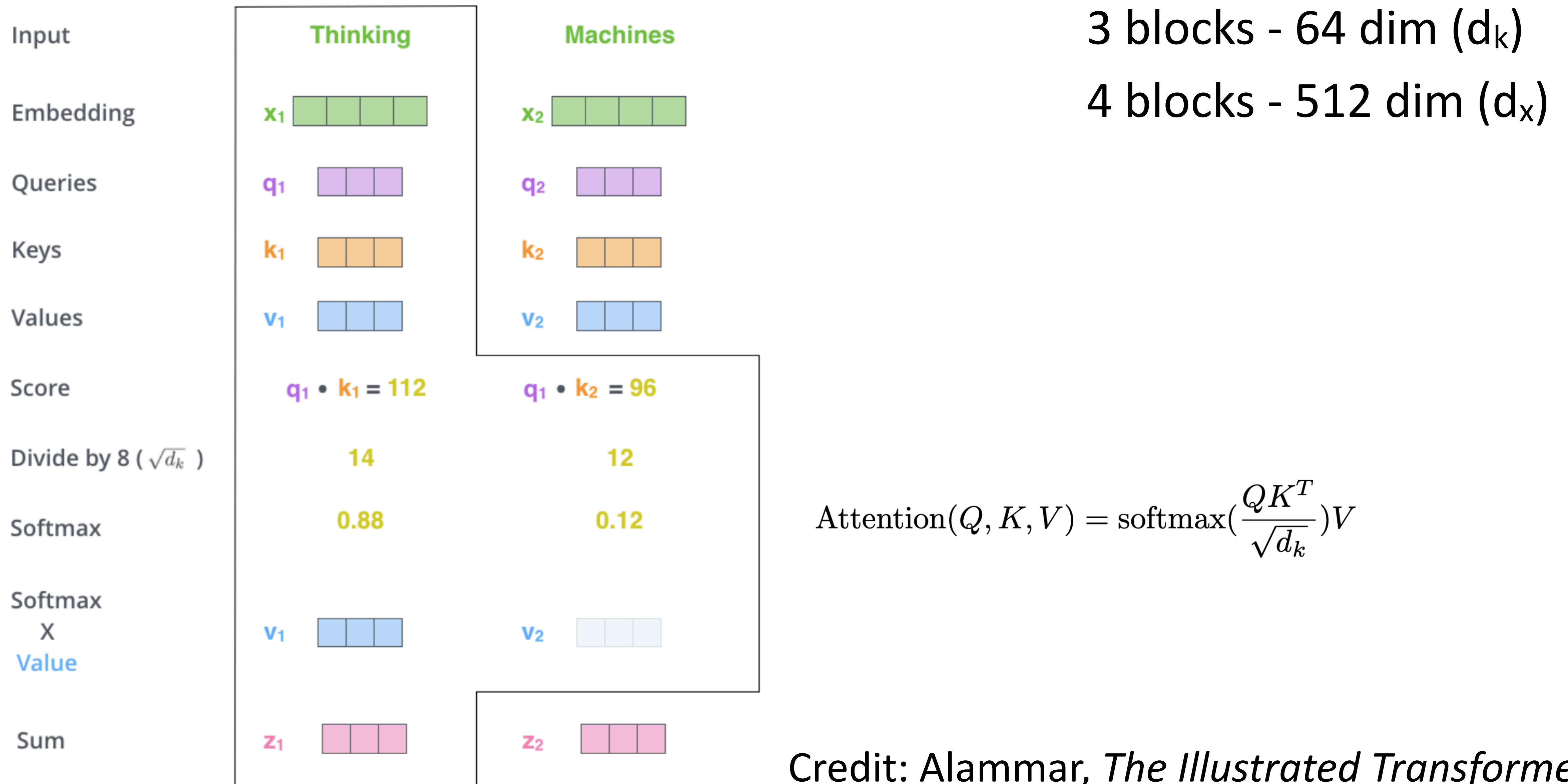
Multi-Head Self Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Credit: Alammam, *The Illustrated Transformer*

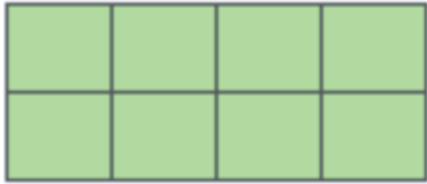
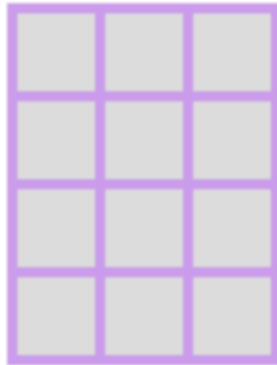
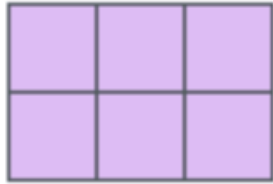
Multi-Head Self Attention



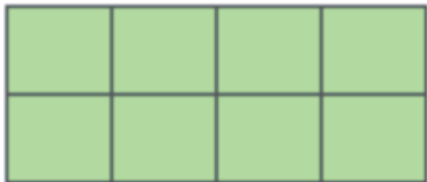
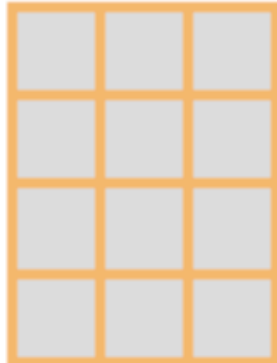
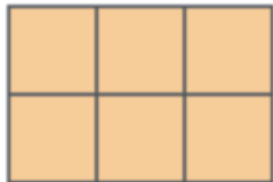
Multi-Head Self Attention

every row in X is a word in input sent

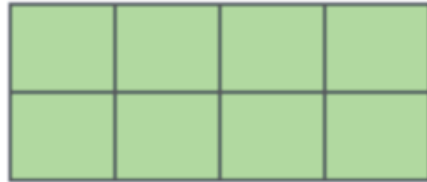
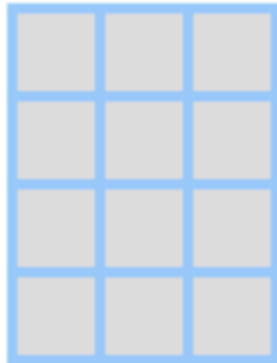
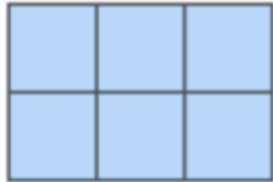
X W^Q Q

Thinking Machines  $\times$  $=$ 

X W^K K

Thinking Machines  $\times$  $=$ 

X W^V V

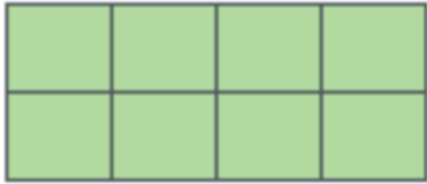
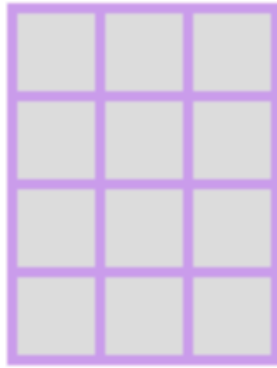
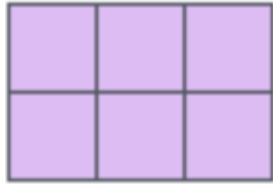
Thinking Machines  $\times$  $=$ 

Credit: Alammam, *The Illustrated Transformer*

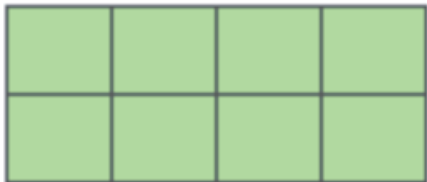
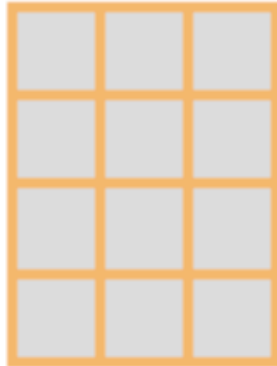
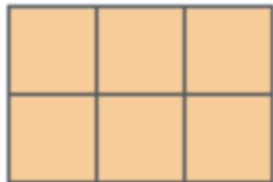
Multi-Head Self Attention

every row in X is a word in input sent

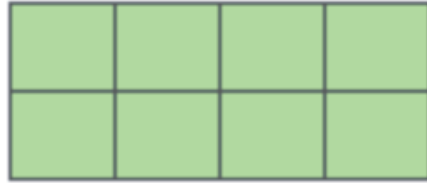
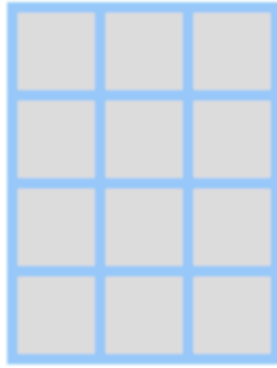
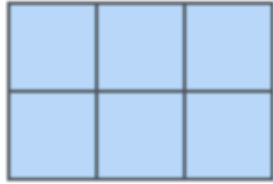
X W^Q Q

Thinking Machines  $\times$  $=$ 

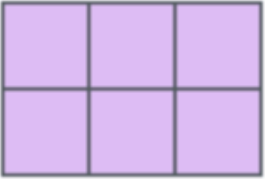
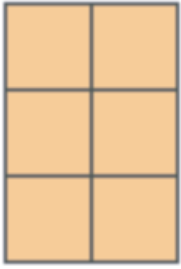
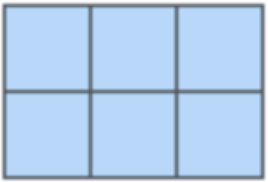
X W^K K

Thinking Machines  $\times$  $=$ 

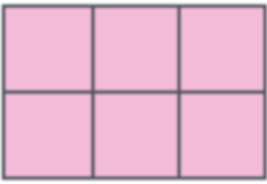
X W^V V

Thinking Machines  $\times$  $=$ 

Q K^T V

$\text{softmax}\left(\frac{\text{ \times \text{}}{\sqrt{d_k}}\right)$ 

Z

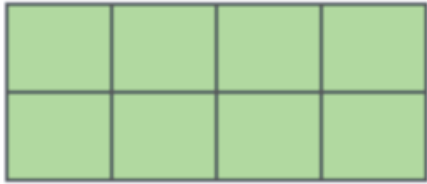
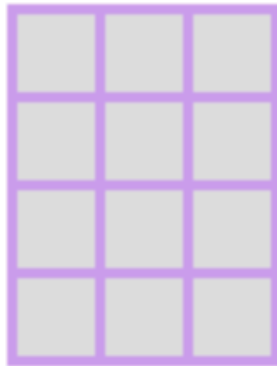
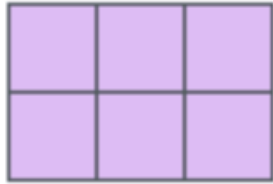
$=$ 

Credit: Alammam, *The Illustrated Transformer*

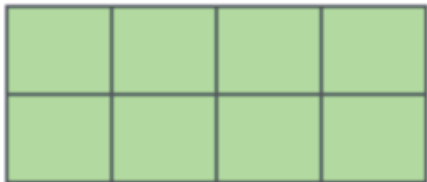
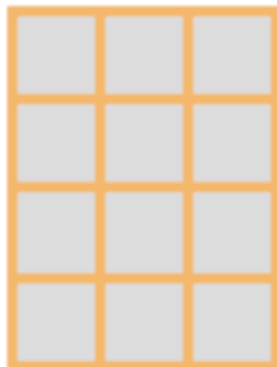
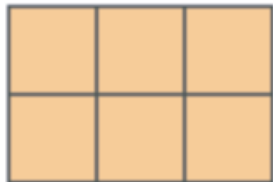
Multi-Head Self Attention

every row in X is a word in input sent

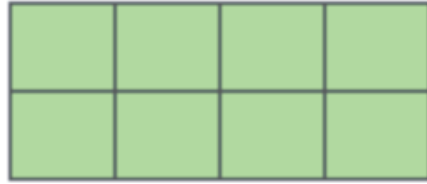
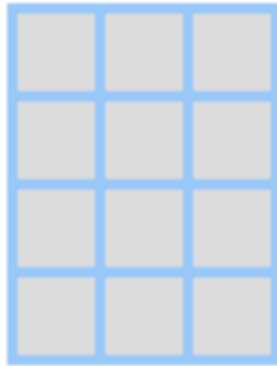
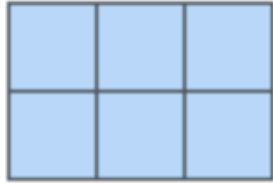
X W^Q Q

Thinking Machines  $\times$  $=$ 

X W^K K

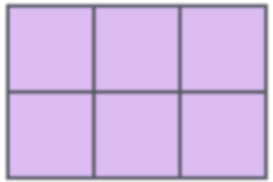
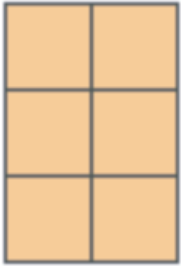
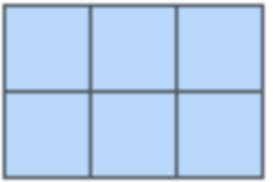
Thinking Machines  $\times$  $=$ 

X W^V V

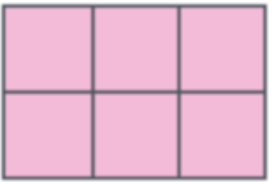
Thinking Machines  $\times$  $=$ 

sent len x sent len (attn for each word to each other)

Q K^T V

$\text{softmax}\left(\frac{\text{ \times \text{}}{\sqrt{d_k}}\right)$ 

Z

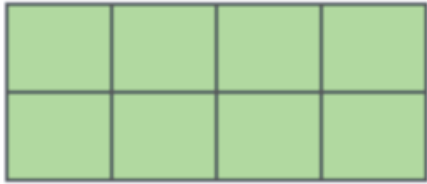
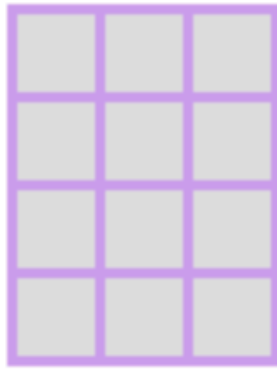
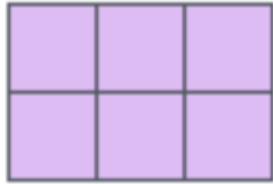
$=$ 

Credit: Alammam, *The Illustrated Transformer*

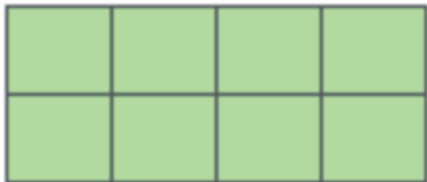
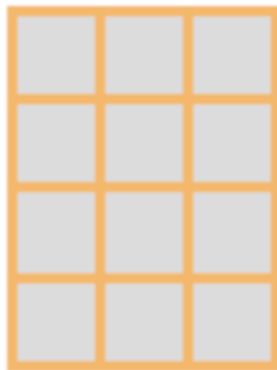
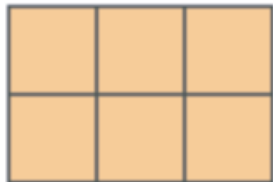
Multi-Head Self Attention

every row in X is a word in input sent

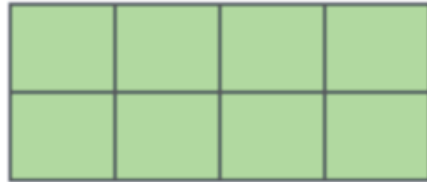
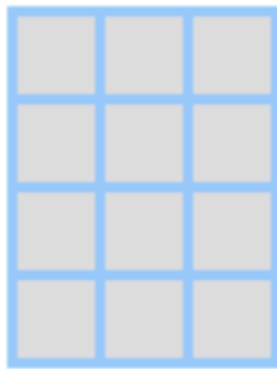
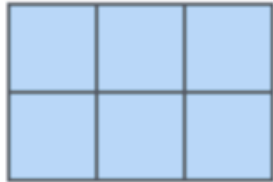
X W^Q Q

Thinking Machines  $\times$  $=$ 

X W^K K

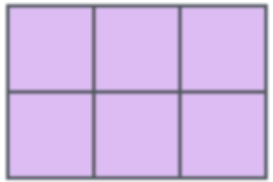
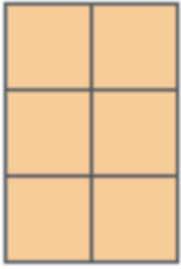
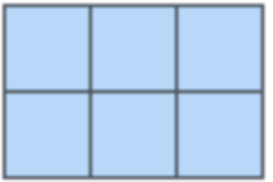
Thinking Machines  $\times$  $=$ 

X W^V V

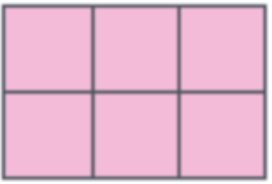
Thinking Machines  $\times$  $=$ 

sent len x sent len (attn for each word to each other)

Q K^T V

$\text{softmax}\left(\frac{\text{ \times \text{}}{\sqrt{d_k}}\right)$ 

Z

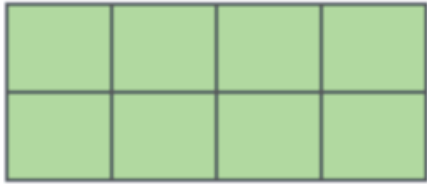
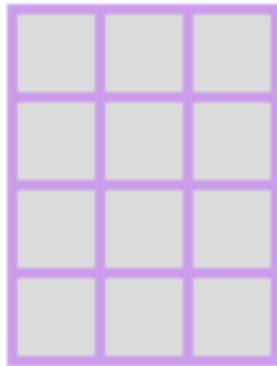
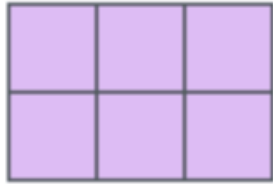
$=$  sent len x hidden dim

Credit: Alammr, *The Illustrated Transformer*

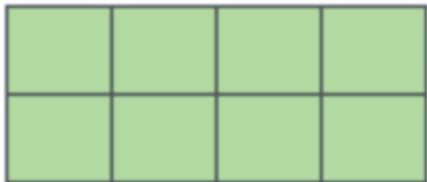
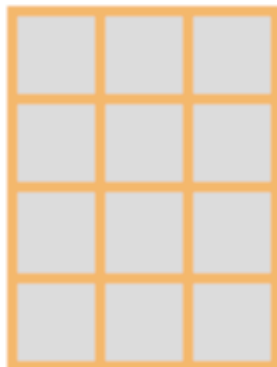
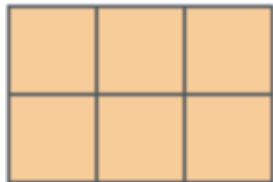
Multi-Head Self Attention

every row in X is a word in input sent

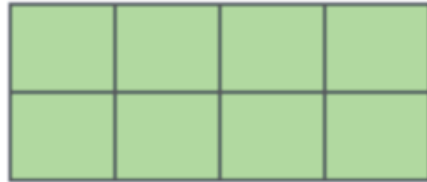
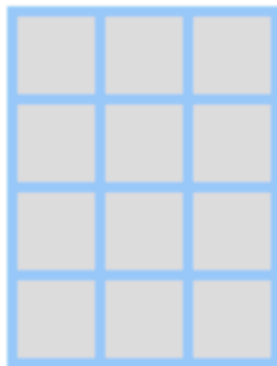
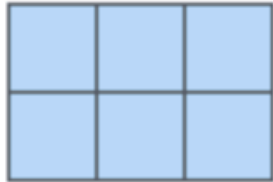
X W^Q Q

Thinking Machines  $\times$  $=$ 

X W^K K

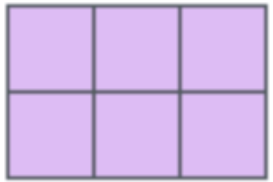
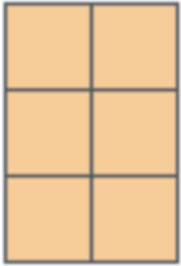
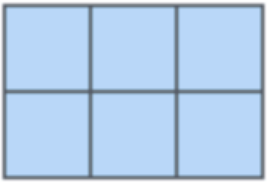
Thinking Machines  $\times$  $=$ 

X W^V V

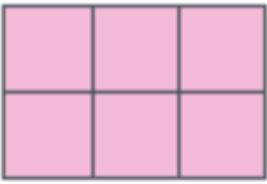
Thinking Machines  $\times$  $=$ 

sent len x sent len (attn for each word to each other)

Q K^T V

$\text{softmax}\left(\frac{\text{ \times \text{}}{\sqrt{d_k}}\right)$ 

Z

$=$ 

sent len x hidden dim

Z is a weighted combination of V rows

Credit: Alammr, *The Illustrated Transformer*

Multi-Head Self Attention

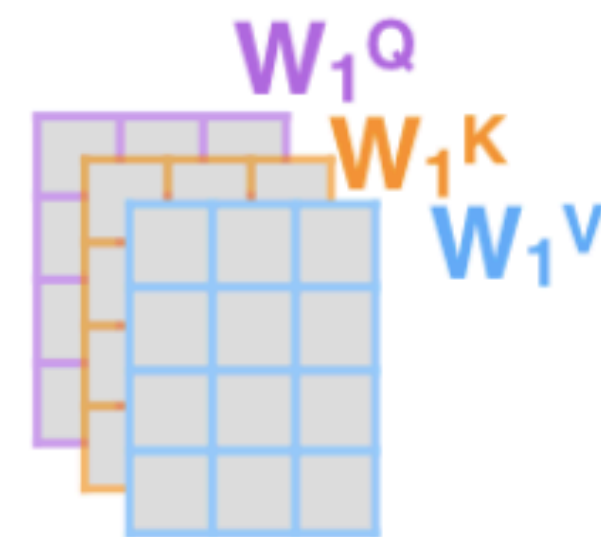
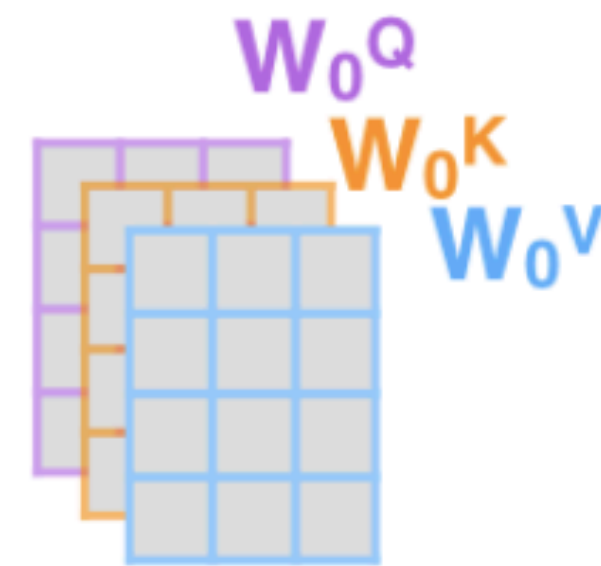
1) This is our input sentence*

Thinking
Machines

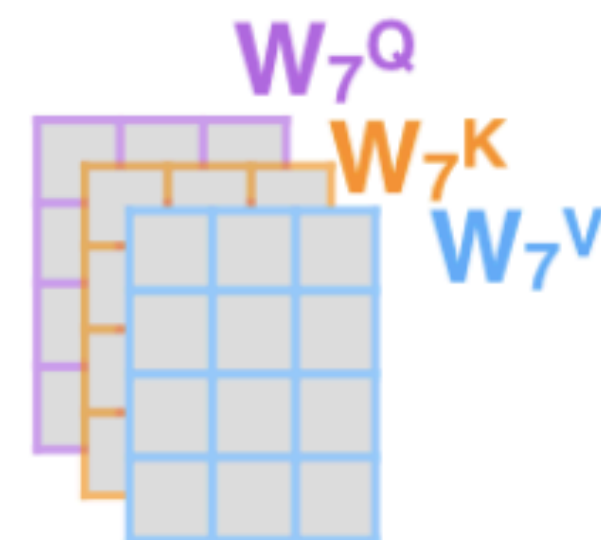
2) We embed each word*



3) Split into 8 heads.
We multiply X or R with weight matrices



...



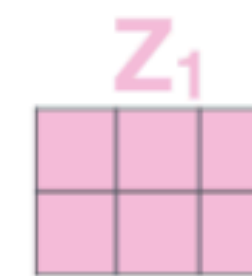
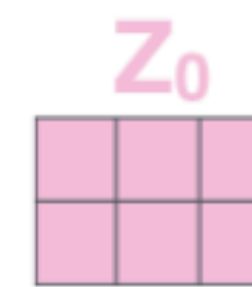
4) Calculate attention using the resulting $Q/K/V$ matrices



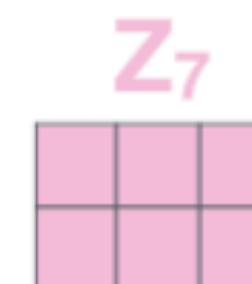
...



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



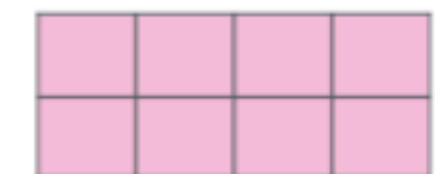
...



W^O



Z



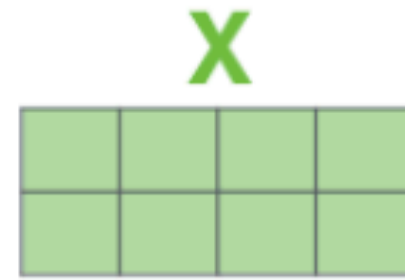
Credit: Alammr, *The Illustrated Transformer*

Multi-Head Self Attention

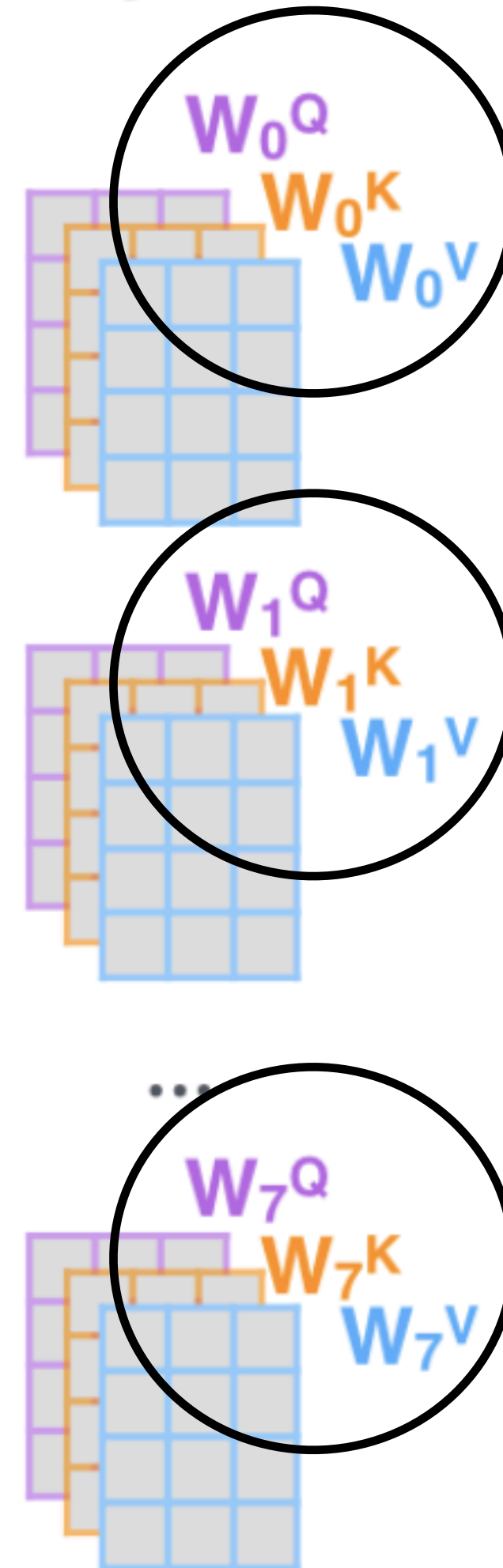
1) This is our input sentence*

Thinking
Machines

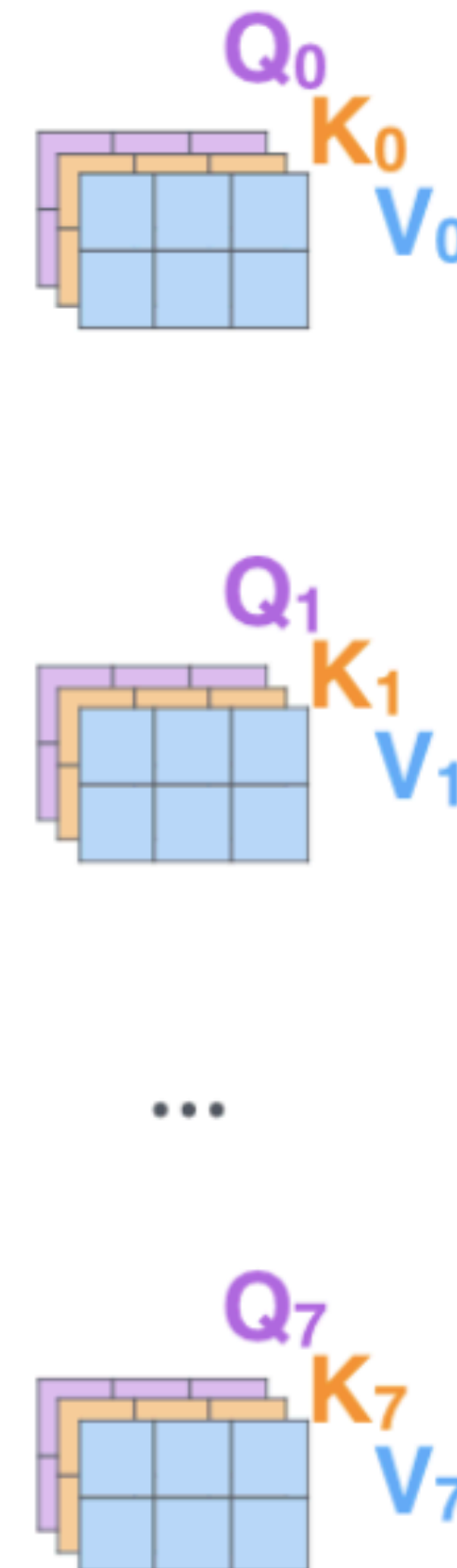
2) We embed each word*



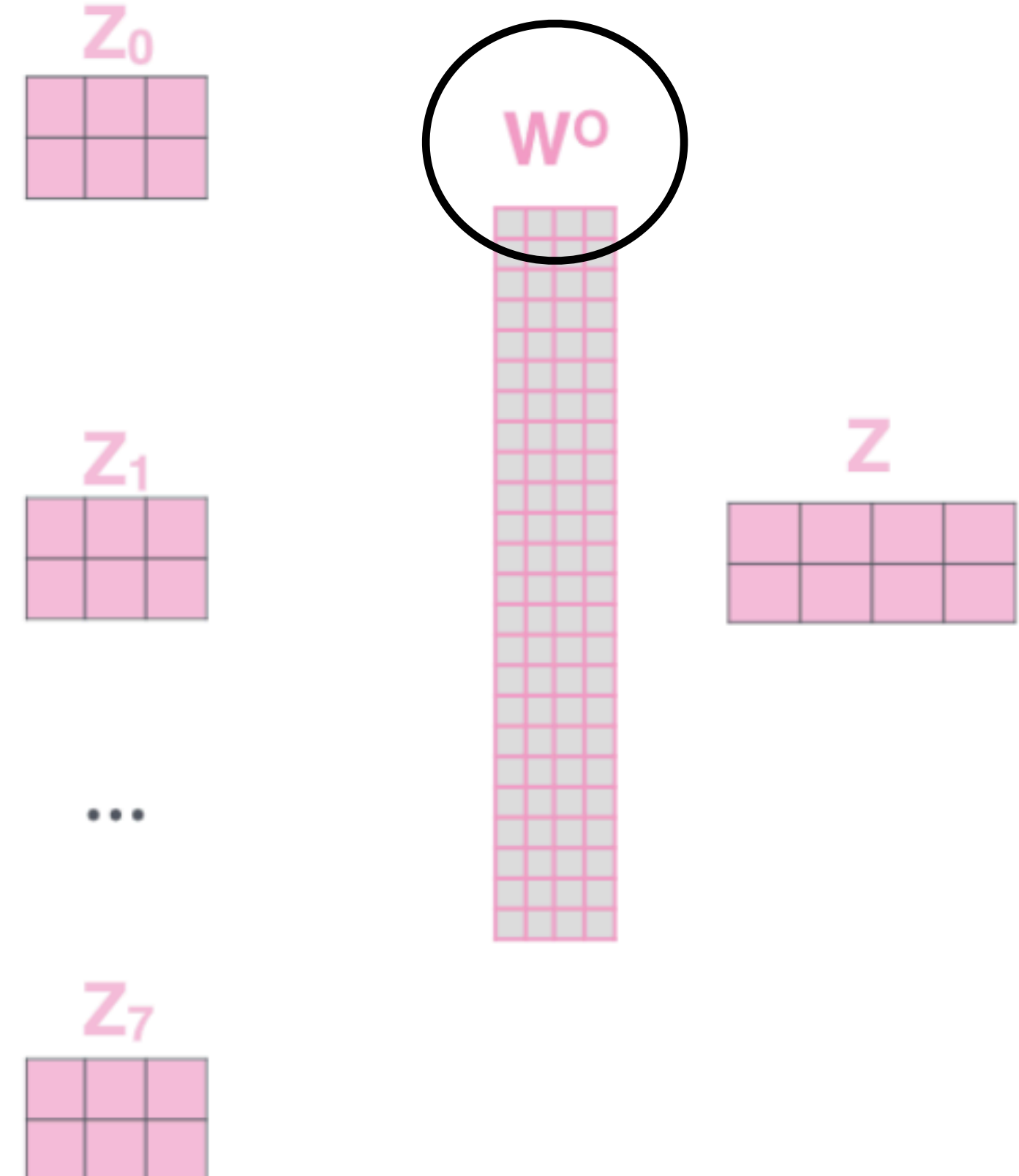
3) Split into 8 heads.
We multiply X or R with weight matrices



4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



Credit: Alammam, *The Illustrated Transformer*

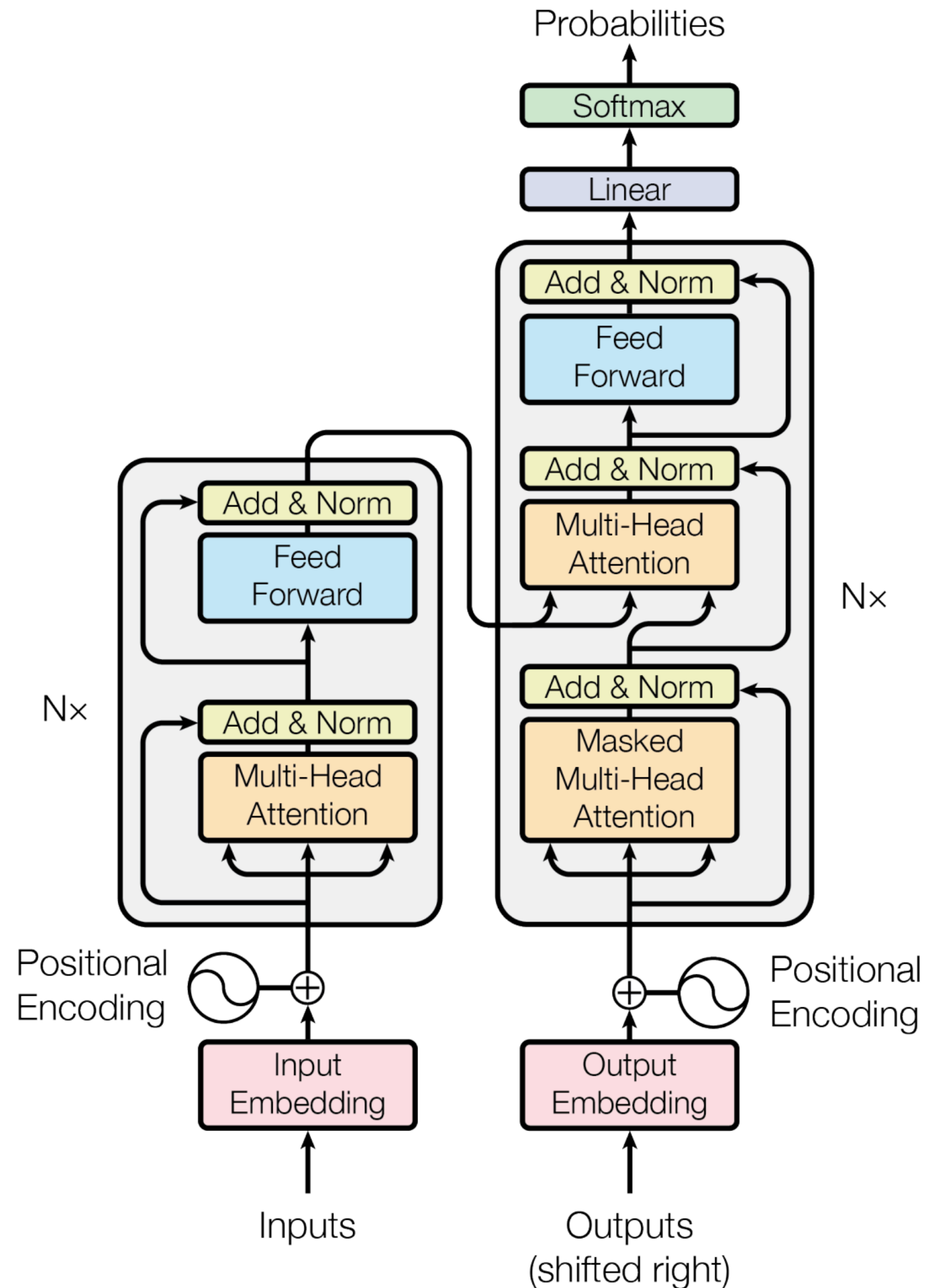
Properties of Self-Attention

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

- ▶ n = sentence length, d = hidden dim, k = kernel size, r = restricted neighborhood size
- ▶ **Quadratic complexity**, but $O(1)$ sequential operations (not linear like in RNNs) and $O(1)$ “path” for words to inform each other

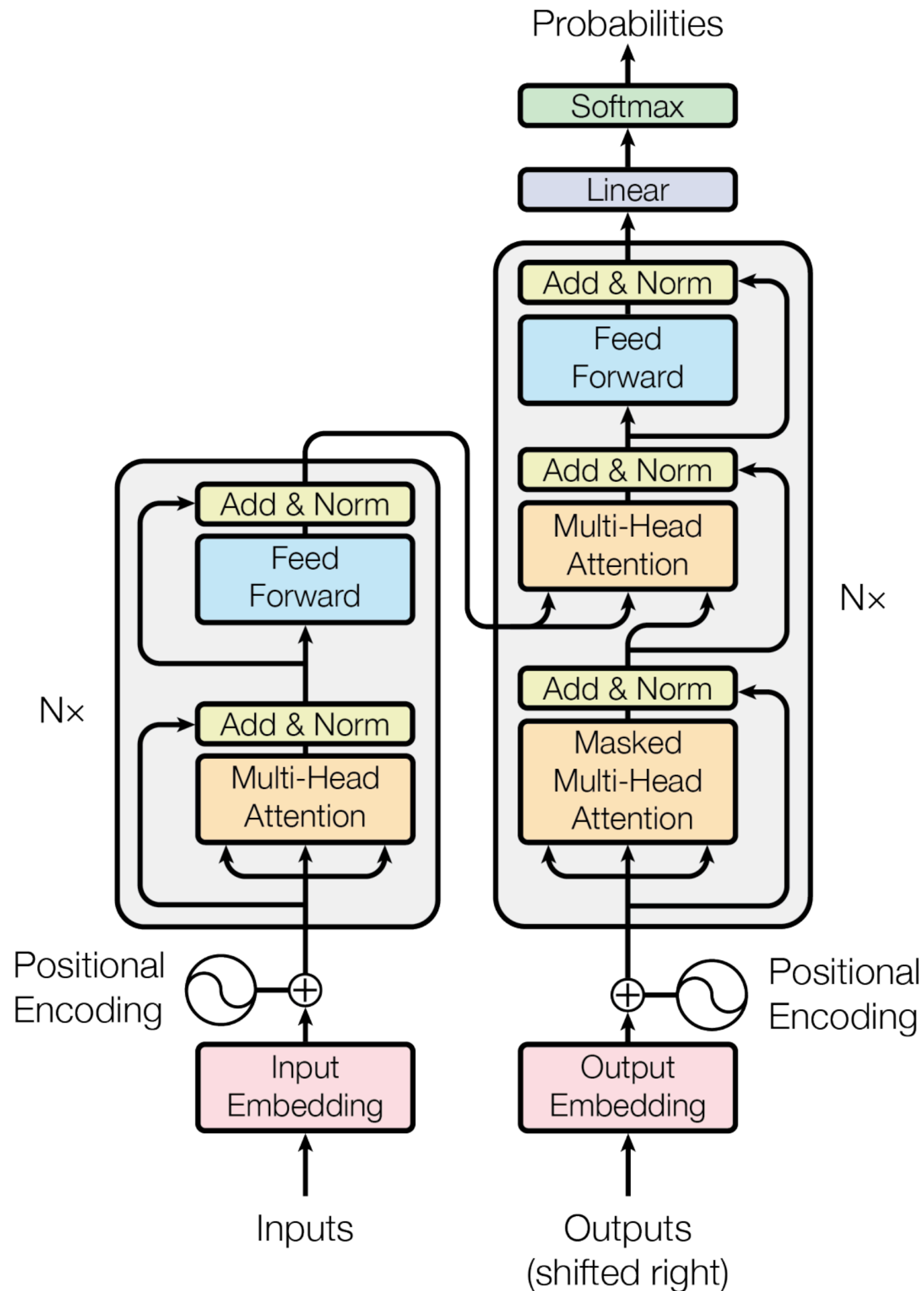
Transformer

Transformers for MT: Complete Model



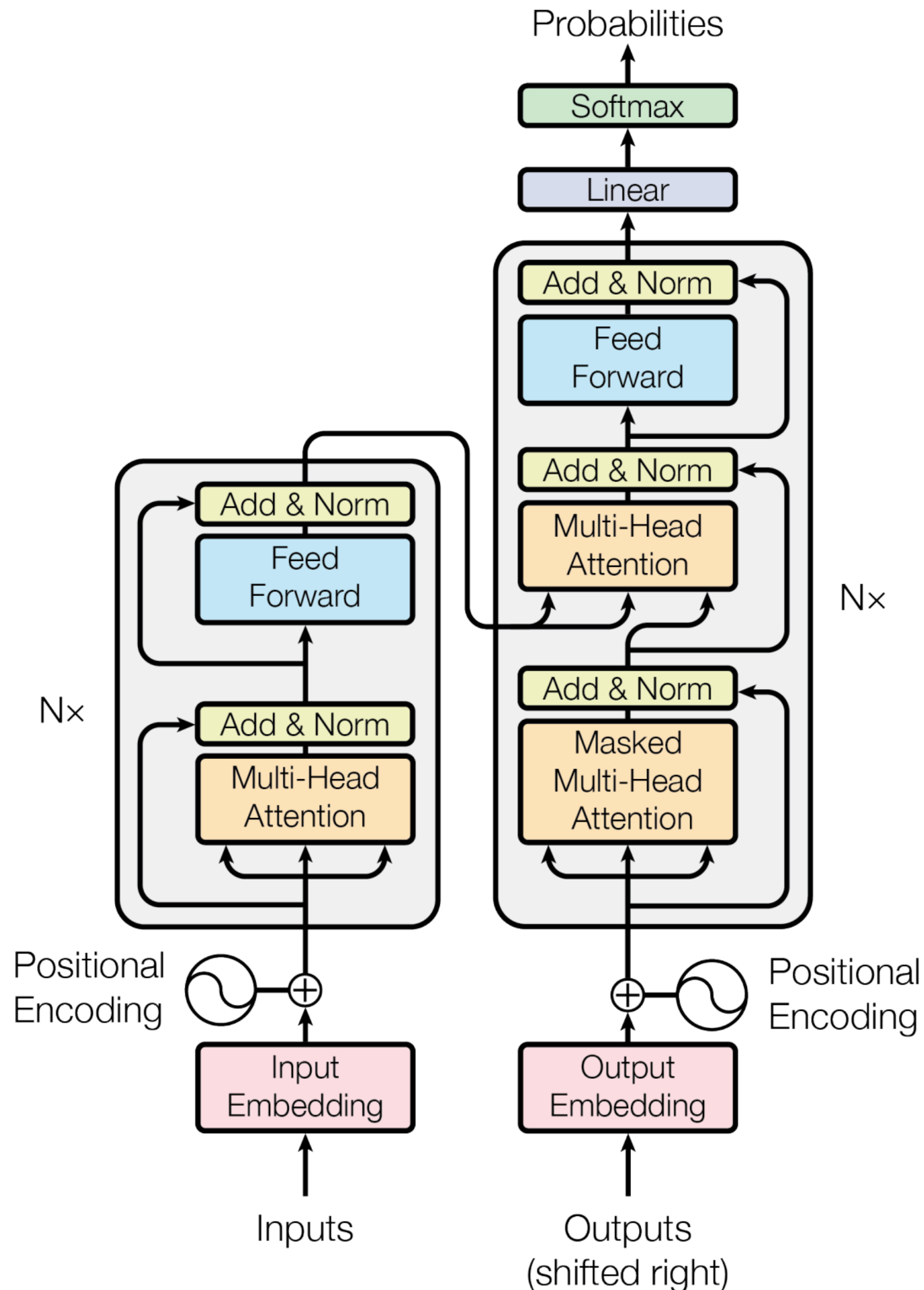
- Encoder and decoder are both transformers

Transformers for MT: Complete Model



- ▶ Encoder and decoder are both transformers
- ▶ Decoder alternates attention over the output and attention over the input as well

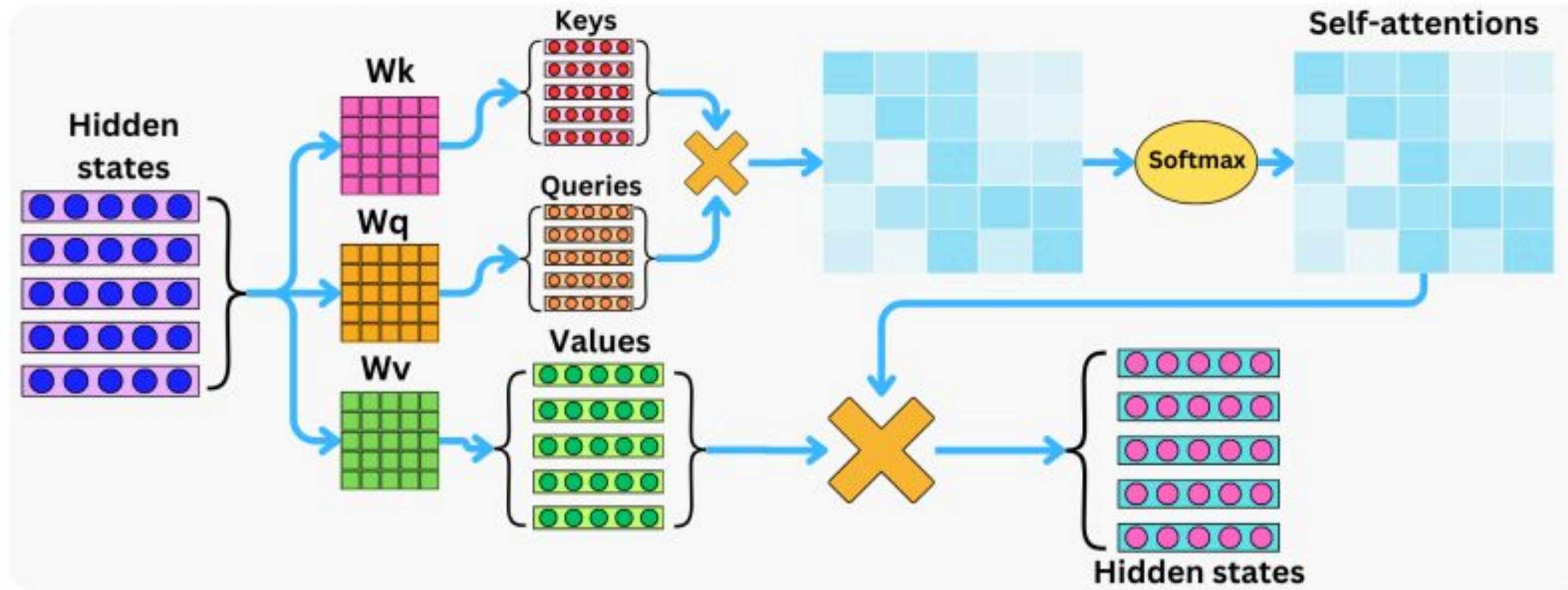
Transformers for MT: Complete Model



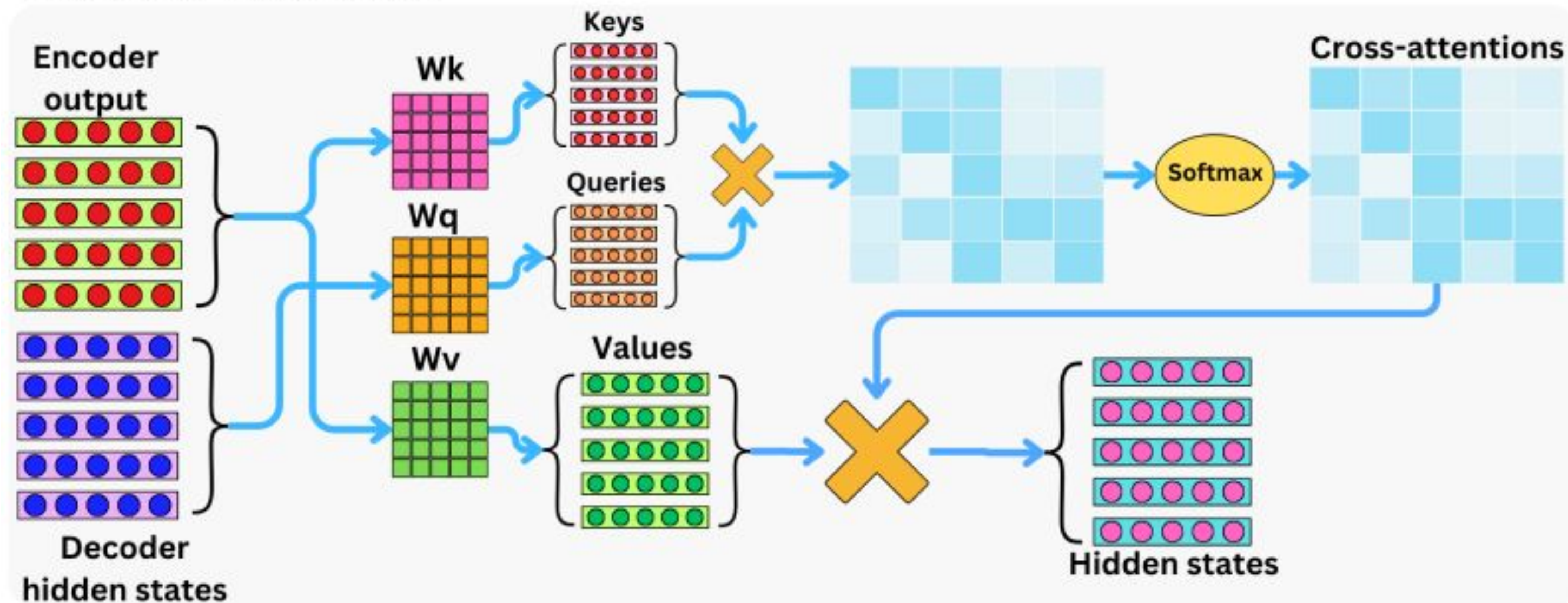
- ▶ Encoder and decoder are both transformers
- ▶ Decoder alternates attention over the output and attention over the input as well
- ▶ Decoder consumes the previous generated tokens but has *no recurrent state*

Self-Attention vs. Cross-Attention

The Self-Attentions

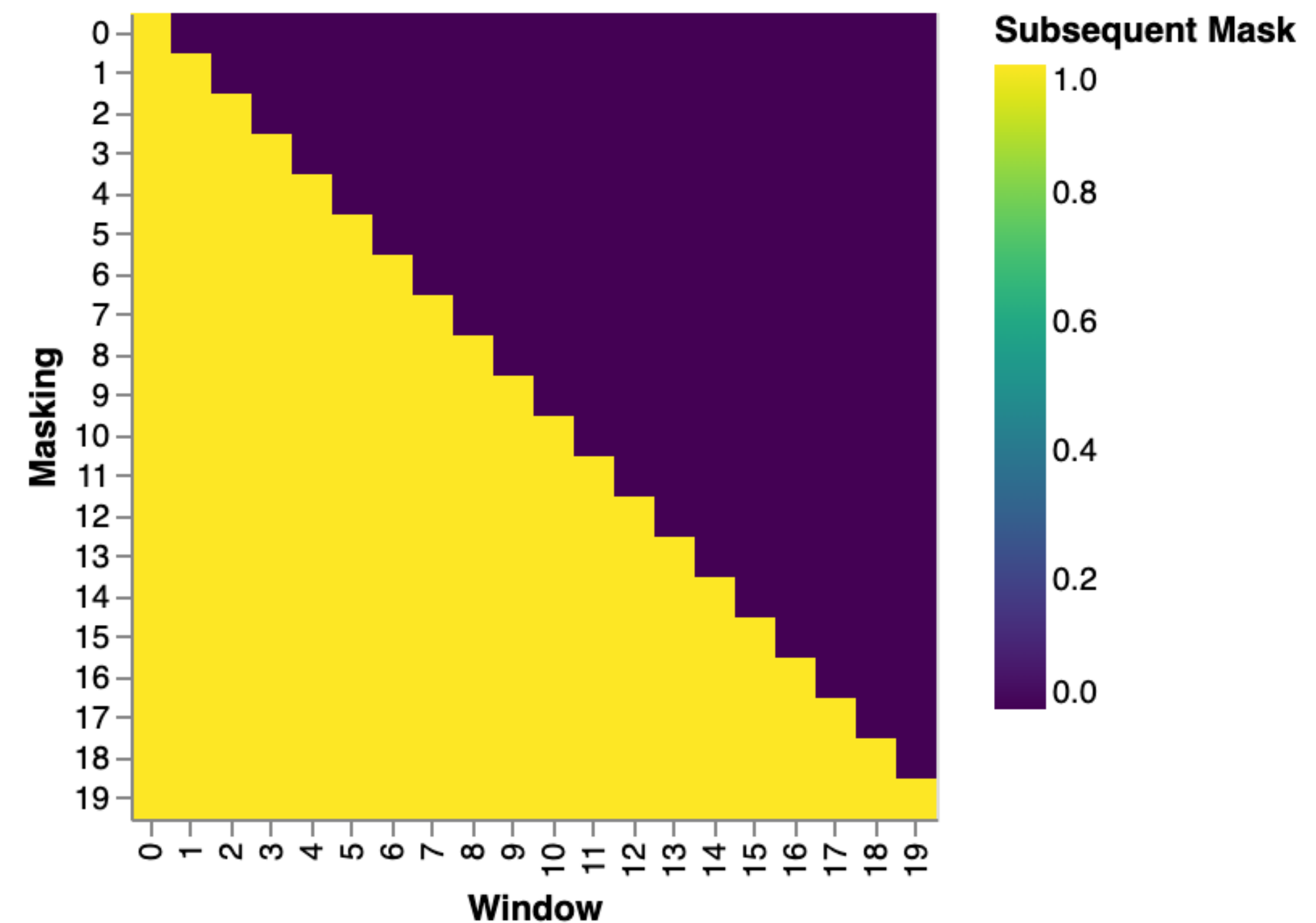


The Cross-Attentions

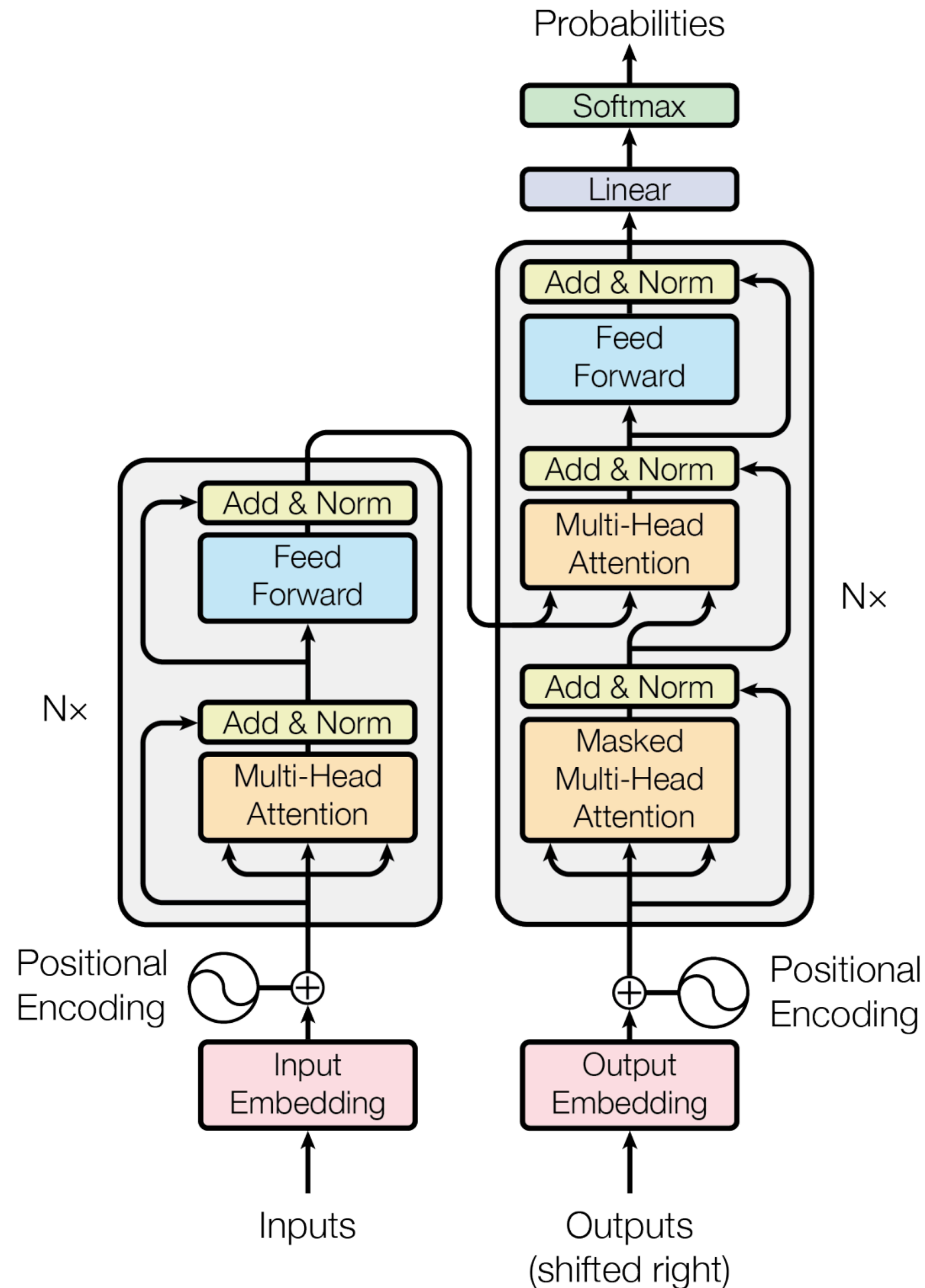


Casual Self-Attention Mask

- ▶ Decoder (by default) is autoregressive, making prediction word by word
- ▶ It should not peek at the right side of the output sentence

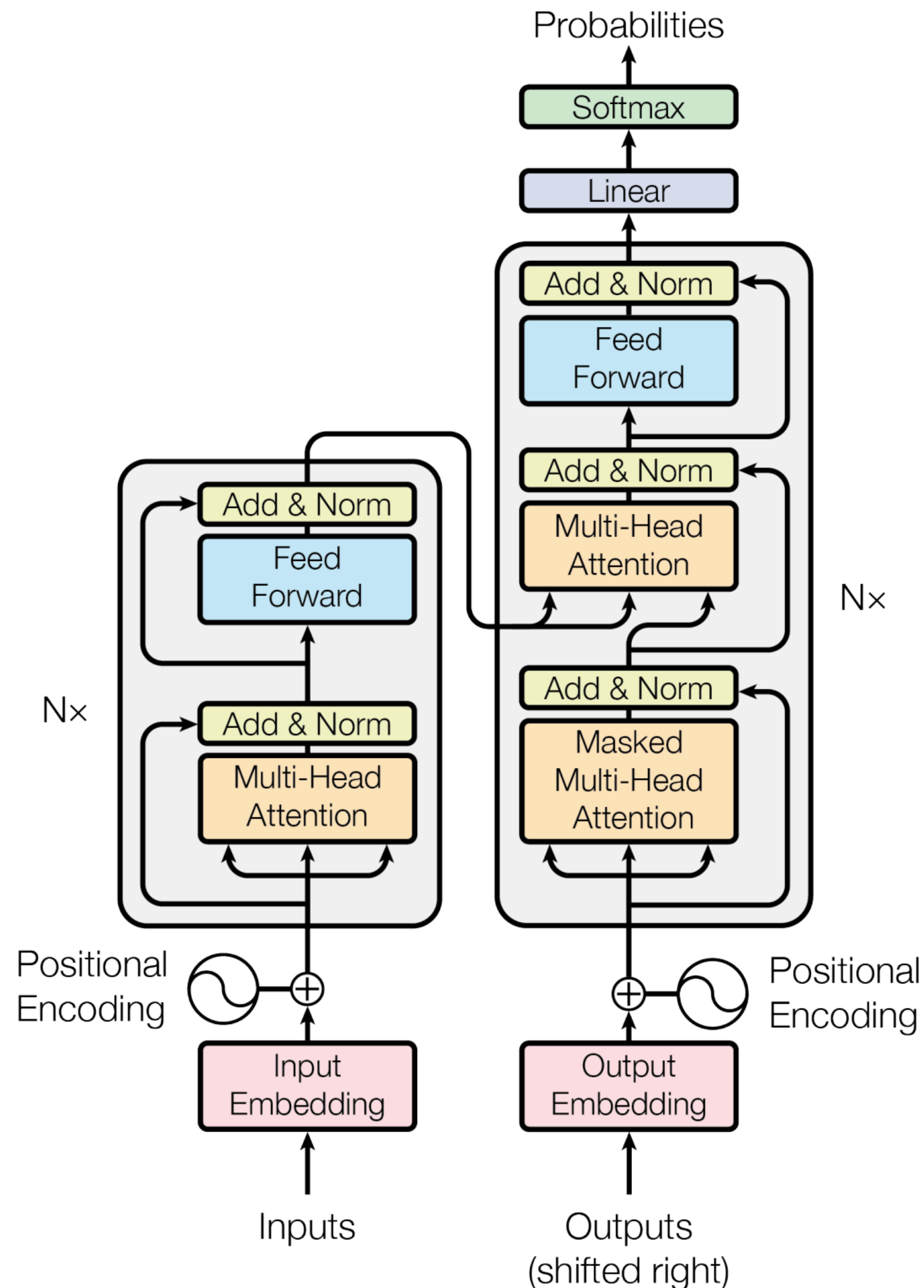


Transformers for MT: Complete Model



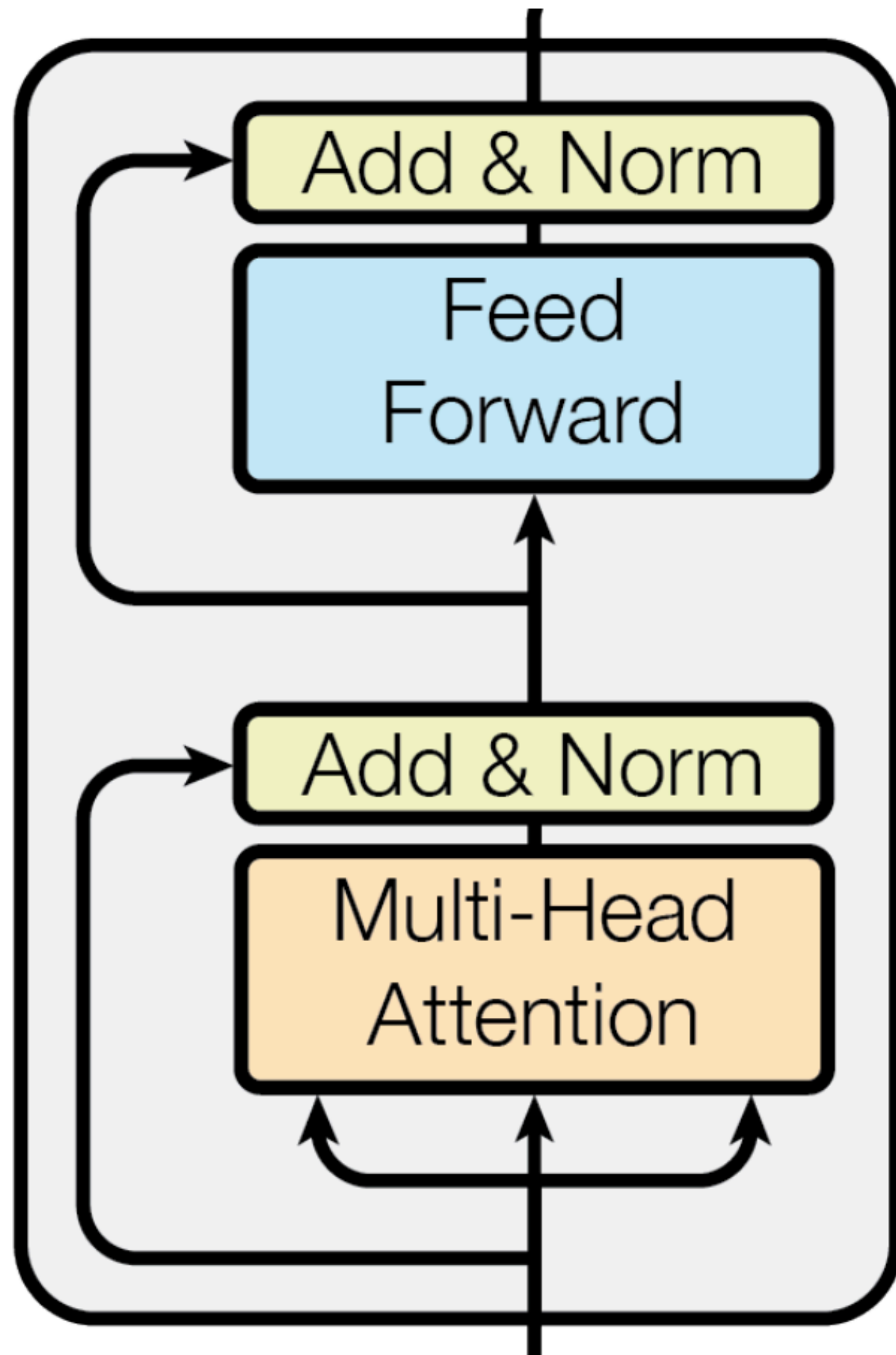
Vaswani et al. (2017)

Transformers for MT: Complete Model



- ▶ Many other details to get it to work: residual connections, layer normalization, positional encoding, optimizer with learning rate schedule, label smoothing

Transformers



- ▶ Alternate multi-head self-attention layers and feedforward layers (w/ ReLU activation) that operate over each word individually
$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$
- ▶ Most of the parameters are in these feedforward layers

Encoder Layer 6

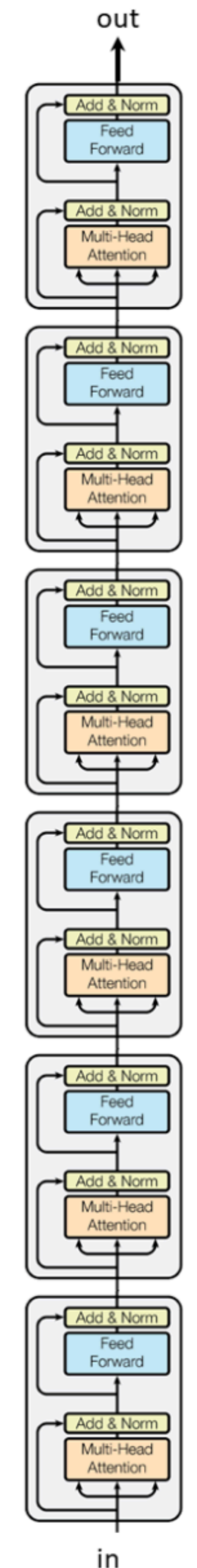
Encoder Layer 5

Encoder Layer 4

Encoder Layer 3

Encoder Layer 2

Encoder Layer 1



Transformers

- ▶ Alternate multi-head self-attention layers and feedforward layers (w/ ReLU activation) that operate over each word individually

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

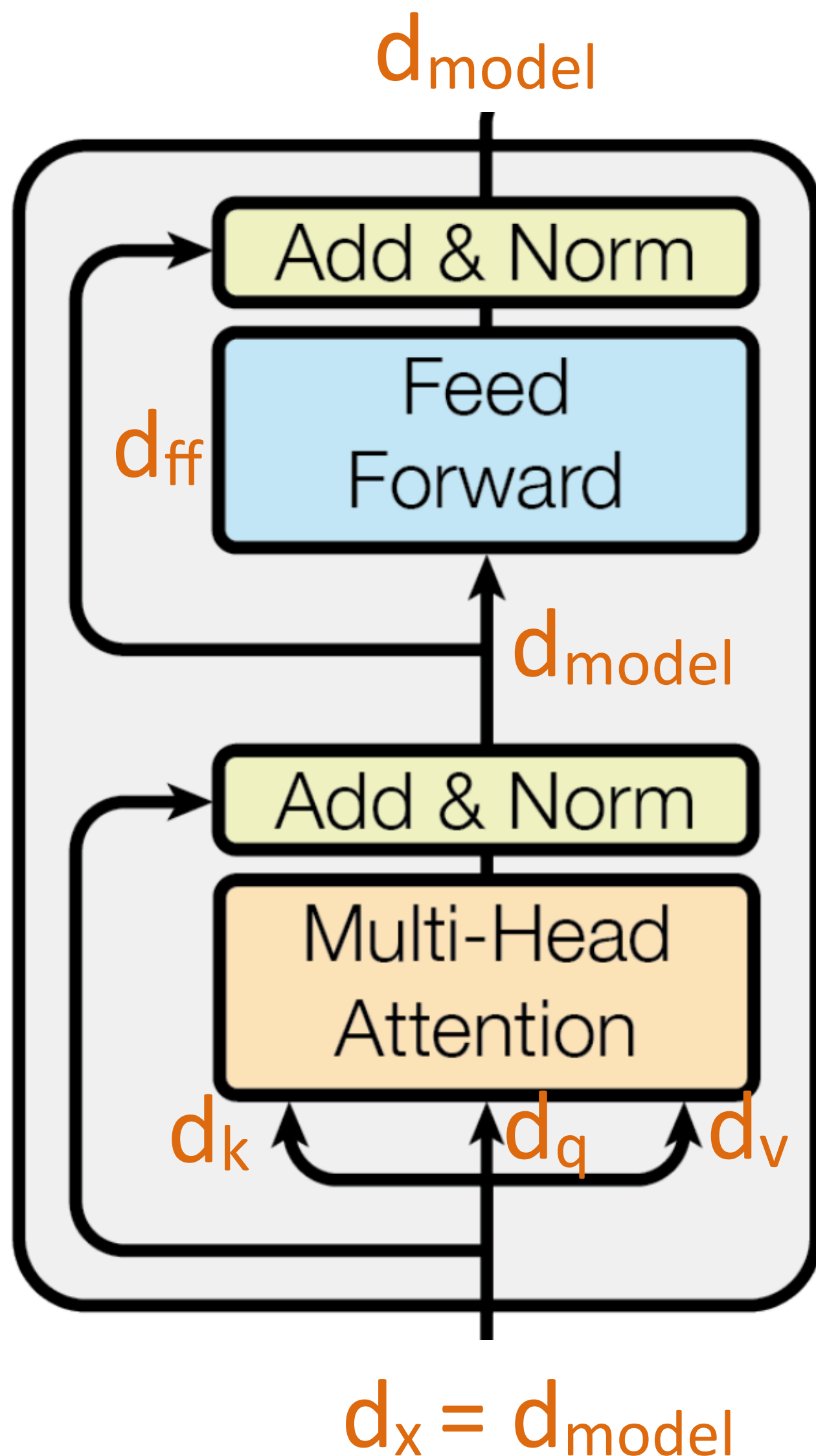
$$h = 8$$

$$d_x = 512$$

$$d_{\text{ff}} = 2048$$

$$d_k = d_q = d_{\text{model}} / h = 64$$

$$d_v = d_{\text{model}} / h = 64$$



$$N = 6$$

Encoder Layer 6

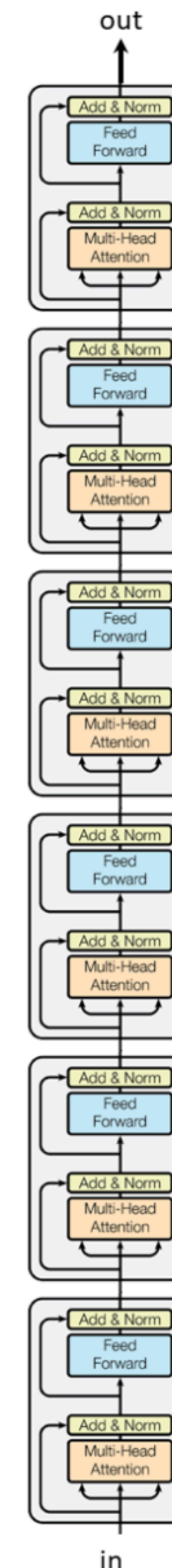
Encoder Layer 5

Encoder Layer 4

Encoder Layer 3

Encoder Layer 2

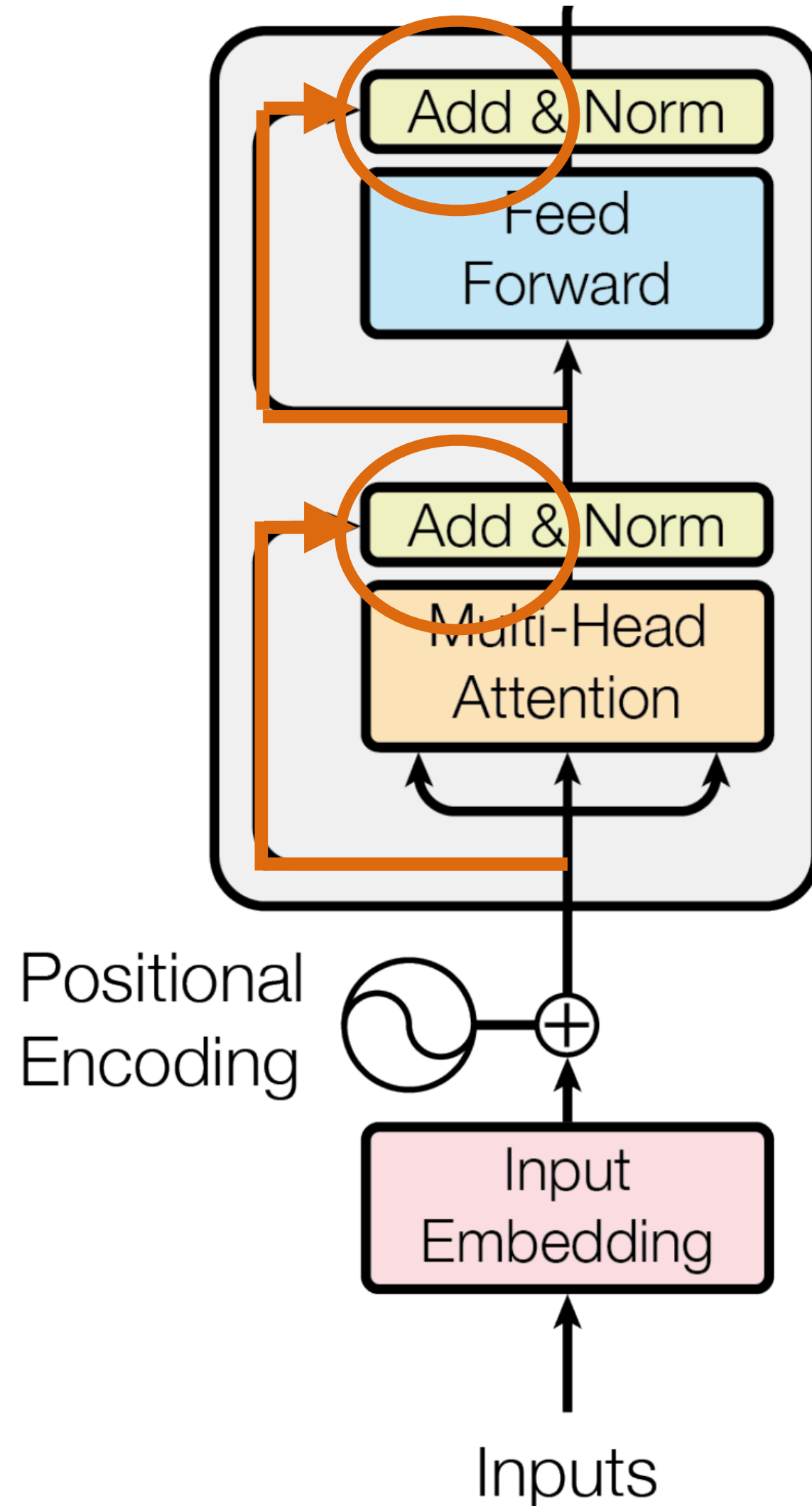
Encoder Layer 1



Vaswani et al. (2017)

Transformers

- ▶ Alternate multi-head self-attention layers and feedforward layers
- ▶ **Residual** connections let the model “skip” each layer — these are particularly useful for training deep networks



Encoder Layer 6

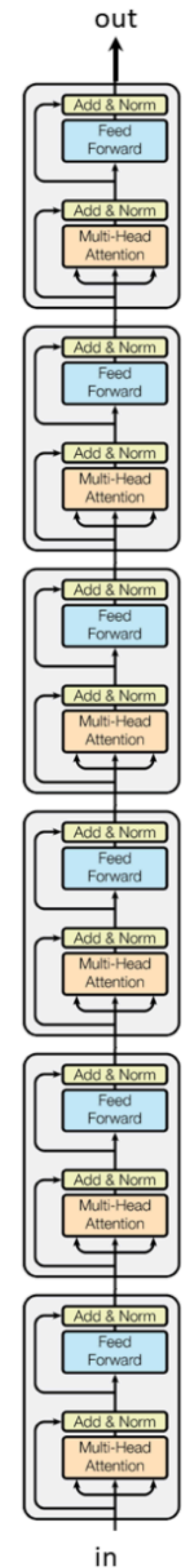
Encoder Layer 5

Encoder Layer 4

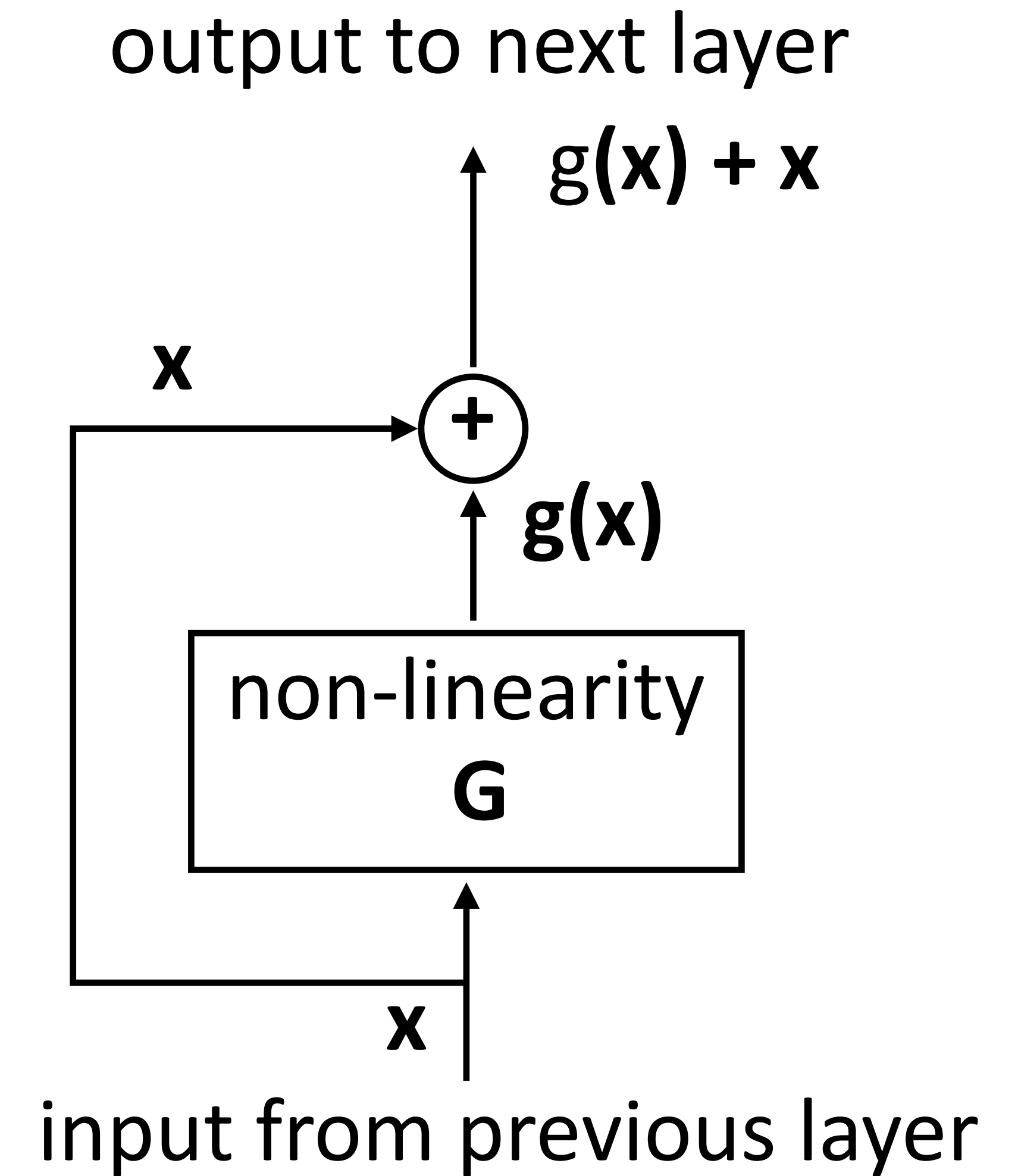
Encoder Layer 3

Encoder Layer 2

Encoder Layer 1



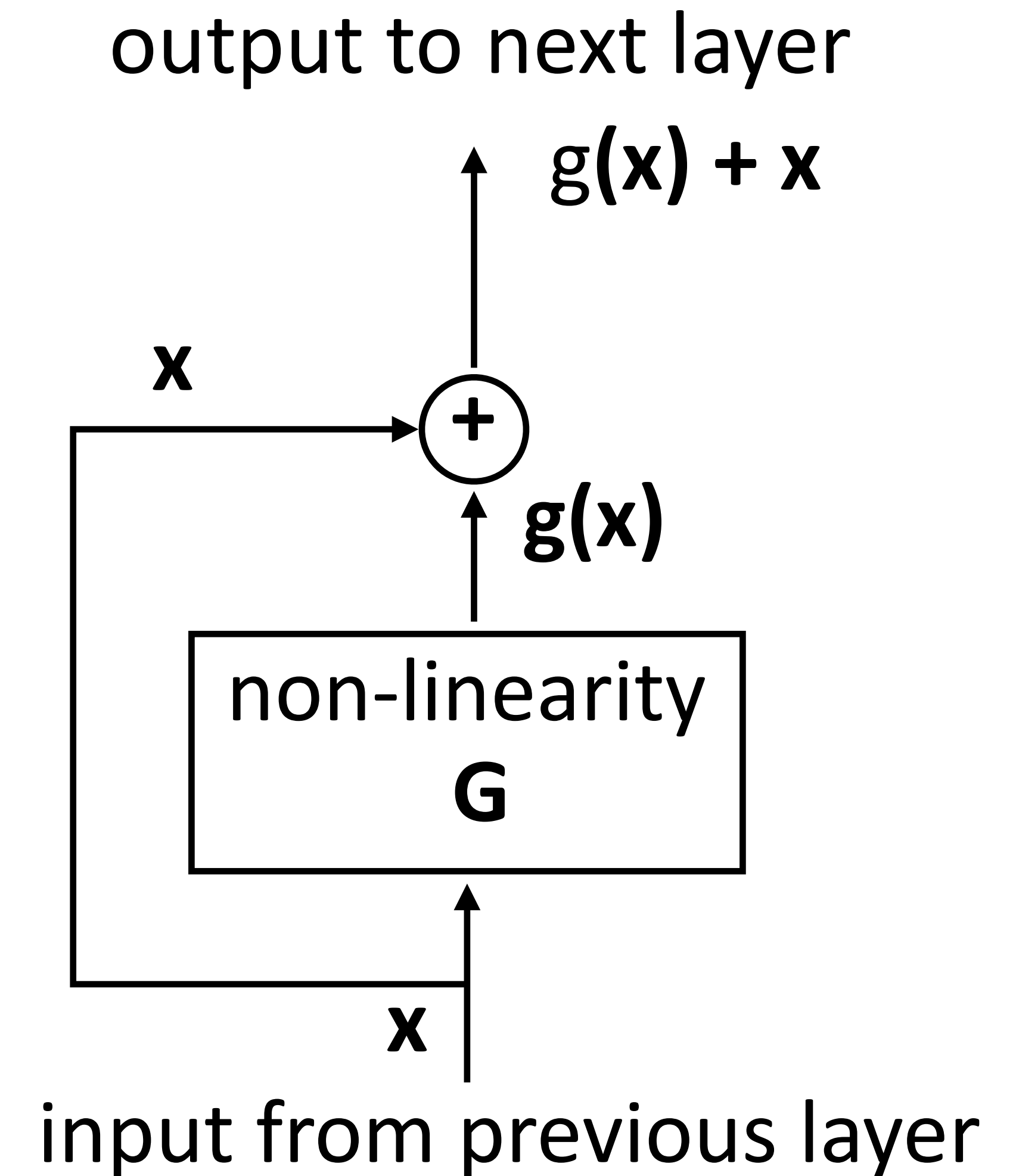
Residual Connections



He et al. (2015)

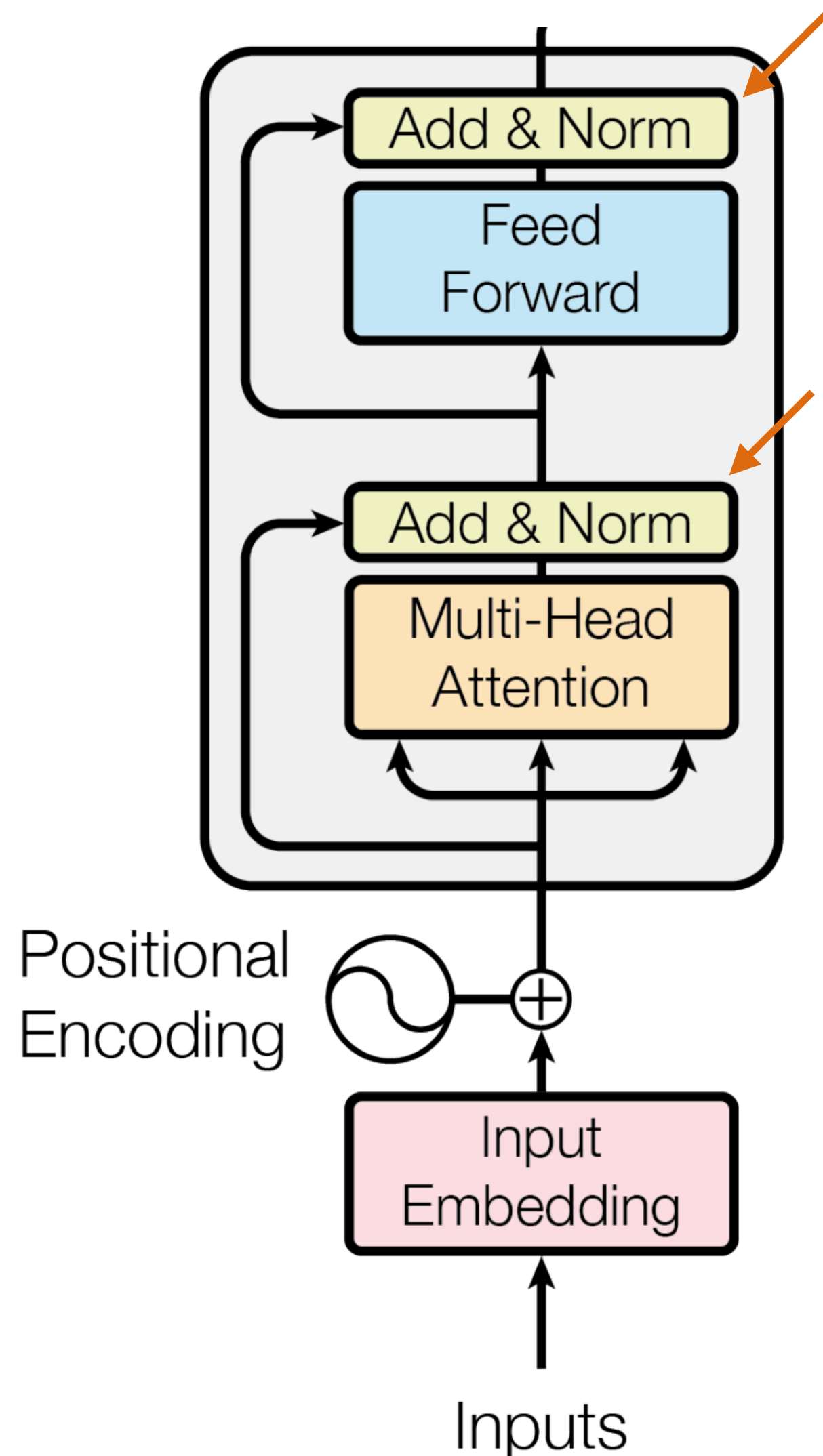
Residual Connections

- ▶ allow gradients to flow through a network directly, without passing through non-linear activation functions



He et al. (2015)

Layer Normalization



Ba et al. (2016)

Ioffe & Szegedy (2015)

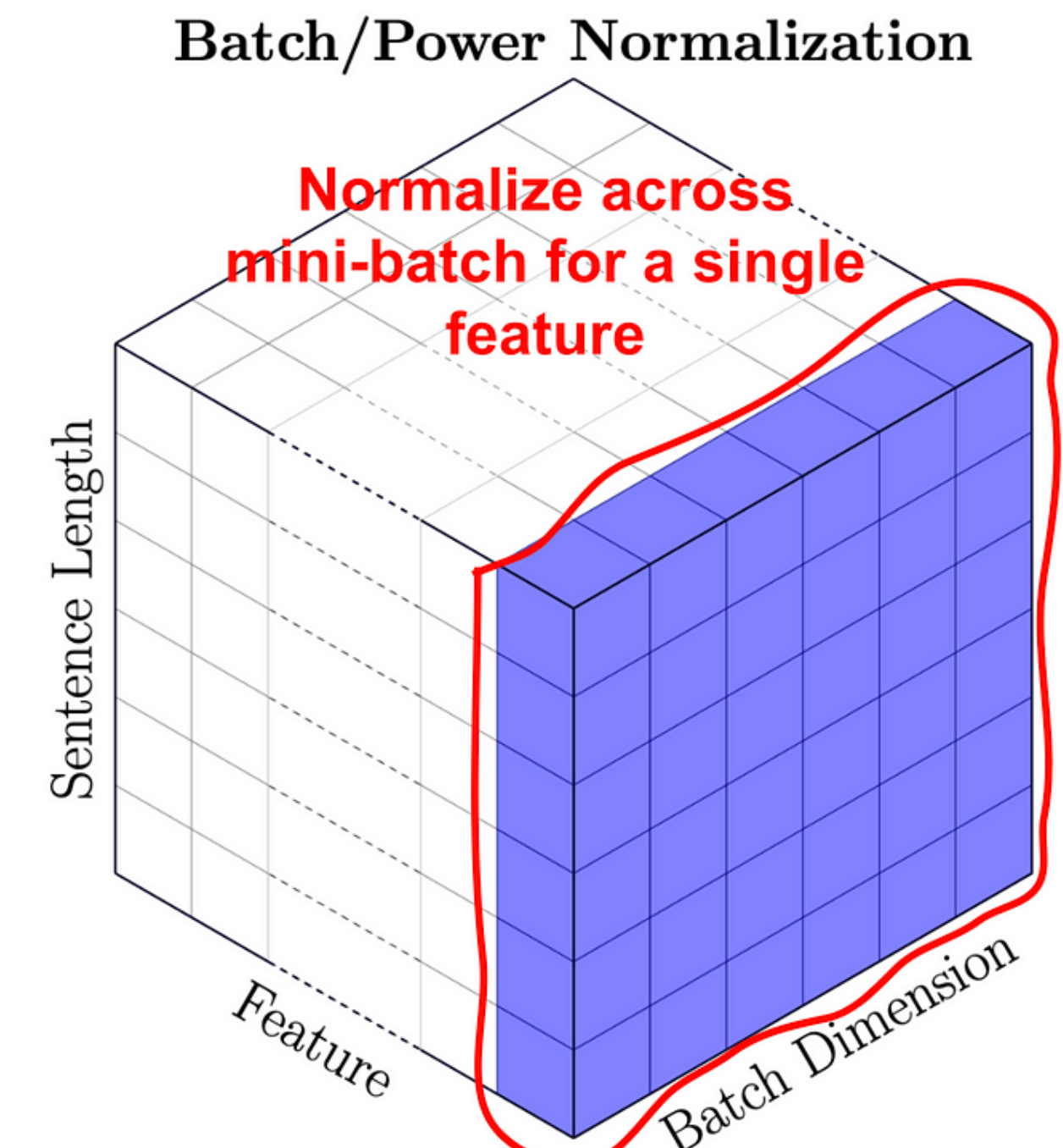
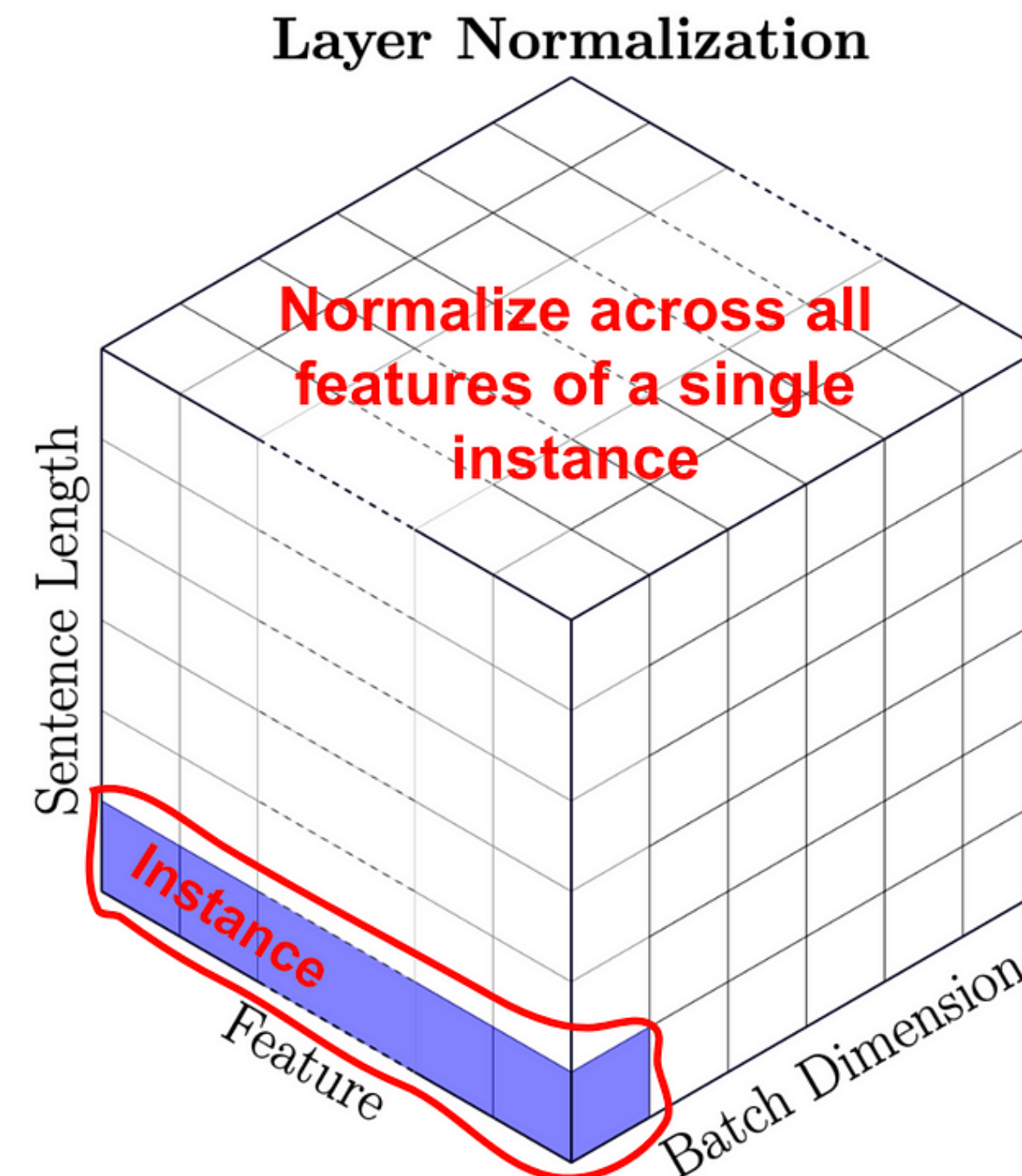
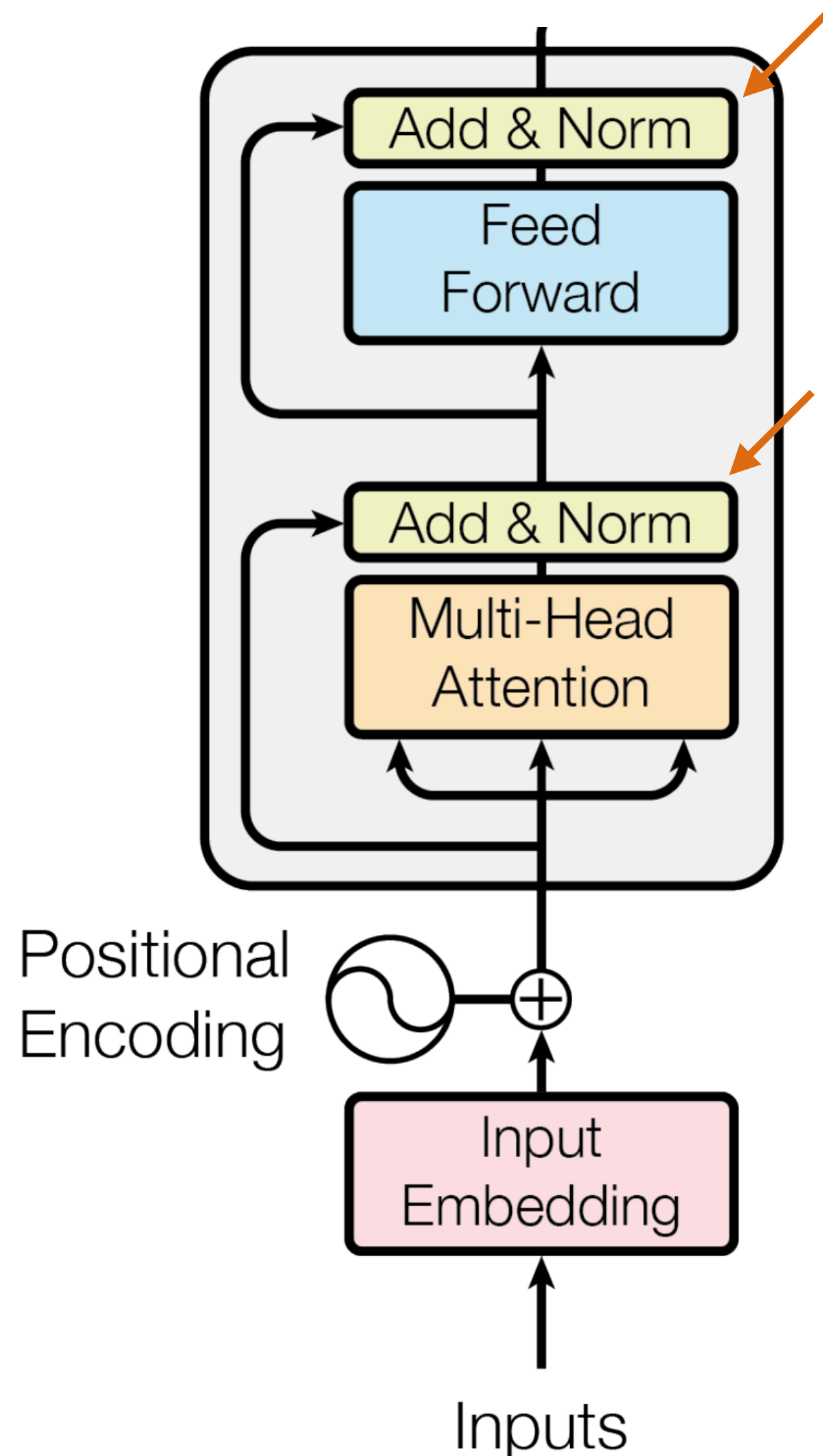


Image Credit: Shen et al. (2020)

Layer Normalization

- ▶ subtract mean, divide by variance



Ba et al. (2016)

Ioffe & Szegedy (2015)

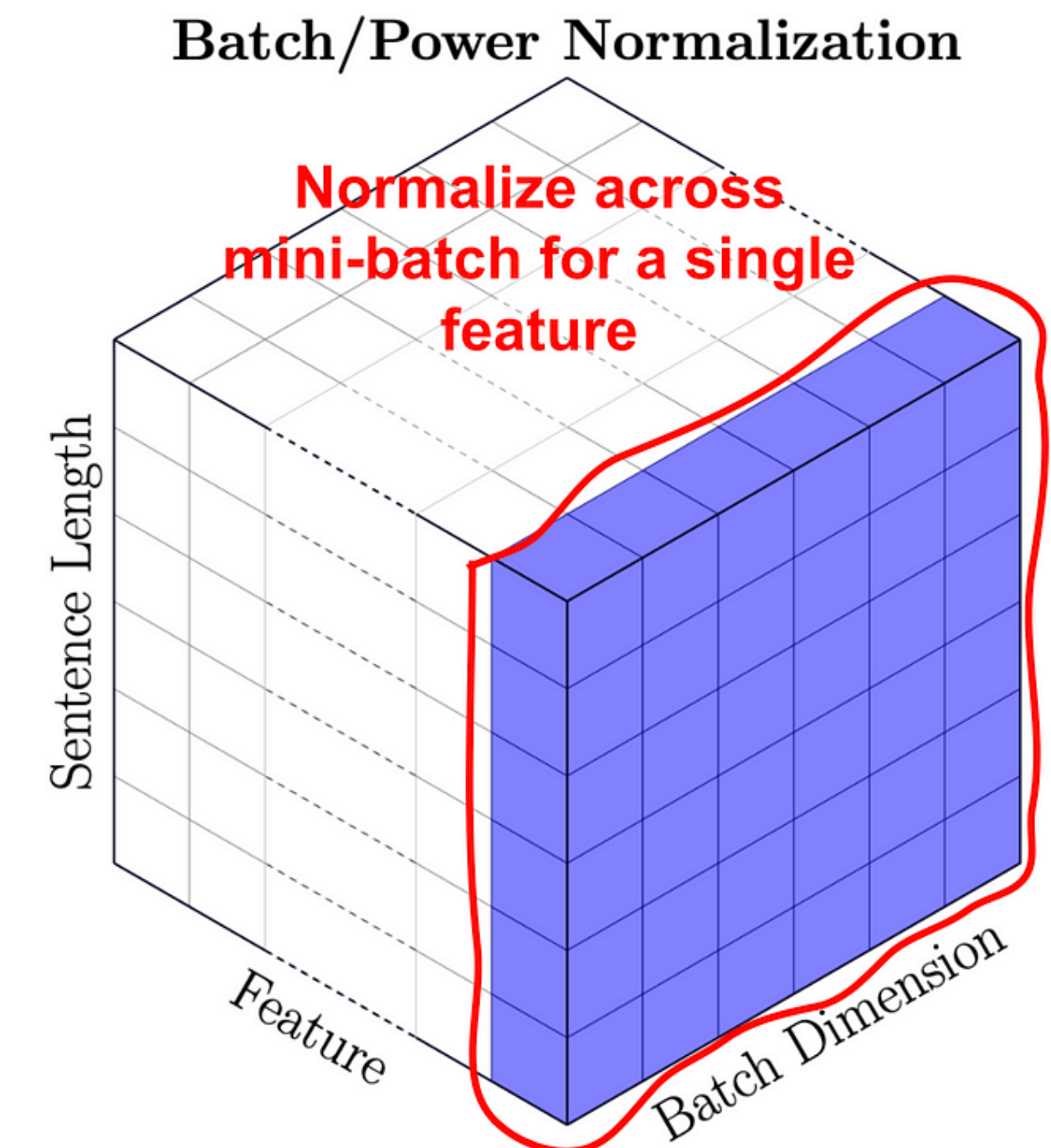
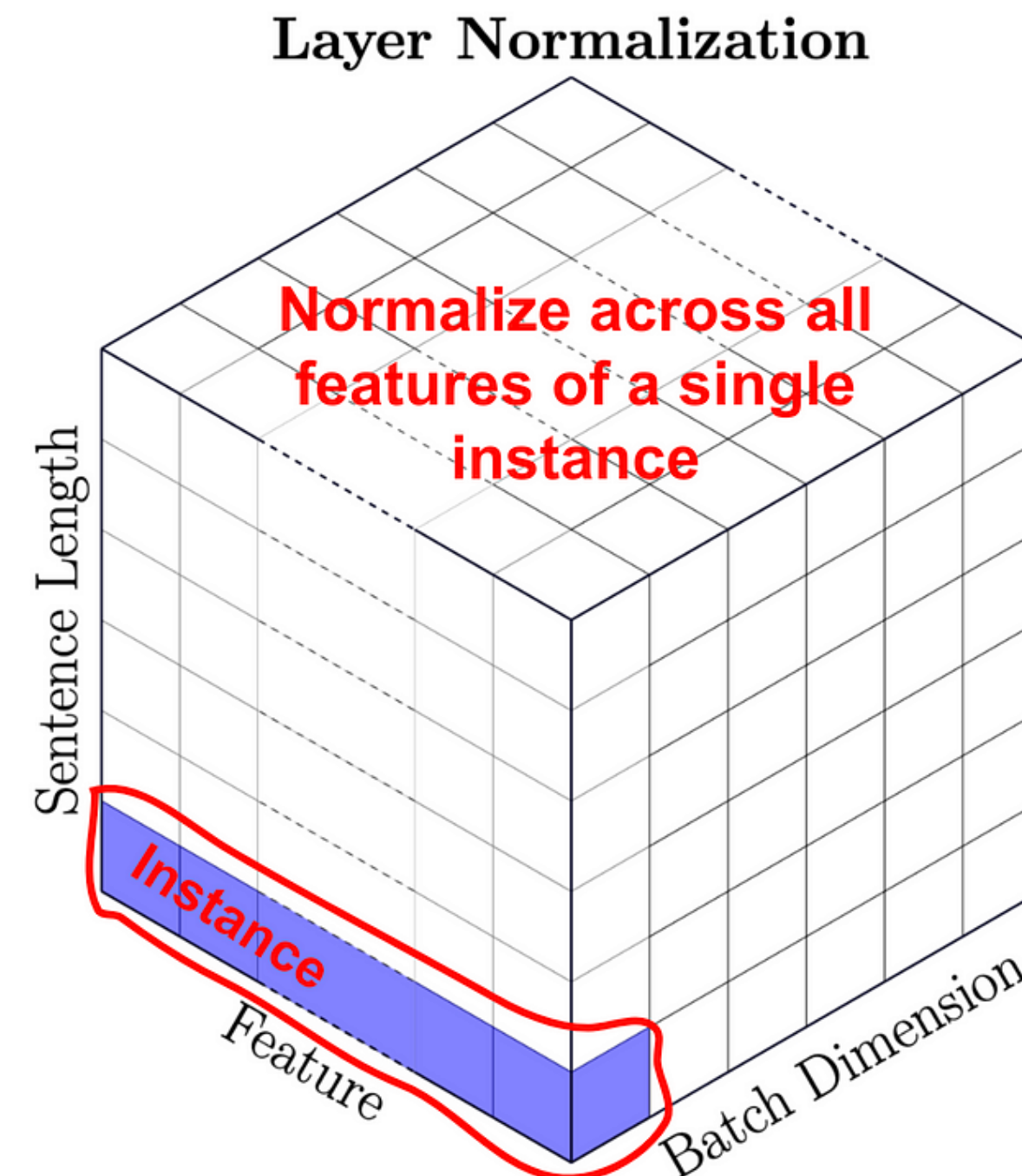
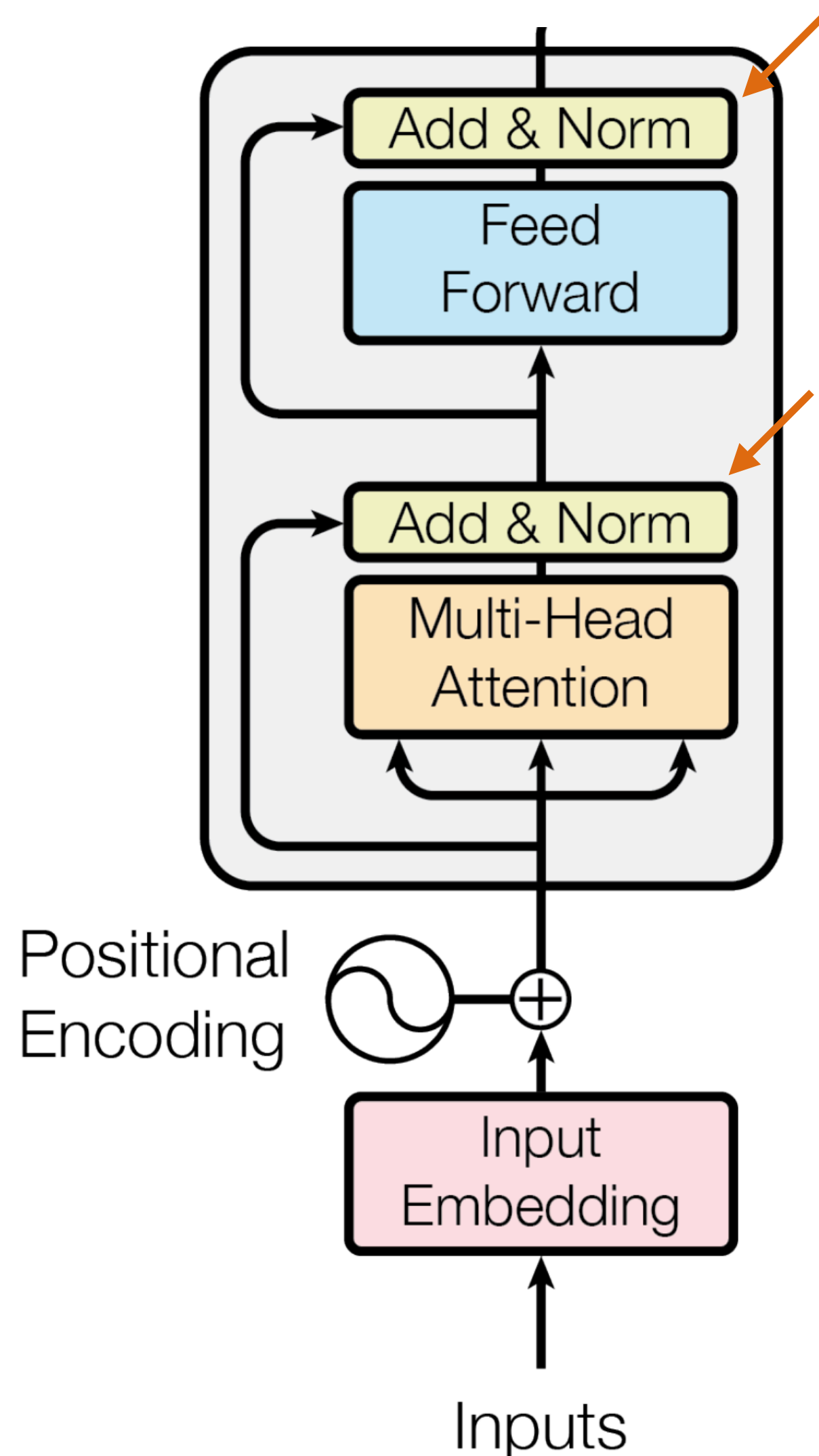


Image Credit: Shen et al. (2020)

Layer Normalization



Ba et al. (2016)

Layer Normalization

1 batch with 3 samples

	r1	r2	r3
c1	2	5	7
c2	6	4	2
c3	8	1	3
c4	9	7	2
c5	3	8	1
Mean	5.6	5	3
Std_dev	2.73	2.45	2.1

Ioffe & Szegedy (2015)

Batch/Power Normalization

1 batch with 3 samples

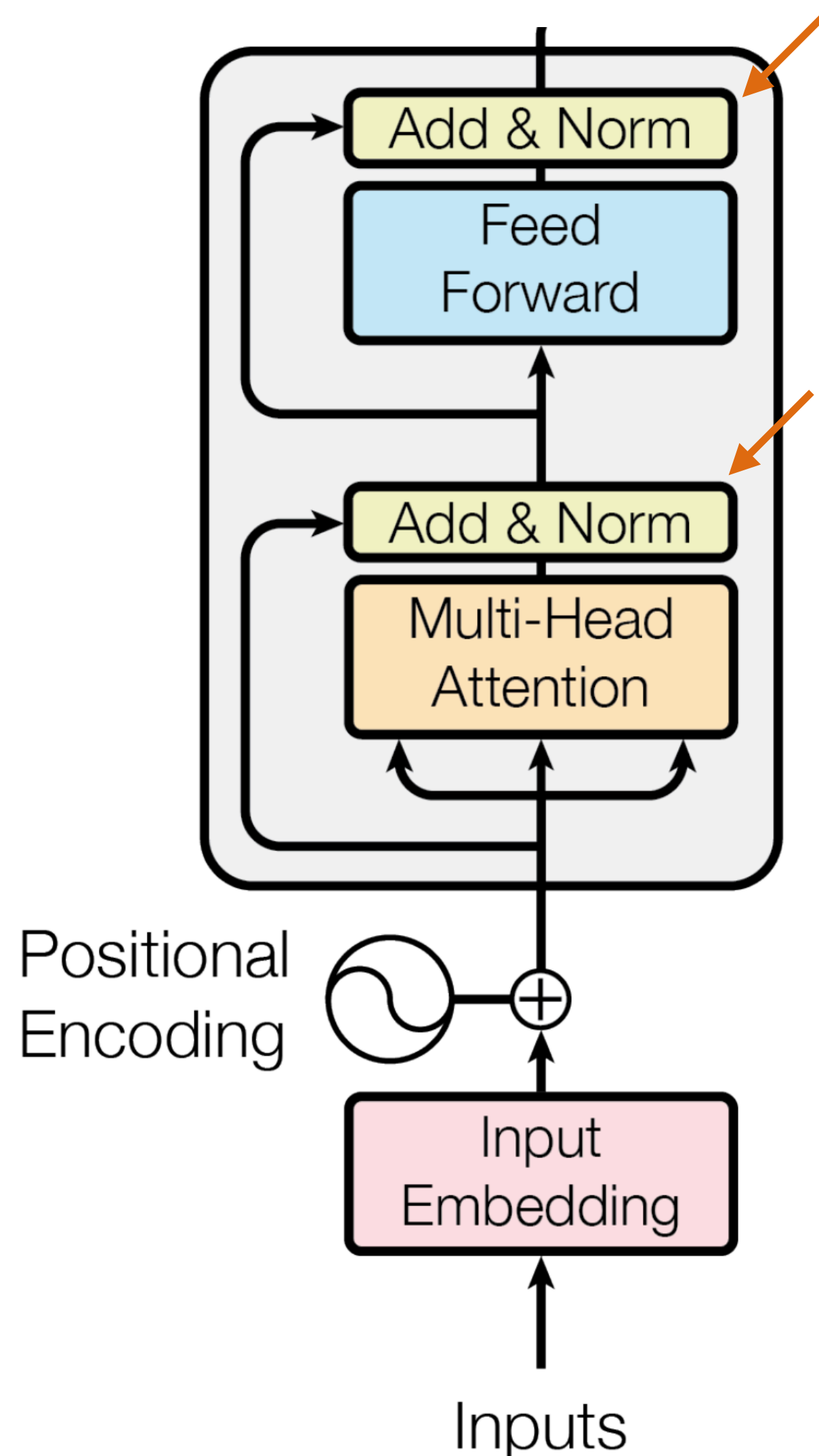
	r1	r2	r3	Mean	Std_dev
c1	2	5	7	4.67	2.05
c2	6	4	2	4	1.36
c3	8	1	3	4	2.95
c4	9	7	2	6	2.95
c5	3	8	1	4	2.95

Layer Normalization

- ▶ LayerNorm normalizes each input independently across its hidden features

Ba et al. (2016)

Ioffe & Szegedy (2015)



Layer Normalization

1 batch with 3 samples

	r1	r2	r3
c1	2	5	7
c2	6	4	2
c3	8	1	3
c4	9	7	2
c5	3	8	1
Mean	5.6	5	3
Std_dev	2.73	2.45	2.1

Batch/Power Normalization

1 batch with 3 samples

	r1	r2	r3	Mean	Std_dev
c1	2	5	7	4.67	2.05
c2	6	4	2	4	1.36
c3	8	1	3	4	2.95
c4	9	7	2	6	2.95
c5	3	8	1	4	2.95

Layer Normalization

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \text{mean}(\mathbf{x})}{\sqrt{\text{variance}(\mathbf{x}) + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}$$


$$\text{mean}(\hat{\mathbf{x}}) \approx 0, \quad \text{var}(\hat{\mathbf{x}}) \approx 1$$

- ▶ For a vector $\mathbf{X} = (x_1, x_2, \dots, x_d)$
- ▶ Learnable parameters $\boldsymbol{\gamma}$, $\boldsymbol{\beta}$ to scale/shift with some flexibility (can be different from layer to layer)

Transformers: Position Sensitivity

*The ballerina is very excited that **she** will dance in the **show**.*

The diagram illustrates the concept of position sensitivity in Transformers. It shows the sentence "The ballerina is very excited that she will dance in the show." with the words "she" and "show" highlighted in blue and red respectively. A blue arc connects "show" to "she", indicating a long-distance dependency. A red arc connects "show" to "The", indicating a short-distance dependency. This visualizes how the model must learn to attend to words based on their relative positions, not just their semantic meaning.

- ▶ If this is in a longer context, we want words to attend *locally*

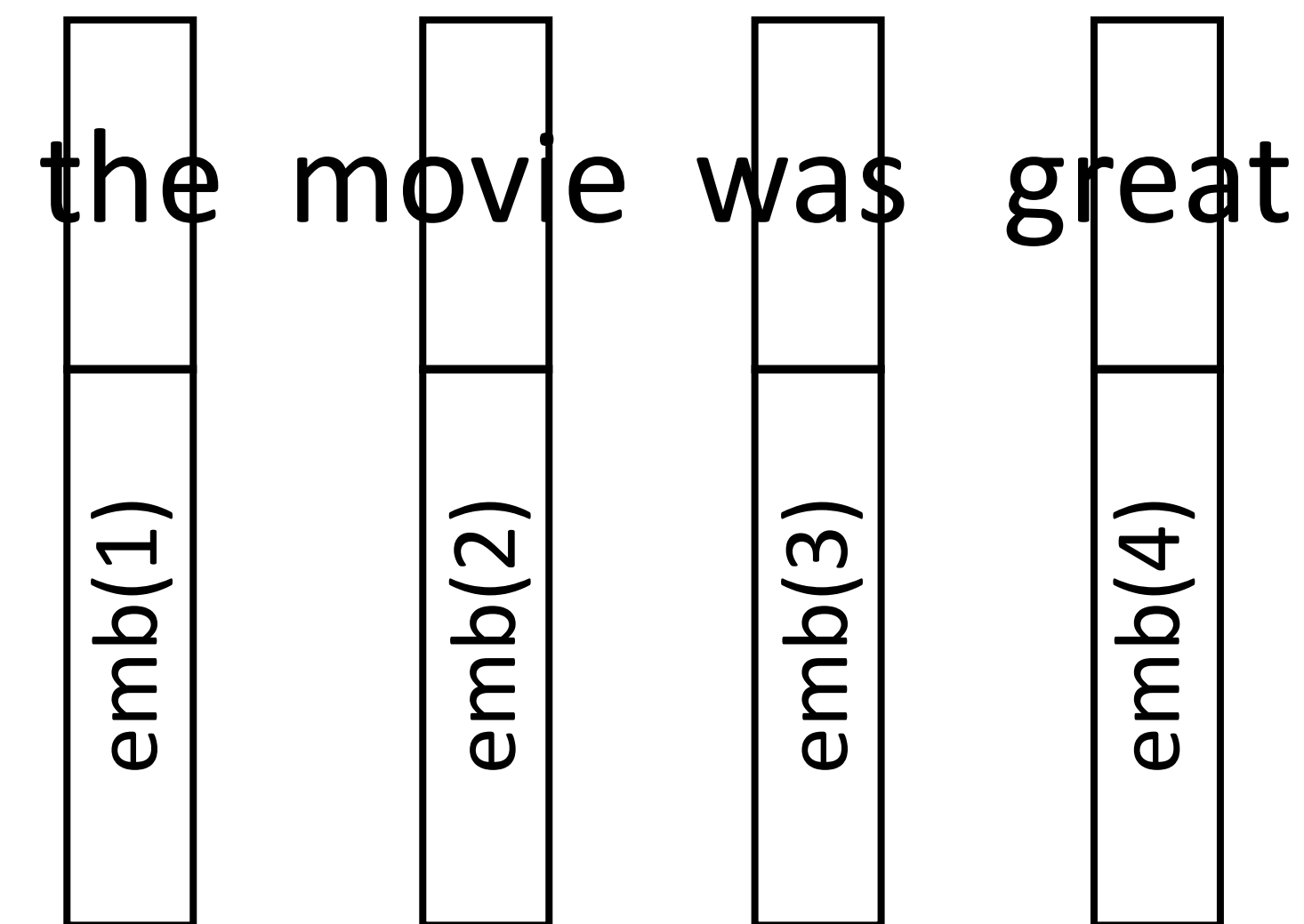
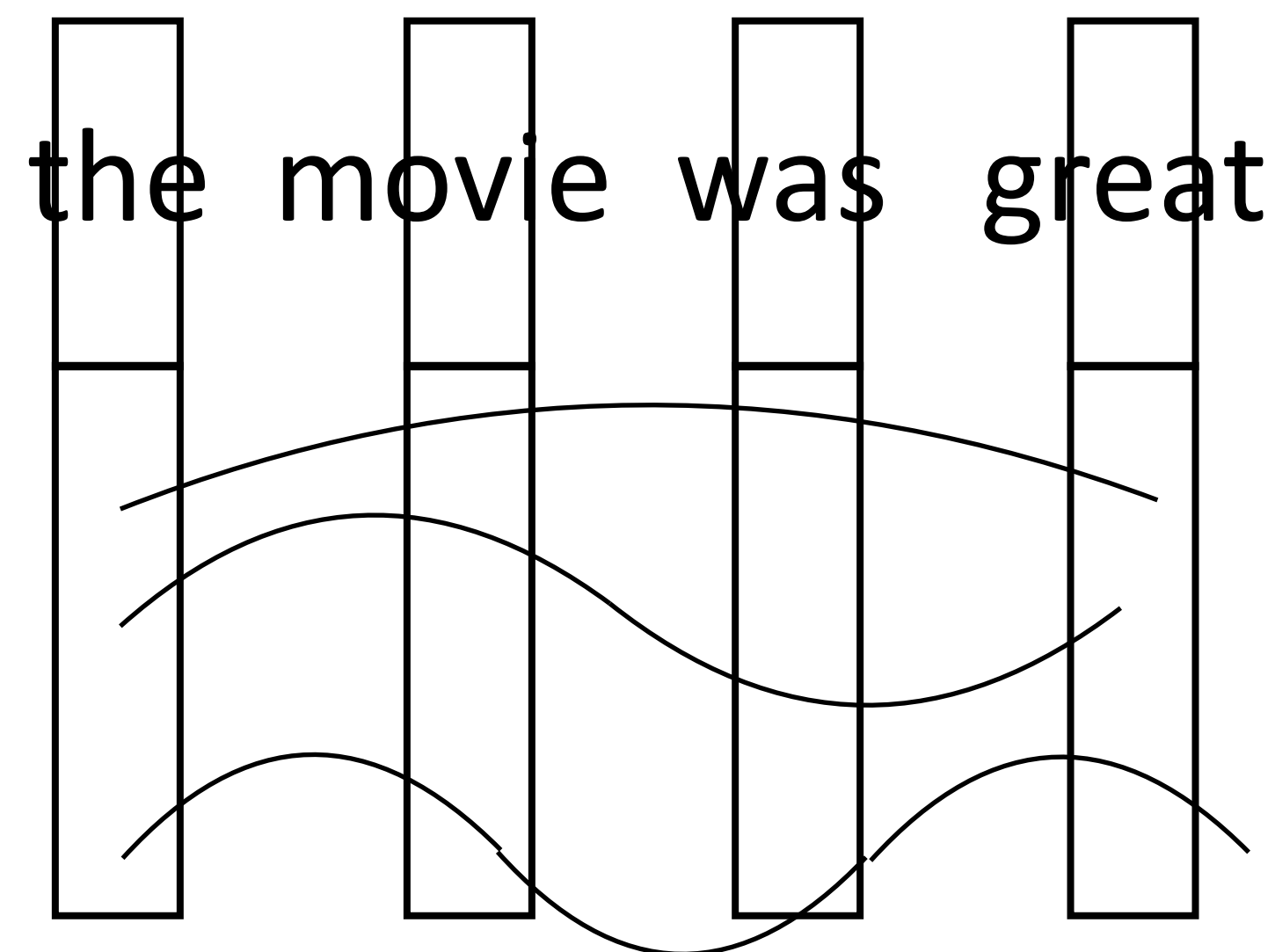
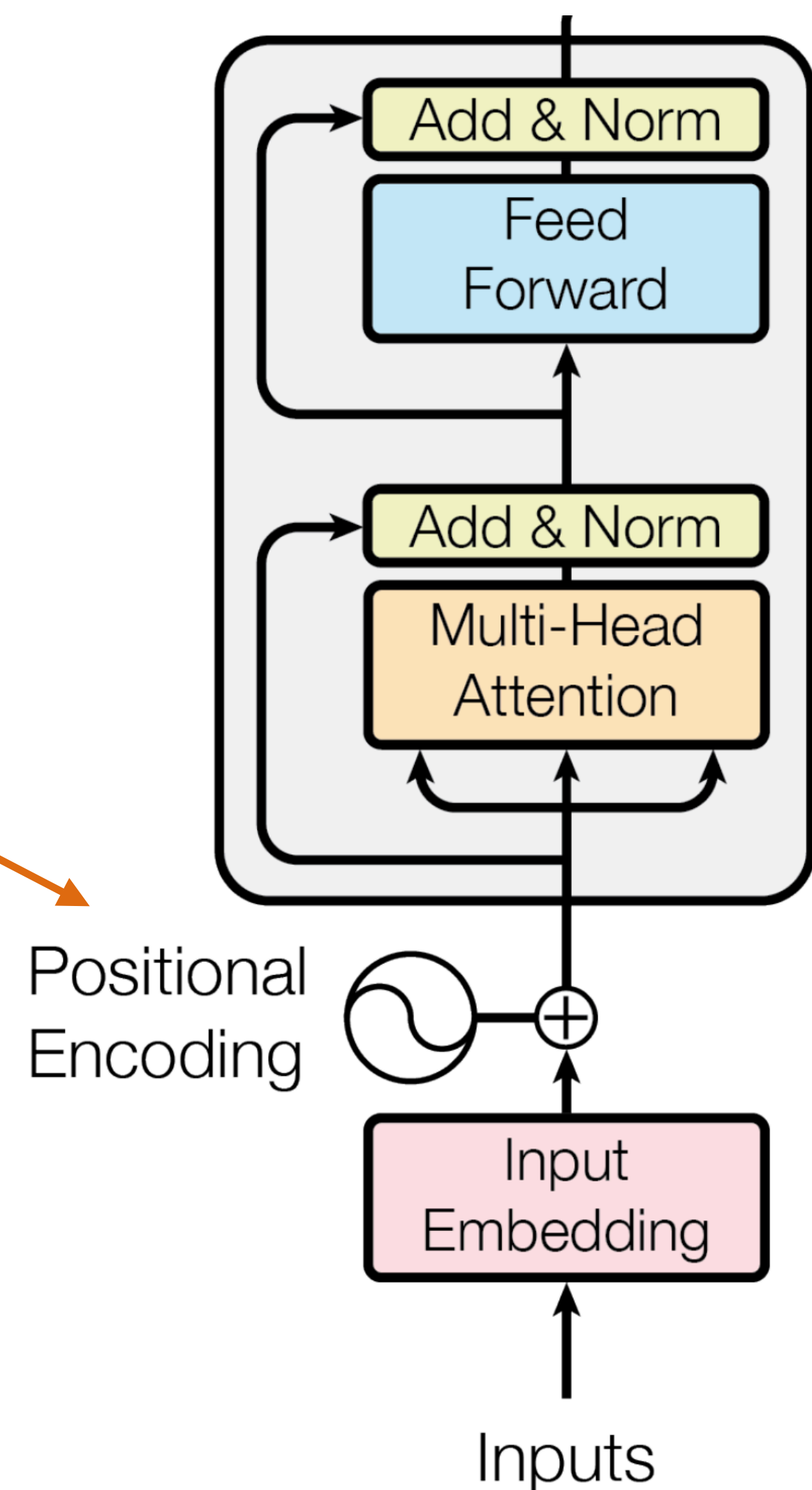
Transformers: Position Sensitivity

*The ballerina is very excited that **she** will dance in the **show**.*

A diagram illustrating attention weights for the word "show". A blue arc connects the word "show" to the word "she", indicating a long-distance dependency. A red arc connects the word "show" to the word "The", indicating a short-distance dependency.

- ▶ If this is in a longer context, we want words to attend *locally*
- ▶ But transformers have *no notion of position* by default

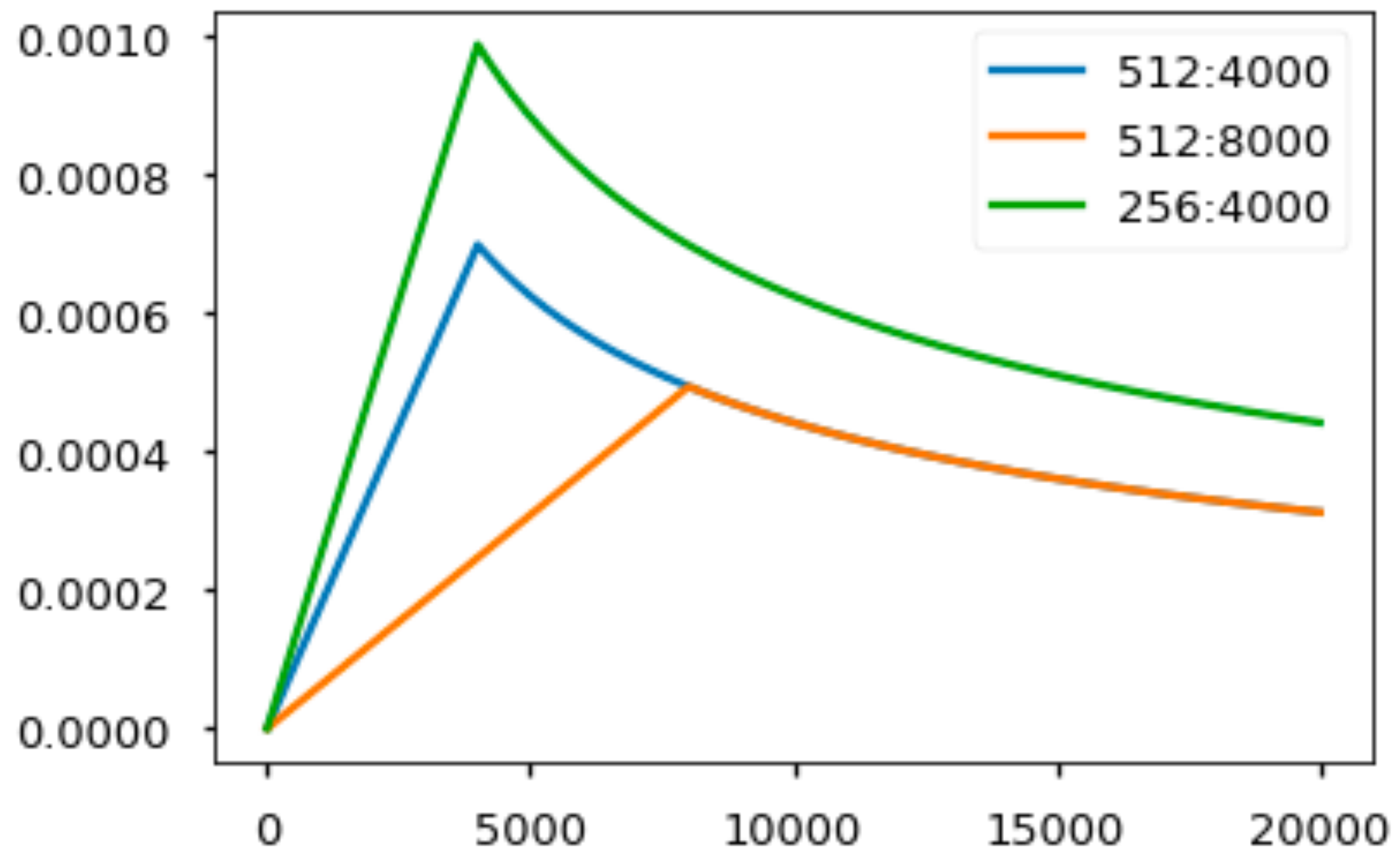
Transformers



- ▶ Augment word embedding with position embeddings, each dim is a sine/cosine wave of a different frequency. Closer points = higher dot products
- ▶ Works essentially as well as just encoding position as a one-hot vector

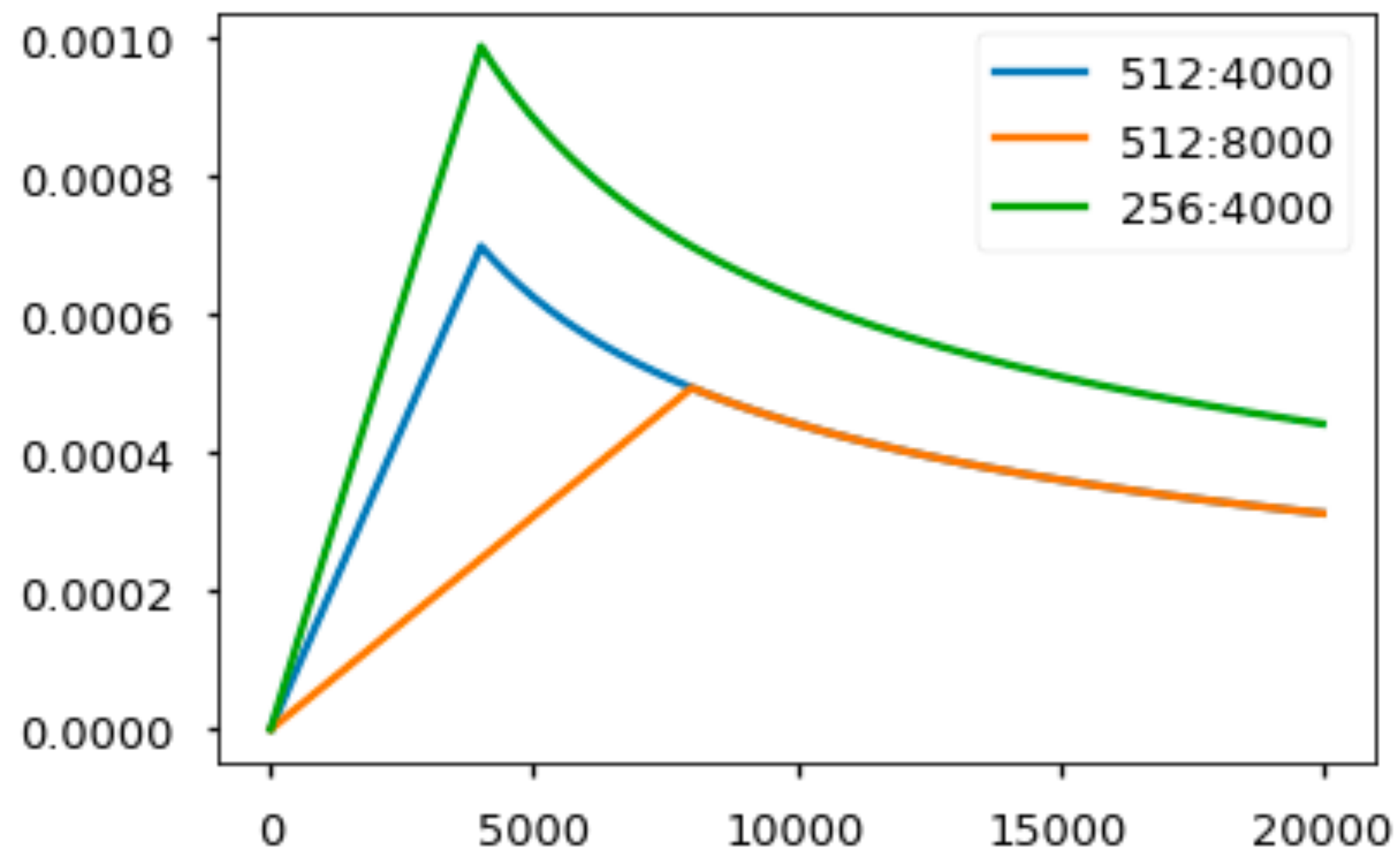
Vaswani et al. (2017)

Transformers

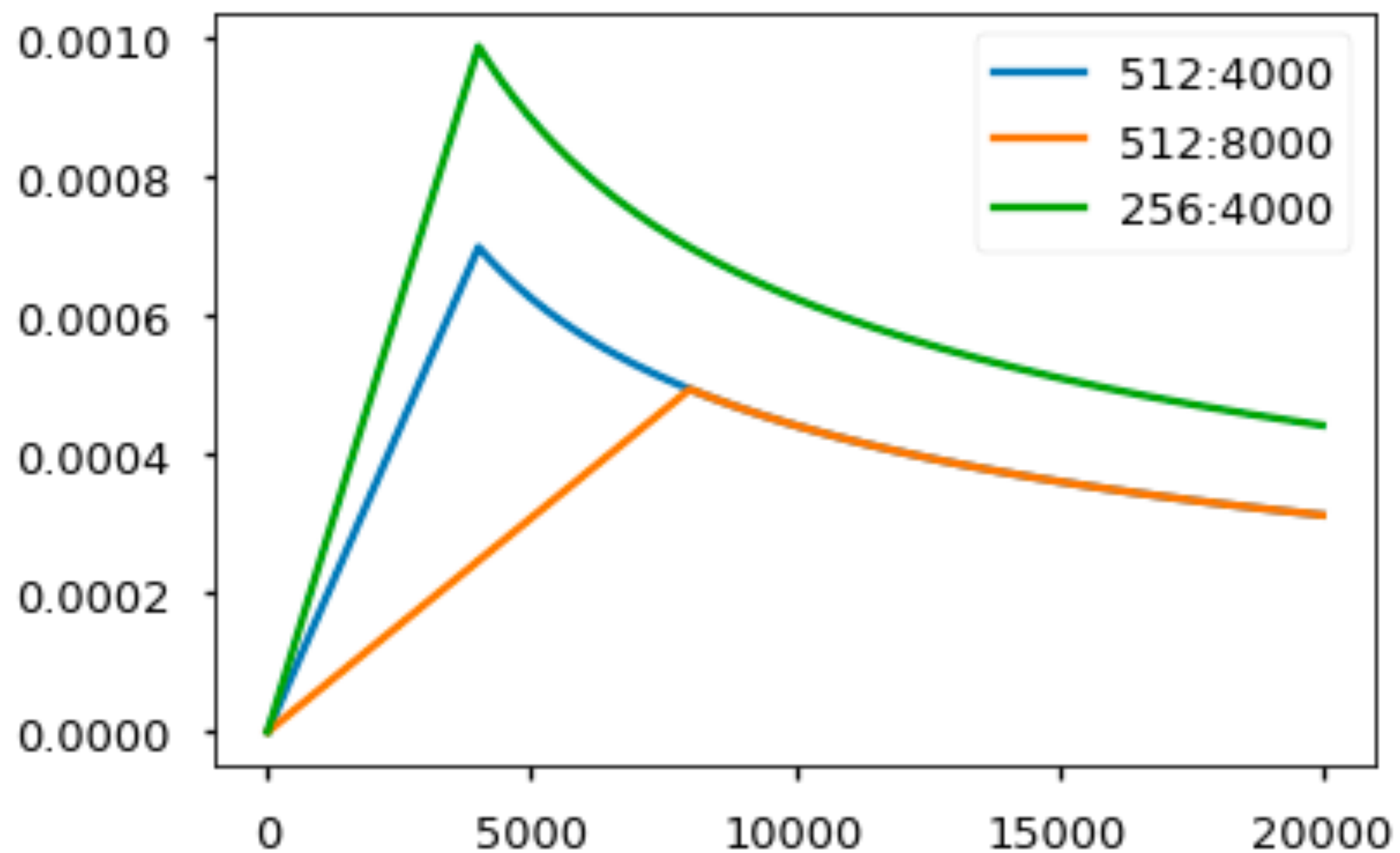


Transformers

- ▶ **Adam** optimizer with varied learning rate over the course of training

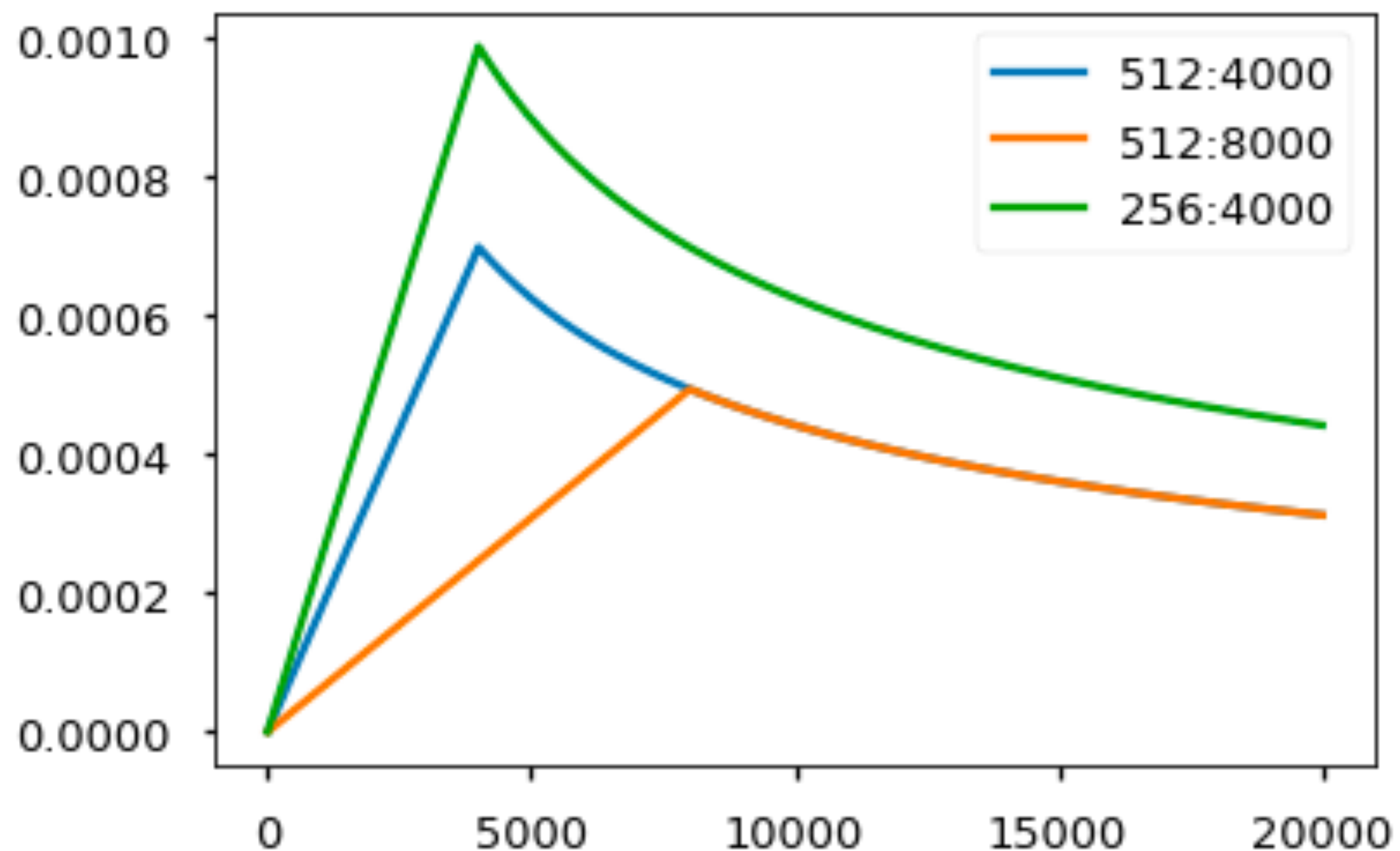


Transformers



- ▶ **Adam** optimizer with varied learning rate over the course of training
- ▶ Linearly increase for warmup, then decay proportionally to the inverse square root of the step number

Transformers



- ▶ **Adam** optimizer with varied learning rate over the course of training
- ▶ Linearly increase for warmup, then decay proportionally to the inverse square root of the step number
- ▶ This part is very important!

Transformer Results

Model	BLEU	
	EN-DE	EN-FR
ByteNet [18]	23.75	
Deep-Att + PosUnk [39]		39.2
GNMT + RL [38]	24.6	39.92
ConvS2S [9]	25.16	40.46
MoE [32]	26.03	40.56
Deep-Att + PosUnk Ensemble [39]		40.4
GNMT + RL Ensemble [38]	26.30	41.16
ConvS2S Ensemble [9]	26.36	41.29
Transformer (base model)	27.3	38.1
Transformer (big)	28.4	41.8

- big = 6 layers, 1000 dim for each token, 16 heads,
base = 6 layers + other params halved

Useful Resources

nn.Transformer:

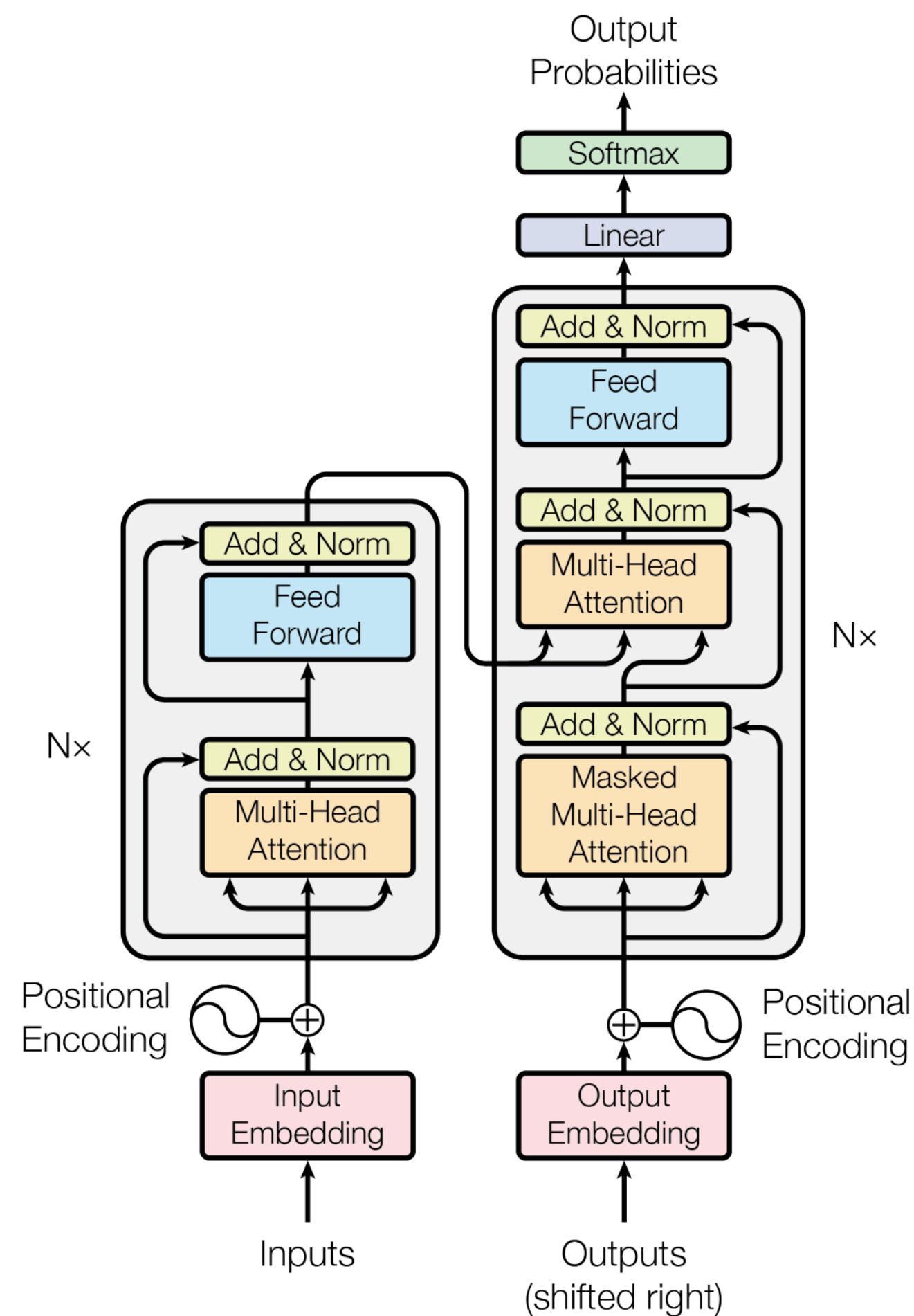
```
>>> transformer_model = nn.Transformer(nhead=16, num_encoder_layers=12)
>>> src = torch.rand((10, 32, 512))
>>> tgt = torch.rand((20, 32, 512))
>>> out = transformer_model(src, tgt)
```

nn.TransformerEncoder:

```
>>> encoder_layer = nn.TransformerEncoderLayer(d_model=512, nhead=8)
>>> transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=6)
>>> src = torch.rand(10, 32, 512)
>>> out = transformer_encoder(src)
```

Summary: Transformer Uses

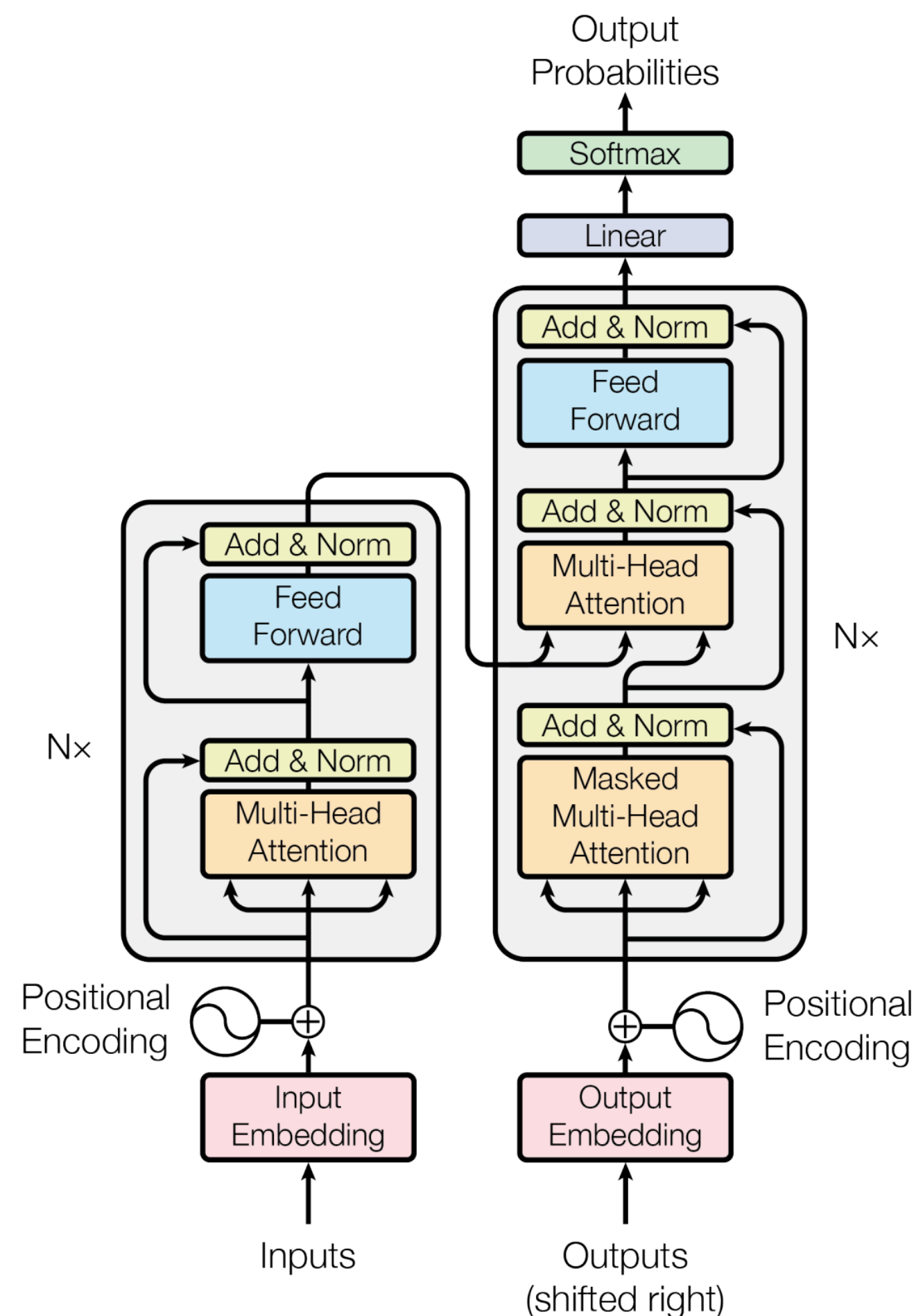
- Supervised: transformer can replace LSTM as encoder, decoder, or both;



Vaswani et al. (2017)

Summary: Transformer Uses

- Supervised: transformer can replace LSTM as encoder, decoder, or both; such as in machine translation and natural language generation tasks.

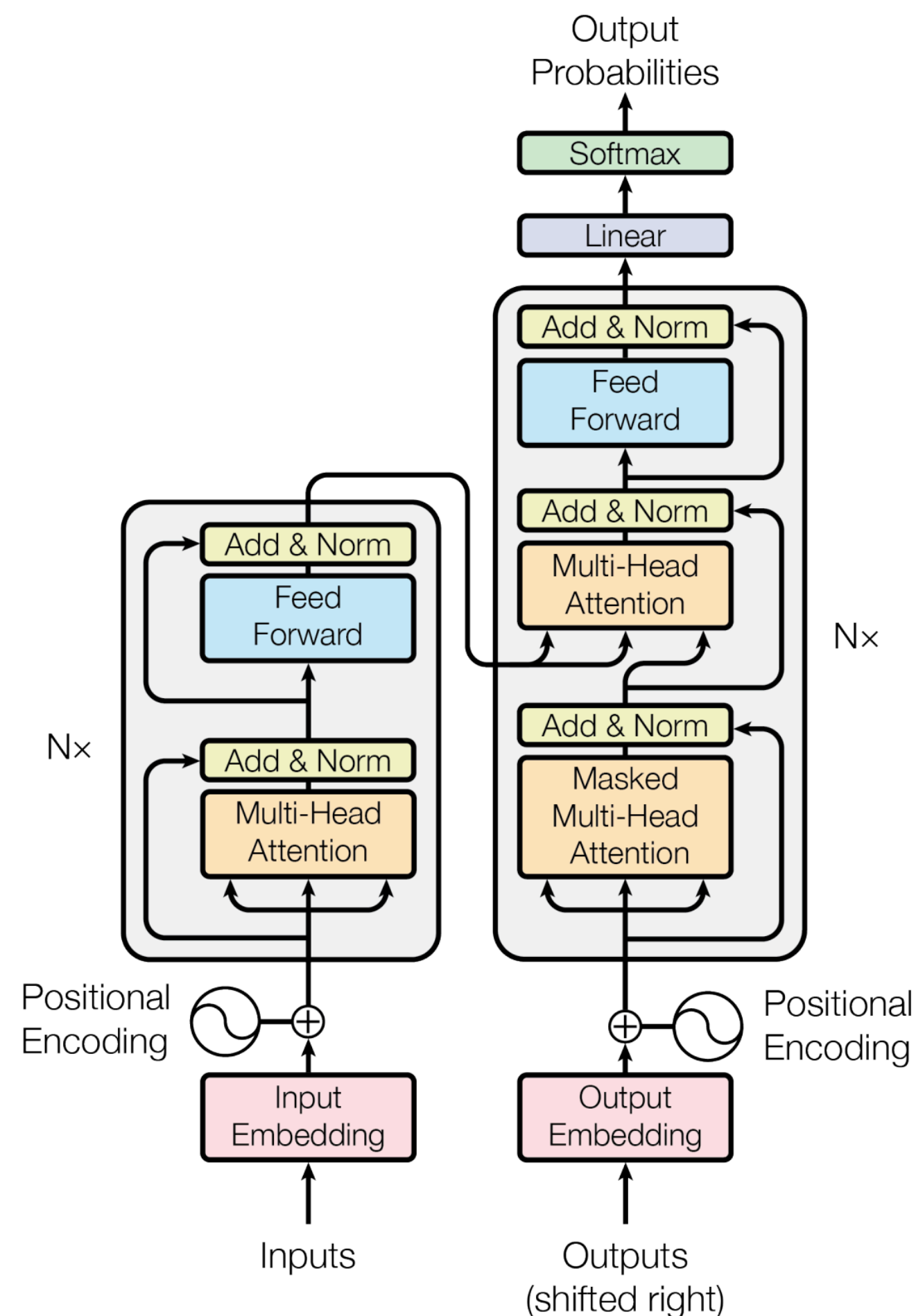


Vaswani et al. (2017)

Summary: Transformer Uses

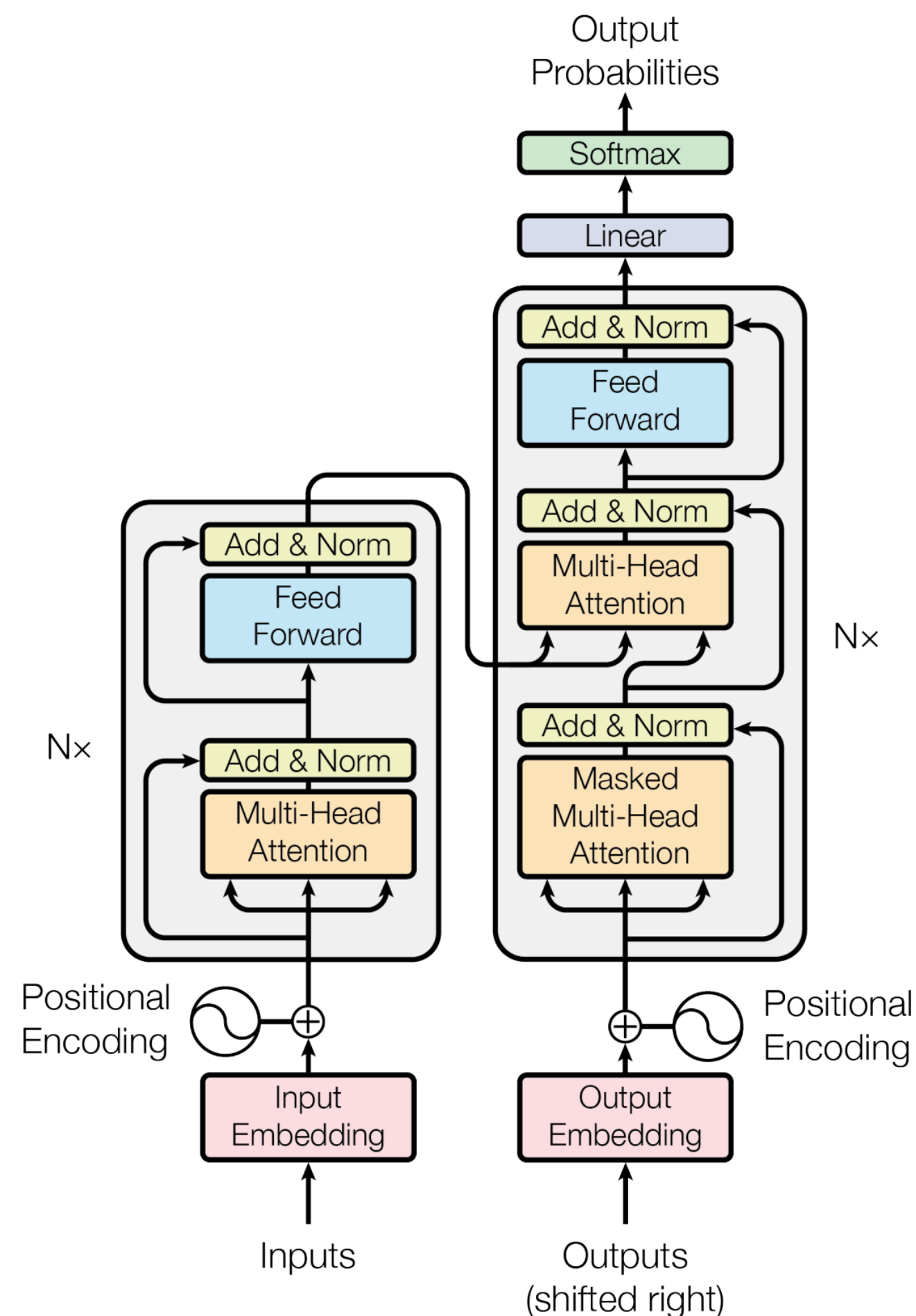
- Supervised: transformer can replace LSTM as encoder, decoder, or both; such as in machine translation and natural language generation tasks.

- Encoder and decoder are both transformers



Summary: Transformer Uses

- Supervised: transformer can replace LSTM as encoder, decoder, or both; such as in machine translation and natural language generation tasks.



- Encoder and decoder are both transformers
- Decoder consumes the previous generated token (and attends to input), but has *no recurrent state*

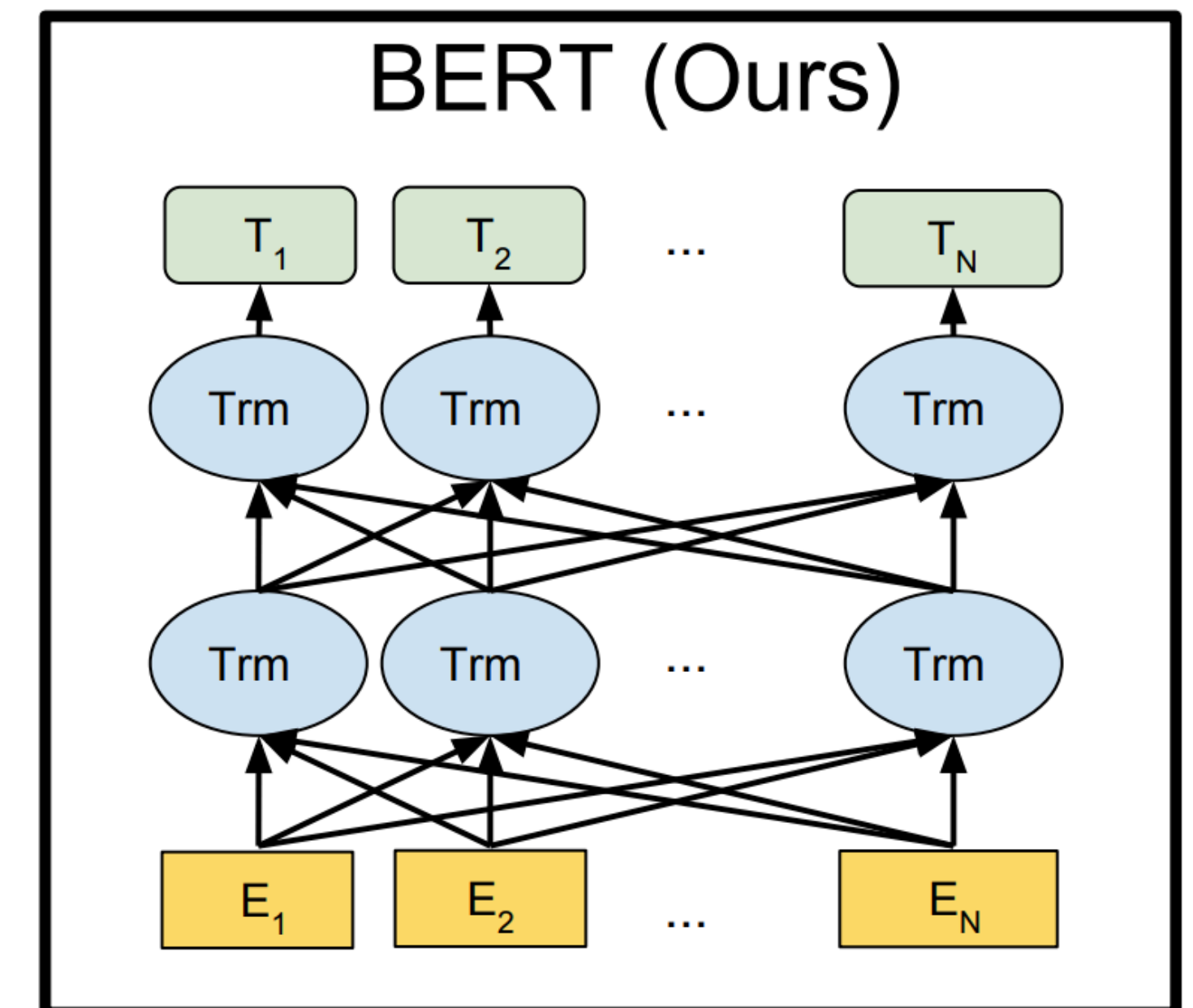
Summary: Transformer Uses

Summary: Transformer Uses

- ▶ Unsupervised: transformers work better than LSTM for unsupervised pre-training of embeddings — predict word given context words

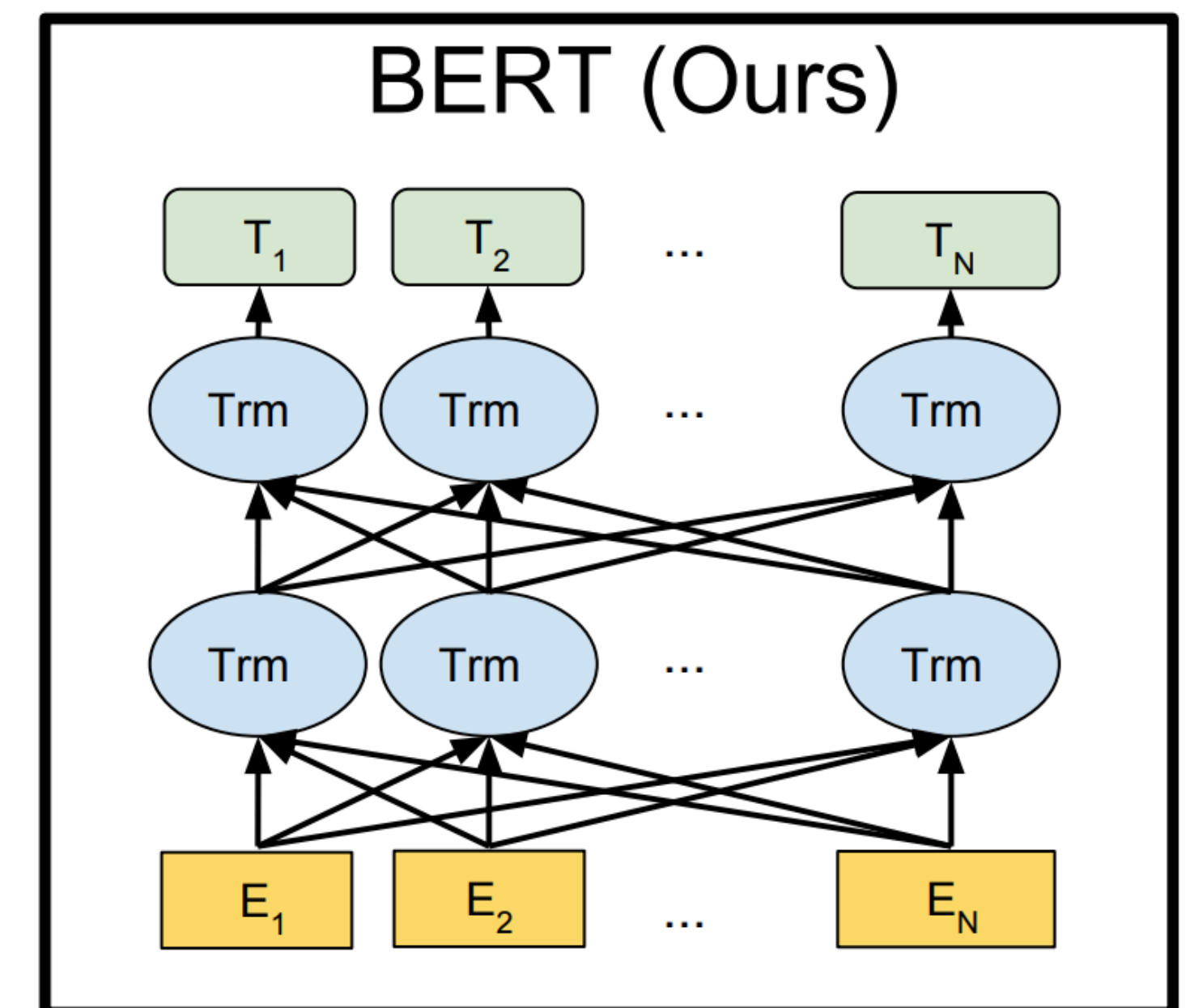
Summary: Transformer Uses

- ▶ Unsupervised: transformers work better than LSTM for unsupervised pre-training of embeddings — predict word given context words
- ▶ BERT (Bidirectional Encoder Representations from Transformers):
pretraining transformer language models similar to ELMo (based on LSTM)



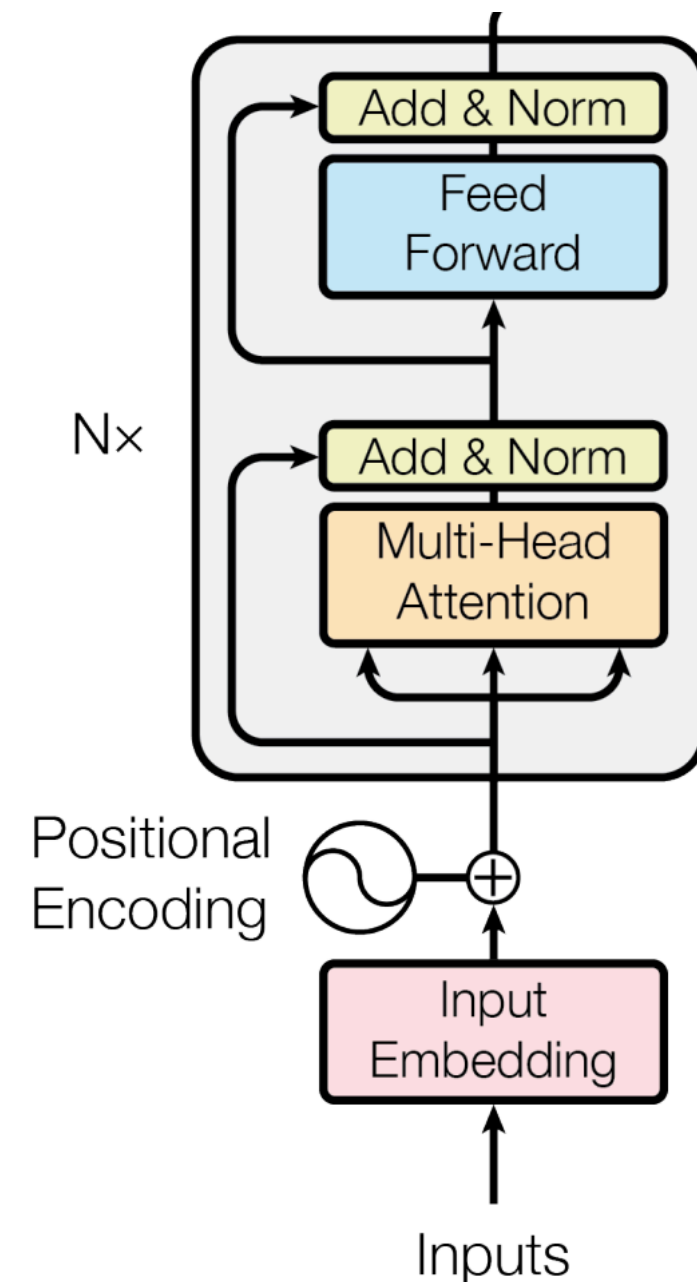
Summary: Transformer Uses

- ▶ Unsupervised: transformers work better than LSTM for unsupervised pre-training of embeddings — predict word given context words
- ▶ BERT (Bidirectional Encoder Representations from Transformers): pretraining transformer language models similar to ELMo (based on LSTM)
- ▶ Stronger than similar methods, SOTA on ~11 tasks (including NER — 92.8 F1)



Other Transformer Variations

- ▶ Multilayer transformer networks consist of interleaved self-attention and feedforward sublayers.
- ▶ Could ordering the sublayers in a different pattern lead to better performance?



s f s f s f s f s f s f s f s f s f s f s f s f

(a) Interleaved Transformer

s s s s s s s f s f s f s f s f s f s f s f f f f f f f

(b) Sandwich Transformer

Figure 1: A transformer model (a) is composed of interleaved self-attention (green) and feedforward (purple) sublayers. Our sandwich transformer (b), a reordering of the transformer sublayers, performs better on language modeling. Input flows from left to right.

Other Transformer Variations

- Mixture of Expert (MoE) Transformer, e.g., used in massively multilingual MT

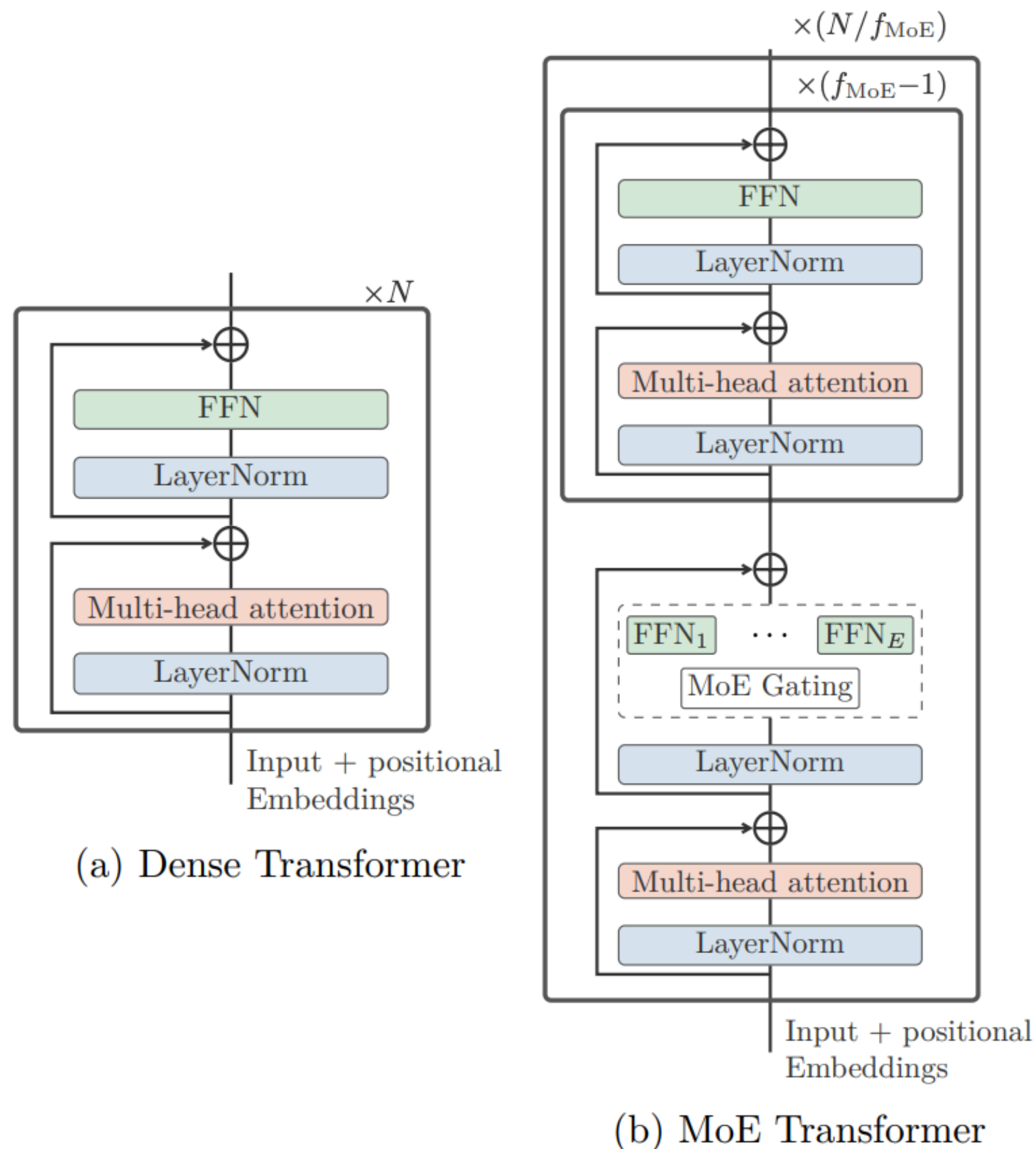
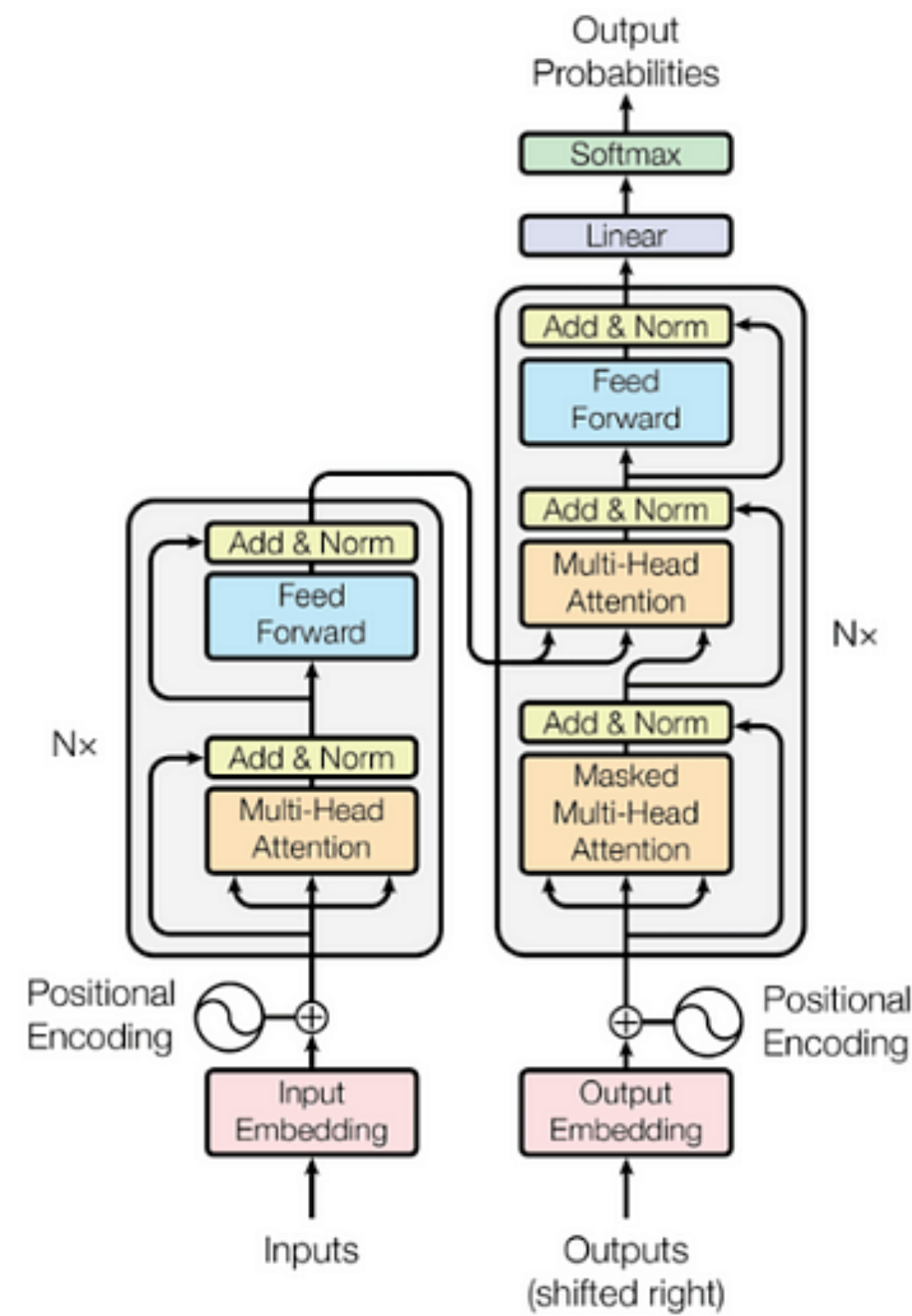


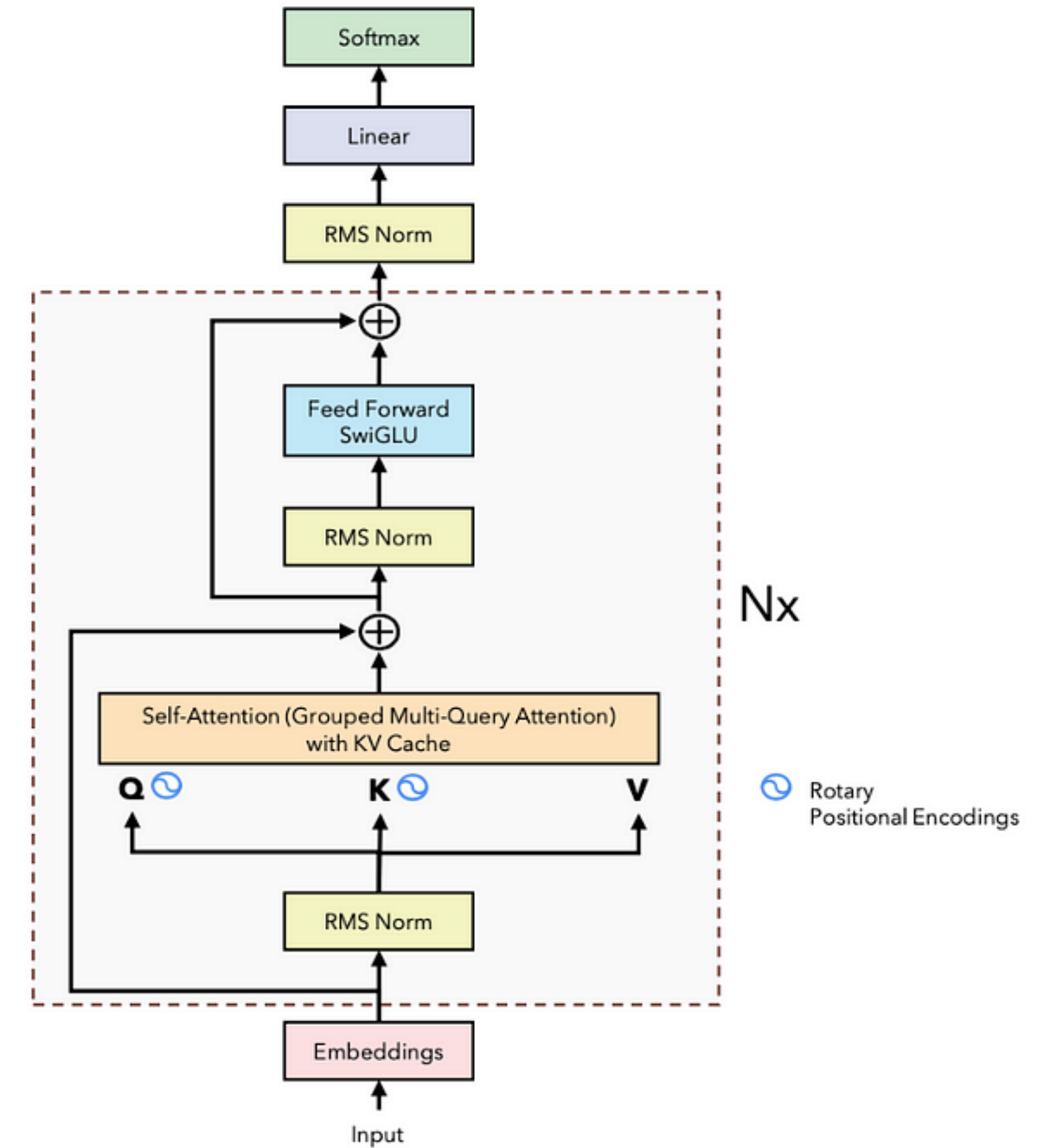
Figure 16: **Illustration of a Transformer encoder with MoE layers** inserted at a $1:f_{\text{MoE}}$ frequency. Each MoE layer has E experts and a gating network responsible for dispatching tokens.

Eigen et al. (2013), Shazeer et al. (2017), NLLB (2022)

Transformer as in LLaMA-3 (later)



Transformer



LLaMA

Takeaways

- ▶ RNN — Sequential model that works well to present language. Attention mechanism is very effective, especially in seq2seq models.
- ▶ CNN — CNNs are a flexible way of extracting features analogous to bag of n-grams, can also encode positional information
- ▶ Transformer — No recurrent units (a computation bottleneck), effectively capturing context and long dependencies. Multi-heads analogous to different convolutional filters.

Normalization

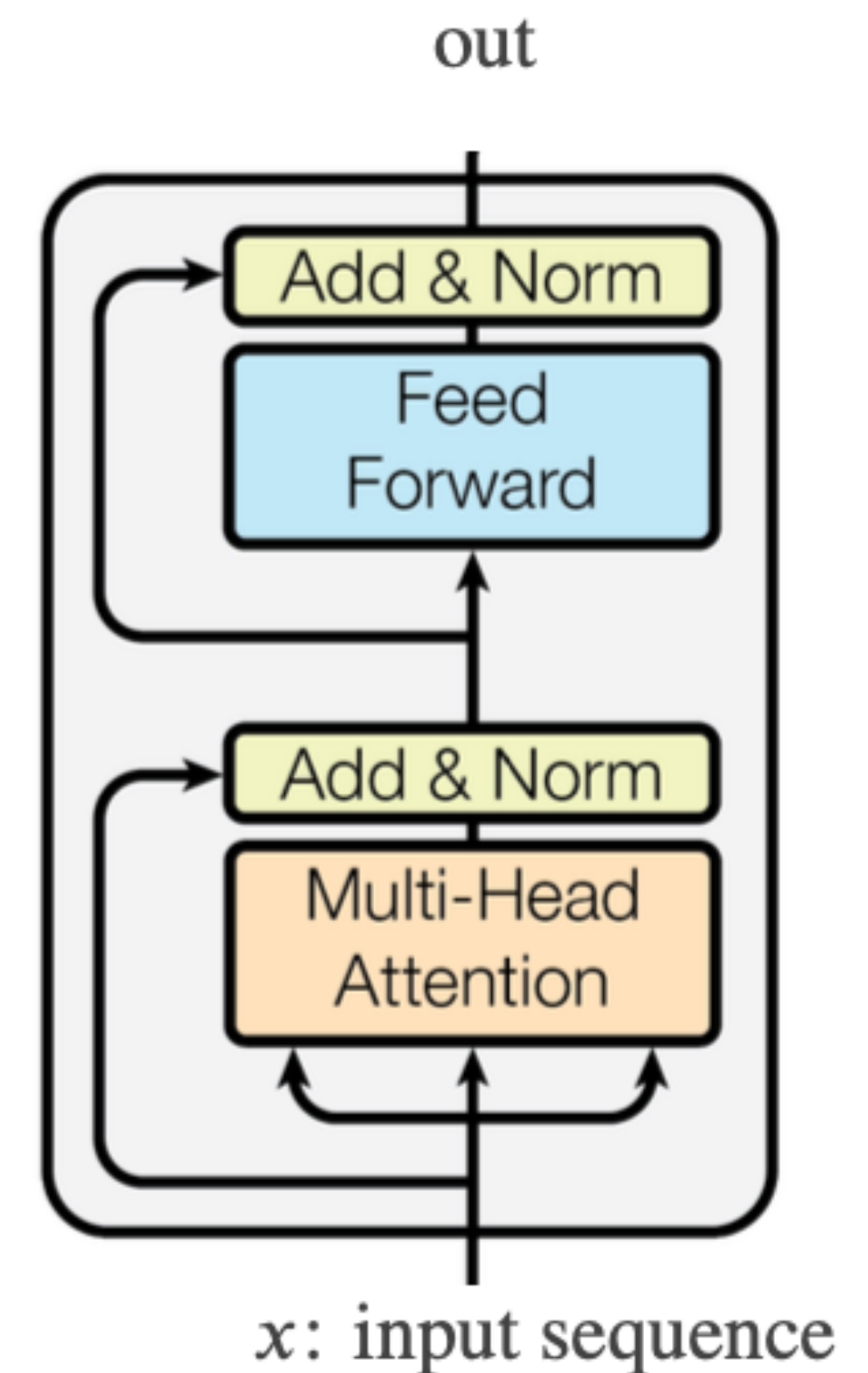
Pre-norm vs. Post-norm

- Original Transformer — “Add” before “Norm”, or “Norm” before “Add”?

$$\textit{LayerNorm}(x + \textit{SubLayer}(x))$$

or

$$x + \textit{SubLayer}(\textit{LayerNorm}(x))$$



(Baevski & Auli, 2018; Child et al., 2019; Wang et al., 2019)

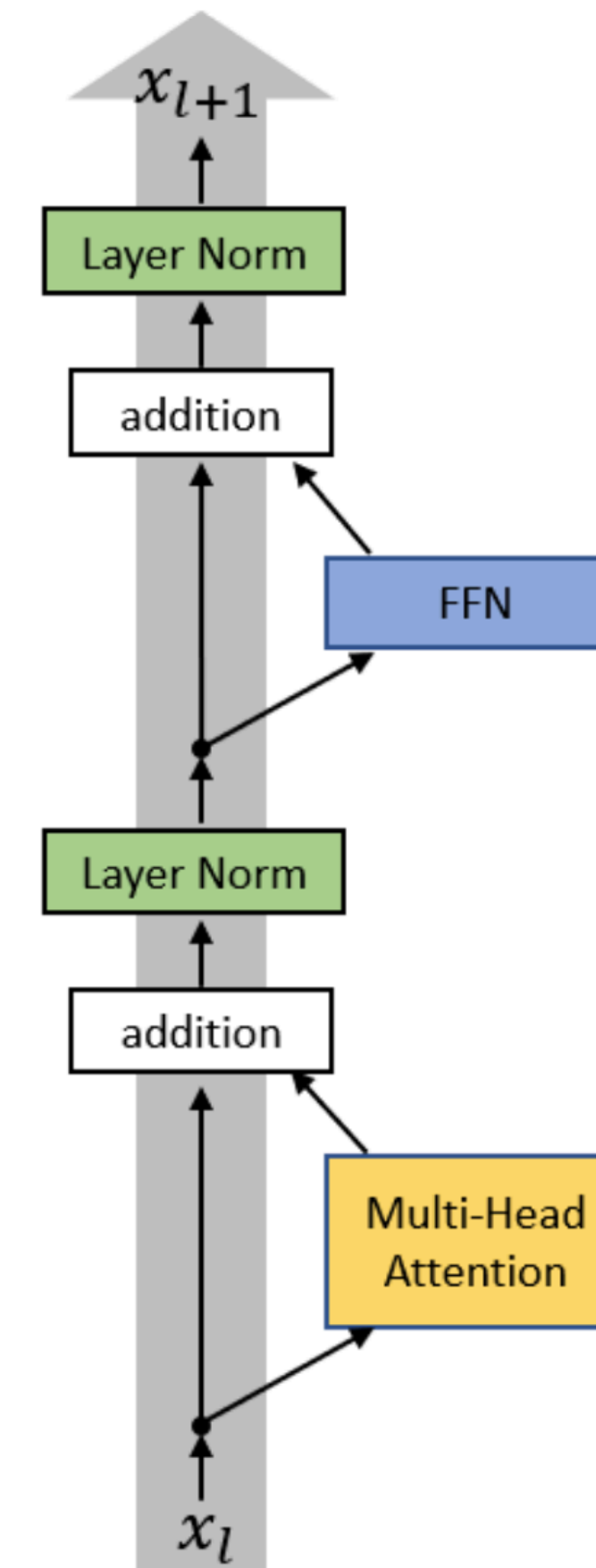
Pre-norm vs. Post-norm

- Original Transformer — “Add” before “Norm”, or “Norm” before “Add”?

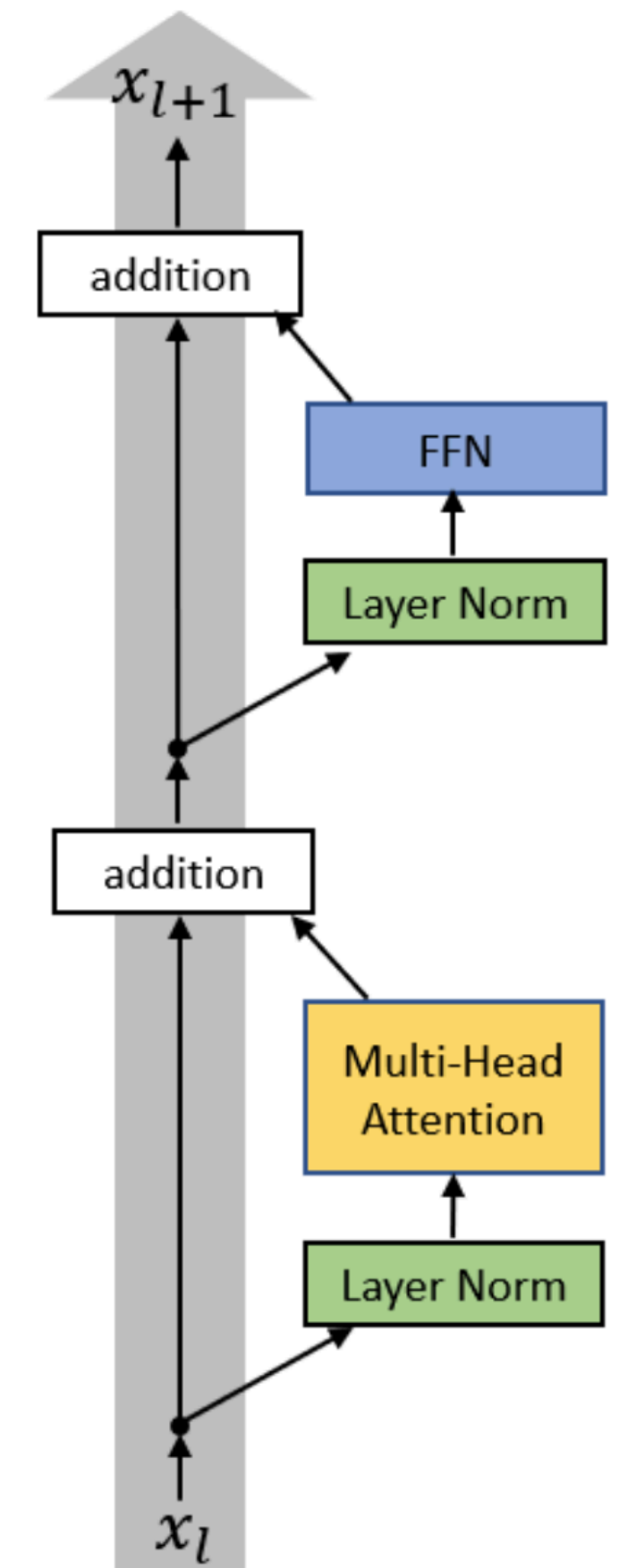
$$\textit{LayerNorm}(x + \textit{SubLayer}(x))$$

or

$$x + \textit{SubLayer}(\textit{LayerNorm}(x))$$



(a)

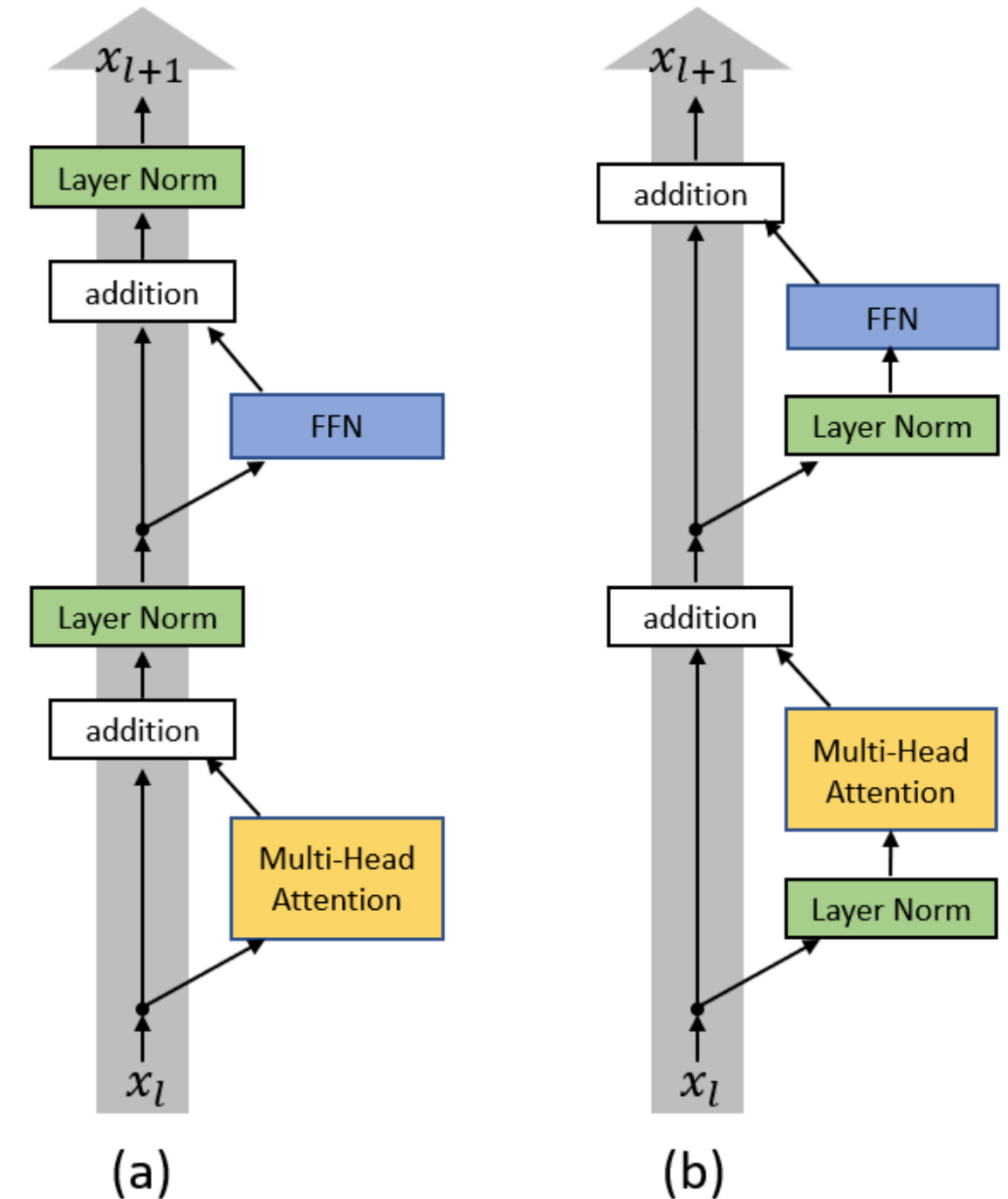


(b)

(Baevski & Auli, 2018; Child et al., 2019; Wang et al., 2019)

Pre-norm vs. Post-norm

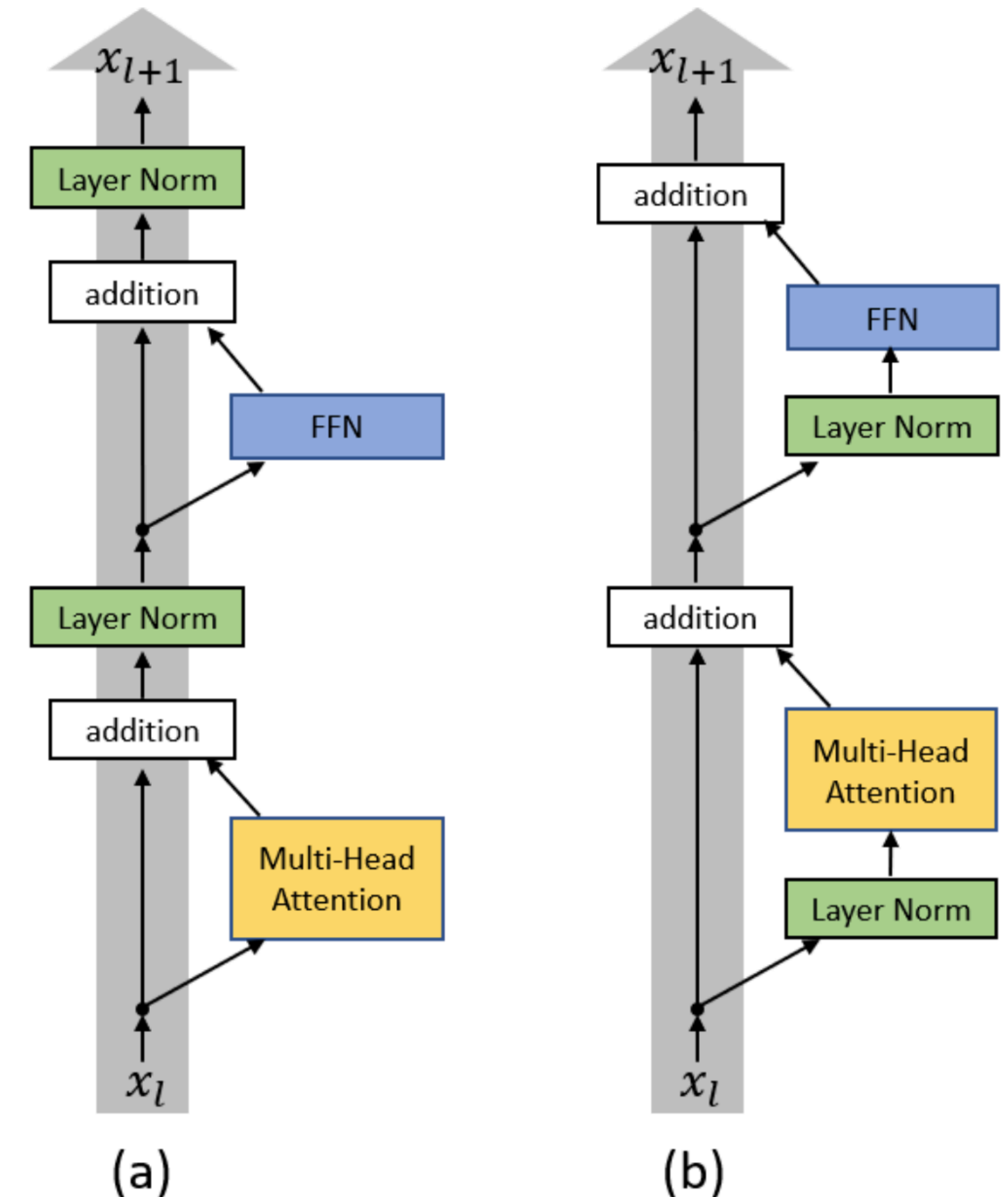
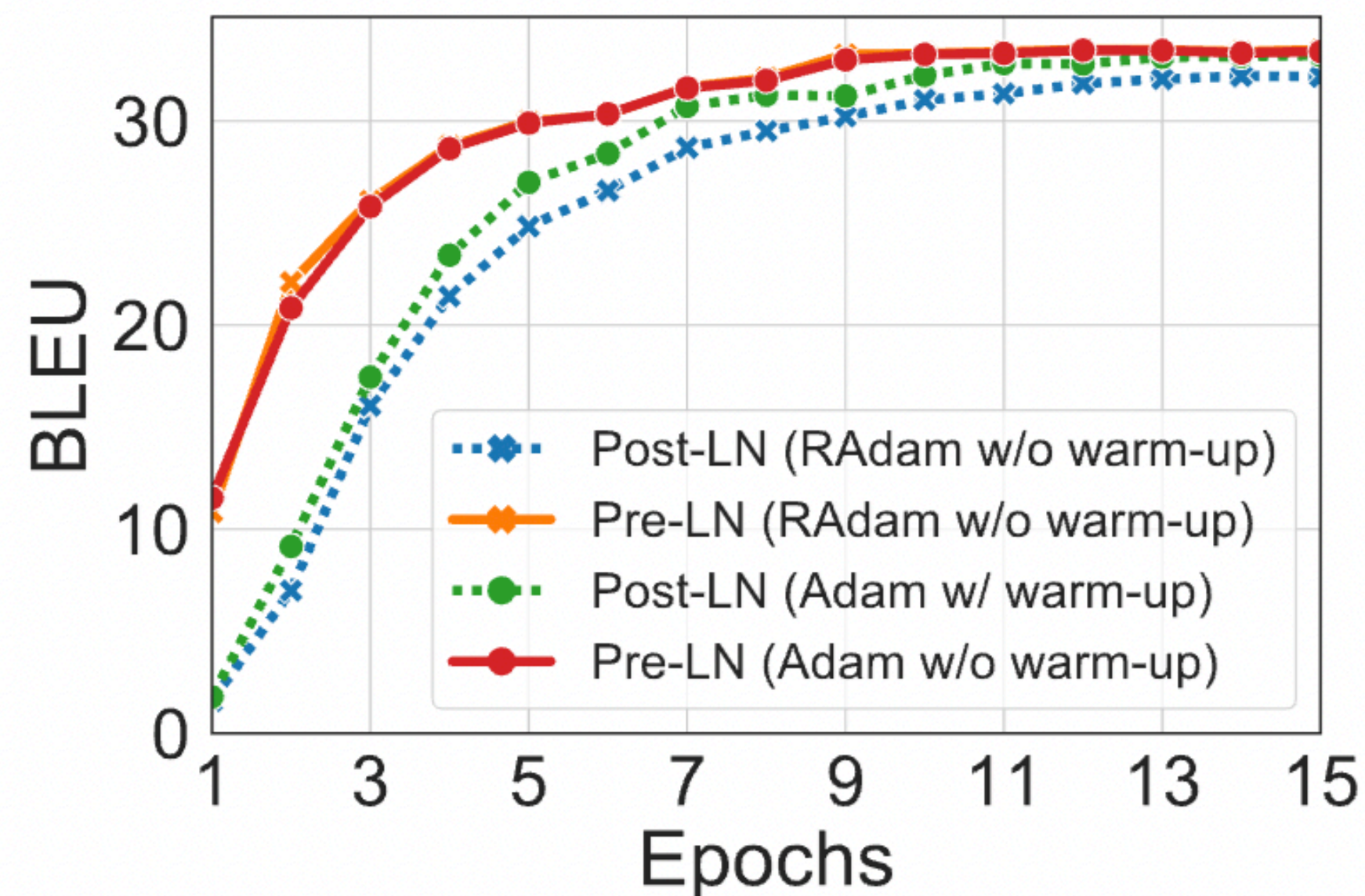
- ▶ Pre-normalization Transformer (b) to have better-behaved gradients at initialization than in the original Transformer (a)



Xiong et al. (2020)

Pre-norm vs. Post-norm

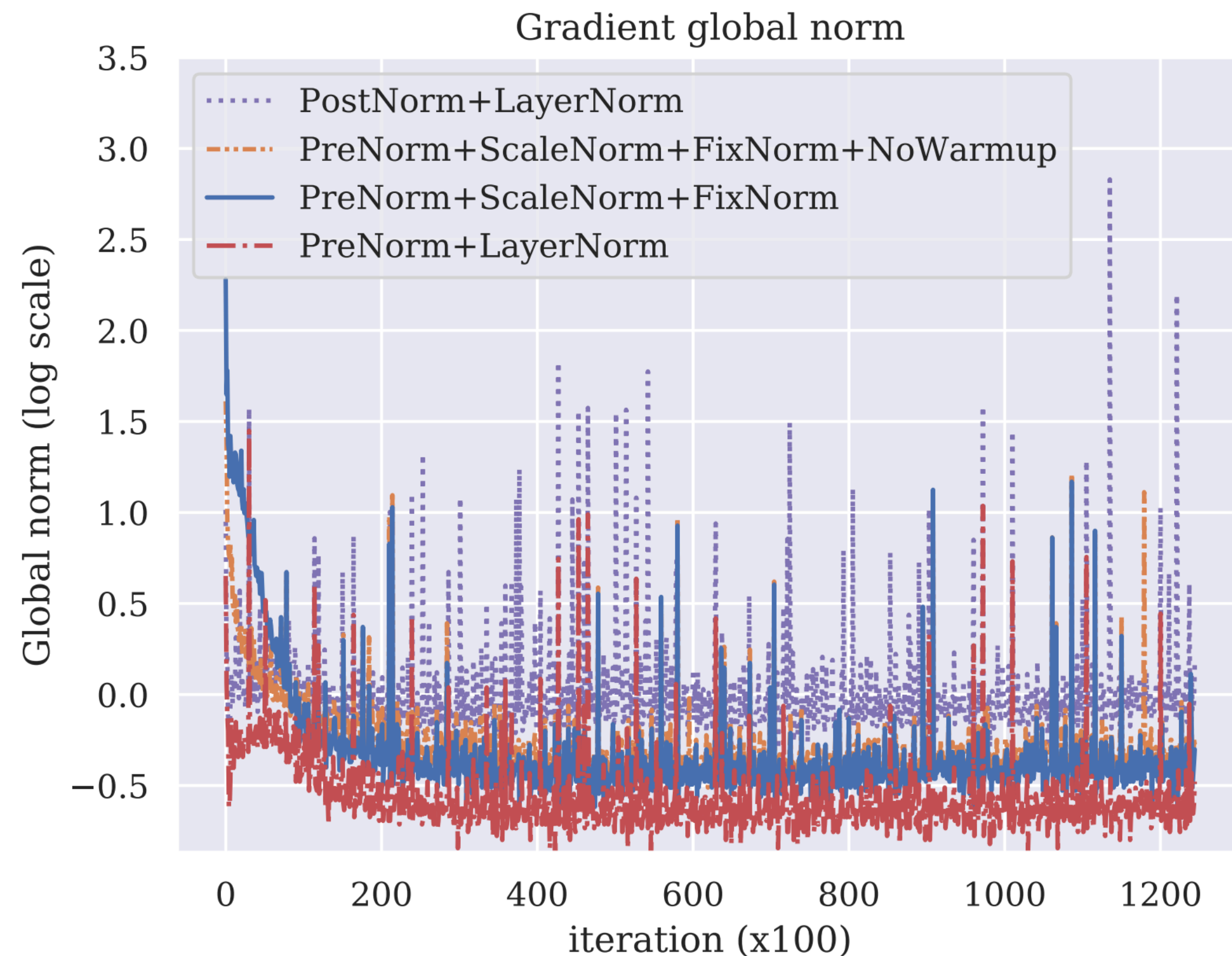
- ▶ Pre-normalization Transformer (b) to have better-behaved gradients at initialization than in the original Transformer (a)
- ▶ In (b), LayerNorm does not disrupt residual



Xiong et al. (2020)

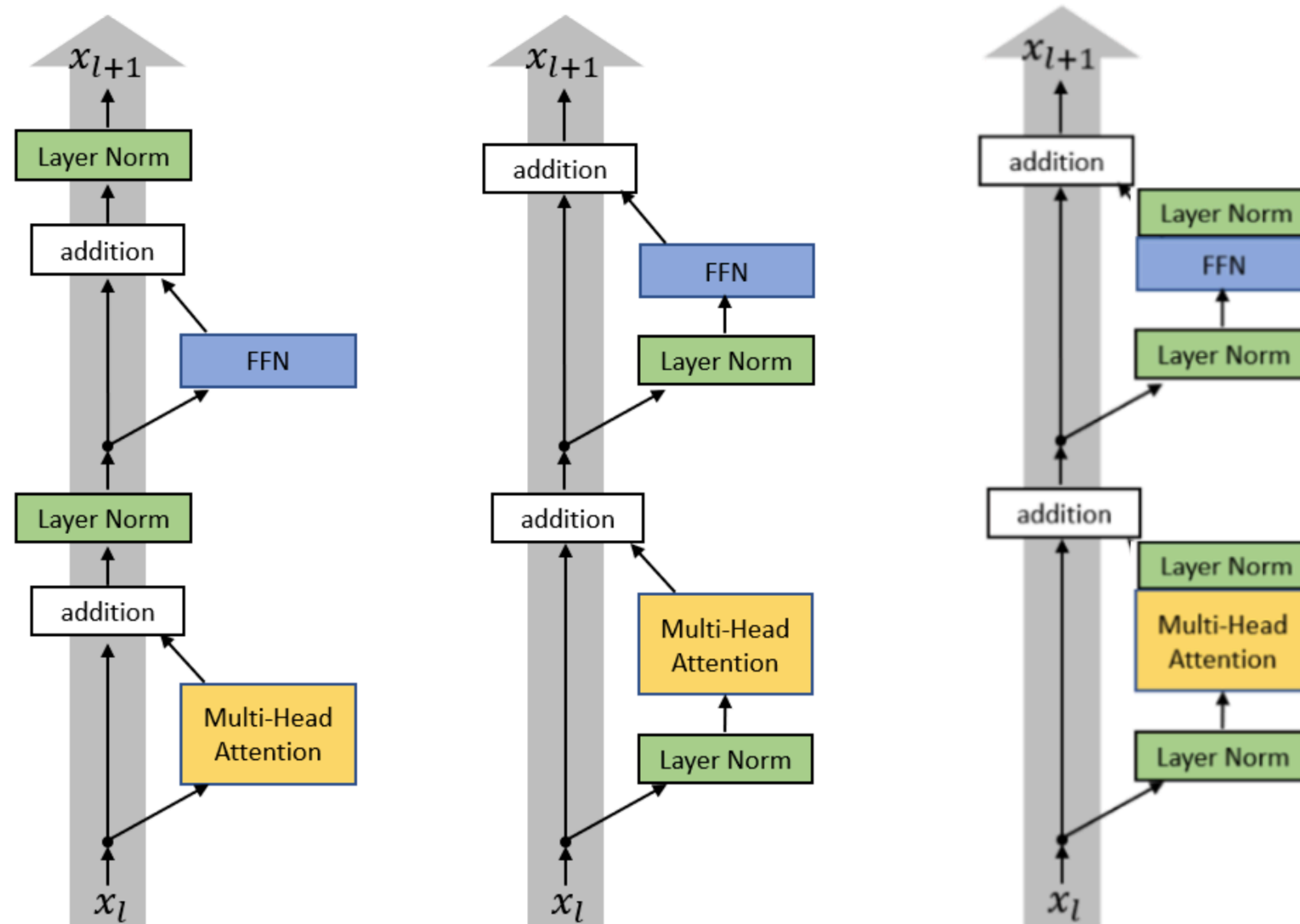
Pre-norm vs. Post-norm

- ▶ Post-norm produces noisy gradients with many tall spikes, needs warm up
- ▶ Pre-norm has fewer noisy gradients with smaller sizes, even without warmup



Pre-norm vs. Post-norm

- ▶ Today, almost all LLMs use pre-norm for stability and larger learning rates for training large networks.
- ▶ Gemma 2/3 used both pre-norm and post-norm.



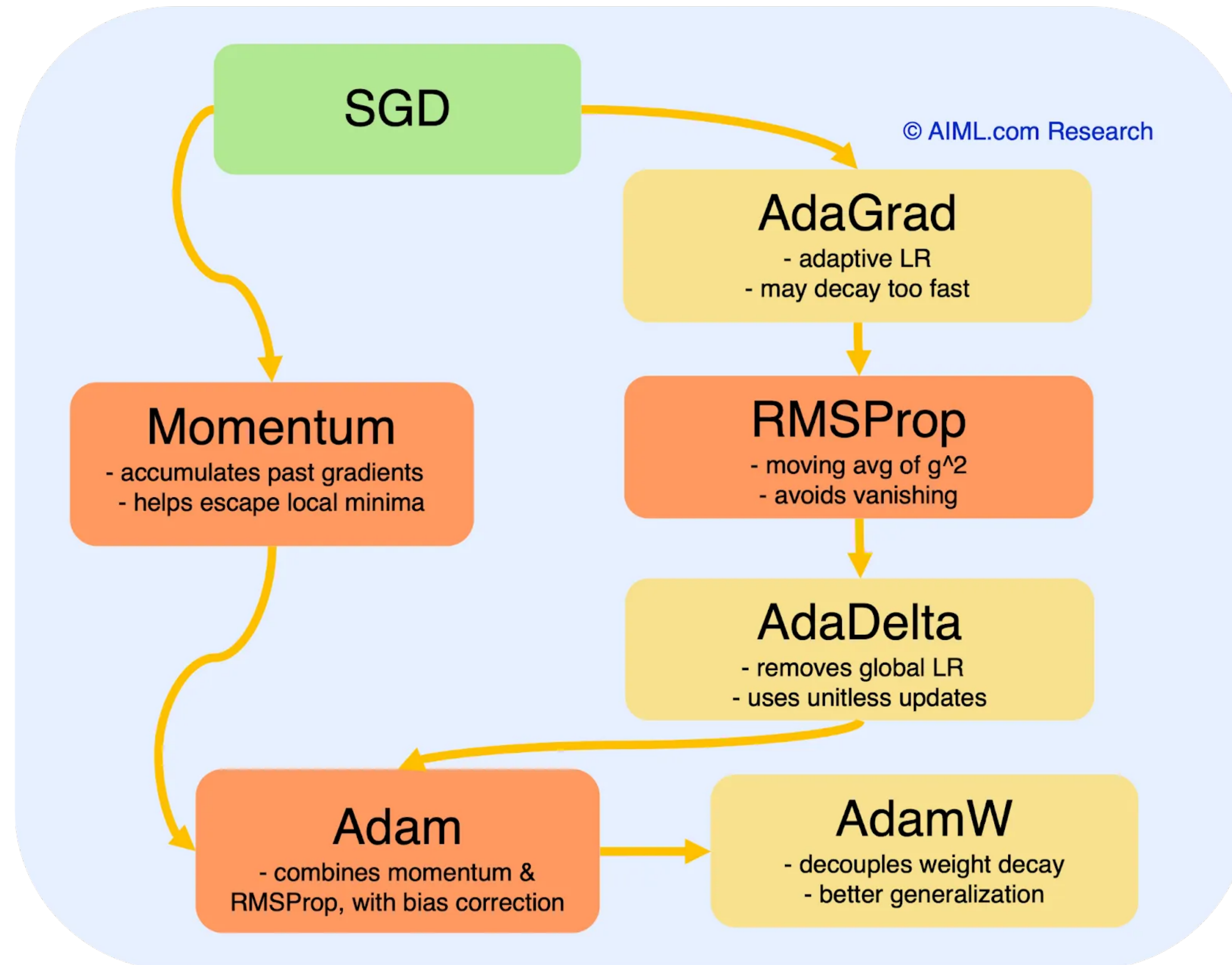
Pre-norm vs. Post-norm

- ▶ Today, almost all LLMs use pre-norm for stability and larger learning rates for training large networks.
- ▶ Gemma 2/3 used both pre-norm and post-norm.

```
class Gemma3DecoderLayer(GradientCheckpointingLayer):  
    def __init__(self, config: Gemma3TextConfig, layer_idx: int):  
        super().__init__()  
        self.config = config  
        self.hidden_size = config.hidden_size  
        self.layer_idx = layer_idx  
        self.attention_type = config.layer_types[layer_idx]  
        self.self_attn = Gemma3Attention(config=config, layer_idx=layer_idx)  
        self.mlp = Gemma3MLP(config)  
        self.input_layernorm = Gemma3RMSNorm(self.hidden_size, eps=config.rms_norm_eps)  
        self.post_attention_layernorm = Gemma3RMSNorm(self.hidden_size, eps=config.rms_norm_eps)  
        self.pre_feedforward_layernorm = Gemma3RMSNorm(self.hidden_size, eps=config.rms_norm_eps)  
        self.post_feedforward_layernorm = Gemma3RMSNorm(self.hidden_size, eps=config.rms_norm_eps)
```

Optimizers

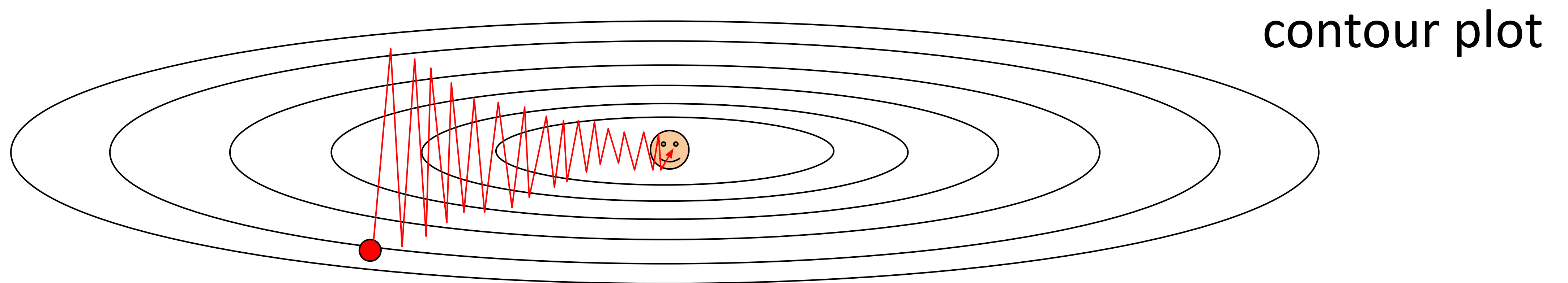
Common Optimizers



(Recap) Problems with SGD

- ▶ Gradient descent $\theta_{t+1} \leftarrow \theta_t - \alpha \nabla f(\theta_t)$

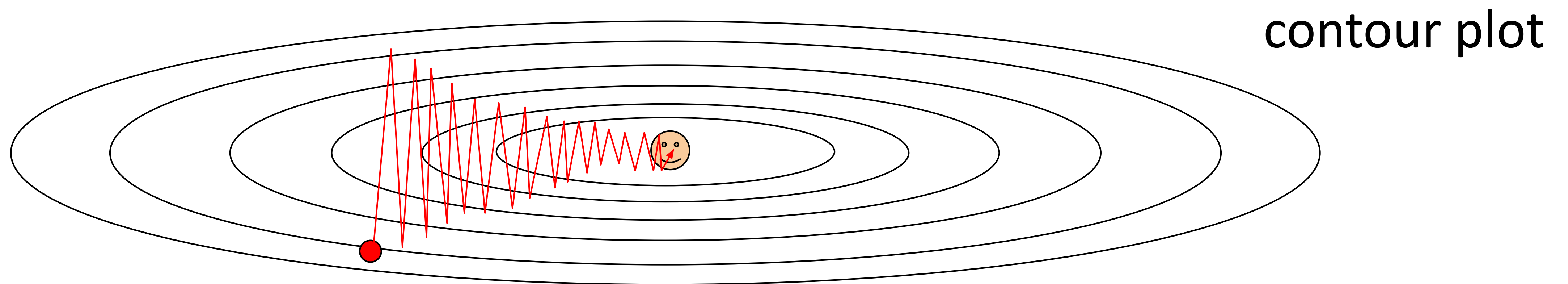
Q: What if loss changes quickly in one direction and slowly in another direction?



(Recap) Problems with SGD

- ▶ Gradient descent $\theta_{t+1} \leftarrow \theta_t - \alpha \nabla f(\theta_t)$

Q: What if loss changes quickly in one direction and slowly in another direction?

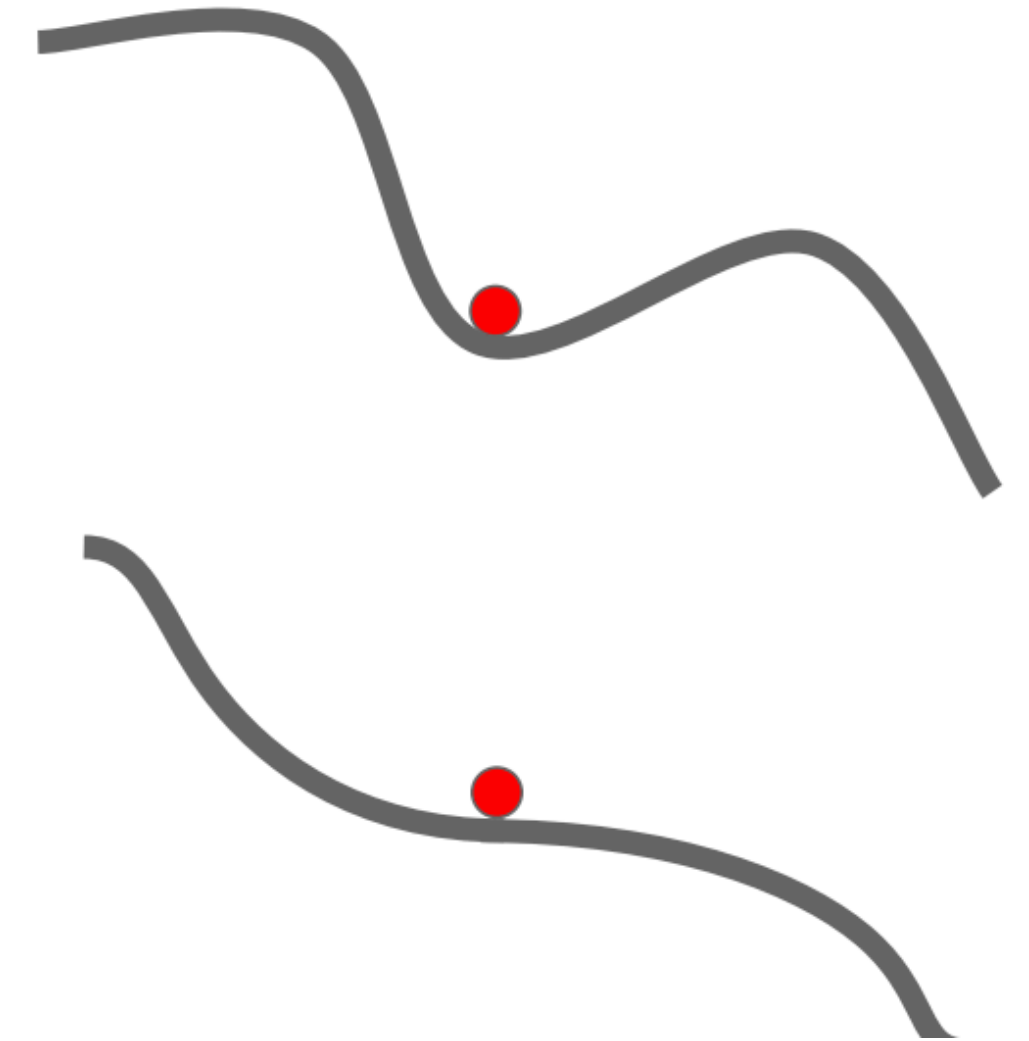


Solution: feature scaling!

Credit: Stanford CS231n

(Recap) Problems with SGD

- ▶ Gradient descent $\theta_{t+1} \leftarrow \theta_t - \alpha \nabla f(\theta_t)$
- ▶ SGD is sensitive to the choice of learning rate
- ▶ Gradients estimated from mini-batches can be noisy
- ▶ Non-convex loss function with local minima, saddle points



Momentum

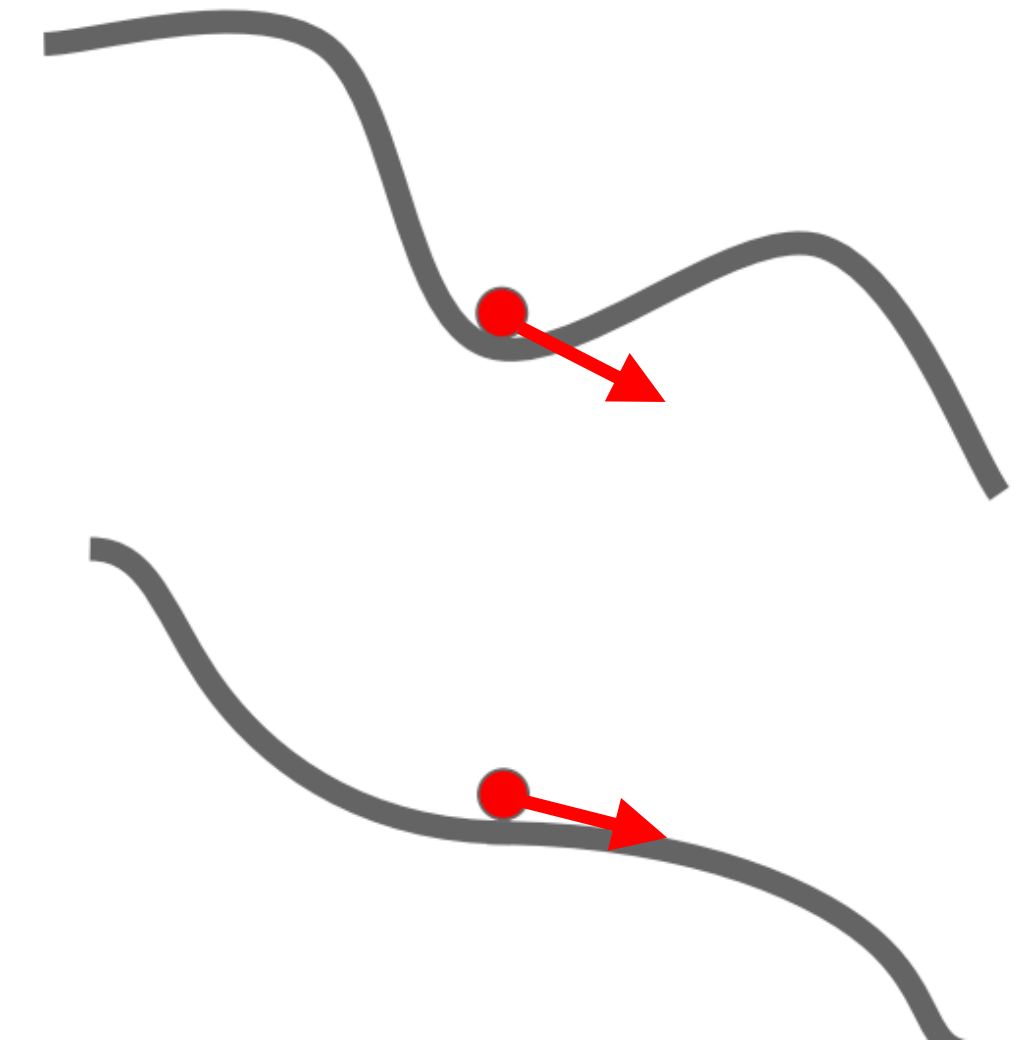
- ▶ Gradients come from a single instance or a mini-batch can be noisy
- ▶ Use “velocity” to accumulate the gradients from the past steps

Standard SGD

$$\theta_{t+1} \leftarrow \theta_t - \alpha \underbrace{\nabla f(\theta_t)}_{\text{current gradient}}$$

SGD with Momentum

$$\begin{aligned} \mathbf{m}_{t+1} &\leftarrow \beta \mathbf{m}_t + \nabla f(\theta_t) \\ \theta_{t+1} &\leftarrow \theta_t - \alpha \underbrace{\mathbf{m}_{t+1}}_{\text{accumulated gradient}} \end{aligned}$$



Polyak (1964), Sutskever et al. (2013)

Image credit: Stanford CS231N

AdaGrad

- ▶ Per-parameter learning rate: parameters that have received large gradients in the past get smaller effective learning rates

Standard SGD

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \underbrace{\alpha \nabla f(\boldsymbol{\theta}_t)}_{\text{current gradient}}$$

AdaGrad

- ▶ accumulate the squared gradients

$$\mathbf{v}_{t+1} \leftarrow \mathbf{v}_t + (\nabla f(\boldsymbol{\theta}_t))^2$$

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\mathbf{v}_{t+1}} + \epsilon} \nabla f(\boldsymbol{\theta}_t)$$

AdaGrad

- ▶ Per-parameter learning rate: parameters that have received large gradients in the past get smaller effective learning rates

Standard SGD

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \underbrace{\alpha \nabla f(\boldsymbol{\theta}_t)}_{\text{current gradient}}$$

AdaGrad

- ▶ accumulate the squared gradients

$$\mathbf{v}_{t+1} \leftarrow \mathbf{v}_t + (\nabla f(\boldsymbol{\theta}_t))^2$$

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \boxed{\frac{\alpha}{\sqrt{\mathbf{v}_{t+1}} + \epsilon}} \nabla f(\boldsymbol{\theta}_t)$$

- ▶ Generally more robust than SGD, requires less tuning of learning rate

Problems with AdaGrad

- ▶ learning rates keep decreasing (too fast) towards 0

AdaGrad

keep increasing


$$\mathbf{v}_{t+1} \leftarrow \mathbf{v}_t + (\nabla f(\boldsymbol{\theta}_t))^2$$

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\mathbf{v}_{t+1}} + \epsilon} \nabla f(\boldsymbol{\theta}_t)$$

RMSProp

- ▶ Using an exponential moving average of squared gradients, instead of accumulating them forever. (typically $\beta \approx 0.9$)

RMSProp

“forget” some past

$$\mathbf{v}_{t+1} \leftarrow \boxed{\beta} \mathbf{v}_t + \boxed{(1 - \beta)} (\nabla f(\boldsymbol{\theta}_t))^2$$

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\mathbf{v}_{t+1}} + \epsilon} \nabla f(\boldsymbol{\theta}_t)$$

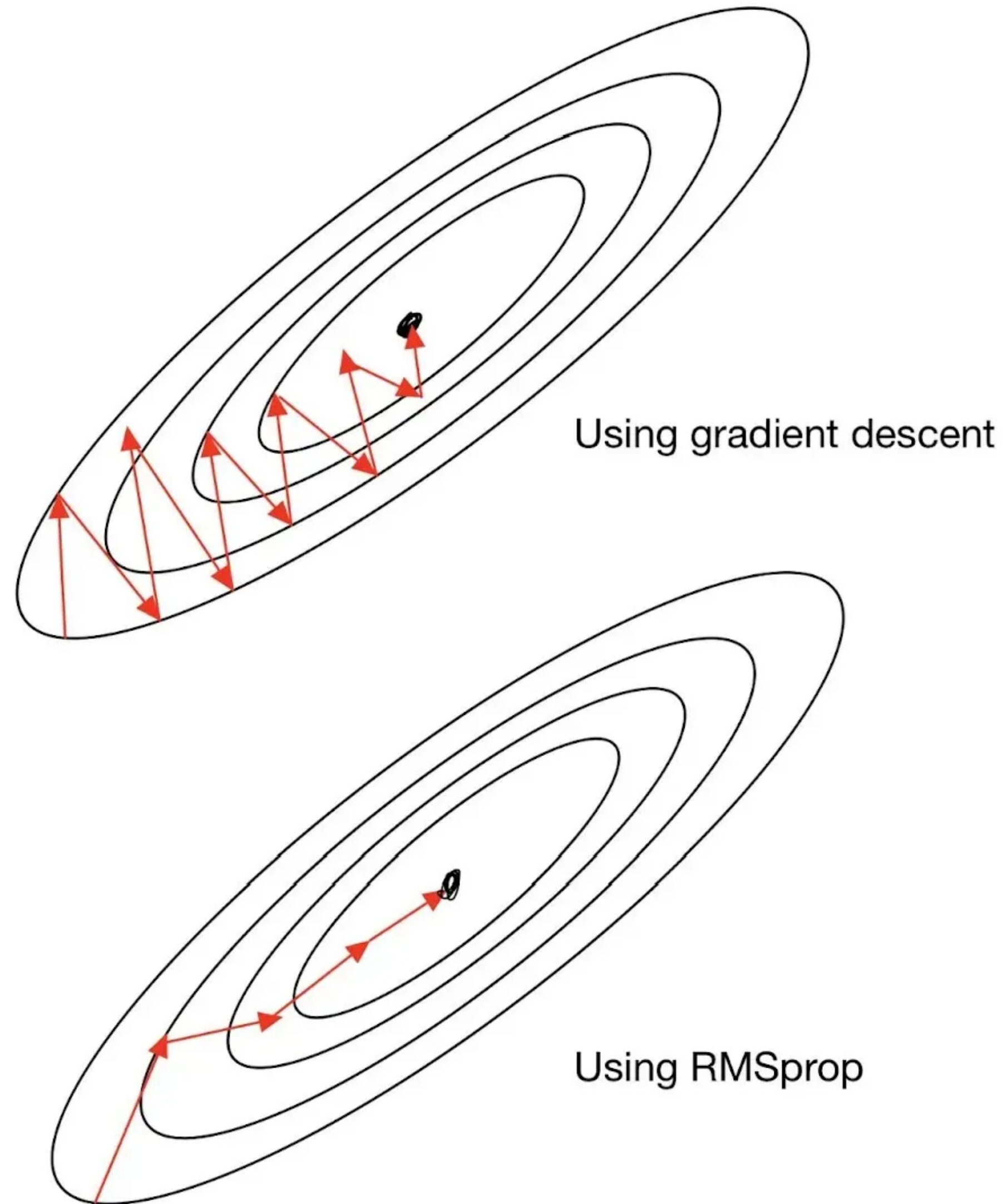
AdaGrad

keep increasing

$$\mathbf{v}_{t+1} \leftarrow \mathbf{v}_t + (\nabla f(\boldsymbol{\theta}_t))^2$$

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\mathbf{v}_{t+1}} + \epsilon} \nabla f(\boldsymbol{\theta}_t)$$

What we have so far?



Standard SGD

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \underbrace{\alpha \nabla f(\boldsymbol{\theta}_t)}_{\text{current gradient}}$$

adaptive
learning rate

RMSProp

$$\mathbf{v}_{t+1} \leftarrow \beta \mathbf{v}_t + (1 - \beta) (\nabla f(\boldsymbol{\theta}_t))^2$$

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\mathbf{v}_{t+1}} + \epsilon} \nabla f(\boldsymbol{\theta}_t)$$

What we have so far?

Standard SGD

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha \underbrace{\nabla f(\boldsymbol{\theta}_t)}_{\text{current gradient}}$$

adaptive
learning rate

RMSProp

$$\begin{aligned} \mathbf{v}_{t+1} &\leftarrow \beta \mathbf{v}_t + (1 - \beta) (\nabla f(\boldsymbol{\theta}_t))^2 \\ \boldsymbol{\theta}_{t+1} &\leftarrow \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\mathbf{v}_{t+1}} + \epsilon} \nabla f(\boldsymbol{\theta}_t) \end{aligned}$$

escape local minima
saddle point

SGD with Momentum

$$\mathbf{m}_{t+1} \leftarrow \beta \mathbf{m}_t + \nabla f(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha \underbrace{\mathbf{m}_{t+1}}_{\text{accumulated gradient}}$$

Adam $\approx$ Momentum + RMSProp

escape local minima
saddle point



SGD with Momentum

$$\mathbf{m}_{t+1} \leftarrow \beta \mathbf{m}_t + \nabla f(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \underbrace{\alpha \mathbf{m}_{t+1}}_{\text{accumulated gradient}}$$

Combine
together!



adaptive
learning rate

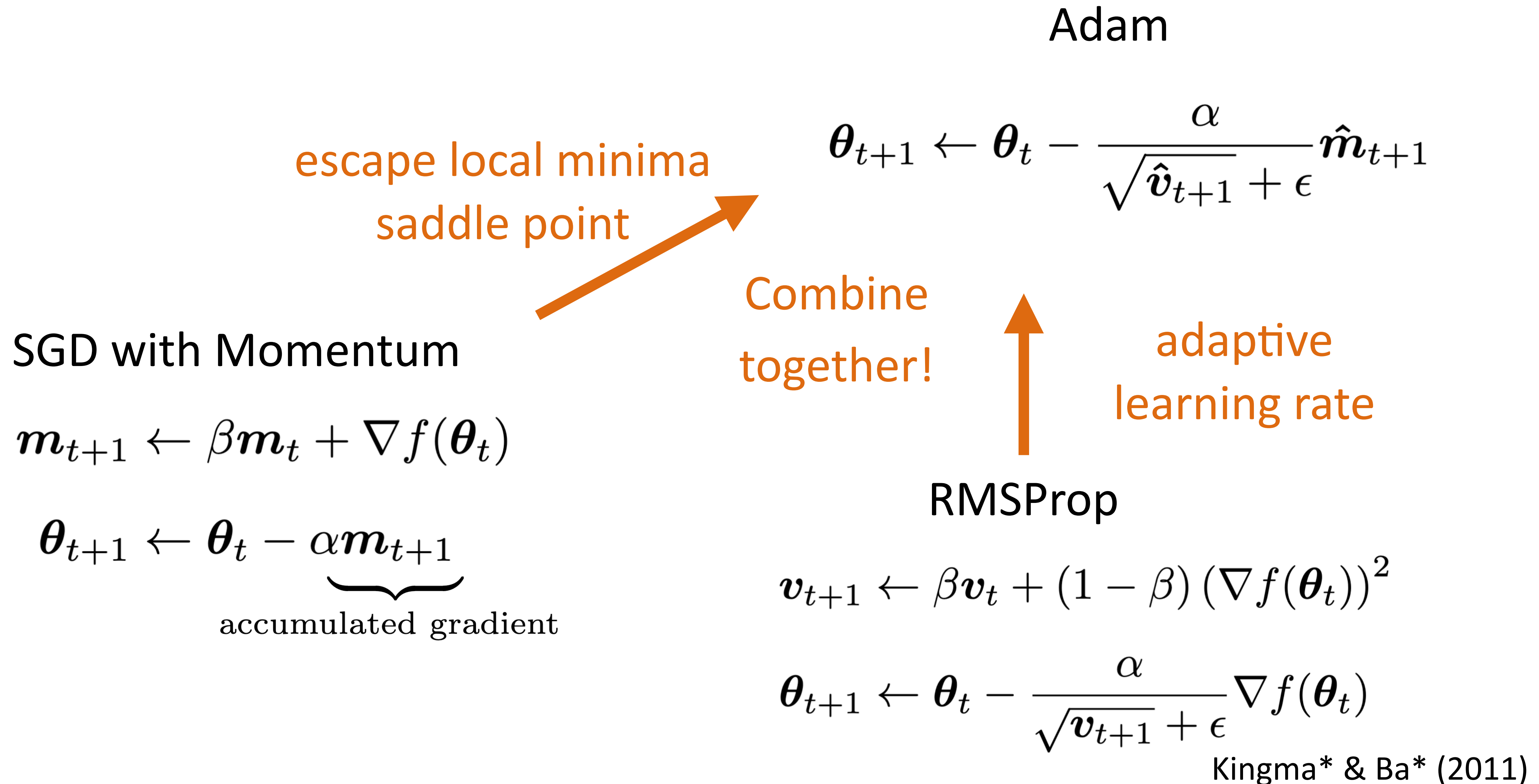
RMSProp

$$\mathbf{v}_{t+1} \leftarrow \beta \mathbf{v}_t + (1 - \beta) (\nabla f(\boldsymbol{\theta}_t))^2$$

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\mathbf{v}_{t+1}} + \epsilon} \nabla f(\boldsymbol{\theta}_t)$$

Kingma* & Ba* (2011)

Adam $\approx$ Momentum + RMSProp



Adam (Adaptive Moment Estimation)

Adam

$$\theta_{t+1} \leftarrow \theta_t - \frac{\alpha}{\sqrt{\hat{v}_{t+1}} + \epsilon} \hat{m}_{t+1}$$

escape local minima
saddle point

Combine
together!

adaptive
learning rate

SGD with Momentum

$$\mathbf{m}_{t+1} \leftarrow \beta \mathbf{m}_t + \nabla f(\theta_t)$$

1st Moment: exponentially decaying
average of past gradients

RMSProp

$$\mathbf{v}_{t+1} \leftarrow \beta \mathbf{v}_t + (1 - \beta) (\nabla f(\theta_t))^2$$

2nd Moment: exponentially decaying
average of past **squared** gradients

Adam (Adaptive Moment Estimation)

Adam

$$\theta_{t+1} \leftarrow \theta_t - \frac{\alpha}{\sqrt{\hat{v}_{t+1}} + \epsilon} \hat{m}_{t+1}$$

bias correction

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$
$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Combine
together!

RMSProp

$$v_{t+1} \leftarrow \beta v_t + (1 - \beta) (\nabla f(\theta_t))^2$$

2nd Moment: exponentially decaying
average of past **squared** gradients

SGD with Momentum

$$m_{t+1} \leftarrow \beta m_t + \nabla f(\theta_t)$$

1st Moment: exponentially decaying
average of past gradients

Adam (Adaptive Moment Estimation)

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

Diederik P. Kingma*
University of Amsterdam, OpenAI
dpkingma@openai.com

Jimmy Lei Ba*
University of Toronto
jimmy@psi.utoronto.ca

ABSTRACT

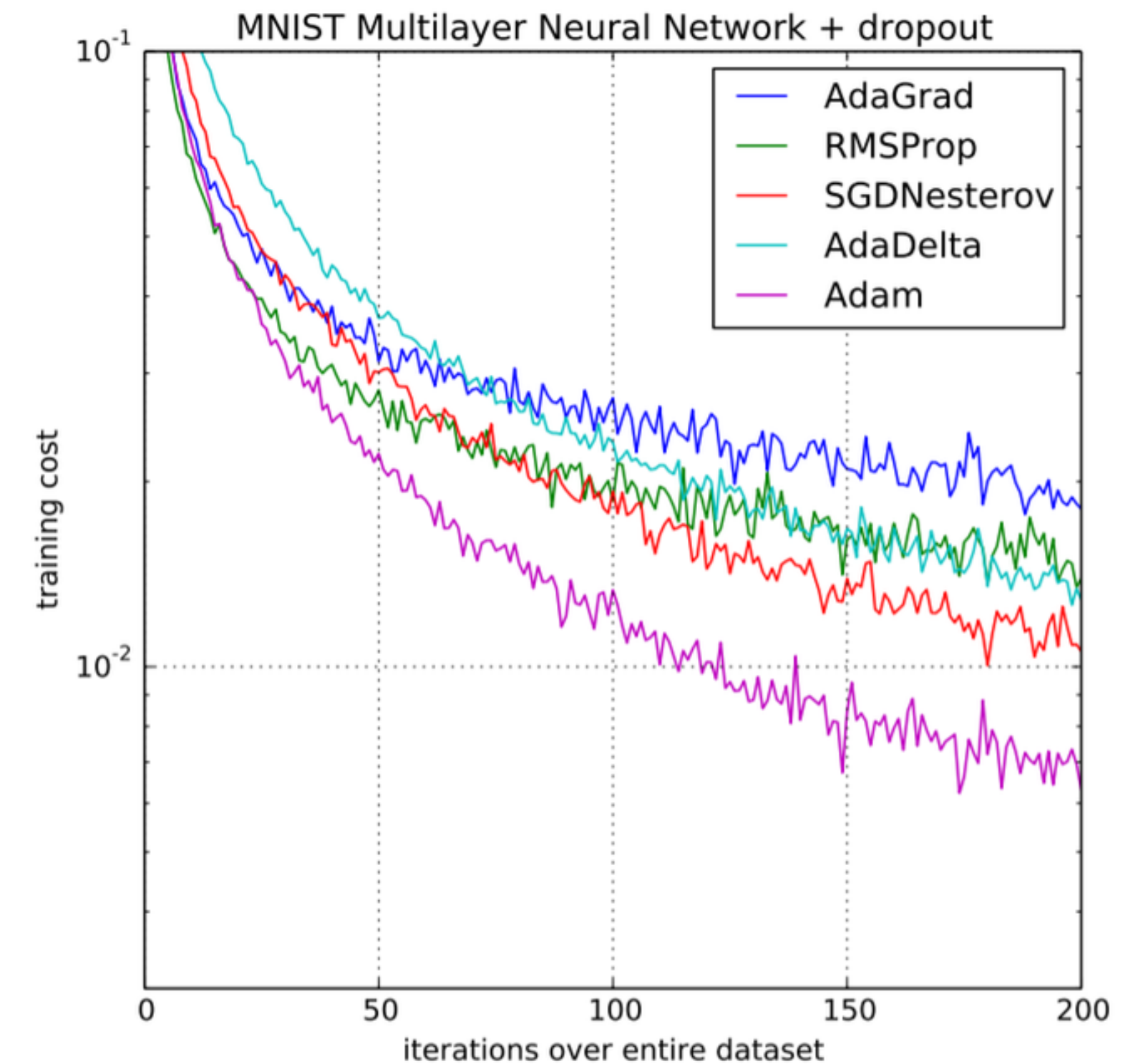
We introduce *Adam*, an algorithm for first-order gradient-based optimization of stochastic objective functions, based on adaptive estimates of lower-order moments. The method is straightforward to implement, is computationally efficient, has little memory requirements, is invariant to diagonal rescaling of the gradients, and is well suited for problems that are large in terms of data and/or parameters. The method is also appropriate for non-stationary objectives and problems with very noisy and/or sparse gradients. The hyper-parameters have intuitive interpretations and typically require little tuning. Some connections to related algorithms, on which *Adam* was inspired, are discussed. We also analyze the theoretical convergence properties of the algorithm and provide a regret bound on the convergence rate that is comparable to the best known results under the online convex optimization framework. Empirical results demonstrate that Adam works well in practice and compares favorably to other stochastic optimization methods. Finally, we discuss *AdaMax*, a variant of *Adam* based on the infinity norm.

1 INTRODUCTION

Stochastic gradient-based optimization is of core practical importance in many fields of science and engineering. Many problems in these fields can be cast as the optimization of some scalar parameterized objective function requiring maximization or minimization with respect to its parameters. If the function is differentiable w.r.t. its parameters, gradient descent is a relatively efficient optimization method, since the computation of first-order partial derivatives w.r.t. all the parameters is of the same computational complexity as just evaluating the function. Often, objective functions are stochastic. For example, many objective functions are composed of a sum of subfunctions evaluated at different subsamples of data; in this case optimization can be made more efficient by taking gradient steps w.r.t. individual subfunctions, i.e. stochastic gradient descent (SGD) or ascent. SGD proved itself as an efficient and effective optimization method that was central in many machine learning success stories, such as recent advances in deep learning (Deng et al., 2013; Krizhevsky et al., 2012; Hinton & Salakhutdinov, 2006; Hinton et al., 2012a; Graves et al., 2013). Objectives may also have other sources of noise than data subsampling, such as dropout (Hinton et al., 2012b) regularization. For all such noisy objectives, efficient stochastic optimization techniques are required. The focus of this paper is on the optimization of stochastic objectives with high-dimensional parameters spaces. In these cases, higher-order optimization methods are ill-suited, and discussion in this paper will be restricted to first-order methods.

We propose *Adam*, a method for efficient stochastic optimization that only requires first-order gradients with little memory requirement. The method computes individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients; the name *Adam* is derived from adaptive moment estimation. Our method is designed to combine the advantages of two recently popular methods: AdaGrad (Duchi et al., 2011), which works well with sparse gradients, and RMSProp (Tieleman & Hinton, 2012), which works well in on-line and non-stationary settings; important connections to these and other stochastic optimization methods are clarified in section 5. Some of Adam's advantages are that the magnitudes of parameter updates are invariant to rescaling of the gradient, its stepsizes are approximately bounded by the stepsize hyperparameter, it does not require a stationary objective, it works with sparse gradients, and it naturally performs a form of step size annealing.

*Equal contribution. Author ordering determined by coin flip over a Google Hangout.



*Equal contribution. Author ordering determined by coin flip over a Google Hangout.

Kingma* & Ba* (2011)